VERILOG PROGRAMMING IN VLSI EXPERT

Hi Sarif,

Please be informed that as part of our infrastructure upgrade, we are shifting your access from **prac2.trg.vlsiexpert.in** to a new server **prac3.trg.vlsiexpert.in.**

Your previous server access will be deactivated on Shortly and going forward you'll need to log in to prac3.trg.vlsiexpert.in

First Step:-
=== cmd or command prompt or terminal Login Credentials ===
ssh -p 23453 -L 7042:localhost:7042 b23n_sarif_727@prac3.trg.vlsiexpert.in

Password: sa78rfi85
7dkhZVm4Wf
ssh -p 23453 -L 7042:localhost:7042 b23n_sarif_727@route.trg.vlsiexpert.in

sa78rfi84
ssh -p 23453 -L 7042:localhost:7042 b23n_sarif_727@route2.trg.vlsiexpert.in

Second Step:-
=== VNC Login Credentials ===
VNC Server: localhost:7042
Password: 7KhdZVn4Wf

PS: See the below link for detail instructions for login
https://helpdesk.vlsiexpert.in/portal/en/kb/articles/userloginsetup-vlsi-expert

Kindly ensure secure and responsible usage of the VNC server.
If you encounter any issues or need assistance, feel free to reach out to our Technical Support Team.

Thanks & Warm Regards,
VLSI Expert IT Support

Email -
itsupport@vlsiexpert.in

Run Command

vcs half_adder.v -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 lab1.size_of_data]$ vcs -sverilog data_size.v -kdb -lca -debug_access+all -full64 && ./simv

./simv -verdi

verdi -cov -covdir simv.vdb/

vcs -sverilog compare.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

`include "uvm_macros.svh"

import uvm_pkg::*;

Dirct simulation use above command----imp

cd /data/pdk/pdk_cc

cd hspice/

vi reference40_models.lib

regex101 -SOFTWARE TOOL IN REGEX

vcs mod_16load_counter.v mod16_load_testbench.v -kdb -lca -debug_access+all -full64 && ./simv

https://support.google.com/a/answer/1371417?hl=en#Match-Word-or-Phrase-in-a-List

1.0 Design a half adder by using the gate level modelling

2.0 Design a half adder by data flow level modelling

3.0 Design a half adder by behavioural level modelling

4.0 Lab1.size_of_data and its value

5.0 Design a 2x1mux by using the behavioral modelling

6.0 Design a 4x1mux by using 2x1mux the behavioral modelling

7.0 Net types-tri

8.0 Net types-wand

9.0 Net types-wor

10.0 Tristate gates

11.0 Delay analysis concept

12.0 Design a Gate level Half Adder (HA) circuit

13.0 Design a Gate level Full Adder (FA) circuit

14.0 Design a Full Adder by using Half adder

15.0 Generating a clock signal

16.0 Design an +ve edge triggered Asynchronous D-FF (Active high reset)

17.0 Design an +ve edge triggered Synchronous D-FF (with synchronous reset)

18.0 Design a 4-bit magnitude comparator

19.0 Design a 1-bit full adder by using Nand gate

20.0 Design an 8x3 encoder by using the behavioral modelling

21.0 Design an 8x3 Priority encoder by using the behavioral modelling

22.0 Named event control-

23.0 Print a data at every +ve clock till i is not print 5 and print data=10*i by using the for loop

24.0 Print a data_ready triggering event at every +ve clock till i is not print 5 and print data=10*i by using @ symbol

25.0 make the unpacked array of size 10. At every index number, print the value should be square in index.

26.0 Design the SR-FF by using the gate level modelling

27.0 Design an 4x2 Priority encoder

28.0 Design a MOD-16 counter (4-bit)

29.0 Design an up-down counter with load data (4-bit)

30.0 Use fork-join (parallel execution) and initial-begin (sequential executions)

31.0 Use fork-join (parallel execution) + initial-begin (sequential executions)

32.0 Perform a factorial of a number by using task and function

33.0 Design a 4-bit F.A. using tasks and function by using 1-bit

34.0 Parameterized module by user using keyword parameter

35.0 Parameterized keyword module by using localparam

36.0 Design a 4-bit Full adder using generate block

37.0 Design a 4-bit Ripple Carry Adder by using generate and parameter statement

38.0 Design a full adder or a half adder with a parameter (sel)

39.0 Design a full adder or a half adder with a parameter (sel)

40.0 Design a 4-bit Binary to Gray code by using data flow

41.0 Design a generic n-bit Binary to Gray code by using data flow by using parameter keyword.

42.0 Design a 4-bit Gray to binary code by using data flow

43.0 Design a generic n-bit Gray to Binary code by using data flow by using parameter keyword.

44.0 Design a 4-bit Parity Checker by using data flow

45.0 Design a Generic N-bit parity checker by using data flow

46.0 Design a 4x1_mux by using User Defined Primitives (UDP)

47.0 Design an edge sensitive D-FF by using UDP

48.0 Design a level sensitive D-latch by using UDP

49.0 Mealy FSM to detect 1010 non_overlapping sequence detector-

50.0 Mealy FSM to detect 1010 Overlapping sequence detector-

51.0 Moore FSM to detect 1010 non_overlapping sequence detector-

52.0 Moore FSM to detect 1010 Overlapping sequence detector-

53.0 Design a CMOS Inverter by switch level modelling

54.0 Design a 2-input NAND gate by switch level modelling

55.0 Design a 2-input NOR gate by switch level modelling

56.0 Design a 2x1mux by switch level modelling by using transmission gate

57.0 Design a 2x1mux by switch level modelling by using Bidirectional switches

58.0 Design a CMOS SWITCH by switch level modelling

59.0 Design a 2-input NAND gate by adding delay concepts- for the simulation purpose only.

60.0 Design a 2x1mux by switch level modelling using mosfet

61.0 Design a Half adder by switch level modelling using mosfet

62.0 1.0 SISO (SERIAL-IN SERIAL-OUT)

63.0 2.0 SIPO (SERIAL-IN PARALLEL-OUT)

64.0 3.0 PISO (PARALLEL-IN SERIAL-OUT)

65.0 4.0 PIPO (PARALLEL-IN PARALLEL-OUT)

66.0 Frequency Divider by 2

66.0 Frequency Divider by f/4

67.0 Frequency Divider by f/3

68.0 Frequency Divider by f/5

69.0 Frequency Divider by f/3.5

70.0 Design a MOD-10 counter (0-9) / A Decade counter

71.0 Design a MOD-5 up-down counter (0-4)

72.0 Verilog code for a mod-8 counter using a parameter

73.0 Verilog code for a mod-N counter

74.0 Design a MOD-47 up-counter (0-46)

75.0 Design a Range up counter (10-46)

76.0 Design a Range (10-40) up-down counter with load data (6-bit) operation-

77.0 8-bit bidirectional barrel shift register (left and right) with a shift control input-

78.0 8-bit twin register set with independent storage (parallel loading) -

79.0 Design an 8-bit Universal shift register

80.0 Design a 4-bit Left shift register (LSR)

81.0 Design a 4-bit Right shift register (RSR)

82.0 Design a universal_shift_register2-

83.0 Left_circular_shift_register_using_loop (load and reset)

84.0 Design a 4-bit Ring Counter (circular left shift)

85.0 Design a 4-bit Johnson Counter (shift right)

86.0 Design a 1x4 D-MUX-

87.0 Design a 4-bit Ripple counter (using D-FFs) (Asynchronous counter)

88.0 Design a 4-bit Ripple Counter (Asynchronous counter) using T_FF -

89.0 Design a D-latch with enable and reset by using behavioral modelling-

90.0 Design a SR-NOR-latch by using gate level

91.0 Design a T (Toggle) FLIP_FLOP

92.0 Design a JK FLIP_FLOP

93.0 Design a Frequency divided by 2,4, 8,16 Circuit WITH 50% DC By using the Asynchronous D-FF -

94.0 Design a 2x1mux by using the dataflow modelling

95.0 Design a 4x1mux by using the dataflow modelling

96.0 Design ALL LOGIC GATES by using dataflow

97.0 Design ALL LOGIC GATES by using gate level modelling

98.0 Swapping of two variables without using the temporary variable by using the blocking assignment

99.0 Swapping of two variables using the third variable by using the blocking assignment

100.0 Swapping of two variables without using the third variable by using the non-blocking assignment

101.0 Swapping of two variables using the third variable by using the non-blocking assignment

102.0 Swapping Using a Third Variable (temp) with Blocking Assignment procedural block (always @(posedge clk))

103.0 Swapping without using a Third Variable with non-Blocking Assignment procedural block (always @(posedge clk))

104.0 Swapping Using a Third Variable (temp) with Non-Blocking Assignment procedural block (always @(posedge clk)) adding delay concept-

……..

1.0 Design a 2x1_mux using bufof1 and bufif0

2.0 Design an 8x1 mux by using data flow

3.0 Design a 4-bit adder and subtractor by using behavioral modelling

4.0 Design a N-bit adder and subtractor by using data flow

5.0 BCD to 7-segment code converter

6.0 Design a Frequency Divider by f/3

7.0 Design a Frequency Divider by f/1.5

8.0 Design a circuit to rising and falling edge detector

9.0 Design a 4-bit Carry lookahead adder (CLA) by using dataflow modelling-

10.0Design a 4-bit Ripple carry adder (RCA) by using data flow

11.0 Design a Full Adder (FA) by using data flow

12.0 Design a Half Subtractor (HS) by using data flow

13.0 Design a Full Subtractor (FS) by using data flow

14.0 Design a 16x1 MUX using 4x1 mux

15.0 Design a 1x4 D-MUX by using behavioural

16.0 Design a 4x16 Decoder using 2x4 decoder behavioural

17.0 Design an 8x3 priority encoder by using if-else

18.0 Design an even and odd parity generation

19.0 Design an even and odd parity checker

20.0 Design a 4-bit BCD adder

21.0 Design a BCD to excess-3 code converter (behavioural modelling)

22.0 Design a BCD (abcd) to excess-3 (wxyz) code converter (DATAFLOW modelling)-

23.0 Design a Tri-state buffer

24.0 Design a SR latch using NOR gates-

25.0 Design a SR latch using NAND gates-

26.0 Design a universal gate by using the behavioral-

27.0 Design a half adder by behavioural level

28.0 Design a Full Subtractor (FS) by using gate level-

29.0 Design a 4-bit Full_adder(FA)_Subtractor (FS) by using gate level-

30.0 Design a 2x4 decoder by using behavioural

31.0 Design a 3x8 decoder by using 2x4 decoder-

32.0 Design an 8x3 Priority encoder using valid-

33.0 Design a 2x1mux by using case in behavioral

34.0 Design a 10x1mux by using 4x1 mux in behavioral

35.0 Design an and gate using 4x1 mux in behavioral

36.0 Design an AND gate using data flow modelling-

37.0 Design an AND gate using case statement-

38.0 Design an 8x1 mux by using 2x1 mux-

39.0 Design a kx1_mux by using parameter -

40.0 Design a parity generator –

41.0 Design a logical operation by using task –

42.0 Design a Reduction operation by using task –

43.0 Design a shift operation by using task –

44.0 Design an equality operation by using task –

45.0 Design a relational operation by using task –

46.0 Design a concatenation operation by using task –

47.0 Design an arithmetic operation by using task –

48.0 Design a bidirectional shift register-

49.0 Design an +ve edge triggered Asynchronous 4-bit D-FF (Active high reset)

50.0 Design an up-counter with load data (8-bit) using task

51.0 Design a synchronous 3-bit up-down counter using JK FF

……

RTL2GDSII FLOW WITH SYNOPSIS TOOLS

1.0 //find the default data size and its value.

2.0 Design an RTL of half adder by using data flow

3.0 Design an RTL of half adder by using behavioral modelling

4.0 Design a 32-bit ALU of RTL code

5.0 Design a 32-bit ALU overflow condition of RTL code

6.0 Design a 1-bit full adder by using the gate level modelling

7.0 Design a 32-bit full adder using 8-bit full adder by using data flow

8.0 Design a 16-bit full adder using 4-bit full adder by using data flow

9.0 Design an RTL code of 4-bit full adder by using behavioural modelling

10.0 Design an RTL of a 4-bit full adder and check the functional coverage

Lab1

1.0 Design a half adder by using the gate level modelling

```
//half_Adder using gate level module

module half_adder(sum,carry,a,b);
output sum;
output carry;
input a,b;
xor my_xor(sum,a,b);
and my_and(carry,a,b);

initial
$display("my result checked ok");
endmodule
```

Simulation Result

Output

```
my result checked ok
```

Lab1.a

2.0 Design a half adder by data flow level modelling

```
// data flow level half adder
module half_adder(sum,carry,a,b);
output sum;
output carry;
input a,b;
assign sum=((~a&b) +(a&~b));
```

```
assign carry=a&b;


initial
$display("my result checked okay");
endmodule
```

Simulation Result

Output

my result checked okay

Lab1.b

3.0 Design a half adder by behavioural level modelling

Design code

```
// adder module half adder behavioural level
module half_adder(sum,carry,a,b);
output sum; reg sum;
output carry; reg carry;
input a,b;


assign a=0;
assign b=1;


always @(a or b)
begin
        {carry,sum}=a+b;


$display("value of sum and carry : %d %d ",sum,carry);
```

```
end

endmodule
```

Run command

```
vcs half_adder.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
value of sum and carry : 1 0
```

4.0 Lab1.size_of_data and its value

Design code

```
// Calculate the data size and it’s default value in verilog

module datasize();

integer a;

reg b;

wire c;

//real d;

time e;


initial begin

    $display("value of integer :%d,value of reg :%d,value of wire :%d,value of time :%d",a,b,c,e);


    $display("value of integer :%b,value of reg :%b,value of wire :%b,value of time :%b",a,b,c,e);

    $display($bits(a),$bits(b),$bits(c),$bits(e),$bits(e));
```

```
end
endmodule
```

Run command

```
vcs -sverilog data_size.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
value of integer :         x,value of reg :x,value of wire :z,value of time :                  x
value of integer :xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx,value of reg :x,value of wire :z,value of time :xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
      32        1        1        64        64
```

Lab1.mux

5.0 Design a 2x1mux by using the behavioral modelling

Design code

```
//mux_2x1 by using behavioral style
module MUX_2X1__1(y,a,b,s);
input a,b,s;
output reg y;

always @(*)
      if (s==0)
            y=a;
      else
            y=b;
endmodule
```

Testbench

```
// testbench of 2x1 mux
module MUX_2X1__1tb();
reg a,b,s;
wire y;

MUX_2X1__1 mux1(.y(y),.a(a),.b(b),.s(s));

initial begin
    $monitor("s=%d,a=%d,b=%d and we get y=%d",s,a,b,y);

    s=0;a=0; b=0;
    #5 s=0;a=1;b=0;
    #5 s=0;a=0;b=1;
    #5 s=1;a=1;b=0;
    #5 s=1;a=1;b=1;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,MUX_2X1__1tb);
end
endmodule
```

Run command

```
vcs mux2x1.v mux_2x1_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

s=0,a=0,b=0 and we get y=0

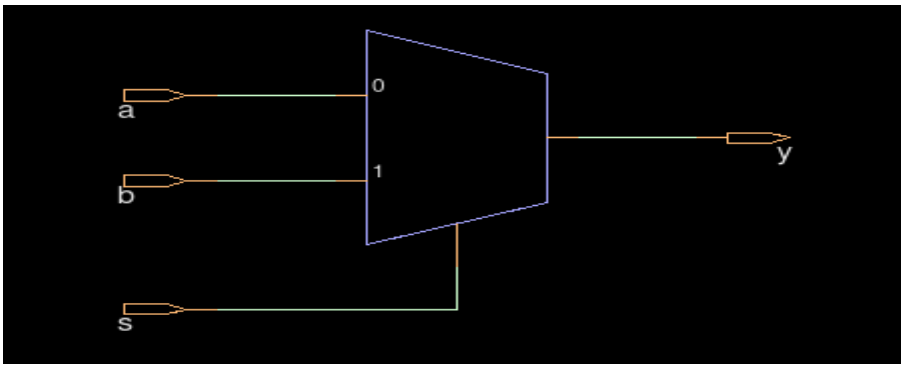
s=0,a=1,b=0 and we get y=1

s=0,a=0,b=1 and we get y=0

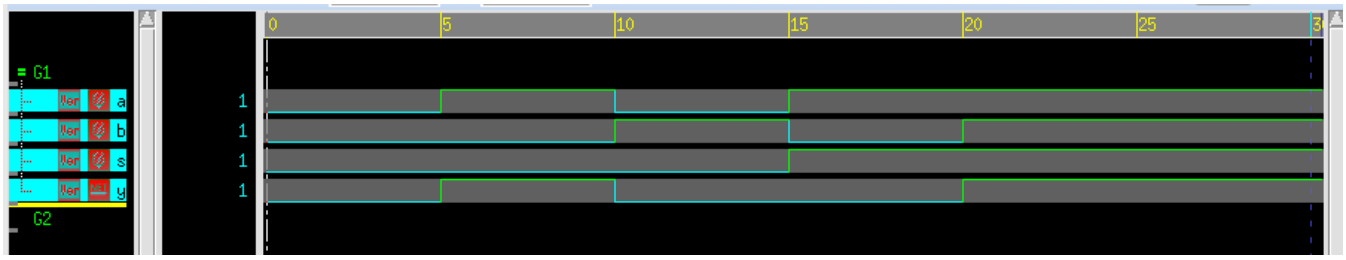
s=1,a=1,b=0 and we get y=0

s=1,a=1,b=1 and we get y=1

Schematic Design



waveform



6.0 Design a 4x1mux by using 2x1mux the behavioral modelling

Design code

//design the 4x1 mux using 2x1 mux

module MUX_2X1__1(y,a,b,s);

input a,b,s;

output reg y;

```
always @(*)
	if (s==0)
		y=a;
	else
		y=b;
endmodule

module MUX_4X1(out,a,b,c,d,s1,s0);
input a,b,c,d,s1,s0;
output out;

wire w1,w2;

MUX_2X1__1 mux1(.y(w1),.a(a),.b(b),.s(s0));
MUX_2X1__1 mux2(.y(w2),.a(c),.b(d),.s(s0));
MUX_2X1__1 mux3(.y(out),.a(w1),.b(w2),.s(s1));
endmodule
```

Testbench

```
//test bench of mux 4x1 by using 2x1
module mux_4x1_tb();
reg a,b,c,d,s0,s1;
wire out;

MUX_4X1 dut(.out(out),.a(a),.b(b),.c(c),.d(d),.s0(s0),.s1(s1));

initial
begin
```

```
        $monitor("for a=%d,b=%d,c=%d,d=%d,s0=%d,s1=%d and we get
out=%d",a,b,c,d,s0,s1,out);


        a = 1; b = 0; c = 1; d = 0;
#5 s1=0;s0=0; a = 0; b = 0; c = 0; d = 0;
#5 s1=1;s0=0; a = 0; b = 0; c = 0; d = 1;
#5 s1=0;s0=1; a = 0; b = 0; c = 1; d = 0;
#5 s1=1;s0=1; a = 0; b = 1; c = 0; d = 0;
#5 s1=0;s0=0; a = 1; b = 0; c = 0; d = 0;
#5 s1=1;s0=0; a = 1; b = 0; c = 1; d = 0;
#5 s1=1;s0=1; a = 0; b = 1; c = 0; d = 1;
#10 $finish();
end


initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,mux_4x1_tb);
end
endmodule
```

Run command

```
vcs mux4x1.v mux4x1_testbench.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
for a=1,b=0,c=1,d=0,s0=x,s1=x and we get out=0
for a=0,b=0,c=0,d=0,s0=0,s1=0 and we get out=0
for a=0,b=0,c=0,d=1,s0=0,s1=1 and we get out=0
```

for a=0,b=0,c=1,d=0,s0=1,s1=0 and we get out=0

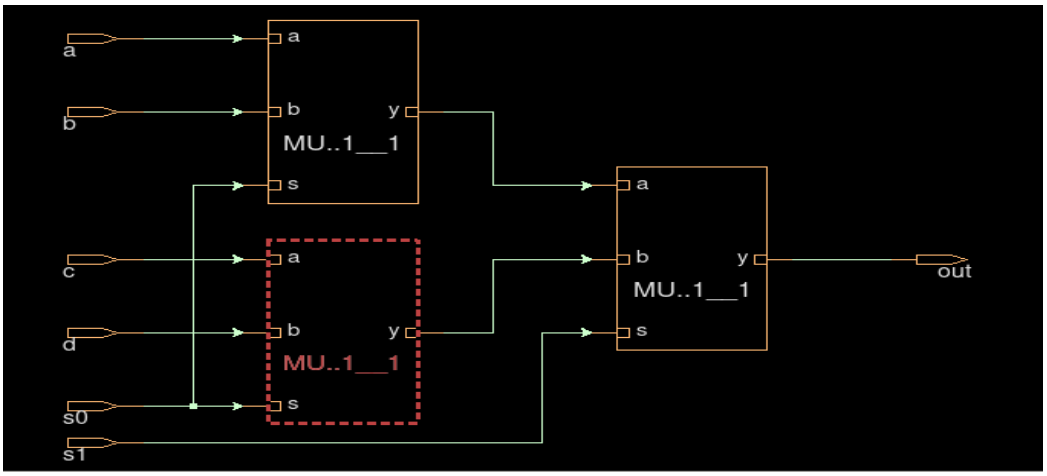
for a=0,b=1,c=0,d=0,s0=1,s1=1 and we get out=0

for a=1,b=0,c=0,d=0,s0=0,s1=0 and we get out=1

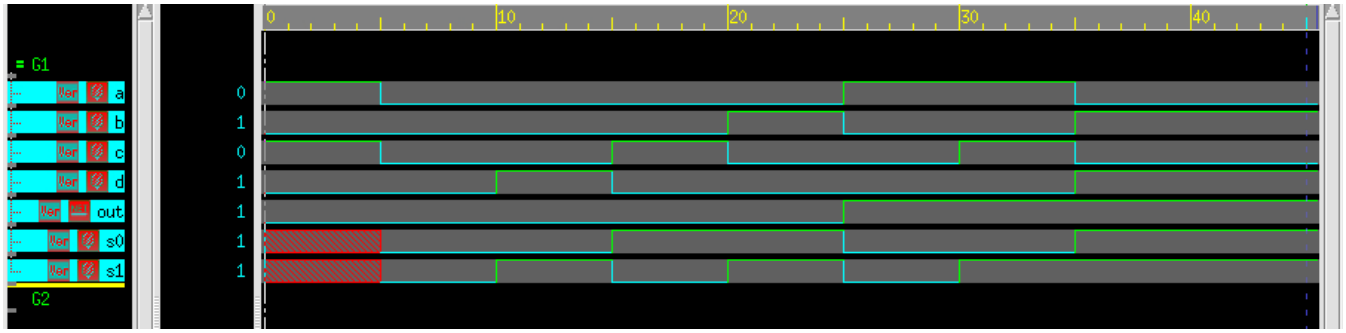
for a=1,b=0,c=1,d=0,s0=0,s1=1 and we get out=1

for a=0,b=1,c=0,d=1,s0=1,s1=1 and we get out=1

Schematic Design



Waveform



Lab2 7.0 Net types-tri

Design code

// test of conditional statement by using **tri**

module test_tri();

tri a;

reg b,c;

```
assign a=(b?c:1'bz);


initial begin
        $monitor("Time=%0d a=%0d b=%0d c=%0d",$time,a,b,c);
        b=0;c=0;
        #10 b=1;
        #10 b=0;
        #10 c=1;
        #10 b=1;
        #10 b=0;
        #10 $finish();
end
endmodule
```

Run command

```
vcs nettype_tri.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
Time=0 a=z b=0 c=0
Time=10 a=0 b=1 c=0
Time=20 a=z b=0 c=0
Time=30 a=z b=0 c=1
Time=40 a=1 b=1 c=1
Time=50 a=z b=0 c=1
```

8.0 Net types-wand

Design code

// test of conditional statement by using **wand**

```
module test_wand();
wand a;
reg b,c;
assign a=b;
assign a=c;

initial begin
    $monitor("Time=%0d a=%0d b=%0d c=%0d",$time,a,b,c);
    #10 b=0;
    #10 c=0;
    #10 b=1;
    #10 c=1;
    #10 b=1;
    #10 $finish();
end
endmodule
```

Simulation Result

Output

```
Time=0 a=x b=x c=x
Time=10 a=x b=0 c=x
Time=20 a=0 b=0 c=0
Time=30 a=x b=1 c=0
Time=40 a=1 b=1 c=1
```

9.0 Net types-wor

Design code

```
//
module test_wor();
wor a;
reg b,c;
assign a=b;
assign a=c;

initial begin
    $monitor("Time=%0d a=%0b b=%0b c=%0b",$time,a,b,c);

    #10 b=0;
    #10 c=0;
    #10 b=1;
    #10 c=1;
    #10 b=1;
    #10 $finish();
end
endmodule
```

Simulation Result

Output

```
Time=0 a=x b=x c=x
Time=10 a=x b=0 c=x
Time=20 a=0 b=0 c=0
Time=30 a=1 b=1 c=0
Time=40 a=1 b=1 c=1
```

10.0 Tristate gates

Design code

```
module test_tristate(
input in,ctrl,
output out1,out2,out3,out4
);
assign in=1;
assign ctrl=0;

bufif1 b1(out1,in,ctrl);
bufif0 b0(out2,in,ctrl);
notif1 n1(out3,in,ctrl);
notif0 n0(out4,in,ctrl);

initial begin
    $monitor("Time=%0d in=%0b ctrl=%0b
out1=%0b,out2=%0b,out3=%0b,out4=%0b",$time,in,ctrl,out1,out2,out3,out4);

    #10 $finish();
end
endmodule
```

Simulation Result

Output

```
Time=0 in=1 ctrl=0 out1=z,out2=1,out3=z,out4=0
```

Lab2.1 11.0 Delay analysis concept

Design code

```
module practice_delay_concept(  );
reg [7:0]a,b;
reg [15:0]r;

initial begin
    $monitor("Time=%0d a=%0d b=%0d r=%0d",$time,a,b,r);

    a=0;b=5;
    #5 a=4;b=2;
    #3 r=(a+b);
    #6 r=a-b;
    #10 r=(a**b);
    #30 $finish();
end

endmodule
```

Simulation Result

Output

```
Time=0 a=0 b=5 r=x
Time=5 a=4 b=2 r=x
Time=8 a=4 b=2 r=6
Time=14 a=4 b=2 r=2
Time=24 a=4 b=2 r=16
```

12.0 Design a Gate level Half Adder (HA) circuit

Design code

```
module half_adder(sum,carry,a,b);
```

```
output sum,carry;
input a,b;

xor my_xor(sum,a,b);
and my_and(carry,a,b);
endmodule
```

Testbench

```
module TB( );
reg a,b;
wire sum,carry;

half_adder dut(.a(a),.b(b),.sum(sum),.carry(carry));

initial begin
    $monitor("Time=%0d a=%0b b=%0b sum=%0b carry=%0b",$time,a,b,sum,carry);
    a=0 ; b=0 ;
    # 10 a=0 ; b=1 ;
    # 10 a=1 ; b=0 ;
    # 10 a=1 ; b=1 ;
    #20 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

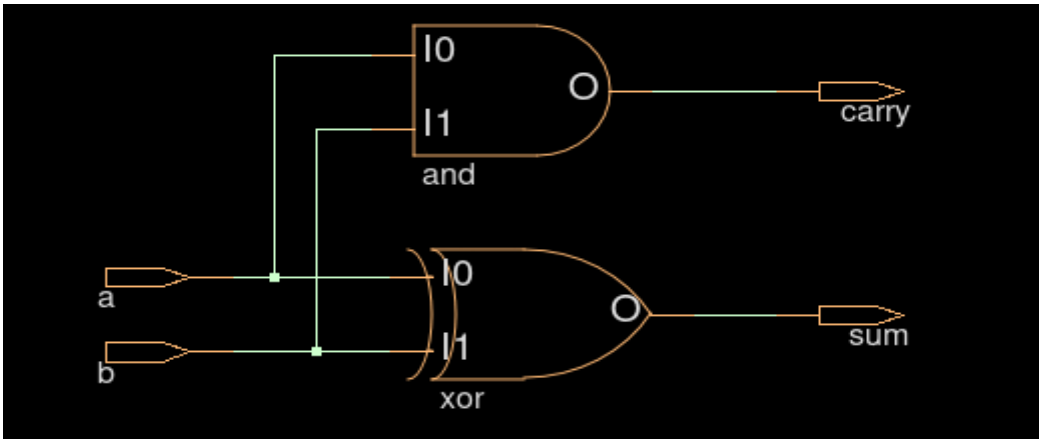
Time=0 a=0 b=0 sum=0 carry=0

Time=20 a=0 b=1 sum=1 carry=0

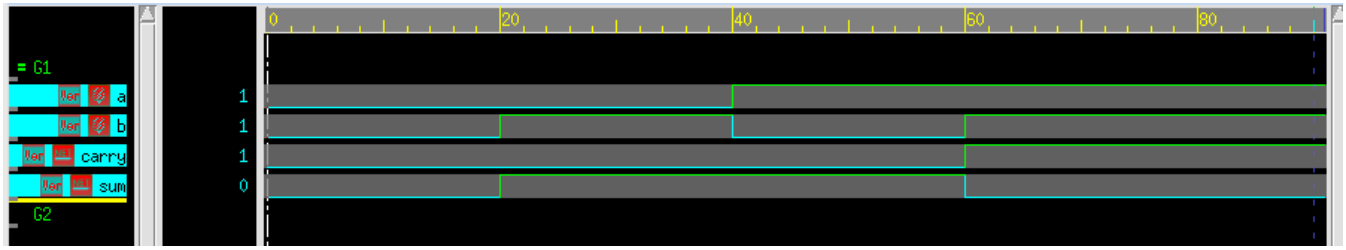
Time=40 a=1 b=0 sum=1 carry=0

Time=60 a=1 b=1 sum=0 carry=1

Schematic Design



waveform



13.0 Design a Gate level Full Adder (FA) circuit

Design code

// Design a full adder by using the gate level modelling

module Full_adder(sum,carry,a,b,c);

output sum,carry;

```
input a,b,c;

wire x1,x2,x3;

xor my_xor1(x1,a,b);
xor my_xor2(sum,x1,c);
and my_and1(x2,x1,c);
and my_and2(x3,a,b);
or my_or(carry,x2,x3);

endmodule
```

Testbench

```
//Testbench of full adder
module TB( );
reg a,b,c;
wire sum,carry;

Full_adder dut(.a(a),.b(b),.c(c),.sum(sum),.carry(carry));

initial begin
    $monitor("Time=%0d a=%0b b=%0b c=%0b sum=%0b carry=%0b",$time,a,b,c,sum,carry);
    a=0 ; b=0 ; c=0 ;
    # 10 a=0 ; b=0 ; c=1 ;
    # 10 a=0 ; b=1 ; c=0 ;
    # 10 a=0 ; b=1 ; c=1 ;
    # 10 a=1 ; b=0 ; c=0 ;
```

```
        # 10 a=1 ; b=0 ; c=1 ;
        # 10 a=1 ; b=1 ; c=0 ;
        # 10 a=1 ; b=1 ; c=1 ;
        #20 $finish();
end

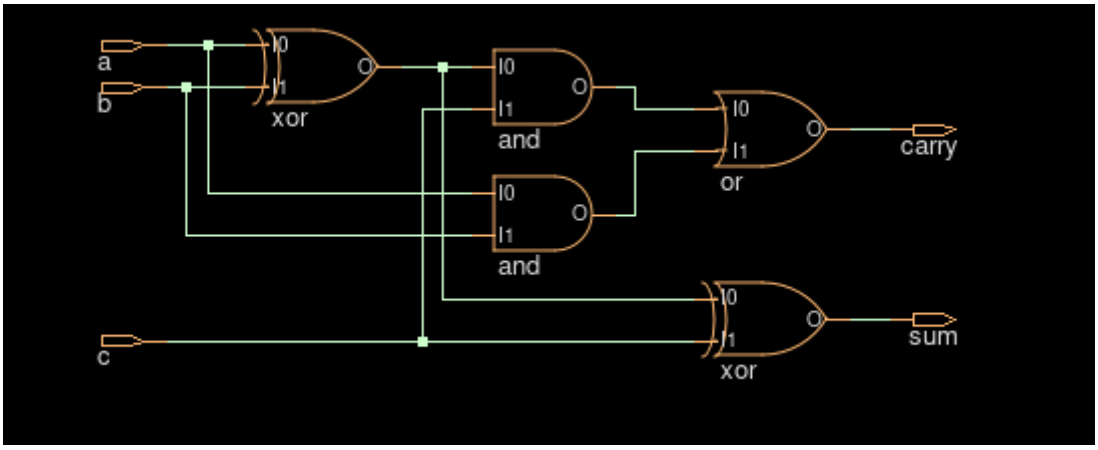
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

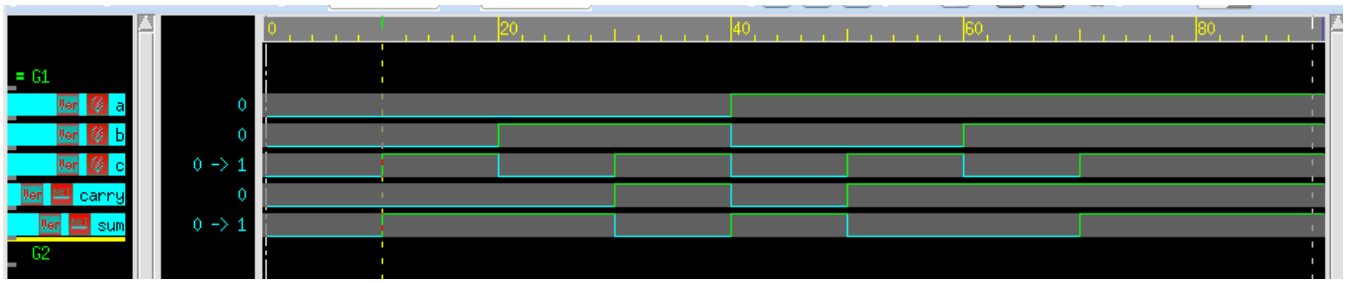
Output

```
Time=0 a=0 b=0 c=0 sum=0 carry=0
Time=10 a=0 b=0 c=1 sum=1 carry=0
Time=20 a=0 b=1 c=0 sum=1 carry=0
Time=30 a=0 b=1 c=1 sum=0 carry=1
Time=40 a=1 b=0 c=0 sum=1 carry=0
Time=50 a=1 b=0 c=1 sum=0 carry=1
Time=60 a=1 b=1 c=0 sum=0 carry=1
Time=70 a=1 b=1 c=1 sum=1 carry=1
```

Schematic Design



waveform



14.0 Design a Full Adder by using Half adder

Design code

```
// Design a full adder by using half adder
module half_adder(sum,carry,a,b);
output sum,carry;
input a,b;
xor my_xor(sum,a,b);
and my_and(carry,a,b);

endmodule

module FA_using_HA(sum, carry,a,b,c );
output sum,carry;
input a,b,c;
```

```
wire x1,x2,x3;

half_adder ha1(x1,x2,a,b);
half_adder ha2(sum,x3,x1,c);
or or3(carry,x3,x2);

endmodule
```

Testbench

```
//testbench of Full adder
module TB( );
reg a,b,c;
wire sum,carry;

FA_using_HA dut(.a(a),.b(b),.c(c),.sum(sum),.carry(carry));
initial begin
    $monitor("Time=%0d a=%0b b=%0b c=%0b sum=%0b carry=%0b",$time,a,b,c,sum,carry);
    a=0 ; b=0 ; c=0 ;
    # 10 a=0 ; b=0 ; c=1 ;
    # 10 a=0 ; b=1 ; c=0 ;
    # 10 a=0 ; b=1 ; c=1 ;
    # 10 a=1 ; b=0 ; c=0 ;
    # 10 a=1 ; b=0 ; c=1 ;
    # 10 a=1 ; b=1 ; c=0 ;
    # 10 a=1 ; b=1 ; c=1 ;
    #20 $finish();
```

```
end

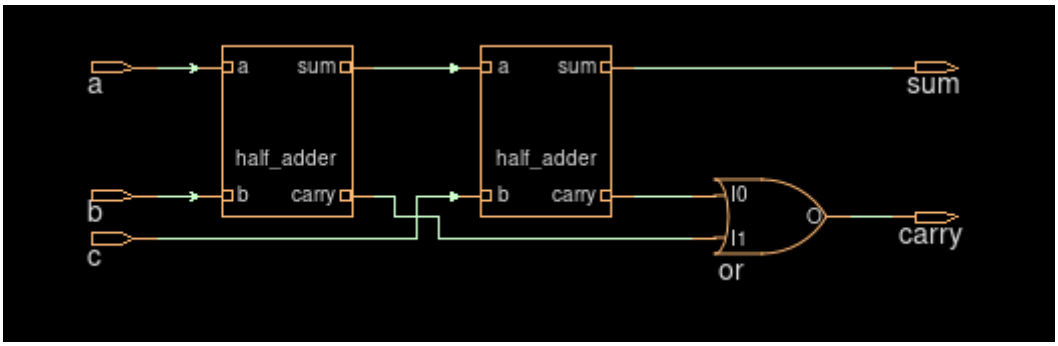

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

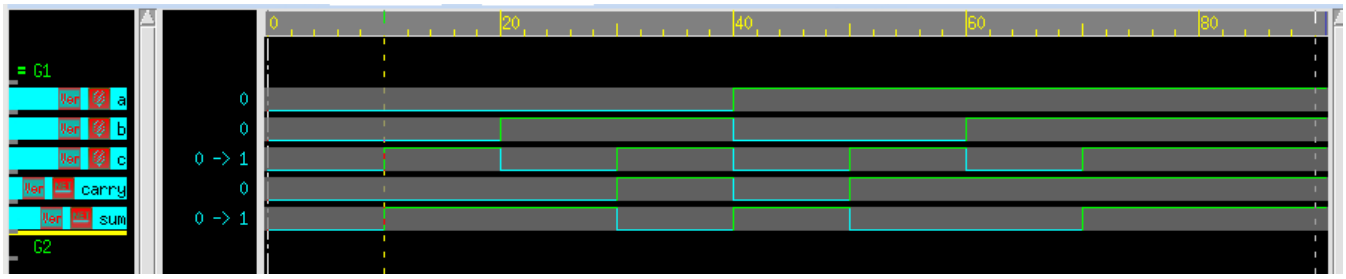
Output

```
Time=0 a=0 b=0 c=0 sum=0 carry=0
Time=10 a=0 b=0 c=1 sum=1 carry=0
Time=20 a=0 b=1 c=0 sum=1 carry=0
Time=30 a=0 b=1 c=1 sum=0 carry=1
Time=40 a=1 b=0 c=0 sum=1 carry=0
Time=50 a=1 b=0 c=1 sum=0 carry=1
Time=60 a=1 b=1 c=0 sum=0 carry=1
Time=70 a=1 b=1 c=1 sum=1 carry=1
```

Schematic Design



waveform



Lab2.2 15.0 Generating a clock signal

Design code

```
module Clock_gen(output reg clk);
initial
    clk=1'b0;
always #5 clk=~clk;

initial begin
    $monitor("Time=%0d clk=%0b",$time,clk);

    #50 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,Clock_gen);
end
endmodule
```

Simulation Result

Output

```
Time=0 clk=0
Time=5 clk=1
```

Time=10 clk=0

Time=15 clk=1

Time=20 clk=0

Time=25 clk=1

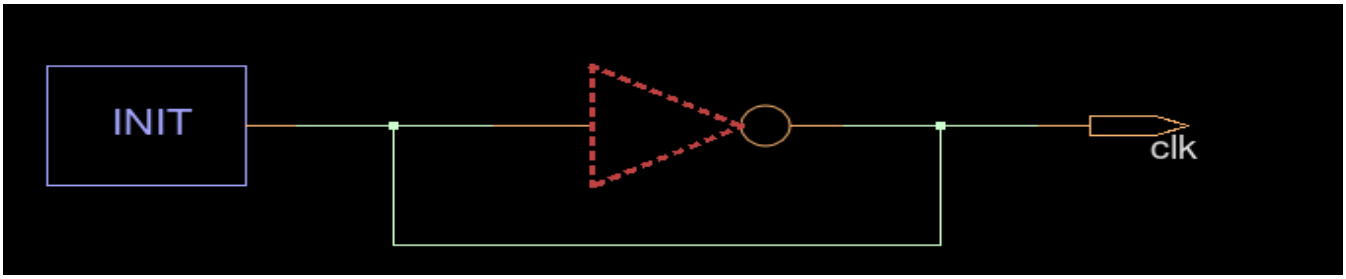
Time=30 clk=0

Time=35 clk=1

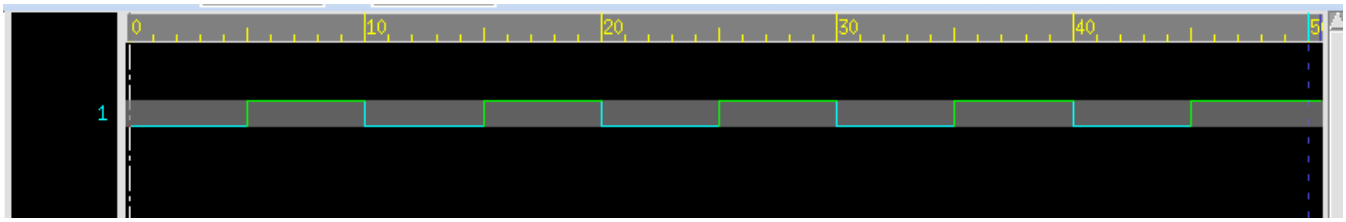
Time=40 clk=0

Time=45 clk=1

Schematic Design



waveform



Lab3

16.0 Design an +ve edge triggered Asynchronous D-FF (Active high reset)

Design code

//Asynchronous d-ff using using positive edge triggering

module d_ff(q,d,clk,rst);

input d,clk,rst;

output reg q;

```
always @(posedge clk or posedge rst)
begin
	if (rst==1)
		q<=0;
	else
		q<=d;
end
endmodule
```

Testbench

```
//Asynchronous d flip flop using positive clock
module d_ff_tb();
reg d,clk,rst;
wire q;

d_ff my_dff(.q(q),.clk(clk),.d(d),.rst(rst));

initial
	clk=0;
always #5 clk=~clk;

initial
begin
	$monitor("Time=%0t for rst=%d,d=%d,q=%d",$time,rst,d,q);

	rst=1; d=0;
```

```
    #10;rst=0; d=1;
    #10; d=1;
    #10; d=0;
    #10; d=1;
    #10; d=0;
    #20 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,d_ff_tb);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1,d=0,q=0

Time=10 for rst=0,d=1,q=0

Time=15 for rst=0,d=1,q=1

Time=30 for rst=0,d=0,q=1

Time=35 for rst=0,d=0,q=0

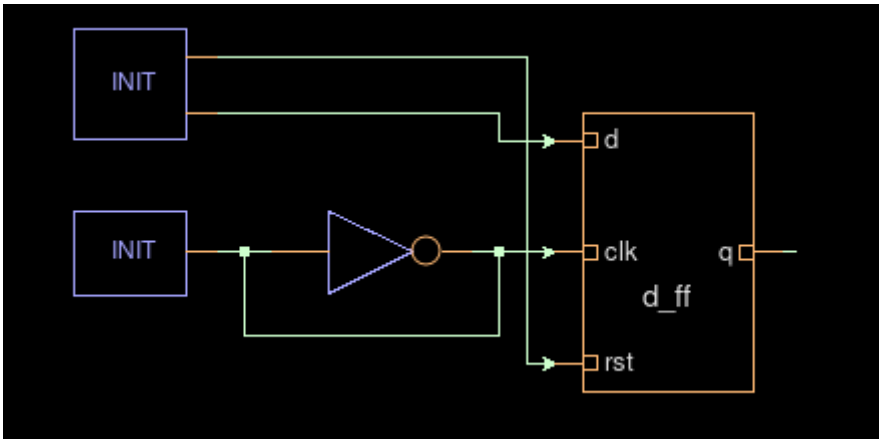
Time=40 for rst=0,d=1,q=0

Time=45 for rst=0,d=1,q=1

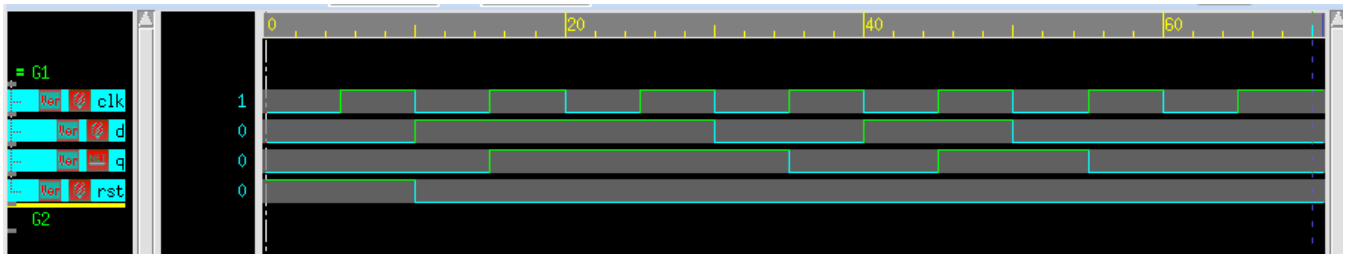
Time=50 for rst=0,d=0,q=1

Time=55 for rst=0,d=0,q=0

Schematic Design



waveform



17.0 Design an +ve edge triggered Synchronous D-FF (with synchronous reset)

Design code

```
module d_ff(q,d,clk,rst);
input d,clk,rst;
output reg q;

always @(posedge clk)
begin
    if (rst==1)
        q<=0;
    else
        q<=d;
end
endmodule
```

Testbench

```
module TB();
reg d,clk,rst;
wire q;

d_ff my_dff(.q(q),.clk(clk),.d(d),.rst(rst));

initial
    clk=0;
always #5 clk=~clk;

initial
begin
    $monitor("Time=%0t for rst=%d,d=%d,q=%d",$time,rst,d,q);

    rst=1; d=0;
    #10;rst=0; d=1;
    #10; d=1;
    #10; d=0;
    #10; d=1;
    #10; d=0;
    #20 $finish();

end

initial begin
```

```
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1,d=0,q=x

Time=5 for rst=1,d=0,q=0

Time=10 for rst=0,d=1,q=0

Time=15 for rst=0,d=1,q=1

Time=30 for rst=0,d=0,q=1

Time=35 for rst=0,d=0,q=0

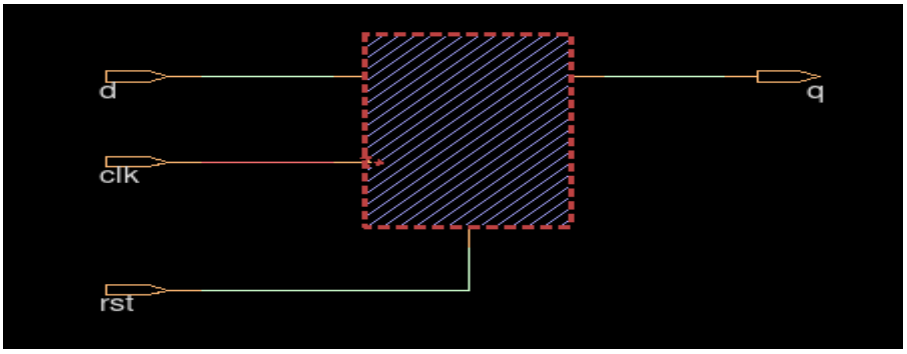
Time=40 for rst=0,d=1,q=0

Time=45 for rst=0,d=1,q=1

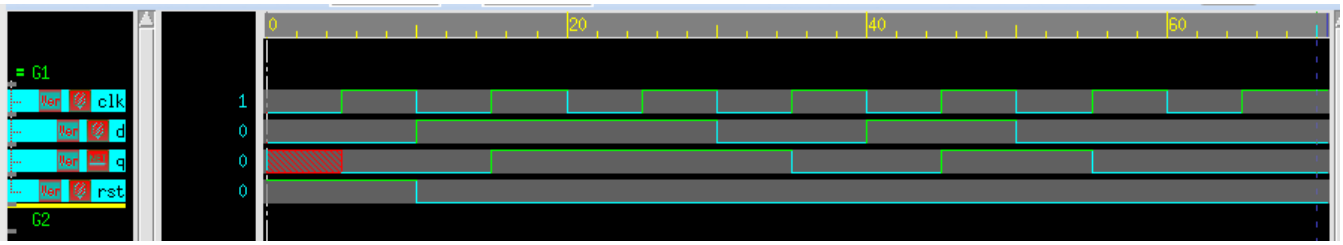
Time=50 for rst=0,d=0,q=1

Time=55 for rst=0,d=0,q=0

Schematic Design



waveform



Lab3.1

18.0 Design a 4-bit magnitude comparator

Design code

```
//magnitude comparator of 4-bit
module comparator(gt,lt,eq,a,b);
input [3:0]a,b;
output gt,lt,eq;

assign gt=(a>b);
assign lt=(a<b);
assign eq=(a==b);

endmodule
```

Testbench

```
//Test bench of 4-bit comparator
module comp_tb();
reg[3:0]a,b;
wire lt,gt,eq;

comparator my_comparator(.lt(lt),.gt(gt),.eq(eq),.a(a),.b(b));

initial begin
```

```
        $monitor("a=%d,b=%d,lt=%d,gt=%d,eq=%d",a,b,lt,gt,eq);
        a=10 ; b=12;
        #10  a=15 ;b=14;
        #10 a=5; b=5;
        #10 $finish();

end

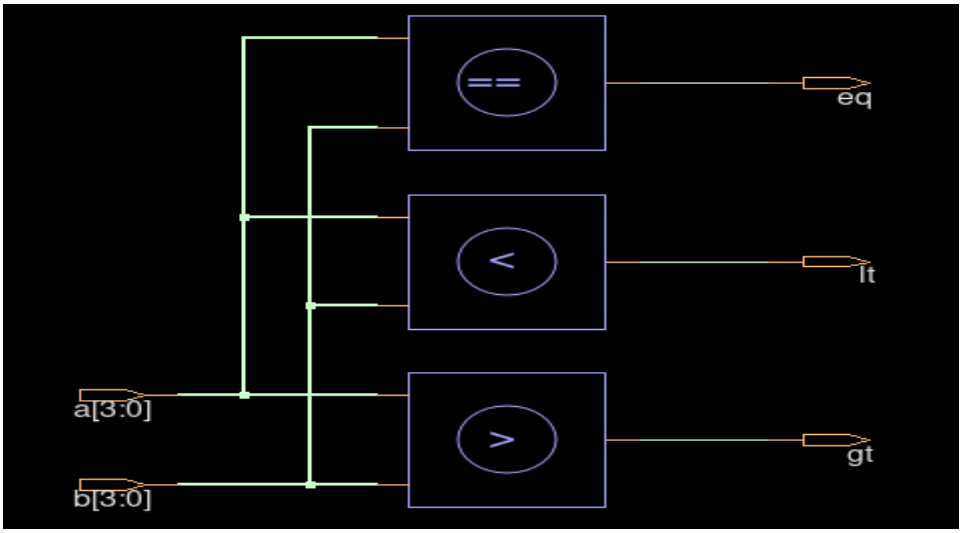
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,comp_tb);
end
endmodule
```

Simulation Result

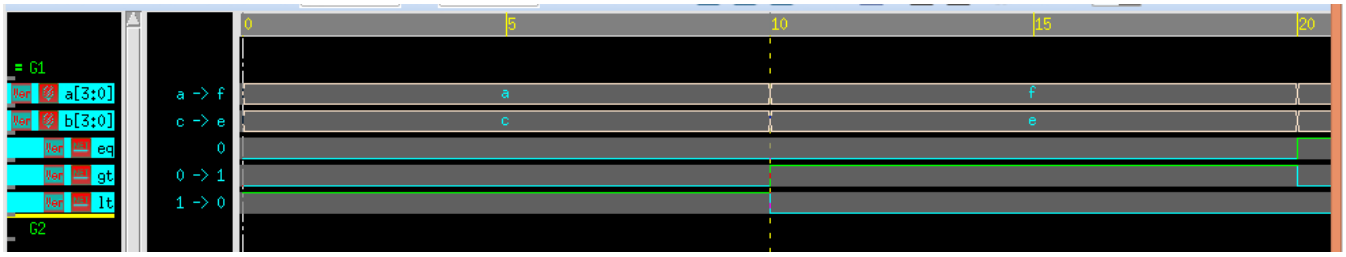
Output

```
a=10,b=12,lt=1,gt=0,eq=0
a=15,b=14,lt=0,gt=1,eq=0
a= 5,b= 5,lt=0,gt=0,eq=1
```

Schematic Design



waveform



Method-2 by using the behavioral modelling

Design code

```
module comp_4bit (a,b,gt,lt,eq);
input [3:0]a,b;
output reg gt,lt,eq;

always @(*)begin
        gt=0;
        lt=0;
        eq=0;
        if (a>b)
                gt=1;
        else if (a<b)
```

```
        lt=1;
    else
        eq=1;
end
endmodule
```

Testbench

```
module comparator_4bit_tb();
reg [3:0] a,b;
wire gt,lt,eq;

comp_4bit dut(.a(a),.b(b),.gt(gt),.lt(lt),.eq(eq));

initial begin
    $display("Time |  A     B     | gt  lt   eq");
    $monitor("%4t  | %b     %b    | %b  %b   %b",$time,a,b,gt,lt,eq);

    repeat(10) begin
        a=$random;
        b=$random;
        #10;
    end
    $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,comparator_4bit_tb);
end
```

endmodule

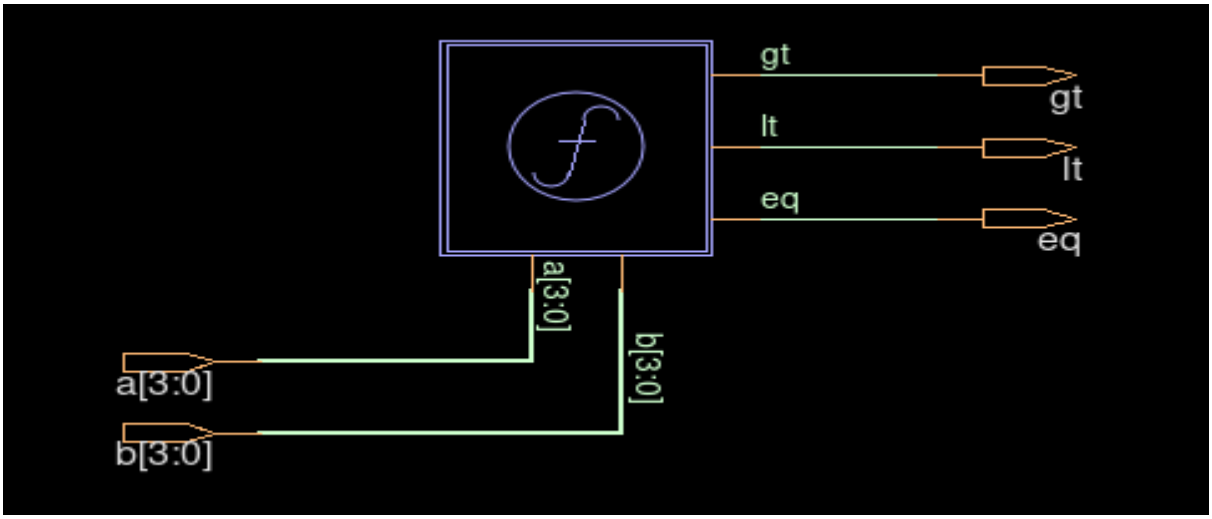
Simulation Result

Output

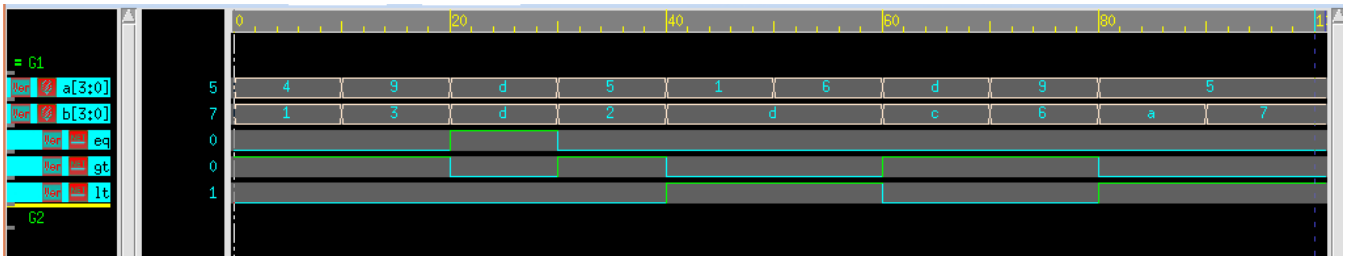
```
 0  | 0100    0001    | 1  0   0
10  | 1001    0011    | 1  0   0
20  | 1101    1101    | 0  0   1
30  | 0101    0010    | 1  0   0
40  | 0001    1101    | 0  1   0
50  | 0110    1101    | 0  1   0
60  | 1101    1100    | 1  0   0
70  | 1001    0110    | 1  0   0
80  | 0101    1010    | 0  1   0
90  | 0101    0111    | 0  1   0
```

$finish at simulation time 100

Schematic Design



waveform



Lab3.2

19.0 Design a 1-bit full adder by using Nand gate

Design code

```
//Design a 1-bit full-adder using nand gate
module fulladder_nand(sum,cout,a,b,c);
input a,b,c;
output sum,cout;

wire w1,w2,w3,w4,w5,w6,w7;

assign w1=~(a & b);
assign w2=~(w1 & a);
assign w3=~(w1 & b);
assign w4=~(w2 & w3);
assign w5=~(w4 & c);
assign w6=~(w4 & w5);
assign w7=~(w5 & c);
assign sum=~(w6 & w7);
assign cout=~(w5 & w1);

endmodule
```

Testbench

```
//Testbench of 1-bit full adder
module fulladder_tb();
reg a,b,c;
wire sum,cout;

fulladder_nand dut(.sum(sum),.cout(cout),.a(a),.b(b),.c(c));

initial
begin
	$monitor("for a=%d,b=%d,c=%d,and we get sum=%d,cout=%d",a,b,c,sum,cout);

	a=0;b=0;c=0;
#5 a=0;b=0;c=1;
#5 a=0;b=1;c=0;
#5 a=0;b=1;c=1;
#5 a=1;b=0;c=0;
#5 a=1;b=0;c=1;
#5 a=1;b=1;c=0;
#5 a=1;b=1;c=1;
#10 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,fulladder_tb);
end
```

endmodule

Simulation Result

Output

for a=0,b=0,c=0,and we get sum=0,cout=0

for a=0,b=0,c=1,and we get sum=1,cout=0

for a=0,b=1,c=0,and we get sum=1,cout=0

for a=0,b=1,c=1,and we get sum=0,cout=1

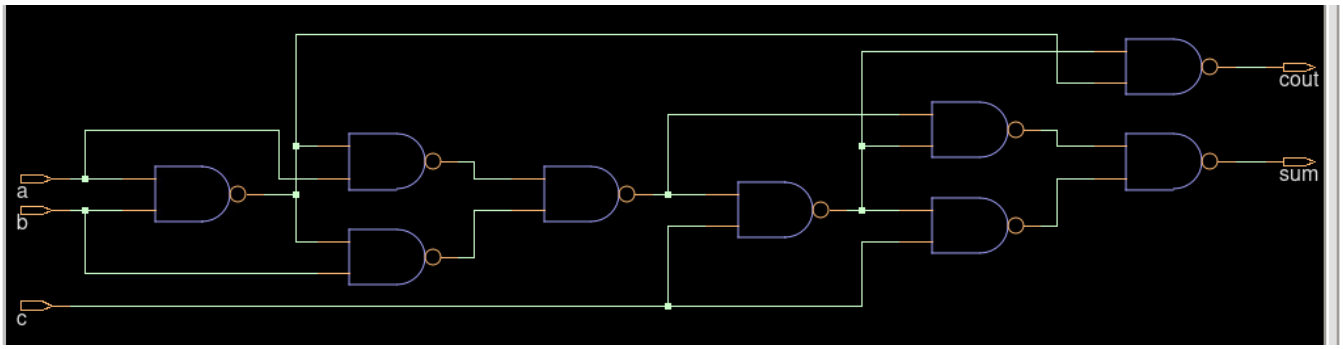
for a=1,b=0,c=0,and we get sum=1,cout=0

for a=1,b=0,c=1,and we get sum=0,cout=1

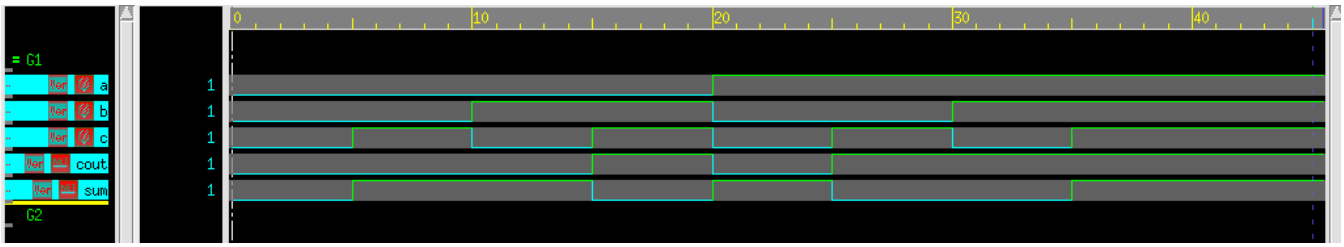
for a=1,b=1,c=0,and we get sum=0,cout=1

for a=1,b=1,c=1,and we get sum=1,cout=1

Schematic Design



waveform



Lab3.3

20.0 Design an 8x3 encoder by using the behavioral modelling

Design code

```
//Design a 8x3 encoder
module encoder_8x3(y,a,en);
input [7:0]a;
input en;
output reg [2:0]y;

always @(a,en) begin
    if (en)
        case(a)
            8'b00000001 : y=3'b000;
            8'b00000010 : y=3'b001;
            8'b00000100 : y=3'b010;
            8'b00001000 : y=3'b011;
            8'b00010000 : y=3'b100;
            8'b00100000 : y=3'b101;
            8'b01000000 : y=3'b110;
            8'b10000000 : y=3'b111;
        endcase

    else
        y=3'bzzz;
end
endmodule
```

Testbench

```
// Testbench of 8x3 encoder
module encoder8x3_tb();
```

```
reg [7:0]a;
reg en;
wire [2:0]y;

encoder_8x3 dut(.y(y),.a(a),.en(en));

initial begin
    $monitor("for a=%b ,en=%b, y=%b",a,en,y);
    en=0;a=8'b00000000;
    en=1;
    #10 a=8'b00000001;
    #10 a=8'b00000010;
    #10 a=8'b00000100;
    #10 a=8'b00001000;
    #10 a=8'b00010000;
    #10 a=8'b00100000;
    #10 a=8'b10000000;
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,encoder8x3_tb);
end
endmodule
```

Simulation Result

Output

```
for a=00000000 ,en=1, y=xxx
```

for a=00000001 ,en=1, y=000

for a=00000010 ,en=1, y=001

for a=00000100 ,en=1, y=010

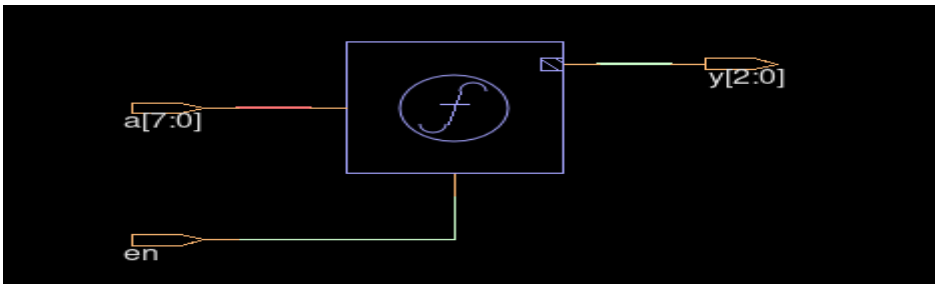
for a=00001000 ,en=1, y=011

for a=00010000 ,en=1, y=100

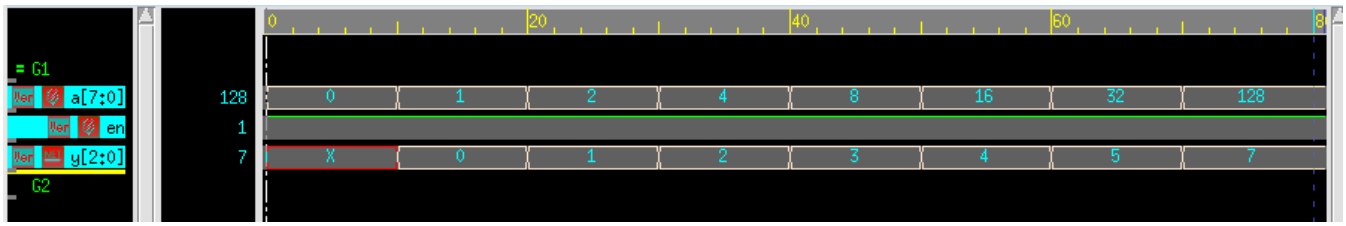
for a=00100000 ,en=1, y=101

for a=10000000 ,en=1, y=111

Schematic Design



waveform



21.0 Design an 8x3 Priority encoder by using the behavioral modelling

Design code

module priority_encoder_8x3(y,a,en);

input [7:0]a;

input en;

output reg [2:0]y;

always @(a,en) begin

```
    if (en)
        casex(a)
            8'b00000001 : y=3'b000;
            8'b0000001x : y=3'b001;
            8'b000001xx : y=3'b010;
            8'b00001xxx : y=3'b011;
            8'b0001xxxx : y=3'b100;
            8'b001xxxxx : y=3'b101;
            8'b01xxxxxx : y=3'b110;
            8'b1xxxxxxx : y=3'b111;
        endcase
    else
        y=3'bzzz;
end
endmodule
```

Testbench

```
module priority_encoder8x3_tb();
reg [7:0]a;
reg en;
wire [2:0]y;

priority_encoder_8x3 dut(.y(y),.a(a),.en(en));

initial begin
    $monitor("for a=%b ,en=%b, y=%b",a,en,y);
    en=0;a=8'b00000000;
    en=1;
```

```
        #10 a=8'b00000001;
        #10 a=8'b00000010;
        #10 a=8'b01010100;
        #10 a=8'b00110000;
        #10 a=8'b00010111;
        #10 a=8'b00100101;
        #10 a=8'b10101010;
        #20 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, priority_encoder8x3_tb);
end
endmodule
```

Simulation Result

Output

for a=00000000 ,en=1, y=xxx

for a=00000001 ,en=1, y=000

for a=00000010 ,en=1, y=001

for a=01010100 ,en=1, y=110

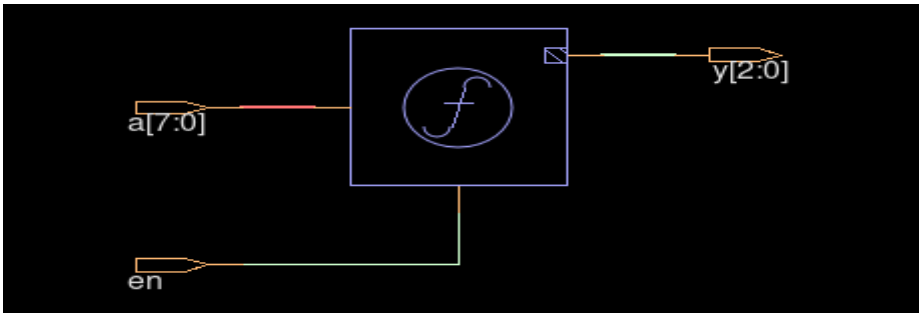
for a=00110000 ,en=1, y=101

for a=00010111 ,en=1, y=100

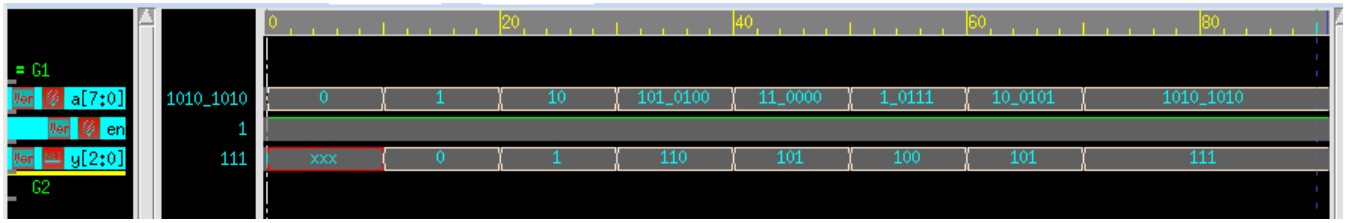
for a=00100101 ,en=1, y=101

for a=10101010 ,en=1, y=111

Schematic Design



waveform



Lab4 22.0 Named event control-

1.0 -> symbol as event triggering

2.0 @Symbol as waiting for event triggering

Design code

```
// named event based control example
module learn( );
integer a=2'd2;
integer b=2'd3;
integer y;

event sarif;

initial begin
    $display("Time=%0t,hello",$time);

    #1 $display("Time=%0t,Hi",$time);
    #2 y=a+b;
```

```
    #1 $display("Time=%0t, %0d",$time,y);
    ->sarif;
end

initial begin
    $display("Time=%0t,soon o/p will be printed",$time);
    @(sarif)
    $display("Time=%0t,this is the o/p a,b %0d %0d",$time,a,b);
end
endmodule
```

Simulation Result

Output

```
Time=0,hello
Time=0,soon o/p will be printed
Time=1,Hi
Time=4, 5
Time=4,this is the o/p a,b 2 3
```

23.0 Print a data at every +ve clock till i is not print 5 and print data=10*i by using the for loop

Design code

```
//example data print at every posedge edge of clock
module data( );
reg clk;
integer y;
integer i;
```

```
initial begin
        clk=0;
end
always #5 clk=~clk;

always @(posedge clk)begin
        for ( i=0; i<=5; i=i+1) begin
        y=10*i;

        $display("new_printed_data y=%0d",y);
end
#10 $finish();
end

endmodule
```

Simulation Result

Output

```
new_printed_data y=0
new_printed_data y=10
new_printed_data y=20
new_printed_data y=30
new_printed_data y=40
new_printed_data y=50
```

24.0 Print a data_ready triggering event at every +ve clock till i is not print 5 and print data=10*i by using @ sumbol

Design code

```
//data_ready event using named event concept and print 10*i
module named_event_data_print( );
integer y;
reg clk;
integer i;

event data_ready;

initial begin
        clk=0;
end
always #5 clk=~clk;

always @(posedge clk) begin
    -> data_ready;  // Trigger event
  end

initial begin
        $display("Time=%0t,soon o/p will be printed",$time);
        @(data_ready)
        $display("Time=%0t, data ready for print",$time);
        for (i=0 ; i<=9 ; i=i+1) begin
                y=10*i;

                $display("new printed_data y=%0d", y);
        end
end
```

endmodule

Simulation Result

Output

Time=0,soon o/p will be printed

Time=5, data ready for print

new printed_data y=0

new printed_data y=10

new printed_data y=20

new printed_data y=30

new printed_data y=40

new printed_data y=50

new printed_data y=60

new printed_data y=70

new printed_data y=80

new printed_data y=90

25.0 make the unpacked array of size 10. At every index number, print the value should be square in index.

Design code

```
// finding the square value by using the for loop and array

module square( );

reg clk;

integer arr[0:9];

integer i;

integer y;


initial begin
```

```
    clk=0;
end
always #5 clk=~clk;

always @(posedge clk)begin
    for ( i=0; i<=10; i=i+1) begin
    y=i*i;
    arr[i]=y;

    $display("new_printed_data y=%d",y);

end
#10 $finish;
end
endmodule
```

Simulation Result

Output

```
new_printed_data y=         0
new_printed_data y=         1
new_printed_data y=         4
new_printed_data y=         9
new_printed_data y=        16
new_printed_data y=        25
new_printed_data y=        36
new_printed_data y=        49
new_printed_data y=        64
```

new_printed_data y= 81

new_printed_data y= 100

Method-2 by using the Array concept

Design code

```
//module square( );
module square_array();
// Declare an integer array of size 10
reg [31:0] arr[0:9];
integer i;

  initial begin
    // Fill array with squares of index values
    for (i = 0; i <= 9; i = i + 1) begin
      arr[i] = i * i;
    end

    // Display the array contents
    for (i = 0; i <= 9; i = i + 1) begin
      $display("arr[%0d] = %0d", i, arr[i]);
    end

    $finish;
  end
endmodule
```

Simulation Result

Output

arr[0] = 0

arr[1] = 1

arr[2] = 4

arr[3] = 9

arr[4] = 16

arr[5] = 25

arr[6] = 36

arr[7] = 49

arr[8] = 64

arr[9] = 81

Lab5

26.0 Design the SR-FF by using the gate level modelling

Design code

```
//SR flip flop by using nand gate at gate level modelling
module SR_flip_flop(q, qbar,s,r,clk);
input s,r,clk;
output q,qbar;
wire w1,w2;

nand n1(w1,s,clk);
nand n2(w2,r,clk);
nand n3(q,w1,qbar);
nand n4(qbar,w2,q);

endmodule
```

Testbench

```
// testbench of SR-Flip Flop
module sr_ff_tb();
reg s,r,clk;
wire q,qbar;

initial begin
	clk=0;
end
always #5 clk=~clk;

SR_flip_flop dut(.q(q),.qbar(qbar),.s(s),.r(r),.clk(clk));

initial begin
	$monitor("Time=%0t for s=%d,r=%0d,clk=%d, we get
q=%d,qbar=%0d",$time,s,r,clk,q,qbar);

	s=0;r=0;
#10 s=0;r=1;
#10 s=1;r=0;
#10 s=1;r=1;

#10 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,sr_ff_tb);
end
```

endmodule

Run command

vcs sr_ff_gatelevel.v sr_ff_testbench.v -kdb -lca -debug_access+all -full64

Simulation Result

Output

Time=0 for s=0,r=0,clk=0, we get q=x,qbar=x

Time=5 for s=0,r=0,clk=1, we get q=x,qbar=x

Time=10 for s=0,r=1,clk=0, we get q=x,qbar=x

Time=15 for s=0,r=1,clk=1, we get q=0,qbar=1

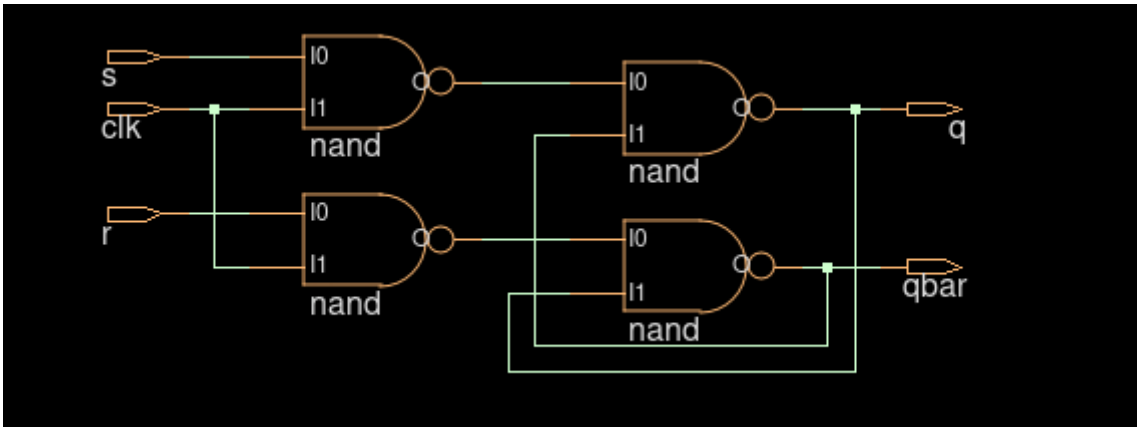
Time=20 for s=1,r=0,clk=0, we get q=0,qbar=1

Time=25 for s=1,r=0,clk=1, we get q=1,qbar=0

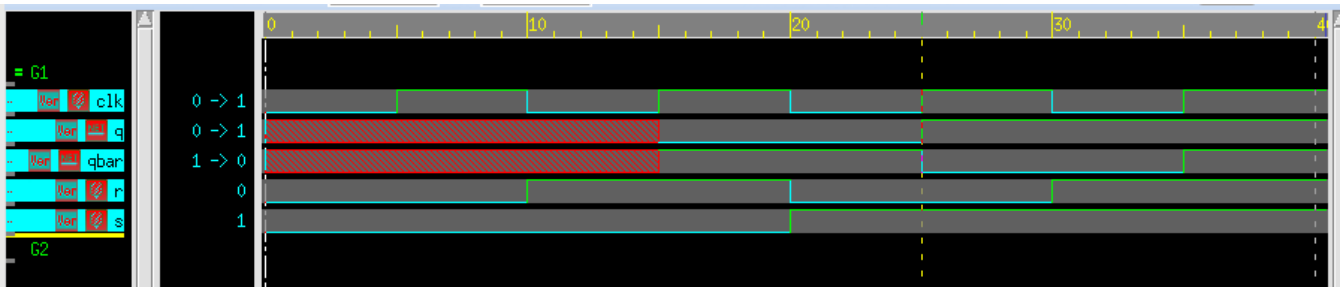
Time=30 for s=1,r=1,clk=0, we get q=1,qbar=0

Time=35 for s=1,r=1,clk=1, we get q=1,qbar=1

Schematic Design



waveform



27.0 Design an 4x2 Priority encoder

Design code

```
// priority encoder 4x2
module priority_encoder_4x2(w,y,z);
input [3:0]w;
output reg [1:0]y;
output reg z;

always @(w) begin
    z=1;
    casex(w)
        4'b1xxx: y=3;
        4'b01xx: y=2;
        4'b001x: y=1;
        4'b0001: y=0;
        default:begin
            z=0;
            y=2'bx;
        end
    endcase
end
endmodule
```

Testbench

```
// desugn an 4x2 priority encoder
module priority_encoder4x2_tb();
reg [3:0]w;
wire z;
wire [1:0]y;

priority_encoder_4x2 dut(.w(w),.z(z),.y(y));

initial begin
	$monitor("for w=%b ,z=%b, y=%b",w,z,y);
	w=4'b0000;
	#10 w=4'b0011;
	#10 w=4'b1000;
	#10 w=4'b0101;
	#10 w=4'b0011;
	#10 w=4'b0001;
	#10 w=4'b0110;
	#10 w=4'b1010;
	#20 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0, priority_encoder4x2_tb);
end
endmodule
```

Run command

vcs priority_encoder4_2.v priority_encoder4_2_tb.v -kdb -lca -debug_access+all -full64

Simulation Result

Output

for w=0000 ,z=0, y=xx

for w=0011 ,z=1, y=01

for w=1000 ,z=1, y=11

for w=0101 ,z=1, y=10

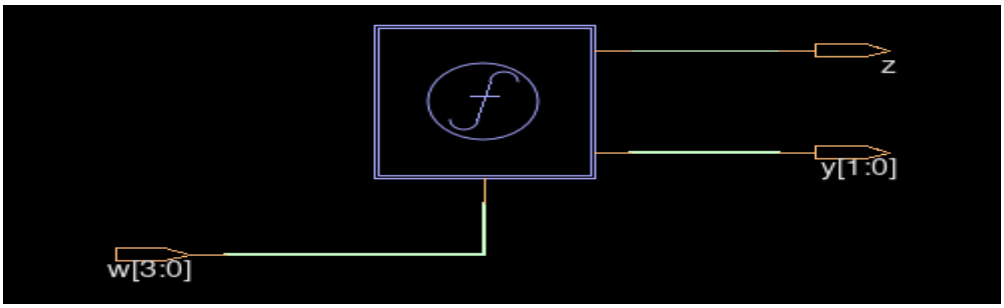
for w=0011 ,z=1, y=01

for w=0001 ,z=1, y=00

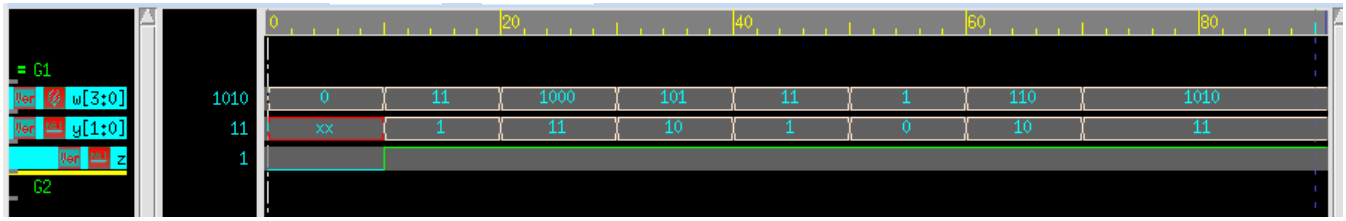
for w=0110 ,z=1, y=10

for w=1010 ,z=1, y=11

Schematic Design



waveform



28.0 Design a MOD-16 counter (4-bit)

Design code

```
//mod-16 counter
module mod_16(count,clk,rst);
input clk,rst;
output reg [3:0]count;

always @(posedge clk)
begin
	if (rst==1)
		count <=4'd0;
	else
		count <= count+1;
end
endmodule
```

Testbench

```
// testbench of mod-16 counter
module mod_16_tb();
reg clk,rst;
wire [3:0] count;

initial begin
	clk=0;
end
always #5 clk=~clk;

mod_16 dut(.clk(clk),.rst(rst),.count(count));

initial begin
```

```
        $monitor("for  rst=%d and we get output count=%d ",rst,count);
        rst=1;
#10 rst=0;


#200 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, mod_16_tb);
end
endmodule
```

Run command

```
vcs mod-16_counter.v mod-16_testbench.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
for  rst=1 and we get output count= x
for  rst=1 and we get output count= 0
for  rst=0 and we get output count= 0
for  rst=0 and we get output count= 1
for  rst=0 and we get output count= 2
for  rst=0 and we get output count= 3
for  rst=0 and we get output count= 4
for  rst=0 and we get output count= 5
for  rst=0 and we get output count= 6
for  rst=0 and we get output count= 7
for  rst=0 and we get output count= 8
```

for rst=0 and we get output count= 9

for rst=0 and we get output count=10

for rst=0 and we get output count=11

for rst=0 and we get output count=12

for rst=0 and we get output count=13

for rst=0 and we get output count=14

for rst=0 and we get output count=15

for rst=0 and we get output count= 0

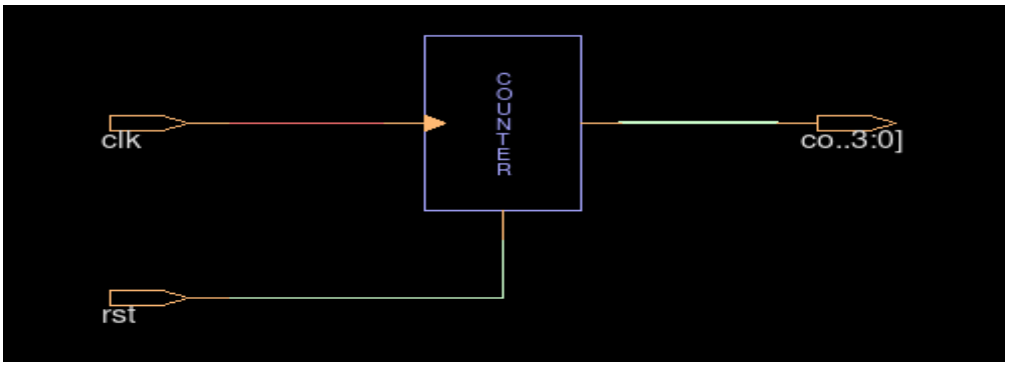
for rst=0 and we get output count= 1

for rst=0 and we get output count= 2

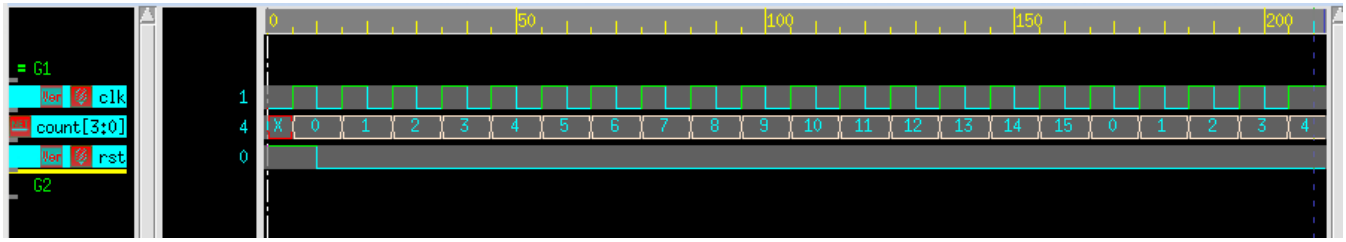
for rst=0 and we get output count= 3

for rst=0 and we get output count= 4

Schematic Design



waveform



29.0 Design an up-down counter with load data (4-bit)

Design code

```
// mod-16 load the data and up_down counter
module mod_16_load(count,u_d,load,clk,rst,data);
input clk,rst,load,u_d;
input [3:0]data;
output reg [3:0]count;

always @(posedge clk )begin

    if (rst==1)
        count <=4'd0;
    else if (load)
        count <=data;
    else if (u_d)
        count <= count +1;
    else
        count <= count -1;
end

endmodule
```

Testbench

```
// mod-16 load data and up-down counter testbench
module mod_16_load_tb();
reg clk,rst,load,u_d;
reg [3:0]data;
wire [3:0] count;

initial begin
```

```
        clk=0;
end
always #5 clk=~clk;

mod_16_load dut(.count(count),.u_d(u_d),.load(load),.clk(clk),.rst(rst),.data(data));

initial begin
        $monitor("Time=%0d for rst=%0d load=%0d data=%d u_d=%0b and we get output count=%d ",$time,rst,load,data,u_d,count);
        rst=1;load=0;u_d=0;data=4'd0;

#10 rst=0;
#10 load=1;data=4'd9;

#10 load=0;
u_d=1;
repeat(5) #10;

u_d=0;
repeat(5) #10;

#100 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, mod_16_load_tb);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1 load=0 data= 0 u_d=0 and we get output count= x

Time=5 for rst=1 load=0 data= 0 u_d=0 and we get output count= 0

Time=10 for rst=0 load=0 data= 0 u_d=0 and we get output count= 0

Time=15 for rst=0 load=0 data= 0 u_d=0 and we get output count=15

Time=20 for rst=0 load=1 data= 9 u_d=0 and we get output count=15

Time=25 for rst=0 load=1 data= 9 u_d=0 and we get output count= 9

Time=30 for rst=0 load=0 data= 9 u_d=1 and we get output count= 9

Time=35 for rst=0 load=0 data= 9 u_d=1 and we get output count=10

Time=45 for rst=0 load=0 data= 9 u_d=1 and we get output count=11

Time=55 for rst=0 load=0 data= 9 u_d=1 and we get output count=12

Time=65 for rst=0 load=0 data= 9 u_d=1 and we get output count=13

Time=75 for rst=0 load=0 data= 9 u_d=1 and we get output count=14

Time=80 for rst=0 load=0 data= 9 u_d=0 and we get output count=14

Time=85 for rst=0 load=0 data= 9 u_d=0 and we get output count=13

Time=95 for rst=0 load=0 data= 9 u_d=0 and we get output count=12

Time=105 for rst=0 load=0 data= 9 u_d=0 and we get output count=11

Time=115 for rst=0 load=0 data= 9 u_d=0 and we get output count=10

Time=125 for rst=0 load=0 data= 9 u_d=0 and we get output count= 9

Time=135 for rst=0 load=0 data= 9 u_d=0 and we get output count= 8

Time=145 for rst=0 load=0 data= 9 u_d=0 and we get output count= 7

Time=155 for rst=0 load=0 data= 9 u_d=0 and we get output count= 6

Time=165 for rst=0 load=0 data= 9 u_d=0 and we get output count= 5

Time=175 for rst=0 load=0 data= 9 u_d=0 and we get output count= 4

Time=185 for rst=0 load=0 data= 9 u_d=0 and we get output count= 3

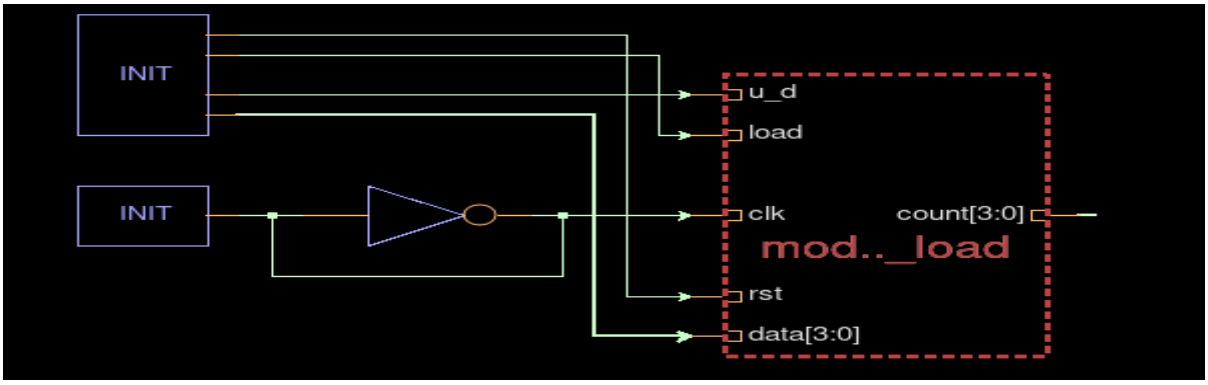
Time=195 for rst=0 load=0 data= 9 u_d=0 and we get output count= 2

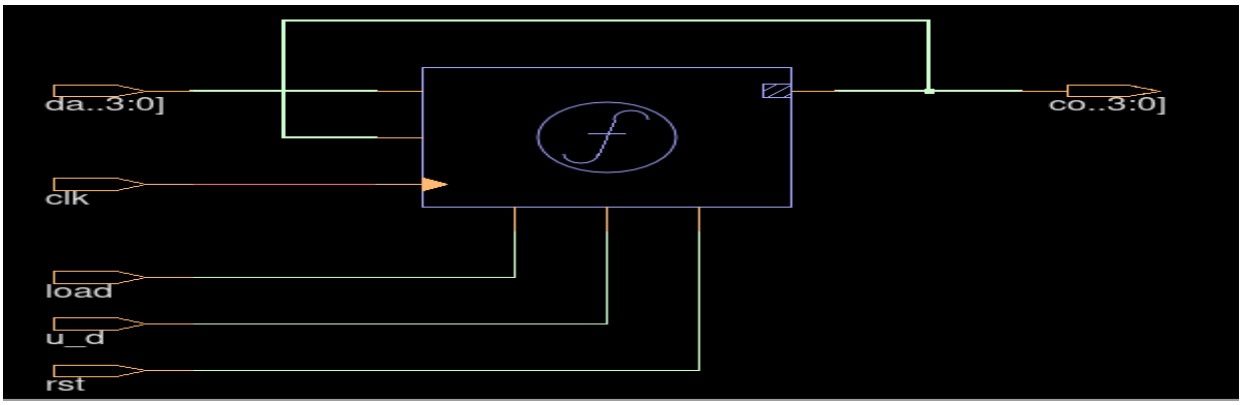
Time=205 for rst=0 load=0 data= 9 u_d=0 and we get output count= 1

Time=215 for rst=0 load=0 data= 9 u_d=0 and we get output count= 0

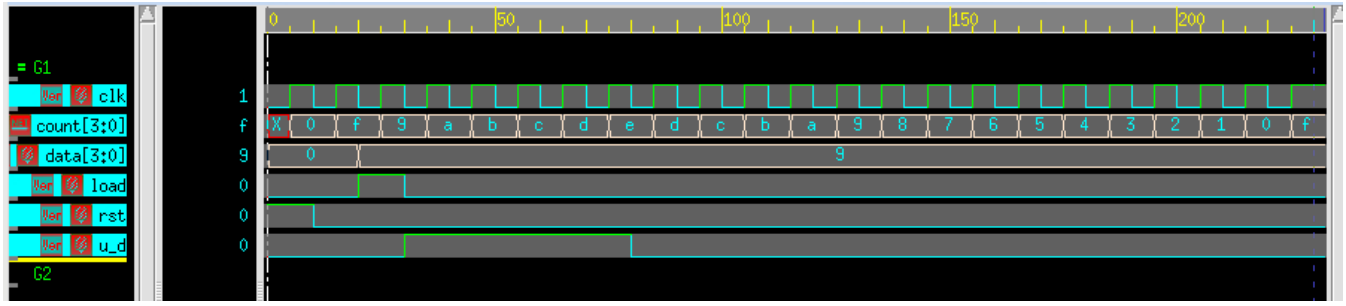
Time=225 for rst=0 load=0 data= 9 u_d=0 and we get output count=15

Schematic Design





waveform



Lab6 fork-join

30.0 Use fork-join (parallel execution) and initial-begin (sequential executions)

Design code

```
//fork- join statement and sequential statement in execution Verilog code
module fork_join_example();
  initial begin
    $display("Time = %0t : Starting fork...join", $time);

    fork
      begin
        #5 $display("Time = %0t : Task 1 done", $time);
      end

      begin
        #10 $display("Time = %0t : Task 2 done", $time);
      end
    join
    $display("Time = %0t : Both tasks completed", $time);
  end
endmodule

module sequential_example();
  initial begin
        $display("Time = %0t : sequential...execution", $time);
        #5 $display("Time = %0t : Task 1 done", $time);
        #10 $display("Time = %0t : Task 2 done", $time);

        #15 $display("Time = %0t : Both tasks completed", $time);
  end
```

```
endmodule
```

Simulation Result

Output

```
Time = 0 : Starting fork...join
Time = 0 : sequential...execution
Time = 5 : Task 1 done
Time = 5 : Task 1 done
Time = 10 : Task 2 done
Time = 10 : Both tasks completed
Time = 15 : Task 2 done
Time = 30 : Both tasks completed
```

31.0 Use fork-join (parallel execution) + initial-begin (sequential executions)

Design code

```
//fork- join statement + sequential statementin verilog code


module fork_join_example();
  initial begin
    $display("Time = %0t : Starting fork...join", $time);

    fork
      begin
        #5 $display("Time = %0t : Task 1 done", $time);
      end

      begin
```

```
      #10 $display("Time = %0t : Task 2 done", $time);
    end
  join

  $display("Time = %0t : Both tasks completed", $time);

  $display("Time = %0t : sequential...execution", $time);
  #5 $display("Time = %0t : Task 1 done", $time);
  #10 $display("Time = %0t : Task 2 done", $time);
  #15 $display("Time = %0t : Both tasks completed", $time);
 end
endmodule
```

Simulation Result

Output

```
Time = 0 : Starting fork...join
Time = 5 : Task 1 done
Time = 10 : Task 2 done
Time = 10 : Both tasks completed
Time = 10 : sequential...execution
Time = 15 : Task 1 done
Time = 25 : Task 2 done
Time = 40 : Both tasks completed
```

Use of task and Function

32.0 Perform a factorial of a number

Design code

```
//find a factorial of a number
module factorial_fun();

integer result;
function integer factorial;
    input integer n;
    integer i;

    begin
        factorial=1;
        for (i=1; i<=n ;i=i+1) begin
            factorial=factorial*i;
        end
    end
endfunction

initial begin
    result=factorial(4);
    $display("Factorial of is: %0d", result);
end
endmodule
```

Simulation Result

Output

Factorial of this number is: 24

Lab7

33.0 Design a 4-bit F.A. using tasks and function by using 1-bit

Design code

```
//4-bit full adder using task and function
module full_4bit_adder();

function [1:0] fulladder_1bit;
        input a, b, cin;
        reg sum, cout;
        begin
                sum  = a ^ b ^ cin;
                cout = (a & b) | (b & cin) | (a & cin);
                fulladder_1bit = {cout, sum};
        end
endfunction
  // Task for 4-bit addition using 1-bit full adder
  task add_4bit;
    input [3:0] in1, in2;
    input cin;
    output [3:0] sum_out;
    output cout_out;

    integer i;
    reg c;
    reg [1:0] temp;

    begin
```

```
    c = cin;
    for (i = 0; i < 4; i = i + 1) begin
      temp = fulladder_1bit(in1[i], in2[i], c);
      sum_out[i] = temp[0];
      c = temp[1];
    end
    cout_out = c;
  end
 endtask
endmodule
```

Testbench

```
//testbench of 4-bit fulladerr using taska nd function
module full_4bit_adder_tb();
reg [3:0] a,b;
reg cin;
reg [3:0] sum;
reg cout;

full_4bit_adder DUT();

initial begin
   a= 4'b1010;  // 10
   b= 4'b0111;  // 7
   cin = 1'b0;

   DUT.add_4bit(a, b, cin, sum, cout);
   $display("Test Case 1:");
```

```
    $display("A    =%b (%0d)",a,a);
    $display("B    =%b (%0d)",b,b);
    $display("Cin    =%b ",cin);

    $display("Sum   = %b (%0d)", sum, sum);
    $display("Cout  = %b", cout);
    $display("------------------------");
    // Test 2
    a= 4'b1111;  // 15
    b= 4'b0001;  // 1
    cin = 1'b1;

    DUT.add_4bit(a, b, cin, sum, cout);
    $display("Test Case 2:");
    $display("A    =%b (%0d)",a,a);
    $display("B    =%b (%0d)",b,b);
    $display("Cin    =%b ",cin);

    $display("Sum   = %b (%0d)", sum, sum);
    $display("Cout  = %b", cout);

    $finish;
  end
endmodule
```

Simulation Result

Output

Test Case 1:

A =1010 (10)

B =0111 (7)

Cin =0

Sum = 0001 (1)

Cout = 1

Test Case 2:

A =1111 (15)

B =0001 (1)

Cin =1

Sum = 0001 (1)

Cout = 1

Lab7

34.0 Parameterized module by user using keyword parameter

Take data and id argument data_width and id_width two parametric value. If (data>id) then perform subtraction and otherwise addition operation

Design code-1

```
//use of parameter by the user
module param_adder_sub #(parameter DATA_WIDTH=4 ,ID_WIDTH=2)(data, id,out);
input [DATA_WIDTH-1:0]data;
input [ID_WIDTH-1:0]id;
output [DATA_WIDTH +ID_WIDTH-1:0]out;

assign out= (DATA_WIDTH > ID_WIDTH)? (data - id) :(data + id);
endmodule
```

Testbench-1

```
// testbench of parameter value by user
module TB();
parameter DATA_WIDTH =4 ,ID_WIDTH =2;
reg [4:0]data;
reg [2:0]id;
wire [7:0]out;

param_adder_sub #(5,3) p2(.data(data),.id(id),.out(out));

initial begin
    data=20;
    id=14;
    #1;
    $display( "DATA_WIDTH=%0d,
ID_WIDTH=%0d,data=%0d,id=%0d,out=%0d",p2.DATA_WIDTH,p2.ID_WIDTH,data
,id,out);
end
endmodule
```

Run command

```
vcs parameter_concept1.v parameter_concept1_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
DATA_WIDTH=5, ID_WIDTH=3,data=20,id=6,out=14
```

Design code-2

```
//use of parameter by the user
```

```
module param_concept #(parameter DATA_WIDTH=5 ,ID_WIDTH=5)(data, id,out);
input [DATA_WIDTH-1:0]data;
input [ID_WIDTH-1:0]id;
output reg [DATA_WIDTH + ID_WIDTH -1:0]out;

always @(*)begin
        if (data > id)
                out=data-id;
        else
                out=data+id;
  end
endmodule
```

Testbench-2

```
// testbench of parameter value by user
module TB();
parameter DATA_WIDTH =5 ,ID_WIDTH =5;
reg [4:0]data;
reg [4:0]id;
wire [9:0]out;

param_concept #(5,5) p2(.data(data),.id(id),.out(out));

initial begin
        data=14;
        id=20;
        #1;
```

```
        $display( "DATA_WIDTH=%0d,
ID_WIDTH=%0d,data=%0d,id=%0d,out=%0d",p2.DATA_WIDTH,p2.ID_WIDTH,data
,id,out);

end

endmodule
```

Run command

```
vcs parameter_concept.v parameter_concept1_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
DATA_WIDTH=5, ID_WIDTH=5,data=14,id=20,out=34
```

35.0 Parameterized keyword module by using localparam

Design code-3

```
//use of parameter by the user

module param_adder_sub #(parameter DATA_WIDTH=4 ,ID_WIDTH=2)(data, id,out);

input [DATA_WIDTH-1:0]data;

input [ID_WIDTH-1:0]id;

output [DATA_WIDTH +ID_WIDTH-1:0]out;


assign out= (DATA_WIDTH > ID_WIDTH) ? data-id :data+id;

endmodule
```

Testbench-3

```
// testbench of parameter by user using localparam

module TB();

genvar i;

generate
```

```
    for (i=0;i<5;i=i+1) begin:gen_list
        localparam DW=4+i;
        localparam IW=2+i;

        reg [DW-1:0] data;
        reg [IW-1:0] id;
        wire [(DW+IW)-1:0]out;

        param_adder_sub #(DW, IW) inst(.data(data),.id(id),.out(out));

        initial begin
            data=i+1;
            id=i;
            #10 $display("INST %0d: DW=%0d IW=%0d data=%0d id=%0d out=%0d",i,DW,IW,data,id,out);
            #1000 $finish;
        end
    end
endgenerate
endmodule
```

Run command

```
vcs parameter_concept1.v parameter_concept3_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
INST 0: DW=4 IW=2 data=1 id=0 out=1
INST 1: DW=5 IW=3 data=2 id=1 out=1
INST 2: DW=6 IW=4 data=3 id=2 out=1
```

INST 3: DW=7 IW=5 data=4 id=3 out=1

INST 4: DW=8 IW=6 data=5 id=4 out=1

Lab8 Generate block statement

36.0 Design a 4-bit Full adder using generate block

Design code

```
// design a full adder by using the generate block
module full_adder (
	input a,b,cin,
	output sum,cout
);
assign sum=a^b^cin;
assign cout=(a & b) | (b & cin) | (cin & a);
endmodule
module adder_4bit (
	input [3:0]a,b,
	input cin,
	output [3:0]sum,
	output cout
);
wire [4:0] carry;

assign carry[0]=cin;
genvar i;
generate
	for (i=0;i<4;i=i+1)begin:fa_loop
		full_adder fa(.a(a[i]),.b(b[i]),.cin(carry[i]),.sum(sum[i]),.cout(carry[i+1]));
```

```
    end
endgenerate

assign cout=carry[4];
endmodule
```

Testbench

```
module TB();
reg [3:0]a,b;
reg cin;

wire [3:0]sum;
wire cout;

adder_4bit dut(a,b,cin,sum,cout);

initial begin
    $monitor("Time=%0t,a=%0d,b=%0d,cin=%0b and we get sum=%0d, cout=%0b",$time,a,b,cin,sum,cout);
    a=4'b0101;b=4'b0011;cin=0;
    #10 a=4'b1000;b=4'b0111;cin=0;
    #10 a=4'b1111;b=4'b0001;cin=0;
    #10 a=4'b0110;b=4'b1001;cin=1;
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
```

end

endmodule

Simulation Result

Output

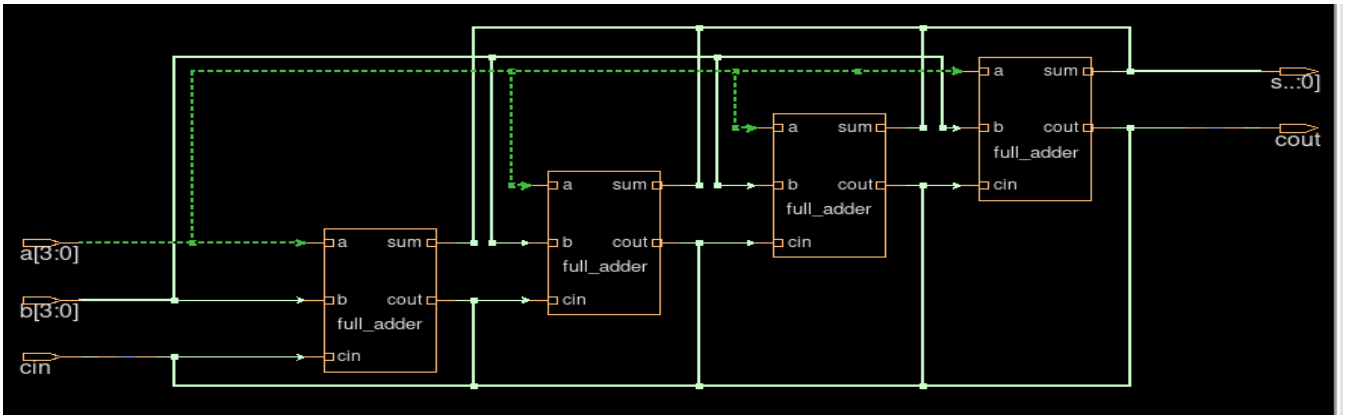
Time=0,a=5,b=3,cin=0 and we get sum=8, cout=0

Time=10,a=8,b=7,cin=0 and we get sum=15, cout=0

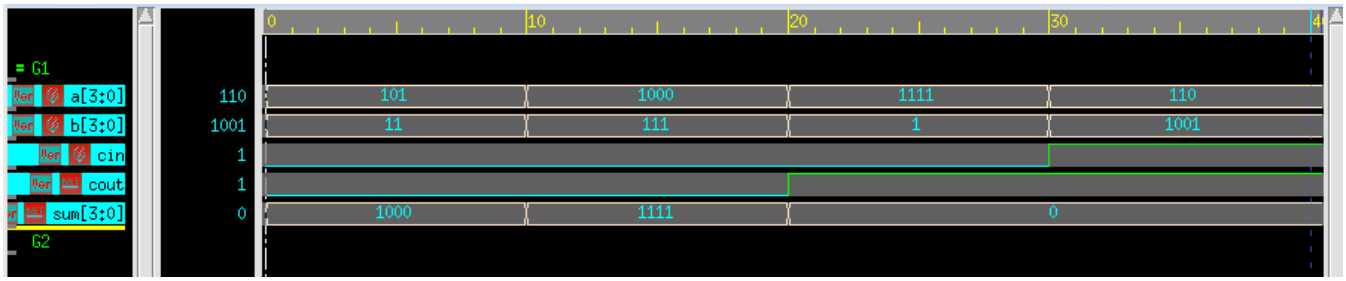
Time=20,a=15,b=1,cin=0 and we get sum=0, cout=1

Time=30,a=6,b=9,cin=1 and we get sum=0, cout=1

Schematic Design



waveform



37.0 Design a 4-bit Ripple Carry Adder by using generate and parameter statement

Design code

module full_adder (

```
        input a,b,cin,
        output sum,cout
);
assign sum=a^b^cin;
assign cout=((a & b) | (b & cin) | (cin & a));
endmodule

module ripple_carry_adder_4bit #(parameter N=4)(
        input [N-1:0]A,B,
        input Cin,
        output [N-1:0]S,
        output Cout
);
wire [N:0]carry;
assign carry[0]=Cin;

genvar g;
generate
        for (g=0;g<N;g=g+1)begin:fa_loop
                full_adder fa(.a(A[g]),.b(B[g]),.cin(carry[g]),.sum(S[g]),.cout(carry[g+1]));
        end
endgenerate

assign Cout=carry[N]
endmodule
```

Testbench

```
module TB();
```

```
parameter N=4;
reg [N-1:0]A,B;
reg Cin;

wire [N-1:0]S;
wire Cout;

ripple_carry_adder_4bit #(N) dut(.A(A),.B(B),.Cin(Cin),.S(S),.Cout(Cout));

initial begin
    $monitor("Time=%0t,A=%0b,B=%0b,Cin=%0b and we get S=%0d, Cout=%0b",$time,A,B,Cin,S,Cout);
    A=0;B=0;Cin=0;
    #10 A=4'b0001;B=4'b0011;Cin=0;
    #10 A=4'b0110;B=4'b0101;Cin=1;
    #10 A=4'b1111;B=4'b1111;Cin=1;
    #10 A=4'b1010;B=4'b0101;Cin=0;
    #10 A=4'b0000;B=4'b0000;Cin=1;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0,A=0,B=0,Cin=0 and we get S=0, Cout=0

Time=10,A=1,B=11,Cin=0 and we get S=4, Cout=0

Time=20,A=110,B=101,Cin=1 and we get S=12, Cout=0

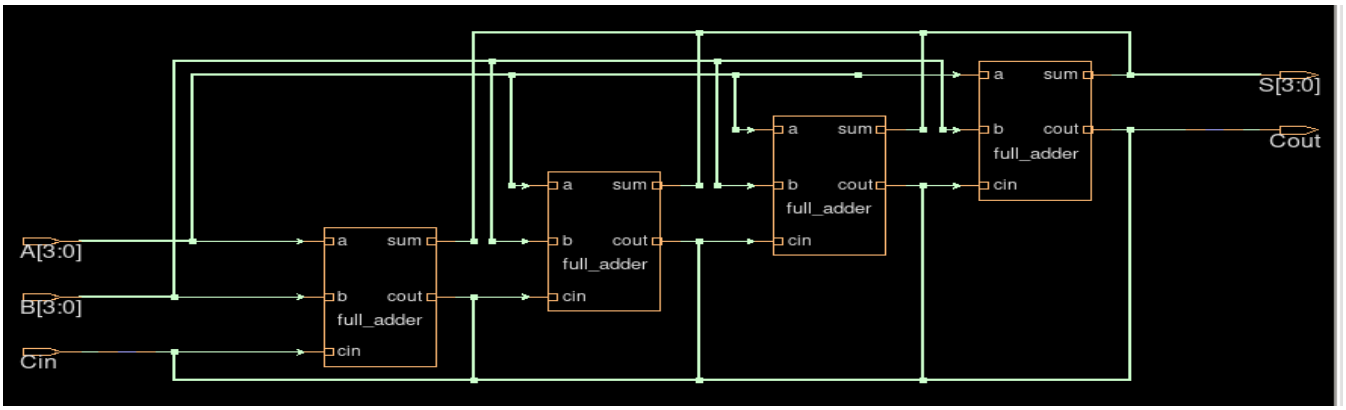
Time=30,A=1111,B=1111,Cin=1 and we get S=15, Cout=1

Time=40,A=1010,B=101,Cin=0 and we get S=15, Cout=0

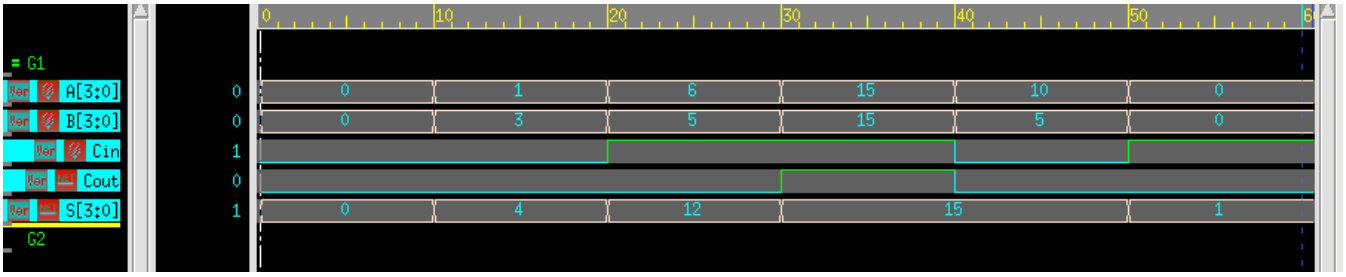
Time=50,A=0,B=0,Cin=1 and we get S=1, Cout=0

$finish at simulation time 60

Schematic Design



waveform



Generate Condition: generate if-else

38.0 Design a full adder or a half adder with a parameter (sel)

Design code

```
module half_adder(
   input a, b,
   output sum, cout
);
assign {sum, cout} = {a^b, (a & b)};
endmodule

module full_adder (
       input a,b,cin,
       output sum,cout
);
assign sum=a^b^cin;
assign cout=((a & b) | (b & cin) | (cin & a));

endmodule
module gen_if_ex #(parameter sel=0)(
       input A,B,Cin,
       output S,Cout
);
generate
       if(sel) begin
          initial $display("Full Adder is selected");
          full_adder fa(A, B, Cin, S, Cout);
       end else begin
          initial $display("Half Adder is selected");
          half_adder ha(A, B, S, Cout);
```

```
        end
    endgenerate
endmodule
```

Testbench

```
module TB();
reg A,B,Cin;
wire S,Cout;
parameter sel=1;

gen_if_ex #(1) dut(.A(A),.B(B),.Cin(Cin),.S(S),.Cout(Cout));

initial begin
        $monitor("Time=%0t,A=%0b,B=%0b,Cin=%0b and we get S=%0d, Cout=%0b",$time,A,B,Cin,S,Cout);
        A=0;B=0;Cin=0;
        #10 A=1;B=1;Cin=0;
        #10 A=1;B=0;Cin=0;
        #10 A=0;B=1;Cin=1;
        #10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Full Adder is selected

Time=0,A=0,B=0,Cin=0 and we get S=0, Cout=0

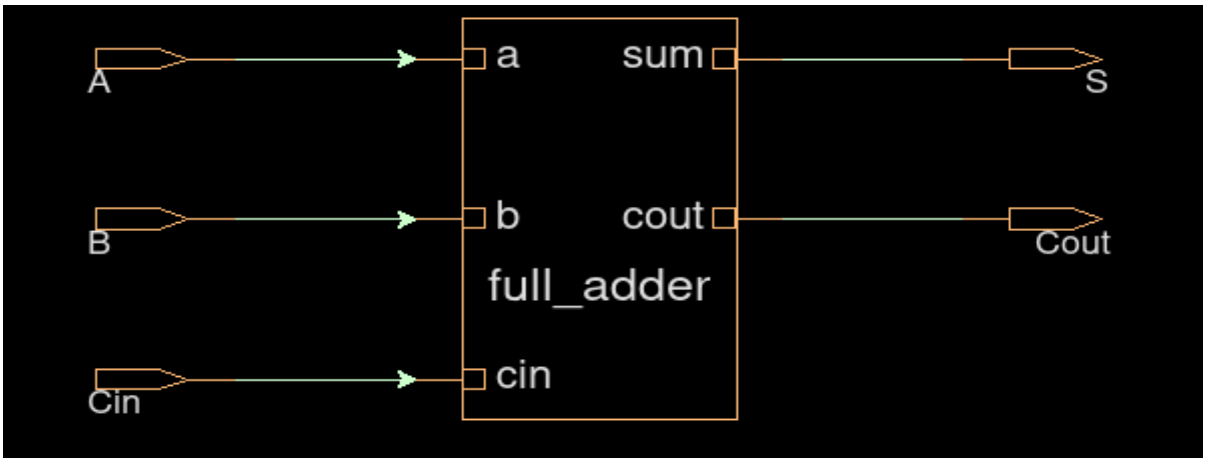
Time=10,A=1,B=1,Cin=0 and we get S=0, Cout=1

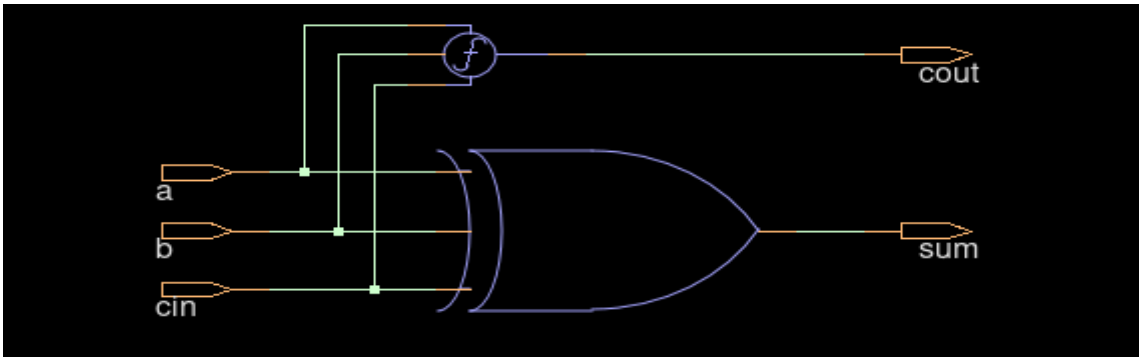
Time=20,A=1,B=0,Cin=0 and we get S=1, Cout=0

Time=30,A=0,B=1,Cin=1 and we get S=0, Cout=1

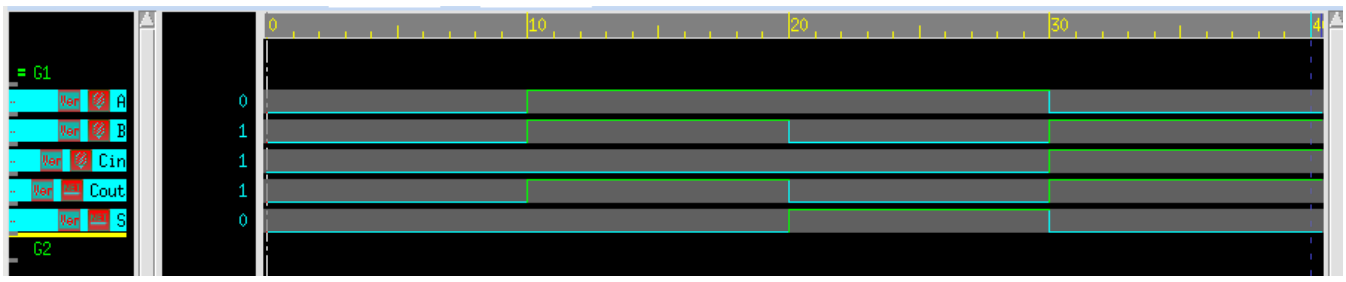
$finish at simulation time 40

Schematic Design





waveform



Generate Condition: generate case (alternatively of if-else)

39.0 Design a full adder or a half adder with a parameter (sel)

Design code

```
module half_adder(
   input a, b,
   output sum, cout
);
assign {sum, cout} = {a^b, (a & b)};
endmodule

module full_adder (
      input a,b,cin,
      output sum,cout
);
assign sum=a^b^cin;
assign cout=((a & b) | (b & cin) | (cin & a));

endmodule
module gen_case #(parameter sel=0)(
      input A,B,Cin,
      output S,Cout
);

generate
      case(sel)
            0: begin
```

```
            initial $display("Half Adder is selected");
            half_adder ha(A, B, S, Cout);end
        1: begin
            initial $display("Full Adder is selected");
            full_adder fa(A, B, Cin, S, Cout);end
    endcase
endgenerate

endmodule
```

Testbench

```
module TB();
reg A,B,Cin;
wire S,Cout;
parameter sel=1;

gen_case #(1) dut(.A(A),.B(B),.Cin(Cin),.S(S),.Cout(Cout));

initial begin
    $monitor("Time=%0t,A=%0b,B=%0b,Cin=%0b and we get S=%0d, Cout=%0b",$time,A,B,Cin,S,Cout);
    A=0;B=0;Cin=0;
    #10 A=0;B=1;Cin=0;
    #10 A=1;B=0;Cin=0;
    #10 A=1;B=1;Cin=0;
    #10 A=1;B=0;Cin=1;
    #10 A=0;B=1;Cin=1;
    #10 $finish();
```

```
end


initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0,A=0,B=0,Cin=0 and we get S=0, Cout=0

Time=10,A=0,B=1,Cin=0 and we get S=1, Cout=0

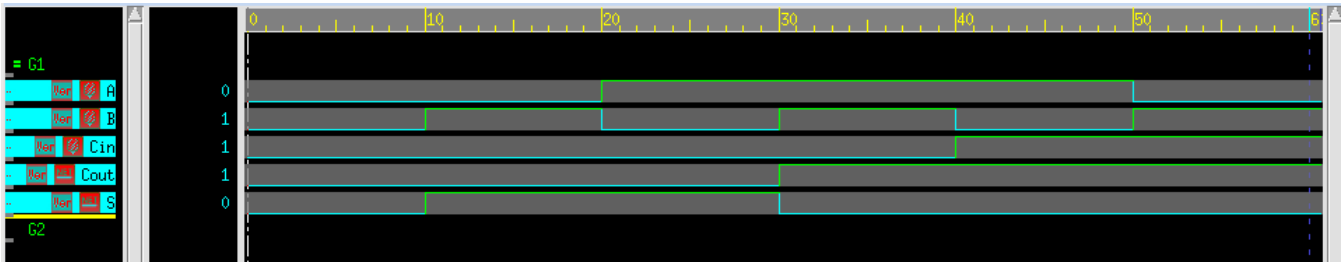
Time=20,A=1,B=0,Cin=0 and we get S=1, Cout=0

Time=30,A=1,B=1,Cin=0 and we get S=0, Cout=1

Time=40,A=1,B=0,Cin=1 and we get S=0, Cout=1

Time=50,A=0,B=1,Cin=1 and we get S=0, Cout=1

waveform



Lab9

40.0 Design a 4-bit Binary to Gray code by using data flow Design code

```
//Design a 4-bit binary to gray code
```

```
module binary_gray(g3,g2,g1,g0,b3,b2,b1,b0);
input b3,b2,b1,b0;
output g3,g2,g1,g0;

assign g3=b3;
assign g2=b3^b2;
assign g1=b2^b1;
assign g0=b1^b0;
endmodule
```

Testbench

```
// design the testbench for binary to gray code
module TB();
reg b3,b2,b1,b0;
wire g3,g2,g1,g0;

binary_gray dut(.g3(g3),.g2(g2),.g1(g1),.g0(g0),.b3(b3),.b2(b2),.b1(b1),.b0(b0));

initial begin
$monitor("Time=%0t,g3=%b,g2=%b,g1=%b,g0=%b,b3=%b,b2=%b,b1=%b,b0=%b",$time,g3,g2,g1,g0,b3,b2,b1,b0);

    b3=1;b2=0;b1=1;b0=0;#10
    b3=1;b2=1;b1=1;b0=0;#10
    b3=0;b2=1;b1=0;b0=1;#10
    b3=1;b2=0;b1=0;b0=1;#10
    $finish();
end
```

```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

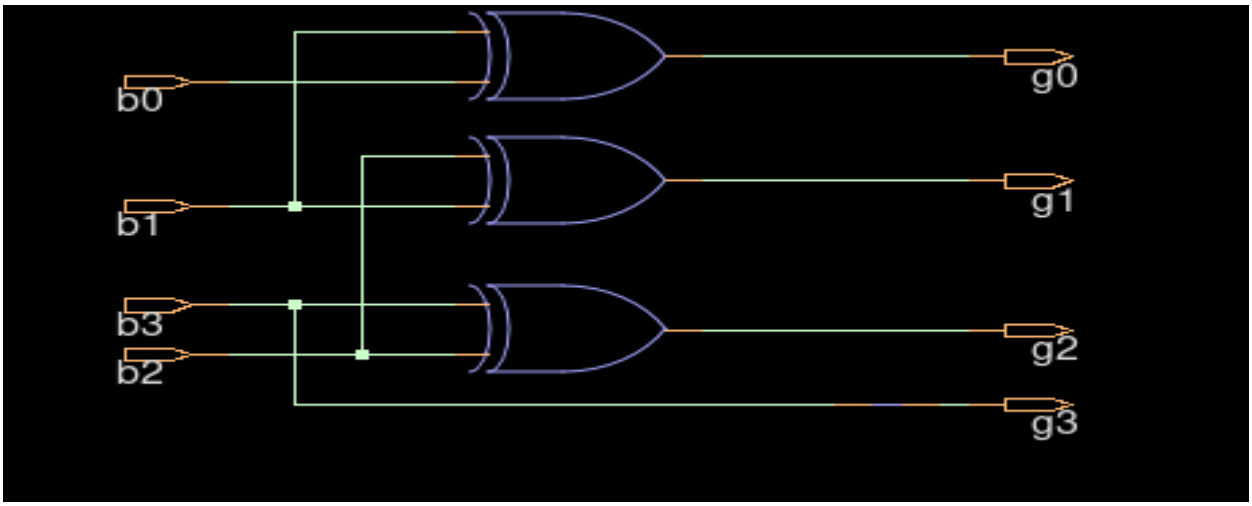
```
vcs binary_to_gray.v binary_to_gray_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

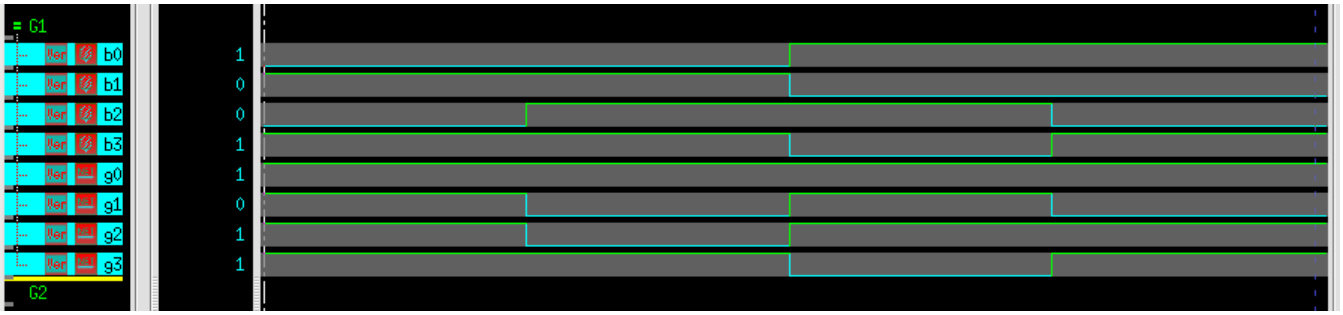
Output

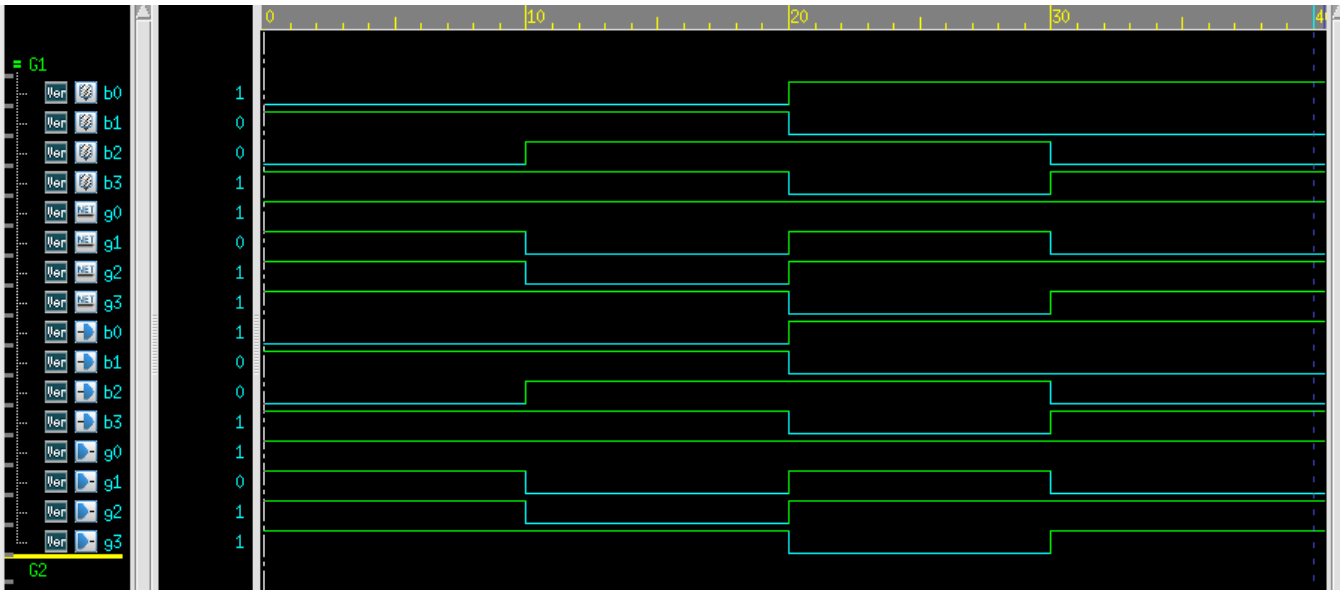
```
Time=0,g3=1,g2=1,g1=1,g0=1,b3=1,b2=0,b1=1,b0=0
Time=10,g3=1,g2=0,g1=0,g0=1,b3=1,b2=1,b1=1,b0=0
Time=20,g3=0,g2=1,g1=1,g0=1,b3=0,b2=1,b1=0,b0=1
Time=30,g3=1,g2=1,g1=0,g0=1,b3=1,b2=0,b1=0,b0=1
```

Schematic Design



Waveform





Method-2

Design code

```
//Design the 4-bit binary to gray code
module binary_gray(binary,gray);
input [3:0] binary;
output [3:0] gray;

assign gray[3]=binary[3];
assign gray[2]=binary[3]^binary[2];
assign gray[1]=binary[2]^binary[1];
assign gray[0]=binary[1]^binary[0];

endmodule
```

Testbench-1

```
// design the binary to gray code of testbench
module TB();
reg [3:0]binary;
wire [3:0]gray;

binary_gray dut(.binary(binary),.gray(gray));
integer i;

initial begin
    $monitor("Time=%0t,binary=%b,gray=%b",$time,binary,gray);
    for (i=0;i<16;i=i+1)begin
        binary=i[3:0];
        #5;
    end
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Testbench-2

```
// design the binary to gray code of testbench
module TB();
reg [3:0]binary;
wire [3:0]gray;
```

```
binary_gray dut(.binary(binary),.gray(gray));
integer i;
initial begin
	$display("binary\tgray code");
	for (i=0;i<16;i=i+1)begin
		binary=i[3:0];
		#5;
		$display("Time=%0t,binary=%b,gray=%b",$time,binary,gray);
	end
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

```
Time=0,binary=0000,gray=0000
Time=5,binary=0001,gray=0001
Time=10,binary=0010,gray=0011
Time=15,binary=0011,gray=0010
Time=20,binary=0100,gray=0110
Time=25,binary=0101,gray=0111
Time=30,binary=0110,gray=0101
Time=35,binary=0111,gray=0100
Time=40,binary=1000,gray=1100
```

Time=45,binary=1001,gray=1101

Time=50,binary=1010,gray=1111

Time=55,binary=1011,gray=1110

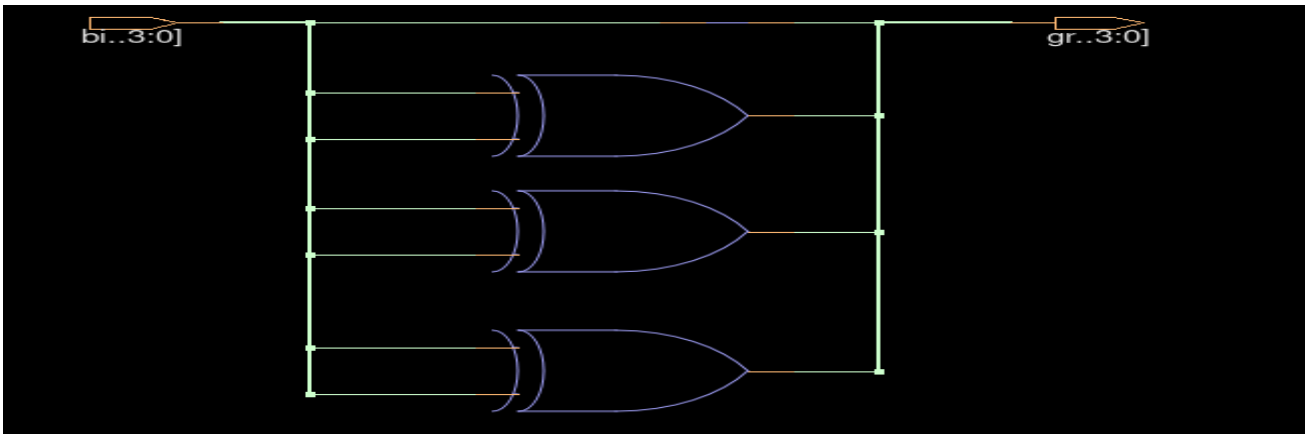
Time=60,binary=1100,gray=1010

Time=65,binary=1101,gray=1011

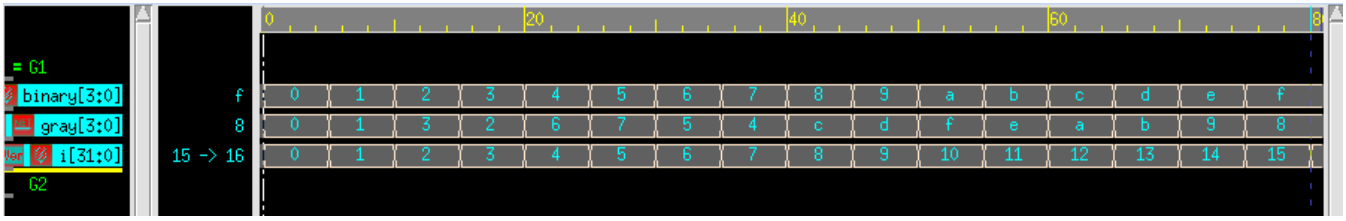
Time=70,binary=1110,gray=1001

Time=75,binary=1111,gray=1000

Schematic Design



Waveform



41.0 Design a generic n-bit Binary to Gray code by using data flow by using parameter keyword.

Method-1

Design code

```
//Design the 4-bit binary to gray code

module binary_gray  #(parameter N=4)(
```

```
input [N-1:0] binary,
output [N-1:0] gray);
assign gray=binary ^ (binary>>1);
endmodule
```

Testbench

```
// design the binary to gray code of testbench
module TB();
parameter N=4;
reg [N-1:0]binary;
wire [N-1:0]gray;

binary_gray #(N) dut(.binary(binary),.gray(gray));
integer i;

initial begin
    for (i=0;i <(1<<N);i=i+1)begin
        binary=i;
        #1;
        $display("Time=%0t,binary=%b,gray=%b",$time,binary,gray);
    end
    #20 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
```

```
end

endmodule
```

Simulation Result

output

```
Time=1,binary=0000,gray=0000

Time=2,binary=0001,gray=0001

Time=3,binary=0010,gray=0011

Time=4,binary=0011,gray=0010

Time=5,binary=0100,gray=0110

Time=6,binary=0101,gray=0111

Time=7,binary=0110,gray=0101

Time=8,binary=0111,gray=0100

Time=9,binary=1000,gray=1100

Time=10,binary=1001,gray=1101

Time=11,binary=1010,gray=1111

Time=12,binary=1011,gray=1110

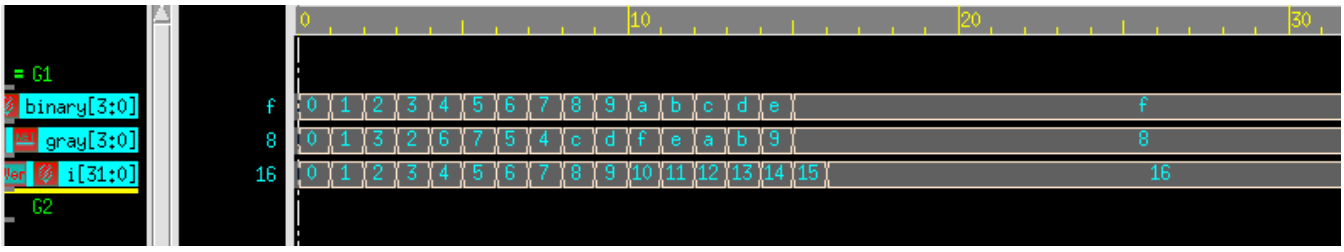
Time=13,binary=1100,gray=1010

Time=14,binary=1101,gray=1011

Time=15,binary=1110,gray=1001

Time=16,binary=1111,gray=1000
```

Waveform



Method-2

Design code

```
//Design the 4-bit binary to gray code
module binary_gray #(parameter N=4)(
input [N-1:0] binary,
output [N-1:0] gray);
assign gray[N-1]=binary[N-1];

genvar i;
generate
	for (i=N-2; i>=0;i=i-1)begin :gray_gen
		assign gray[i]=binary[i+1]^binary[i];
	end
endgenerate
endmodule
```

Testbench

```
// Design the binary to gray code of testbench
module TB();
parameter N=4;
reg [N-1:0]binary;
wire [N-1:0]gray;

binary_gray #(N) dut(.binary(binary),.gray(gray));
integer i;

initial begin
	$display("binary\tgray code");
```

```
    for (i=0;i <(1<<N);i=i+1)begin
        binary=i[N-1:0];
        #1;
        $display("Time=%0t,binary=%b,gray=%b",$time,binary,gray);
    end
    #20 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

```
Time=1,binary=0000,gray=0000
Time=2,binary=0001,gray=0001
Time=3,binary=0010,gray=0011
Time=4,binary=0011,gray=0010
Time=5,binary=0100,gray=0110
Time=6,binary=0101,gray=0111
Time=7,binary=0110,gray=0101
Time=8,binary=0111,gray=0100
Time=9,binary=1000,gray=1100
Time=10,binary=1001,gray=1101
Time=11,binary=1010,gray=1111
```

Time=12,binary=1011,gray=1110

Time=13,binary=1100,gray=1010

Time=14,binary=1101,gray=1011

Time=15,binary=1110,gray=1001

Time=16,binary=1111,gray=1000

42.0 Design a 4-bit Gray to binary code by using data flow

Design code

```
//Design the 4-bit  gray to binary code
module gray_binary(binary,gray);
input [3:0] gray;
output [3:0] binary;

assign binary[3]=gray[3];
assign binary[2]=gray[3]^gray[2];
assign binary[1]=gray[3]^gray[2]^gray[1];
assign binary[0]=gray[3]^gray[2]^gray[1]^gray[0];
endmodule
```

Testbench

```
// Design the gray to binary code of testbench
module TB();
reg [3:0]gray;
wire [3:0]binary;

gray_binary dut(.gray(gray),.binary(binary));
integer i;
```

```
initial begin
    $monitor("Time=%0t,gray=%b,binary=%b",$time,gray,binary);
    for (i=0;i<16;i=i+1)begin
        gray=i[3:0];
        #1;
    end
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

```
vcs gray_binary.v gray_binary_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
Time=0,gray=0000,binary=0000
Time=1,gray=0001,binary=0001
Time=2,gray=0010,binary=0011
Time=3,gray=0011,binary=0010
Time=4,gray=0100,binary=0111
Time=5,gray=0101,binary=0110
Time=6,gray=0110,binary=0100
Time=7,gray=0111,binary=0101
Time=8,gray=1000,binary=1111
Time=9,gray=1001,binary=1110
```

Time=10,gray=1010,binary=1100

Time=11,gray=1011,binary=1101

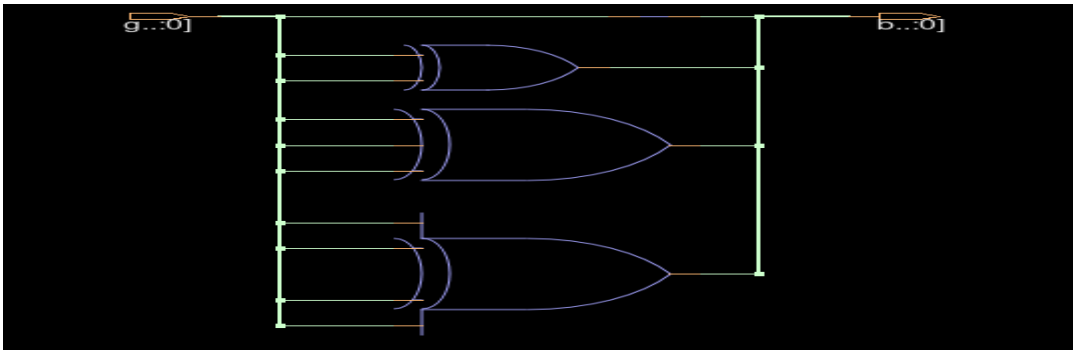
Time=12,gray=1100,binary=1000

Time=13,gray=1101,binary=1001

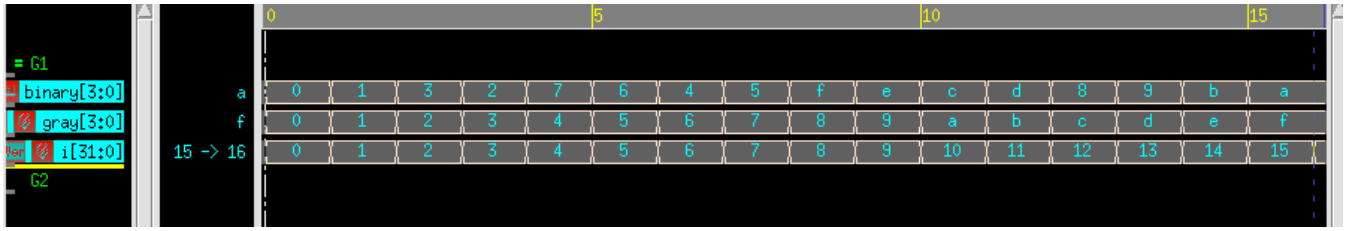
Time=14,gray=1110,binary=1011

Time=15,gray=1111,binary=1010

Schematic Design



waveform



43.0 Design a generic n-bit Gray to Binary code by using data flow by using parameter keyword.

Design code

//Design the n-bit binary to gray code

module gray_binary #(parameter N=4)(

input [N-1:0] gray,

output [N-1:0] binary);

assign binary[N-1]=gray[N-1];

```
genvar i;
generate
	for (i=N-2; i>=0;i=i-1)begin :binary_gen
		assign binary[i]=binary[i+1]^gray[i];
	end
endgenerate
endmodule
```

Testbench

```
// design the n-bit gray to binary code of testbench
module TB();
parameter N=4;
reg [N-1:0]gray;
wire [N-1:0]binary;

gray_binary #(N) dut(.gray(gray),.binary(binary));
integer i;

initial begin
	$display("gray\tbinary code");
	for (i=0;i <(1<<N);i=i+1)begin
		gray=i[N-1:0];
		#1;
		$display("Time=%0t,gray=%b,binary=%b",$time,gray,binary);
	end
	#20 $finish();
end
```

```
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

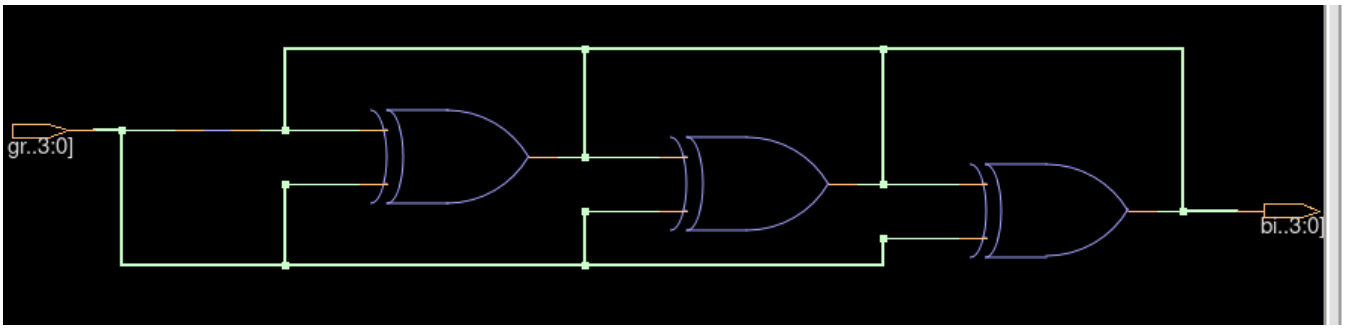
```
vcs gray_binary.v gray_binary_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

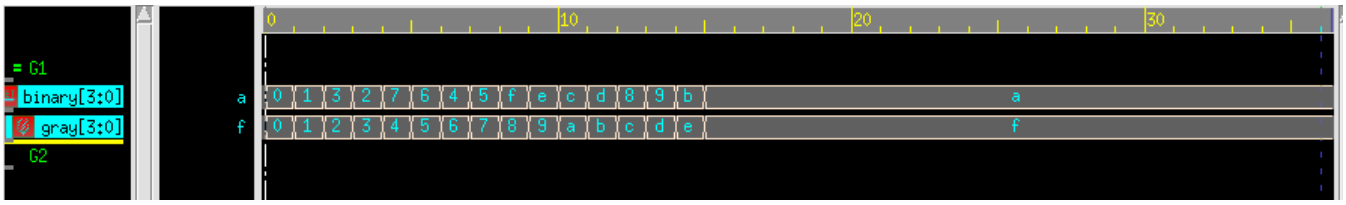
Output

```
Time=1,gray=0000,binary=0000
Time=2,gray=0001,binary=0001
Time=3,gray=0010,binary=0011
Time=4,gray=0011,binary=0010
Time=5,gray=0100,binary=0111
Time=6,gray=0101,binary=0110
Time=7,gray=0110,binary=0100
Time=8,gray=0111,binary=0101
Time=9,gray=1000,binary=1111
Time=10,gray=1001,binary=1110
Time=11,gray=1010,binary=1100
Time=12,gray=1011,binary=1101
Time=13,gray=1100,binary=1000
Time=14,gray=1101,binary=1001
Time=15,gray=1110,binary=1011
Time=16,gray=1111,binary=1010
```

Schematic Design



Waveform



44.0 Design a 4-bit Parity Checker by using data flow

Design code

```
// design the 4-bit parity checker

module parity_bit(parity_even,parity_odd,data_x);
input [3:0]data_x;
output parity_even,parity_odd;

assign parity_even=^data_x;
assign parity_odd=~^data_x;

endmodule
```

Testbench

```
/testbench of 4-bit parity checker
module TB();
```

```
reg [3:0]data_x;
wire parity_even;
wire parity_odd;
integer i;
parity_bit dut (.data_x(data_x),.parity_even(parity_even),.parity_odd(parity_odd));

initial begin
    $display("show the Data  Parity");
    for ( i = 0; i < 16; i = i + 1)begin
        data_x = i;
        #1 $display("for %b|parity_even=%b |even_odd= %b", data_x,
parity_even,parity_odd);
    end
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

```
vcs parity_bit.v parity_bit_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
show the Data  Parity
for 0000|parity_even=0 |even_odd= 1
```

for 0001|parity_even=1 |even_odd= 0

for 0010|parity_even=1 |even_odd= 0

for 0011|parity_even=0 |even_odd= 1

for 0100|parity_even=1 |even_odd= 0

for 0101|parity_even=0 |even_odd= 1

for 0110|parity_even=0 |even_odd= 1

for 0111|parity_even=1 |even_odd= 0

for 1000|parity_even=1 |even_odd= 0

for 1001|parity_even=0 |even_odd= 1

for 1010|parity_even=0 |even_odd= 1

for 1011|parity_even=1 |even_odd= 0

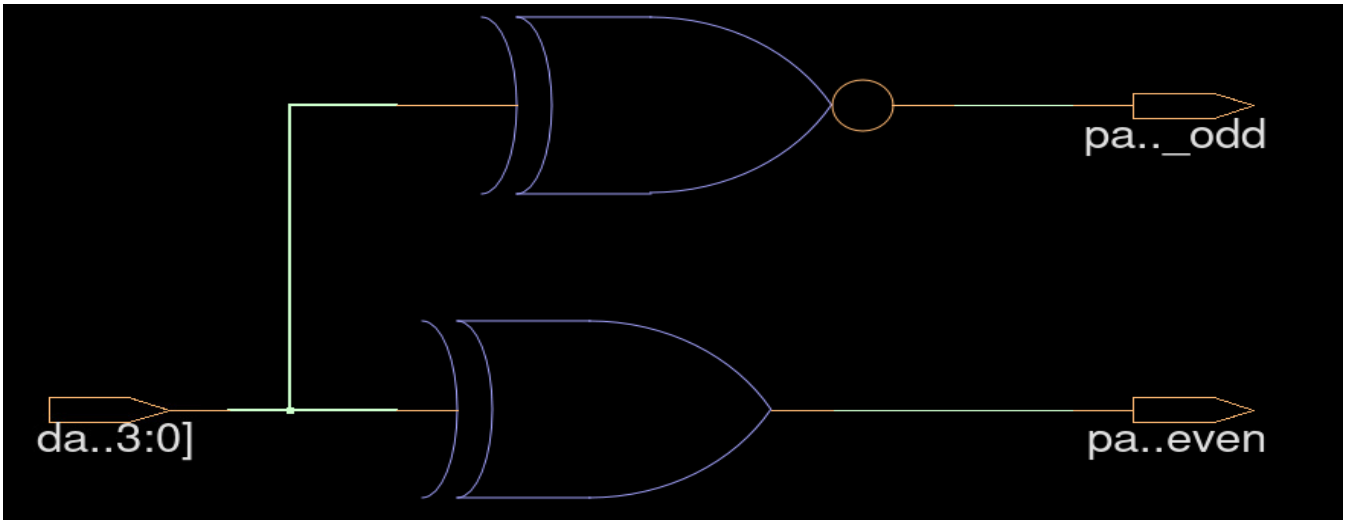
for 1100|parity_even=0 |even_odd= 1

for 1101|parity_even=1 |even_odd= 0

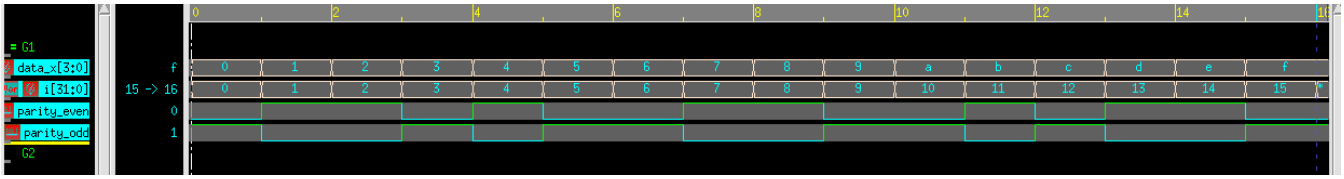
for 1110|parity_even=1 |even_odd= 0

for 1111|parity_even=0 |even_odd= 1

Schematic Design



waveform



45.0 Design a Generic N-bit parity checker by using data flow

Design code

```
// design the 4-bit parity checker                    # in testbench N=2^n -1
module parity_bit #(parameter N=4)(
input [N-1:0]data_x,
output parity_even,parity_odd
);

assign parity_even=^data_x;
assign parity_odd=~^data_x;
endmodule
```

Testbench

```
//testbench of 4-bit parity checker
module TB();
parameter N=4;
reg [N-1:0]data_x;
wire parity_even;
wire parity_odd;

integer i;
parity_bit #N dut (.data_x(data_x),.parity_even(parity_even),.parity_odd(parity_odd));

initial begin
```

```
        $display("show the Data  Parity");
        for ( i = 0; i < (1<<N); i = i + 1)begin
                data_x = i[N-1:0];
                #1 $display("for %b|parity_even=%b |even_odd= %b", data_x,
parity_even,parity_odd);
        end
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

```
vcs paritybit.v paritybit_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
for 0000|parity_even=0 |even_odd= 1
for 0001|parity_even=1 |even_odd= 0
for 0010|parity_even=1 |even_odd= 0
for 0011|parity_even=0 |even_odd= 1
for 0100|parity_even=1 |even_odd= 0
for 0101|parity_even=0 |even_odd= 1
for 0110|parity_even=0 |even_odd= 1
for 0111|parity_even=1 |even_odd= 0
for 1000|parity_even=1 |even_odd= 0
```

for 1001|parity_even=0 |even_odd= 1

for 1010|parity_even=0 |even_odd= 1

for 1011|parity_even=1 |even_odd= 0

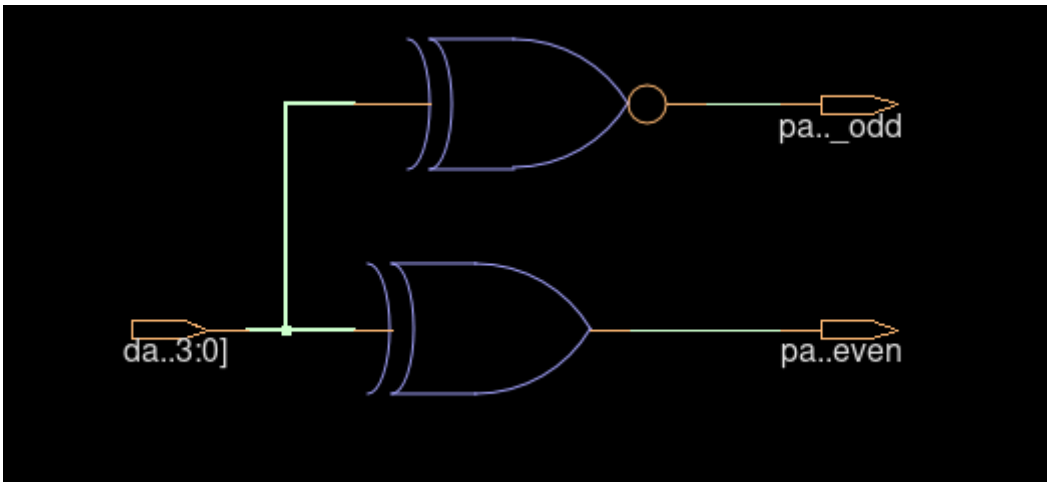
for 1100|parity_even=0 |even_odd= 1

for 1101|parity_even=1 |even_odd= 0

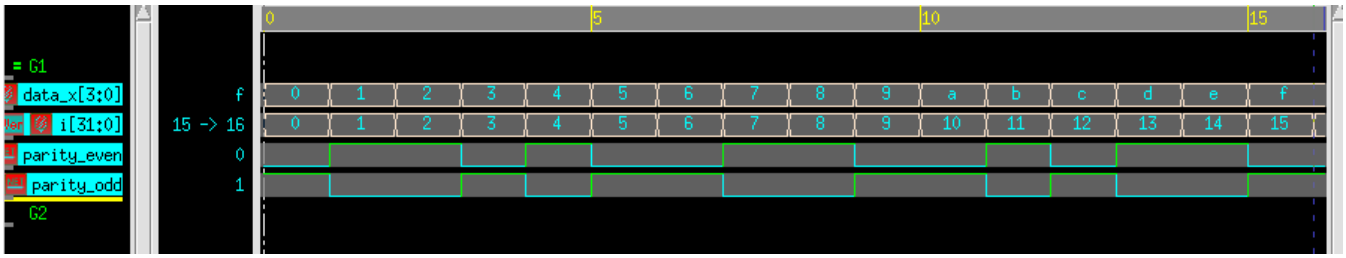
for 1110|parity_even=1 |even_odd= 0

for 1111|parity_even=0 |even_odd= 1

Schematic Design



waveform



Lab10 UDP

46.0 Design a 4x1_mux by using User Defined Primitives (UDP)

Design code

//udp combinational_circuit

```
primitive mux4x1(y,s1,s0,a,b,c,d);
        output y;
        input s1,s0,a,b,c,d;


        table
              // s1 s0 a b c d  y
                 0  0  0 ? ? ? :0;
                 0  0  1 ? ? ? :1;
                 0  1  ? 0 ? ? :0;
                 0  1  ? 1 ? ? :1;
                 1  0  ? ? 0 ? :0;
                 1  0  ? ? 1 ? :1;
                 1  1  ? ? ? 0 :0;
                 1  1  ? ? ? 1 :1;
                 ?  x  ? ? ? ? :x;
                 x  ?  ? ? ? ? :x;
           endtable
   endprimitive
```

Testbench

```
// testbench of using udp concept in the designing of 4x1mux
module TB();
reg s1,s0,a,b,c,d;
wire y;
mux4x1 dut(y,s1,s0,a,b,c,d);


initial begin
        $monitor("we get y=%d,s1=%d,s0=%d,a=%d,b=%d,c=%d,d=%d",y,s1,s0,a,b,c,d);
```

```
        s1=0; s0=0; s0=0;a = 1; b = 0; c = 1; d = 0;
        #5 s1=0;s0=1;
        #5 s1=1;s0=0;
        #5 s1=1;s0=1;
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

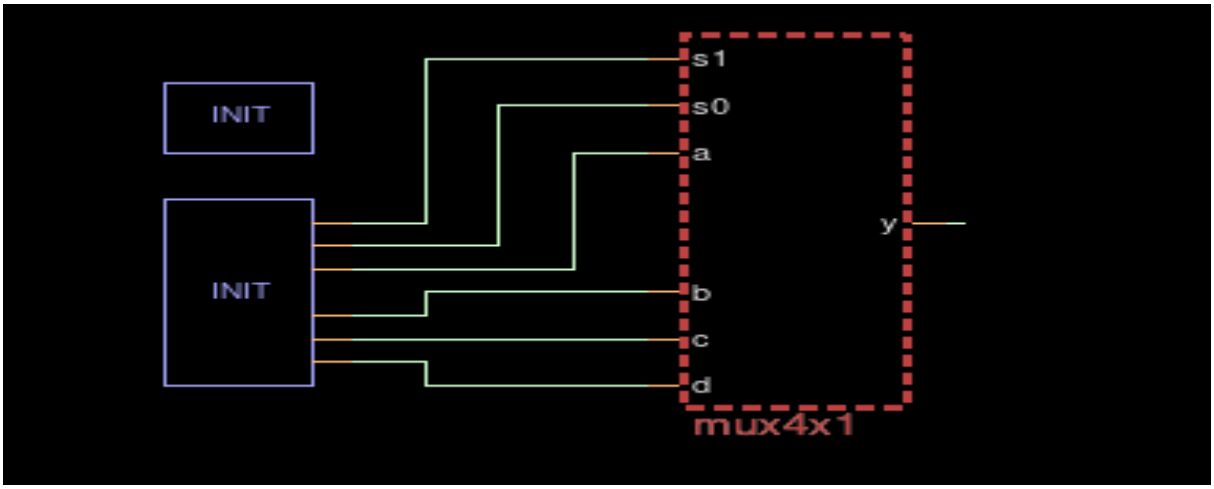
```
vcs udp_mux4x1.v udp_mux4x1_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

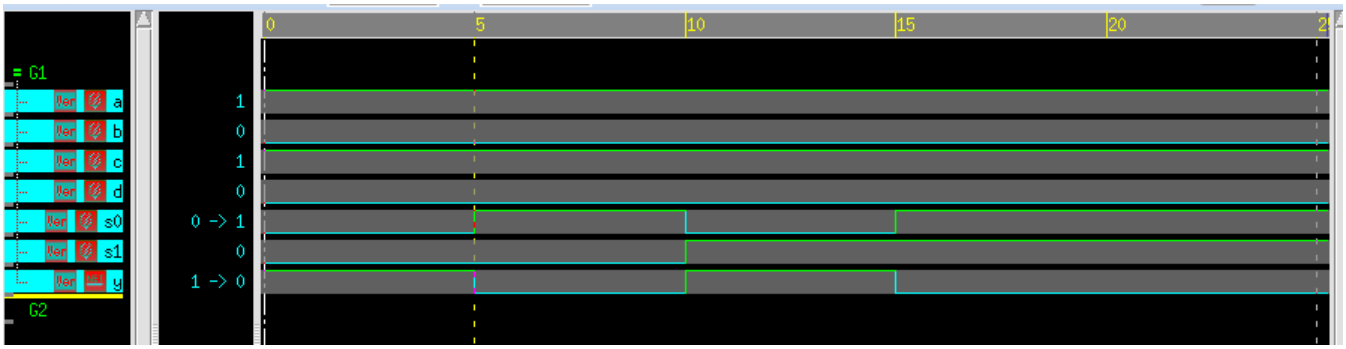
Output

```
we get y=1,s1=0,s0=0,a=1,b=0,c=1,d=0
we get y=0,s1=0,s0=1,a=1,b=0,c=1,d=0
we get y=1,s1=1,s0=0,a=1,b=0,c=1,d=0
we get y=0,s1=1,s0=1,a=1,b=0,c=1,d=0
```

Schematic Design



Waveform



47.0 Design an edge sensitive D-FF by using UDP

Design code

```
//Design a D-FF by using UDP
primitive D_FF_udp_rst (q,d,clk,rst);
    output q;
    reg q;
    input d, clk,rst;
    initial q = 0;
    table
    // d clk  rst: q : q+
      ?  ?   1  : ? : 0;   // async reset
      ? (10) 0  : ? : -;
      1 (10) 0  : ? : 1;
```

```
        0 (10) 0  : ? : 0;
        ? (1x) 0  : ? : -;
        ? (0?) 0  : ? : -;
        ? (x1) 0  : ? : -;
       (??) ?  0  : ? : -;

  endtable
endprimitive
```

Testbench

```
//Testbench of D_ff by using the UDP
module TB();
reg d,clk,rst;
wire q;

D_FF_udp_rst dut(q,d,clk,rst);

initial begin
      clk=0;
end
always #5 clk=~clk;

initial begin
      $monitor("Time=%0t for rst=%b ,d=%b, q=%b",$time,rst,d,q);

      rst=1;d=0;
      #10 rst=0;
      #10 d=1;
```

```
        #10 d=0;
        #10 d=1;
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1 ,d=0, q=0

Time=10 for rst=0 ,d=0, q=0

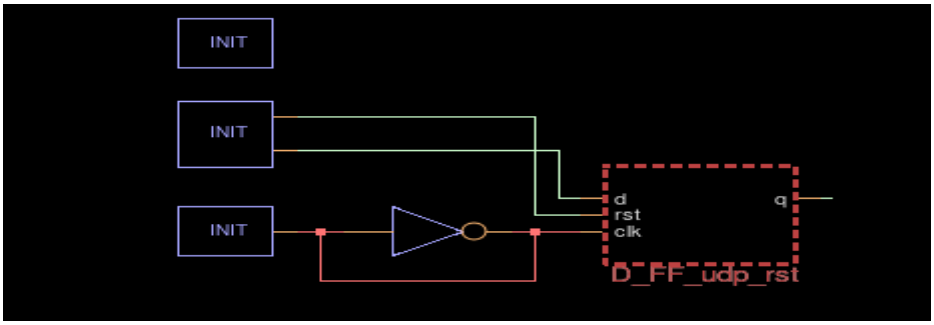
Time=20 for rst=0 ,d=1, q=1

Time=30 for rst=0 ,d=0, q=0

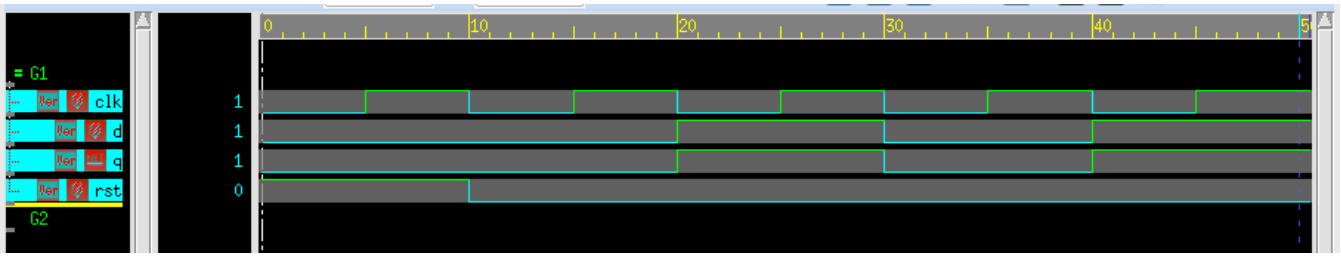
Time=40 for rst=0 ,d=1, q=1

$finish at simulation time 50

Schematic Design



waveform



48.0 Design a level sensitive D-latch by using UDP

Design code

```
// level sensitive D-latch using UDP
primitive D_latch_udp (q,d,clk,rst);
        output q;
        reg q;
        input d, clk,rst;
        initial q = 0;
        table
              // d clk  rst: q : q+
                 ?  ?   1  : ? : 0;
                 1  1   0  : ? : 1;
                 0  1   0  : ? : 0;
                 ?  0   0  : ? : -;
          endtable
endprimitive
```

Testbench

```
// testbench of D-latch
module TB();
reg d,clk,rst;
wire q;
```

```
D_latch_udp dut(q,d,clk,rst);

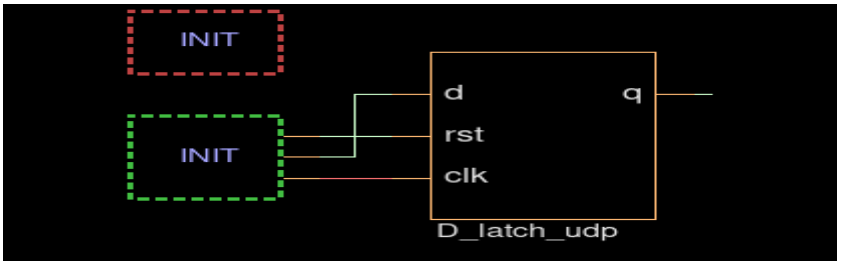

initial begin
    $monitor("Time=%0t for rst=%b clk=%0b ,d=%b, q=%b",$time,rst,clk,d,q);
    rst=1;d=0;clk=0;
    #10 rst=0;clk=1;
    #10 d=1;
    #10 d=0;
    #10 d=1;
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

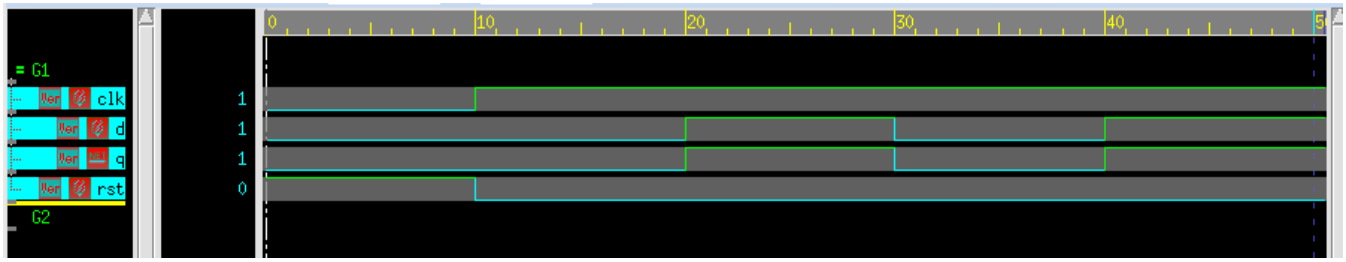
Output

```
Time=0 for rst=1 clk=0 ,d=0, q=0
Time=10 for rst=0 clk=1 ,d=0, q=0
Time=20 for rst=0 clk=1 ,d=1, q=1
Time=30 for rst=0 clk=1 ,d=0, q=0
Time=40 for rst=0 clk=1 ,d=1, q=1
$finish at simulation time                50
```

Schematic Design



waveform



Lab11

49.0 Mealy FSM to detect 1010 non_overlapping sequence detector-

Design code-1

```
//mealy FSM to detect nonoverlapping 1010
module mealy_1010_non (clk ,rst,in,out);
input clk,rst,in;
output reg out;

parameter S0=2'b00,S1=2'b01,S2=2'b10,S3=2'b11;
reg [1:0]CST,NST;

always @(posedge clk or posedge rst)begin
        if (rst)
                CST<=S0;
        else
```

```
            CST<=NST;
end

always @(CST or in)begin
        NST=CST;
        out=1'b0;

        case(CST)
            S0:begin
                if (in)
                    NST=S1;
                else
                    NST=S0;
            end

            S1:begin
              if (in)
                    NST=S1;
              else
                    NST=S2;
          end

            S2:begin
              if (in)
                    NST=S3;
              else
                    NST=S0;
```

```
            end

                S3:begin
                   if (in)
                         NST=S1;
                        else begin

                         NST=S0;
                               out=1'b1;
                        end
            end
                default:NST=S0;
        endcase
end
endmodule
```

Testbench

```
// testbench of mealy 1010 nonoverlapping detection
module TB();
reg clk,rst,in;
wire out;

mealy_1010_non dut(.clk(clk),.rst(rst),.in(in),.out(out));

initial begin
        clk=0;
end
always #5 clk=~clk;
```

```
initial begin
        $monitor("Time=%0d,rst=%0b,in=%0b,out=%0b",$time,rst,in,out);
        rst=1;in=0;
        #10;
        rst=0;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=1; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=0; #10;
        $finish();
end

initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

vcs mealy_1010non_ov.v mealy_1010non_ov_tb.v -kdb -lca -debug_access+all -full64

Simulation Result

Output

Time=0,rst=1,in=0,out=0

Time=10,rst=0,in=1,out=0

Time=20,rst=0,in=0,out=0

Time=30,rst=0,in=1,out=0

Time=40,rst=0,in=0,out=1

Time=45,rst=0,in=0,out=0

Time=50,rst=0,in=1,out=0

Time=60,rst=0,in=0,out=0

Time=70,rst=0,in=1,out=0

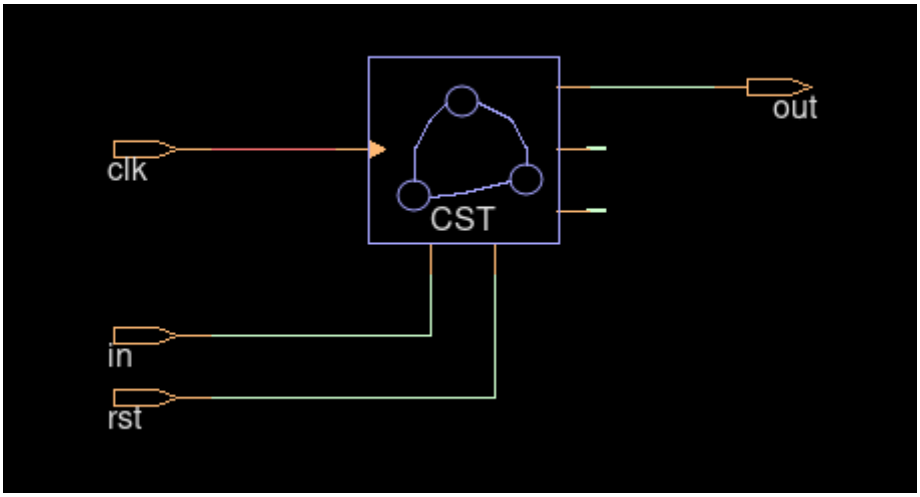
Time=100,rst=0,in=0,out=0

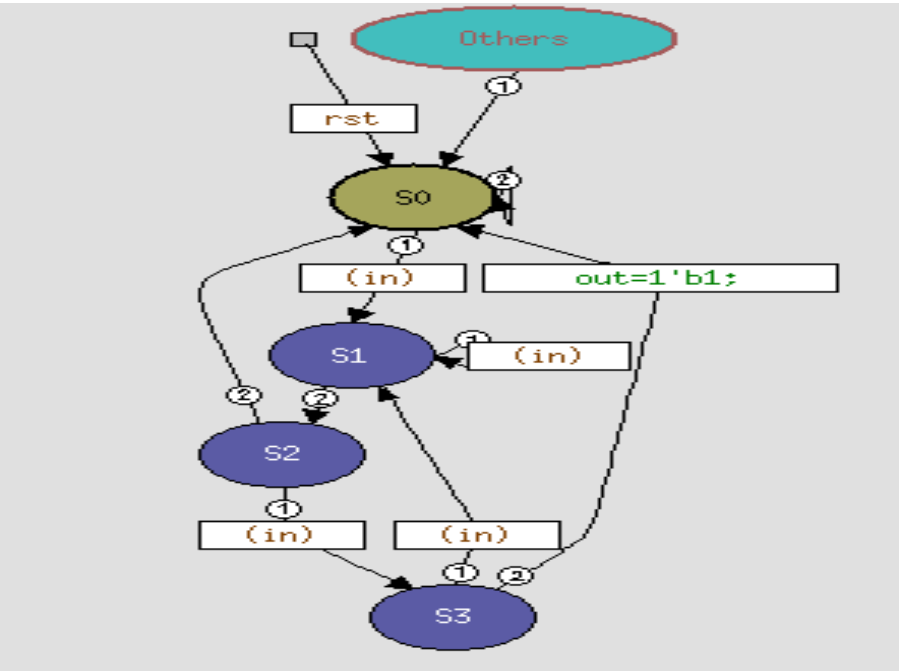
Time=110,rst=0,in=1,out=0

Time=120,rst=0,in=0,out=1

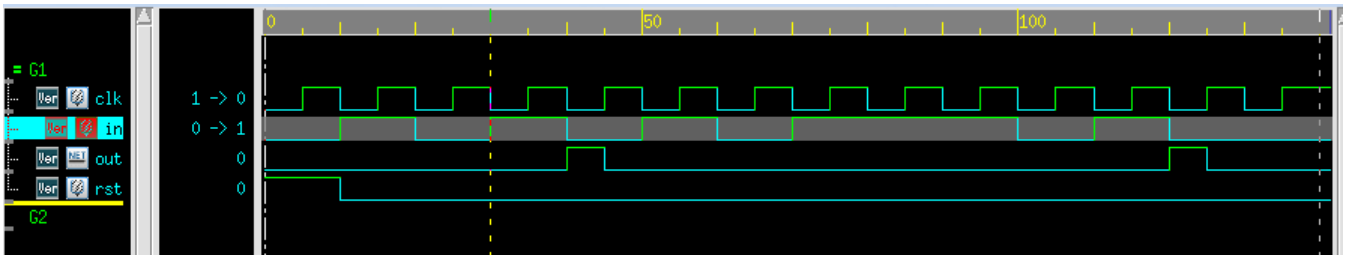
Time=125,rst=0,in=0,out=0

Schematic Design





Waveform



Design code-2

```
//mealy fsm to detect nonoverlapping 1010

module mealy_1010_non (clk ,rst,in,out);

input clk,rst,in;

output reg out;


parameter S0=2'b00,S1=2'b01,S2=2'b10,S3=2'b11;

reg [1:0]CST,NST;


always @(posedge clk or negedge rst)begin
```

```
        if (!rst)
                CST<=S0;
        else
                CST<=NST;
end

always @(*)begin
        NST=CST;
        out=1'b0;
        case(CST)
                S0:NST = in ? S1:S0;
                S1:NST = in ? S1:S2;
                S2:NST = in ? S3:S0;
                S3:begin
                        NST = in ? S1:S0;
                        out= in ? 1'b0: 1'b1;
                end
                default:begin
                        NST=S0;
                        out= 1'b0;
                end
        endcase
end
endmodule
```

Testbench

// testbench of mealy 1010 non_overlapping detection

```
module TB();
reg clk,rst,in;
wire out;

mealy_1010_non dut(.clk(clk),.rst(rst),.in(in),.out(out));

initial begin
	clk=0;
end
always #5 clk=~clk;

initial begin
	$monitor("Time=%0d,rst=%0b,in=%0b,out=%0b",$time,rst,in,out);

	rst=0;in=0;
	#10;
	rst=1;
	in=1; #10;
	in=0; #10;
	in=1; #10;
	in=0; #10;
	in=1; #10;
	in=0; #10;
	in=1; #10;
	in=1; #10;
	in=1; #10;
	in=0; #10;
```

```
        in=1; #10;
        in=0; #10;
        in=0; #10;
        $finish();
end

initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,TB);
end
endmodule
```

50.0 Mealy FSM to detect 1010 Overlapping sequence detector-

Design code

```
//mealy fsm to detect overlapping 1010
module mealy_1010_ov (clk ,rst,in,out);
input clk,rst,in;
output reg out;

parameter S0=2'b00,S1=2'b01,S2=2'b10,S3=2'b11;
reg [1:0]CST,NST;

always @(posedge clk or negedge rst)begin
        if (!rst)
                CST<=S0;
        else
                CST<=NST;
```

```
end

always @(*)begin
        NST=CST;
        out=1'b0;

        case(CST)
                S0:NST = in ? S1:S0;
                S1:NST = in ? S1:S2;
                S2:NST = in ? S3:S0;
                S3:begin
                        NST = in ? S1:S2;
                        out= in ? 1'b0: 1'b1;
                end

                default:begin
                        NST=S0;
                        out= 1'b0;
                end
        endcase
end
endmodule
```

Testbench

```
// testbench of mealy 1010 overlapping detection
module TB();
reg clk,rst,in;
wire out;
```

```
mealy_1010_ov dut(.clk(clk),.rst(rst),.in(in),.out(out));

initial begin
        clk=0;
end
always #5 clk=~clk;

initial begin
        $monitor("Time=%0d,rst=%0b,in=%0b,out=%0b",$time,rst,in,out);

        rst=0;in=0;
        #10;
        rst=1;

        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=1; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
```

```
        in=0; #10;
        $finish();
end

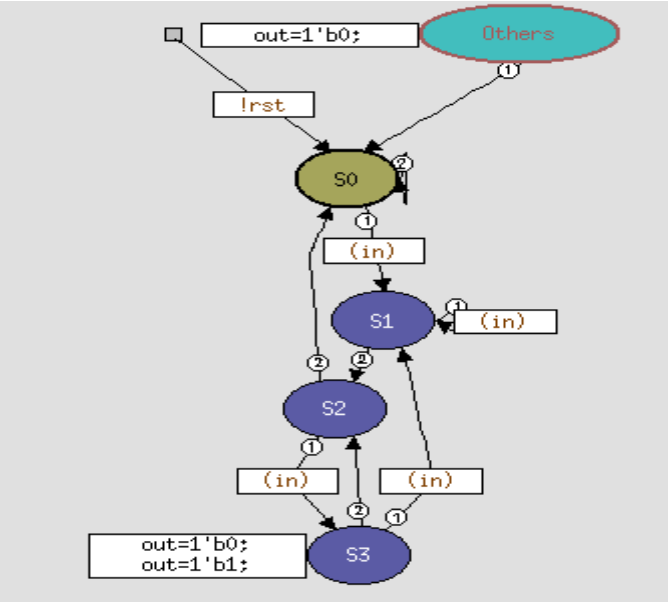
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,TB);
end
endmodule
```

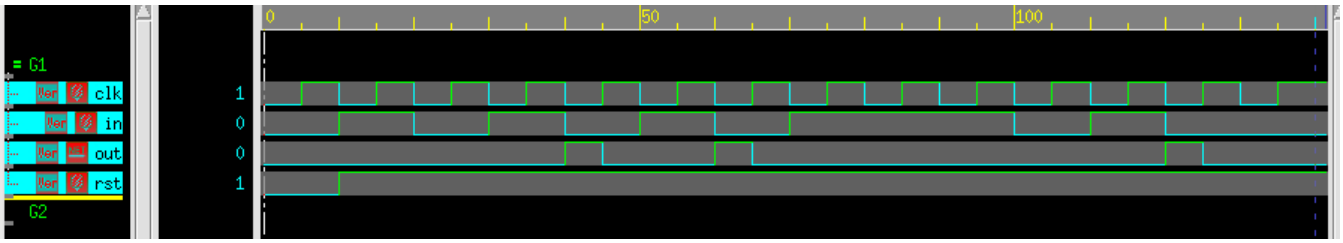
Simulation Result

Output

```
Time=0,rst=0,in=0,out=0
Time=10,rst=1,in=1,out=0
Time=20,rst=1,in=0,out=0
Time=30,rst=1,in=1,out=0
Time=40,rst=1,in=0,out=1
Time=45,rst=1,in=0,out=0
Time=50,rst=1,in=1,out=0
Time=60,rst=1,in=0,out=1
Time=65,rst=1,in=0,out=0
Time=70,rst=1,in=1,out=0
Time=100,rst=1,in=0,out=0
Time=110,rst=1,in=1,out=0
Time=120,rst=1,in=0,out=1
Time=125,rst=1,in=0,out=0
```

waveform





51.0 Moore FSM to detect 1010 non_overlapping sequence detector-

Design code-1

```
//moore fsm to detect nonoverlapping 1010
module moore_1010_non (clk ,rst,in,out);
input clk,rst,in;
output reg out;

parameter S0=0,S1=1,S2=2,S3=3,S4=4;
reg [2:0]CST,NST;

always @(posedge clk )begin
        if (rst)
                CST<=S0;
```

```
        else
                CST<=NST;
end

always @(*)begin
        case(CST)
                S0:NST = in ? S1:S0;
                S1:NST = in ? S1:S2;
                S2:NST = in ? S3:S0;
                S3:NST = in ? S1:S4;
                S4:NST = in ? S1:S0;
                default:NST=S0;
        endcase
end

always @(*)begin
        case(CST)
                S4:out=1;
                default:out=0;
        endcase
end
endmodule
```

Testbench

```
// testbench of moore 1010 nonoverlapping detection

module TB();
reg clk,rst,in;
```

```
wire out;

moore_1010_non dut(.clk(clk),.rst(rst),.in(in),.out(out));

initial begin
        clk=0;
end
always #5 clk=~clk;

initial begin
        $monitor("Time=%0d,rst=%0b,in=%0b,out=%0b",$time,rst,in,out);
        rst=1;in=0;
        #10;
        rst=0;

        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
```

```
        in=0; #10;
        $finish();
end

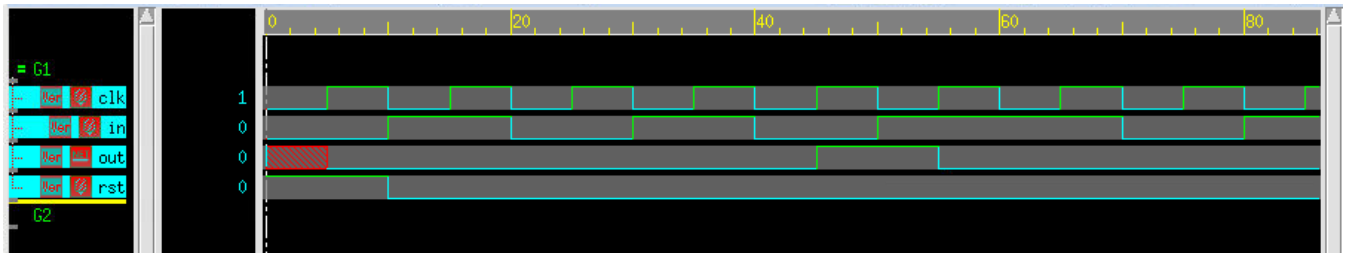
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

```
Time=0,rst=1,in=0,out=x
Time=5,rst=1,in=0,out=0
Time=10,rst=0,in=1,out=0
Time=20,rst=0,in=0,out=0
Time=30,rst=0,in=1,out=0
Time=40,rst=0,in=0,out=0
Time=45,rst=0,in=0,out=1
Time=50,rst=0,in=1,out=1
Time=55,rst=0,in=1,out=0
Time=70,rst=0,in=0,out=0
Time=80,rst=0,in=1,out=0
Time=100,rst=0,in=0,out=0
Time=110,rst=0,in=1,out=0
Time=120,rst=0,in=0,out=0
Time=125,rst=0,in=0,out=1
Time=135,rst=0,in=0,out=0
```

waveform



52.0 Moore FSM to detect 1010 Overlapping sequence detector-

Design code

```
//moore fsm to detect overlapping 1010
module moore_1010_ov (clk ,rst,in,out);
input clk,rst,in;
output reg out;

parameter S0=0,S1=1,S2=2,S3=3,S4=4;
reg [2:0]CST,NST;

always @(posedge clk )begin
        if (rst)
                CST<=S0;
        else
                CST<=NST;
end

always @(*)begin
        case(CST)
                S0:NST = in ? S1:S0;
```

```
            S1:NST = in ? S1:S2;
            S2:NST = in ? S3:S0;
            S3:NST = in ? S1:S4;
            S4:NST = in ? S3:S0;
            default:NST=S0;
        endcase
end

always @(*)begin
        case(CST)
            S4:out=1;
            default:out=0;
        endcase
end
```

Testbench

```
// testbench of moore 1010 overlapping detection
module TB();
reg clk,rst,in;
wire out;

moore_1010_ov dut(.clk(clk),.rst(rst),.in(in),.out(out));

initial begin
        clk=0;
end
always #5 clk=~clk;
```

```
initial begin
        $monitor("Time=%0d,rst=%0b,in=%0b,out=%0b",$time,rst,in,out);
        rst=1;in=0;
        #10;
        rst=0;

        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=0; #10;
        in=1; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=0; #10;
        $finish();
end

initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0,rst=1,in=0,out=x

Time=5,rst=1,in=0,out=0

Time=10,rst=0,in=1,out=0

Time=20,rst=0,in=0,out=0

Time=30,rst=0,in=1,out=0

Time=40,rst=0,in=0,out=0

Time=45,rst=0,in=0,out=1

Time=50,rst=0,in=1,out=1

Time=55,rst=0,in=1,out=0

Time=60,rst=0,in=0,out=0

Time=65,rst=0,in=0,out=1

Time=75,rst=0,in=0,out=0

Time=80,rst=0,in=1,out=0

Time=100,rst=0,in=0,out=0

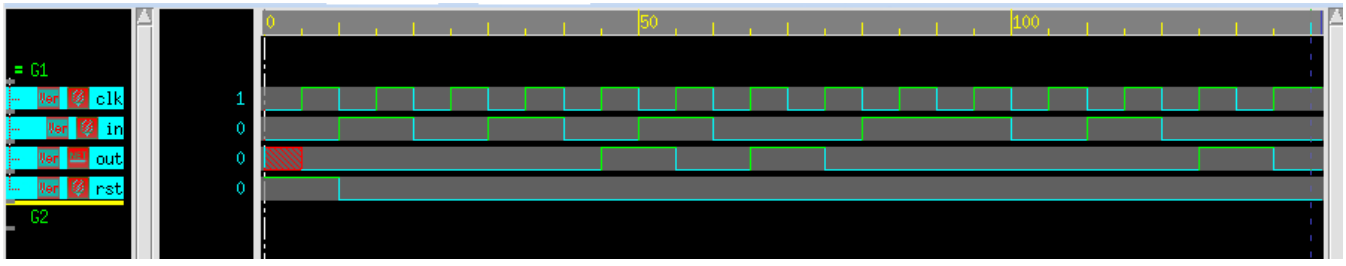
Time=110,rst=0,in=1,out=0

Time=120,rst=0,in=0,out=0

Time=125,rst=0,in=0,out=1

Time=135,rst=0,in=0,out=0

waveform



Design code-2

```
module moore_1010_ov (clk ,rst,in,out);
input clk,rst,in;
output reg out;

parameter S0=0,S1=1,S2=2,S3=3,S4=4;
reg [2:0]CST,NST;

always @(posedge clk )begin
        if (rst)
                CST<=S0;
        else
                CST<=NST;
end

always @(*)begin
        case(CST)

                S0:NST = in ? S1:S0;
                S1:NST = in ? S1:S2;
                S2:NST = in ? S3:S0;
                S3:NST = in ? S1:S4;
                S4:NST = in ? S3:S0;
                default:NST=S0;
        endcase
end
```

```
always @(*)begin
        case(CST)
                S4:out=1;
                default:out=0;
        endcase
end
endmodule
```

Testbench-2

```
module TB();
reg clk,rst,in;
wire out;
integer i;

reg [15:0] data = 16'b1010_1010_1001_1010;

moore_1010_ov dut(.clk(clk),.rst(rst),.in(in),.out(out));

initial begin
        clk=0;
end
always #5 clk=~clk;

initial begin
        $monitor("Time=%0d,rst=%0b,in=%0b,out=%0b",$time,rst,in,out);
        rst=1;in=0;
        #10;
        rst=0;
```

```
        i=0;
        for (i=0;i<16;i=i+1)begin
                in=data[i];
                #10;
        end
        #50 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

```
Time=0,rst=1,in=0,out=x
Time=5,rst=1,in=0,out=0
Time=10,rst=0,in=0,out=0
Time=20,rst=0,in=1,out=0
Time=30,rst=0,in=0,out=0
Time=40,rst=0,in=1,out=0
Time=60,rst=0,in=0,out=0
Time=80,rst=0,in=1,out=0
Time=90,rst=0,in=0,out=0
Time=100,rst=0,in=1,out=0
Time=110,rst=0,in=0,out=0
Time=115,rst=0,in=0,out=1
```

Time=120,rst=0,in=1,out=1

Time=125,rst=0,in=1,out=0

Time=130,rst=0,in=0,out=0

Time=135,rst=0,in=0,out=1

Time=140,rst=0,in=1,out=1

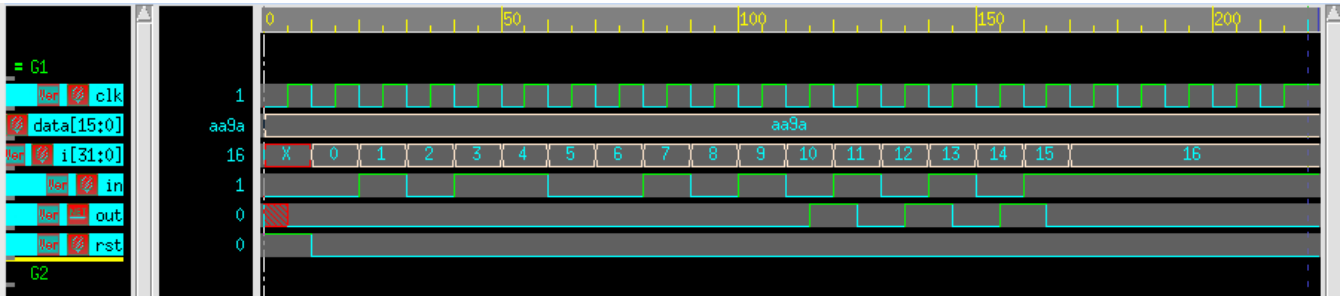
Time=145,rst=0,in=1,out=0

Time=150,rst=0,in=0,out=0

Time=155,rst=0,in=0,out=1

Time=160,rst=0,in=1,out=1

waveform



SWITCH LEVEL MODELLING

53.0 Design a CMOS Inverter by switch level modelling

Design code

//Design a cmos inverter by using the switch level modelling by **using mosfet**

module inverter(out,in);

input in;

output out;

supply0 a;

supply1 b;

nmos n1(out,a,in);

```
pmos p1(out,b,in);

endmodule
```

Testbench

```
// Testbench of cmos_inverter
module TB();
reg in;
wire out;

inverter dut(.in(in),.out(out));

initial
begin
    $monitor("Time=%0t,for in=%0b,out=%0b",$time,in,out);
    in=0;
#10 in=1;
#10 in=0;

#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

vcs cmos_inverter.v cmos_inverter_tb.v -kdb -lca -debug_access+all -full64

Simulation Result

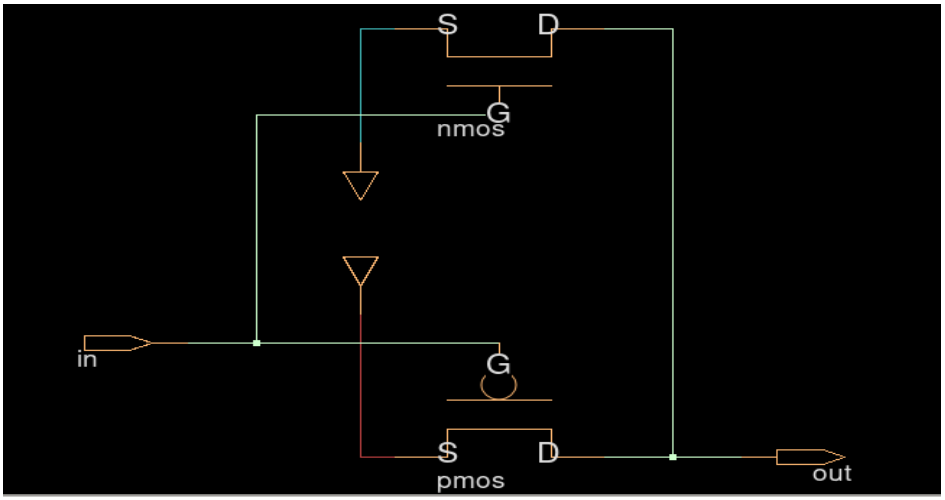
Output

Time=0,for in=0,out=1

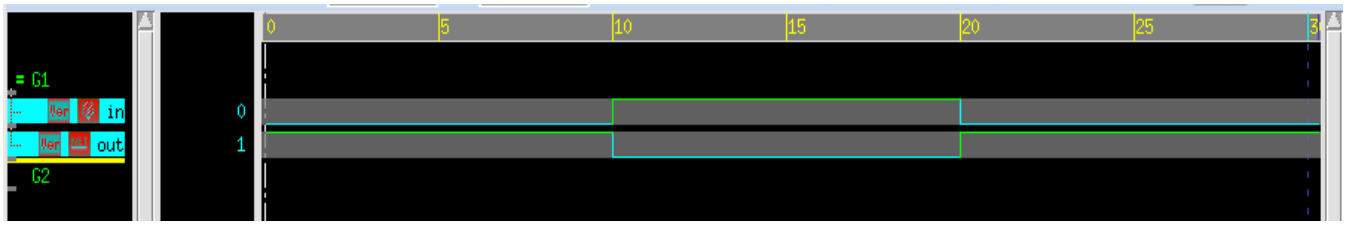
Time=10,for in=1,out=0

Time=20,for in=0,out=1

Schematic Design



waveform



54.0 Design a 2-input NAND gate by switch level modelling

Design code

//Design a 2-input nand gate by using the switch level modelling(mosfet)

module nand_gate(out,in1,in2);

input in1,in2;

```
output out;
supply0 a;
supply1 b;

wire x;

pmos p1(out,b,in1);
pmos p2(out,b,in2);
nmos n1(out,x,in1);
nmos n2(x,a,in2);

endmodule
```

Testbench

```
// Testbench of 2-input nand gate
module TB();
reg in1,in2;
wire out;

nand_gate dut(.out(out),.in1(in1),.in2(in2));

initial
begin
    $monitor("Time=%0t,for in1=%0b,in2=%0b,out=%0b",$time,in1,in2,out);

    in1=0;in2=0;
#10 in1=0;in2=1;
#10 in1=1;in2=0;
```

```
#10 in1=1;in2=1;

#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

```
vcs nand_gate.v nand_gate_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

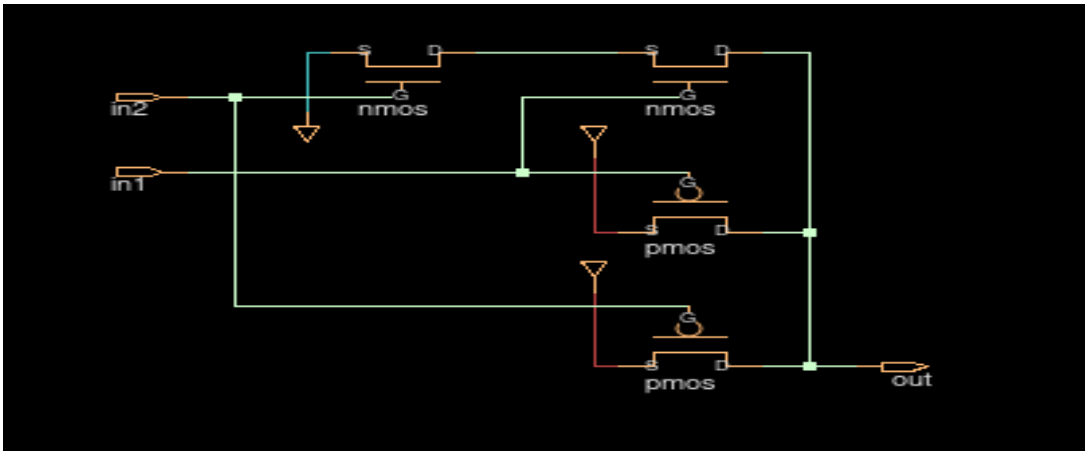
Time=0,for in1=0,in2=0,out=1

Time=10,for in1=0,in2=1,out=1

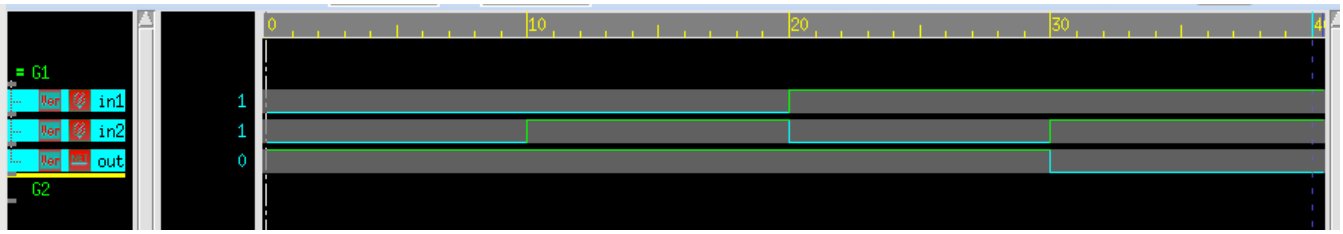
Time=20,for in1=1,in2=0,out=1

Time=30,for in1=1,in2=1,out=0

Schematic Design



waveform



55.0 Design a 2-input NOR gate by switch level modelling

Design code

```
//Design a 2-input nor gate by using the switch level modelling
module nor_gate(out,in1,in2);
input in1,in2;
output out;
supply0 a;
supply1 b;

wire x;

pmos p1(x,b,in1);
pmos p2(out,x,in2);
nmos n1(out,a,in1);
nmos n2(out,a,in2);

endmodule
```

Testbench

```
// Testbench of 2-input nor_gate
module TB();
reg in1,in2;
```

```
wire out;

nor_gate dut(.out(out),.in1(in1),.in2(in2));

initial
begin
    $monitor("Time=%0t,for in1=%0b,in2=%0b,out=%0b",$time,in1,in2,out);
    in1=0;in2=0;
#10 in1=0;in2=1;
#10 in1=1;in2=0;
#10 in1=1;in2=1;

#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

```
vcs nor_gate.v nor_gate_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

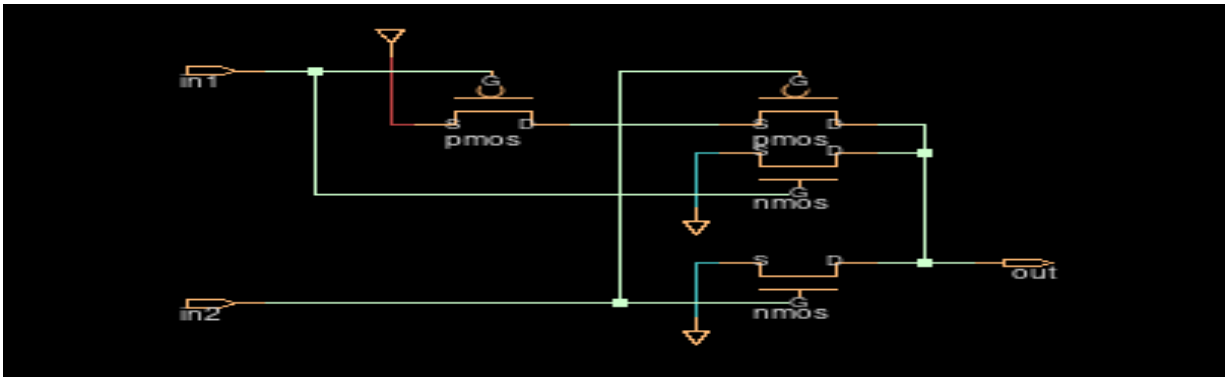
Output

```
Time=0,for in1=0,in2=0,out=1
Time=10,for in1=0,in2=1,out=0
```

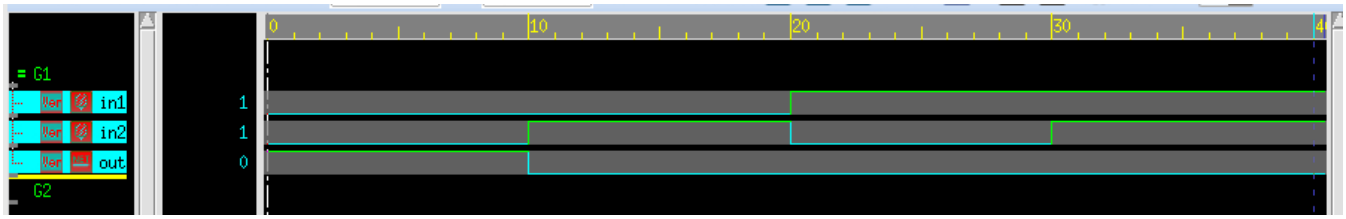
Time=20,for in1=1,in2=0,out=0

Time=30,for in1=1,in2=1,out=0

Schematic Design



waveform



56.0 Design a 2x1mux by switch level modelling by using transmission gate

Design code

//Design a 2x1mux_cmos by using the switch level modelling

module mux_cmos(out,in1,in2,s);

input in1,in2,s;

output out;

supply0 a;

supply1 b;

wire sbar;

not n1(sbar,s);

```
cmos c1(out,in1,sbar,s);
cmos c2(out,in2,s,sbar);
endmodule
```

Testbench

```
// Testbench of 2x1mux_cmos
module TB();
reg in1,in2,s;
wire out;
mux_cmos dut(.out(out),.in1(in1),.in2(in2),.s(s));

initial
begin
        $monitor("Time=%0t,for in1=%0b,in2=%0b,s=%0b,out=%0b",$time,in1,in2,s,out);
        in1=0;in2=0;s=0;
#10 in1=0;in2=1;s=0;
#10 in1=1;in2=0;s=0;
#10 in1=1;in2=1;s=0;
#10 in1=0;in2=0;s=1;
#10 in1=0;in2=1;s=1;
#10 in1=1;in2=0;s=1;
#10 in1=1;in2=1;s=1;

#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
```

```
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0,for in1=0,in2=0,s=0,out=0

Time=10,for in1=0,in2=1,s=0,out=0

Time=20,for in1=1,in2=0,s=0,out=1

Time=30,for in1=1,in2=1,s=0,out=1

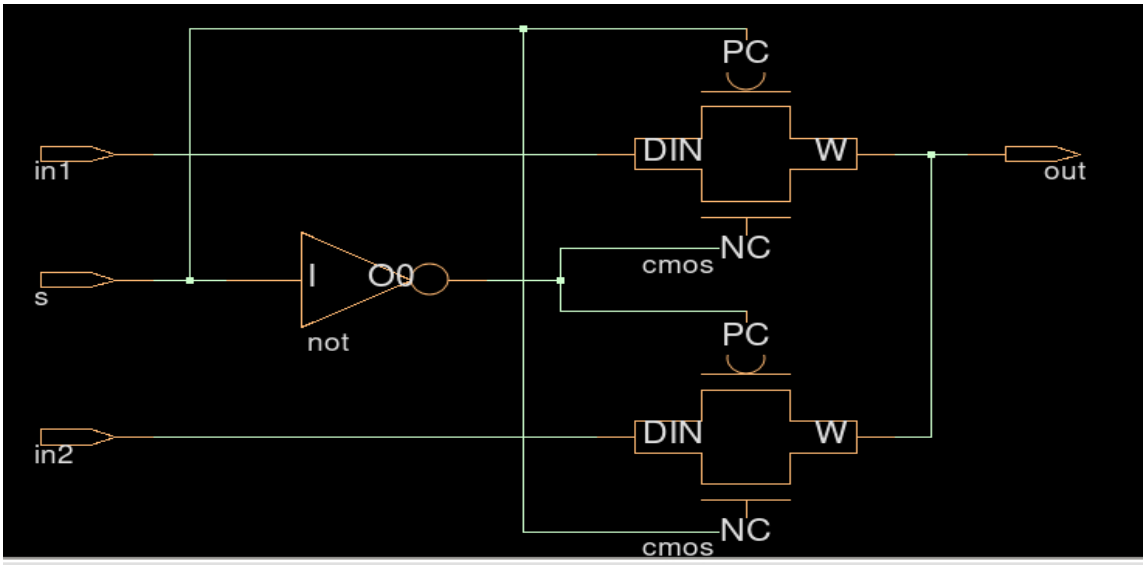
Time=40,for in1=0,in2=0,s=1,out=0

Time=50,for in1=0,in2=1,s=1,out=1

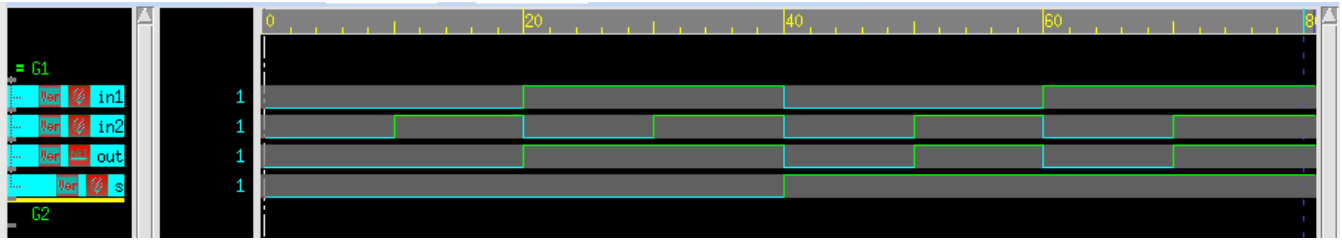
Time=60,for in1=1,in2=0,s=1,out=0

Time=70,for in1=1,in2=1,s=1,out=1

Schematic Design



waveform



Method-2

Design 2x1 mux by using CMOS transmission gates

Design code

```
module cmos_mux(out,in1,in2,s);
input in1,in2,s;
output out;
supply0 a;
supply1 b;

wire sbar;

pmos p1(sbar,b,s);
nmos n1(sbar,a,s);
cmos c1(out,in1,sbar,s);
cmos c2(out,in2,s,sbar);
endmodule
```

Testbench

```
module TB();
reg in1,in2,s;
wire out;
cmos_mux dut(.out(out),.in1(in1),.in2(in2),.s(s));
```

```
initial
begin
        $monitor("Time=%0t,for in1=%0b,in2=%0b,s=%0b,out=%0b",$time,in1,in2,s,out);
        in1=0;in2=0;s=0;
#10 in1=0;in2=1;s=0;
#10 in1=1;in2=0;s=0;
#10 in1=1;in2=1;s=0;
#10 in1=0;in2=0;s=1;
#10 in1=0;in2=1;s=1;
#10 in1=1;in2=0;s=1;
#10 in1=1;in2=1;s=1;

#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

```
Time=0,for in1=0,in2=0,s=0,out=0
Time=10,for in1=0,in2=1,s=0,out=0
Time=20,for in1=1,in2=0,s=0,out=1
Time=30,for in1=1,in2=1,s=0,out=1
```

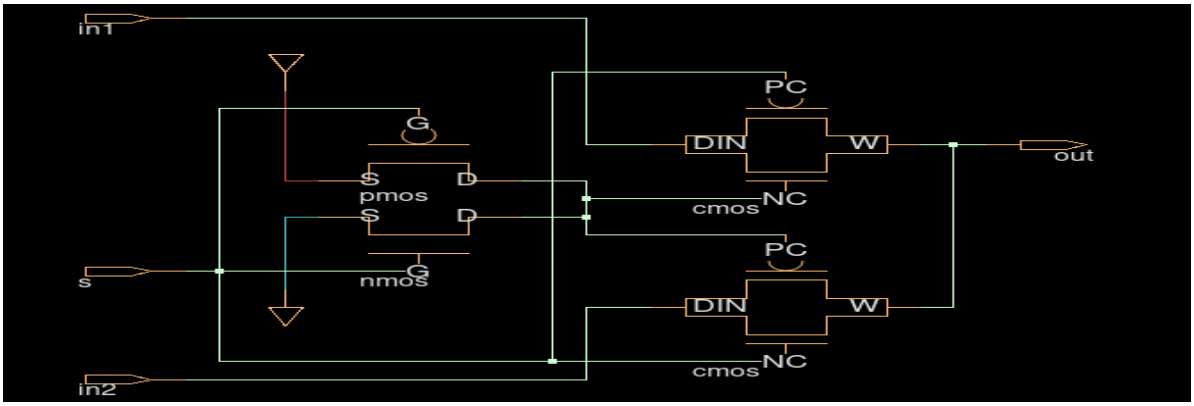
Time=40,for in1=0,in2=0,s=1,out=0

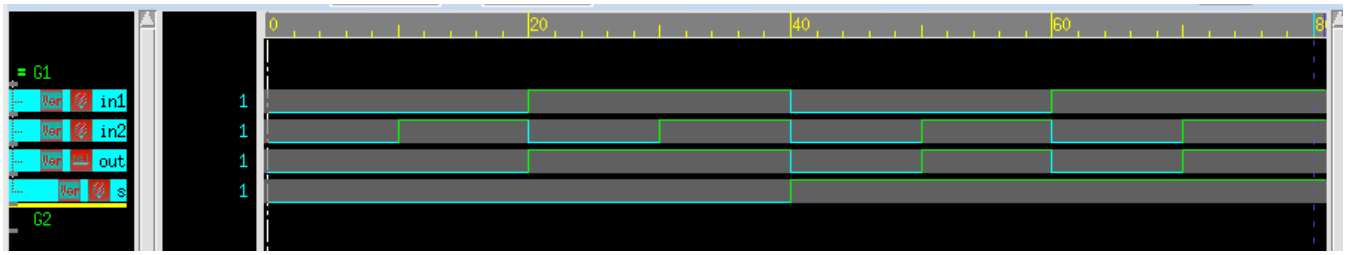
Time=50,for in1=0,in2=1,s=1,out=1

Time=60,for in1=1,in2=0,s=1,out=0

Time=70,for in1=1,in2=1,s=1,out=1

Schematic Design





57.0 Design a 2x1mux by switch level modelling by using Bidirectional switches

Design code

module cmos_mux(out,in1,in2,s);

input in1,in2,s;

output out;

supply0 a;

supply1 b;

wire sbar;

```
pmos p1(sbar,b,s);
nmos n1(sbar,a,s);
tranif1 t1(out,in1,sbar);
tranif1 t2(out,in2,s);

endmodule
```

Testbench

```
module TB();
reg in1,in2,s;
wire out;
cmos_mux dut(.out(out),.in1(in1),.in2(in2),.s(s));

initial
begin
    $monitor("Time=%0t,for in1=%0b,in2=%0b,s=%0b,out=%0b",$time,in1,in2,s,out);
    in1=0;in2=0;s=0;
#10 in1=0;in2=1;s=0;
#10 in1=1;in2=0;s=0;
#10 in1=1;in2=1;s=0;
#10 in1=0;in2=0;s=1;
#10 in1=0;in2=1;s=1;
#10 in1=1;in2=0;s=1;
#10 in1=1;in2=1;s=1;

#10 $finish();
end
```

```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0,for in1=0,in2=0,s=0,out=0

Time=10,for in1=0,in2=1,s=0,out=0

Time=20,for in1=1,in2=0,s=0,out=1

Time=30,for in1=1,in2=1,s=0,out=1

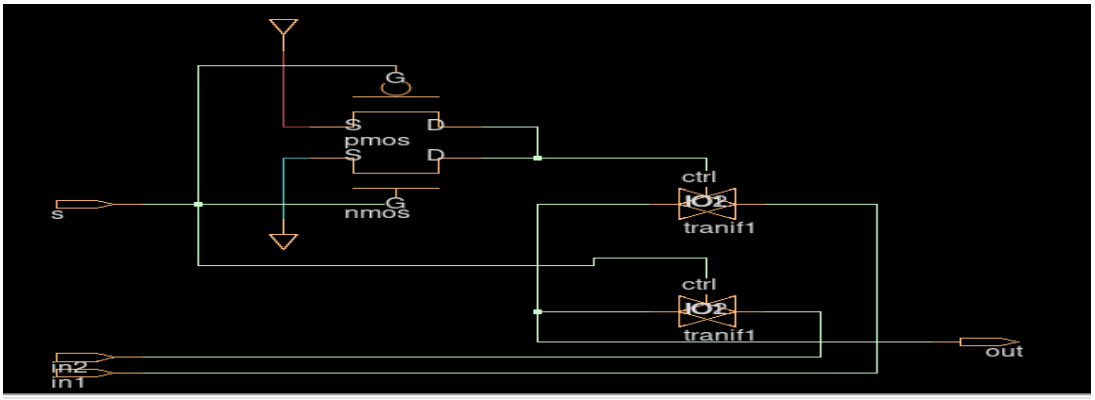
Time=40,for in1=0,in2=0,s=1,out=0

Time=50,for in1=0,in2=1,s=1,out=1

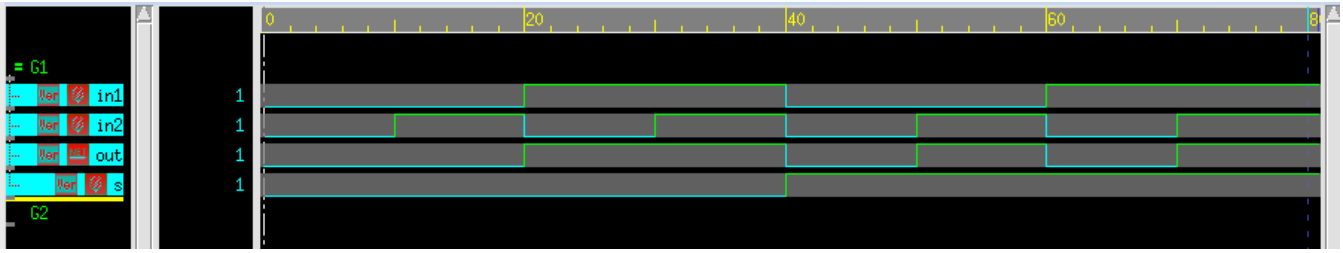
Time=60,for in1=1,in2=0,s=1,out=0

Time=70,for in1=1,in2=1,s=1,out=1

Schematic Design



waveform



58.0 Design a CMOS SWITCH by switch level modelling

Design code

```
//Design a cmos_switch by using the switch level modelling
module cmos_switch(out,in,s);
input in,s;
output out;
supply0 a;
supply1 b;

wire sbar;

not n1(sbar,s);
cmos c1(out,in,s,sbar);

endmodule
```

Testbench

```
// Testbench of cmos_switch
module TB();
reg in,s;
wire out;

cmos_switch dut(.out(out),.in(in),.s(s));
```

```
initial
begin
	$monitor("Time=%0t,for in=%0b,s=%0b,out=%0b",$time,in,s,out);
	in=0;s=0;
#10 in=1;s=0;
#10 in=0;s=1;
#10 in=1;s=1;

#10 $finish();
end

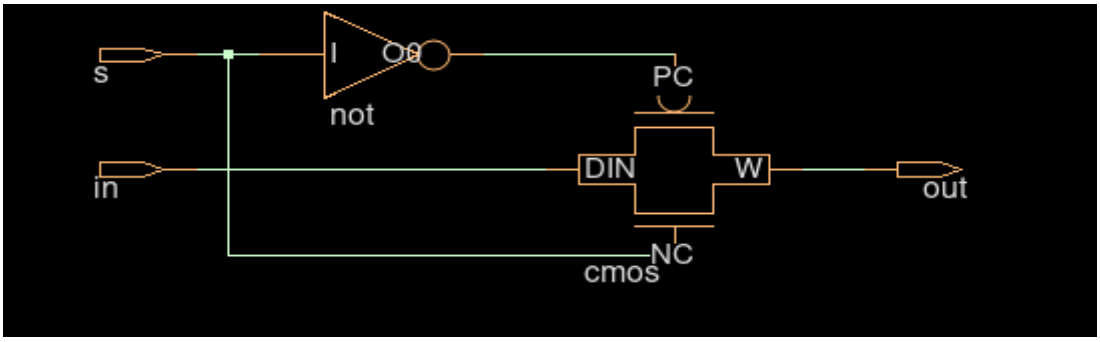
initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

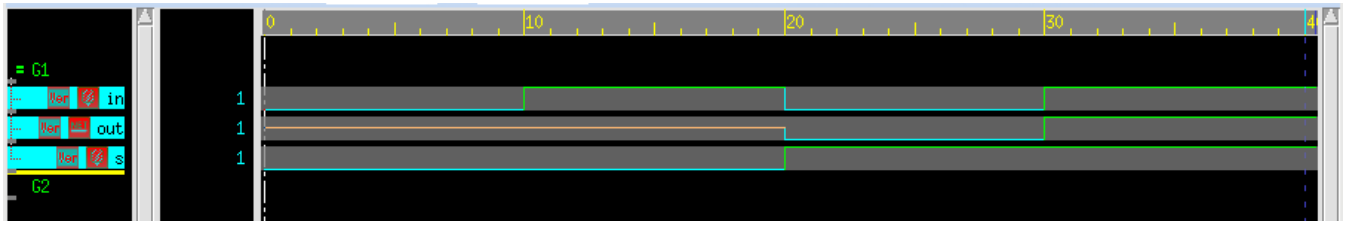
Output

```
Time=0,for in=0,s=0,out=z
Time=10,for in=1,s=0,out=z
Time=20,for in=0,s=1,out=0
Time=30,for in=1,s=1,out=1
```

Schematic Design



waveform



59.0 Design a 2-input NAND gate by adding delay concepts- for the simulation purpose only.

Design code

```
module nand_gate(out,in1,in2);
input in1,in2;
output out;
supply0 a;
supply1 b;

wire x;

pmos p1(out,b,in1);
pmos #(5)p2(out,b,in2);
nmos #(5,10)n1(out,x,in1);
nmos #(5,10,15)n2(x,a,in2);

endmodule
```

Testbench

```
module TB();
reg in1,in2;
wire out;

nand_gate dut(.out(out),.in1(in1),.in2(in2));

initial
begin
    $monitor("Time=%0t,for in1=%0b,in2=%0b,out=%0b",$time,in1,in2,out);
    in1=0;in2=0;
#10 in1=0;in2=1;
#10 in1=1;in2=0;
#10 in1=1;in2=1;

#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0,for in1=0,in2=0,out=x

Time=5,for in1=0,in2=0,out=1

Time=10,for in1=0,in2=1,out=1

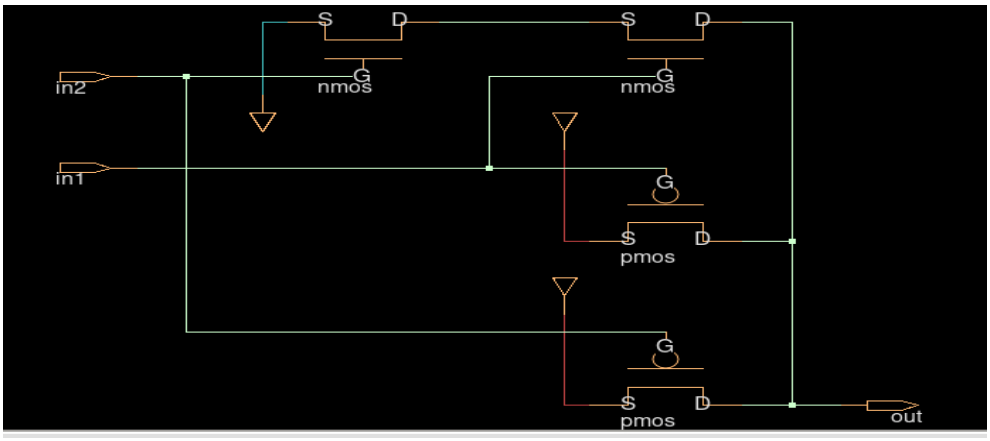
Time=20,for in1=1,in2=0,out=z

Time=25,for in1=1,in2=0,out=1

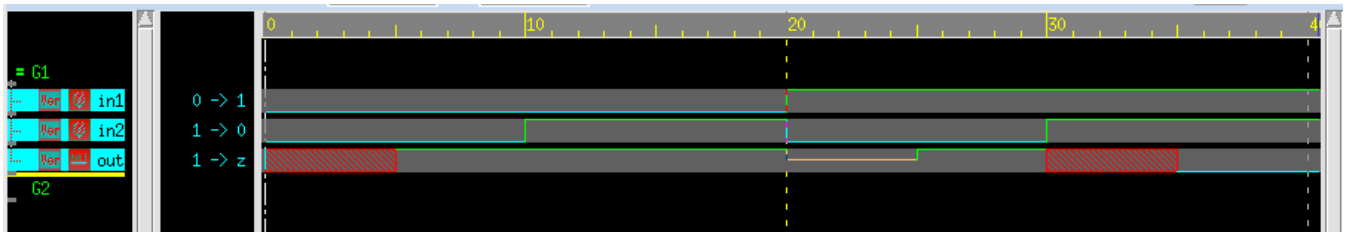
Time=30,for in1=1,in2=1,out=x

Time=35,for in1=1,in2=1,out=0

Schematic Design



waveform



60.0 Design a 2x1mux by switch level modelling using mosfet

Design code

module cmos_mux(out,in1,in2,s);

input in1,in2,s;

output out;

```
supply0 a;
supply1 b;

wire x1,x2,x3,sbar,ybar;
not no(sbar,s);

pmos p1(x1,b,in1);
pmos p2(x1,b,sbar);
pmos p3(ybar,x1,in2);
pmos p4(ybar,x1,s);
pmos p5(out,b,ybar);

nmos n1(ybar,x2,in1);
nmos n2(ybar,x3,in2);
nmos n3(x2,a,sbar);
nmos n4(x3,a,s);
nmos n5(out,a,ybar);

endmodule
```

Testbench

```
module TB();
reg in1,in2,s;
wire out;

cmos_mux dut(.out(out),.in1(in1),.in2(in2),.s(s));

initial
```

```
begin
    $monitor("Time=%0t,for in1=%0b,in2=%0b,s=%0b,out=%0b",$time,in1,in2,s,out);

    in1=0;in2=0;s=0;
#10 in1=0;in2=1;s=0;
#10 in1=1;in2=0;s=0;
#10 in1=1;in2=1;s=0;
#10 in1=0;in2=0;s=1;
#10 in1=0;in2=1;s=1;
#10 in1=1;in2=0;s=1;
#10 in1=1;in2=1;s=1;
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

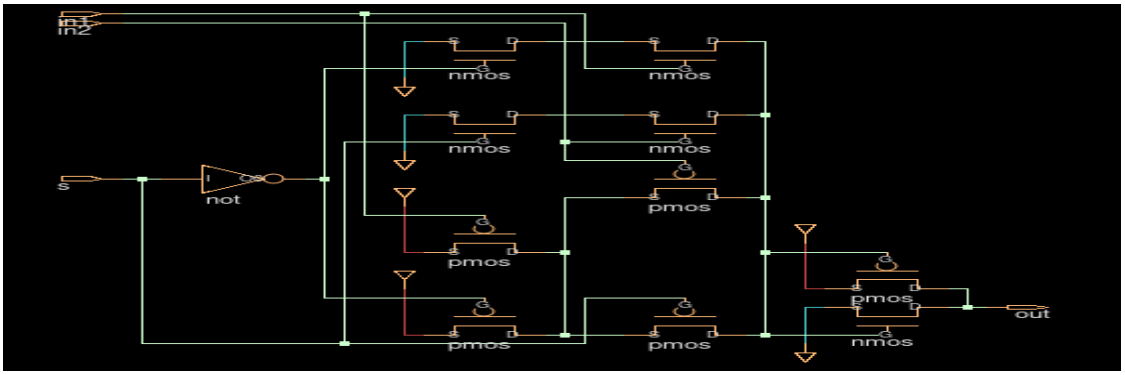
```
Time=0,for in1=0,in2=0,s=0,out=0
Time=10,for in1=0,in2=1,s=0,out=0
Time=20,for in1=1,in2=0,s=0,out=1
Time=30,for in1=1,in2=1,s=0,out=1
Time=40,for in1=0,in2=0,s=1,out=0
```

Time=50,for in1=0,in2=1,s=1,out=1

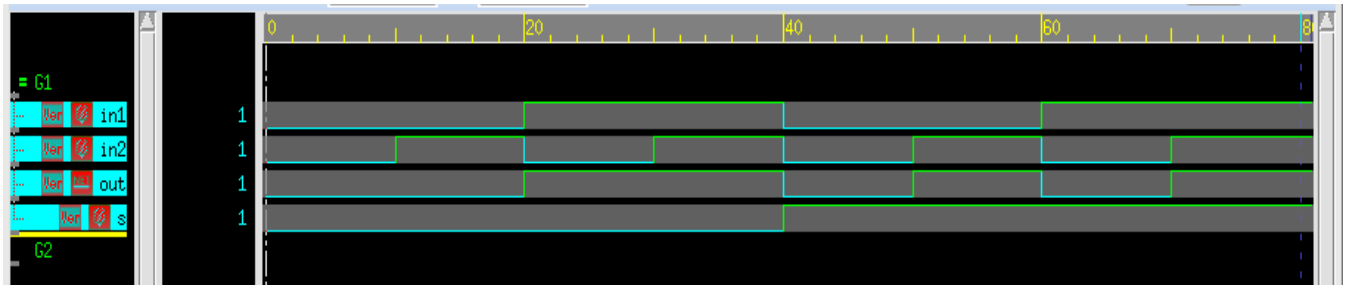
Time=60,for in1=1,in2=0,s=1,out=0

Time=70,for in1=1,in2=1,s=1,out=1

Schematic Design



waveform



61.0 Design a Half adder by switch level modelling using mosfet

Design code

```
module cmos_half_adder(out,in1,in2);
input in1,in2;
output out;
supply0 a;
supply1 b;

wire x1,x2,x3,x4,in1bar,in2bar;
not (in1bar,in1);
```

```
not (in2bar,in2);

pmos p1(x1,b,in1);
pmos p2(x2,b,in1bar);
pmos p3(out,x1,in2bar);
pmos p4(out,x2,in2);

nmos n1(out,x3,in1);
nmos n2(out,x4,in1bar);
nmos n3(x3,a,in2);
nmos n4(x4,a,in2bar);

endmodule
```

Testbench

```
module TB();
reg in1,in2;
wire out;

cmos_half_adder dut(.out(out),.in1(in1),.in2(in2));

initial
begin
    $monitor("Time=%0t,for in1=%0b,in2=%0b,out=%0b",$time,in1,in2,out);
    in1=0;in2=0;
#10 in1=0;in2=1;
#10 in1=1;in2=0;
#10 in1=1;in2=1;
```

```
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

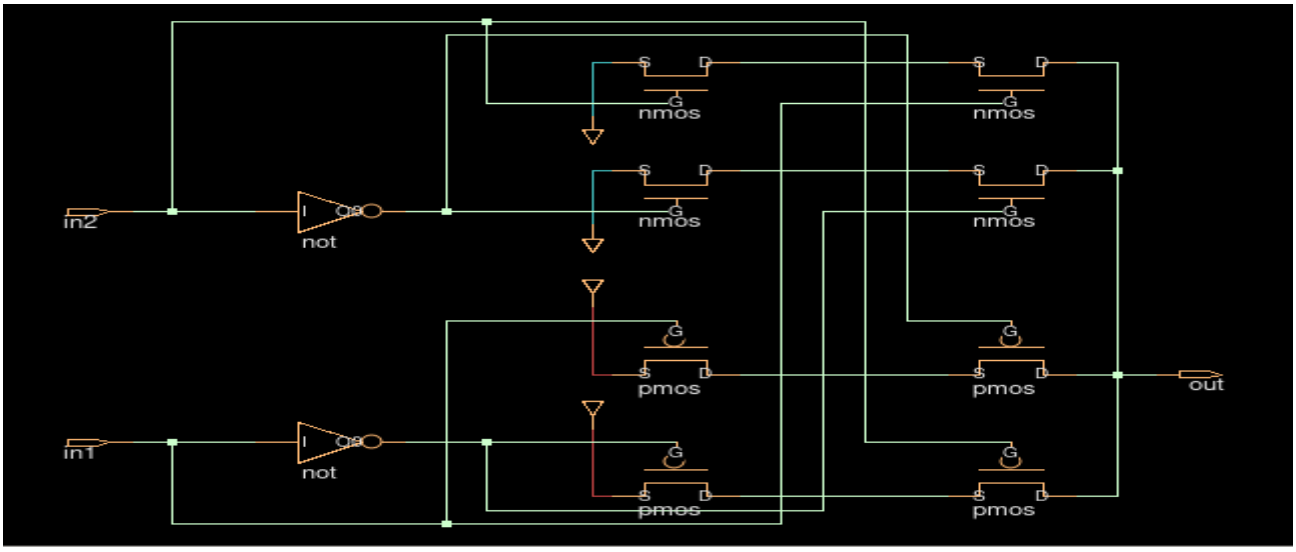
Time=0,for in1=0,in2=0,out=0

Time=10,for in1=0,in2=1,out=1

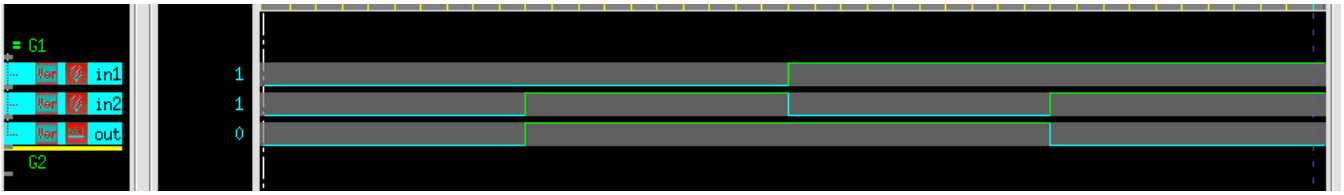
Time=20,for in1=1,in2=0,out=1

Time=30,for in1=1,in2=1,out=0

Schematic Design



waveform



Lab12

DATA SHIFT REGISTER

62.0 1.0 SISO (SERIAL-IN SERIAL-OUT)

Design code

```
module shift_R_SISO(so,sin,clk,rst);
output so;
input sin,clk,rst;

reg [3:0]q;

always@(posedge clk)
begin
	if (rst)
		q=4'b0000;
	else begin
		q[3]<=sin;
		q[2]<=q[3];
		q[1]<=q[2];
		q[0]<=q[1];
	end
end

assign so = q[0];
```

```
endmodule
```

Testbench

```
module SISO_testbench();
reg clk,rst;
reg sin;
wire so;

shift_R_SISO dut(.so(so),.sin(sin),.clk(clk),.rst(rst));
initial
    clk=0;
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t rst=%0b sin=%0b so=%0b",$time,rst,sin,so);
    rst=1; sin=0;
    #10 rst=0;
    sin=1;#10;
    sin=0;#10;
    sin=1;#10;
    sin=0;#10;
    #85 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,SISO_testbench);
end
```

endmodule

Simulation Result

Output

Time=0 rst=1 sin=0 so=x

Time=5 rst=1 sin=0 so=0

Time=10 rst=0 sin=1 so=0

Time=20 rst=0 sin=0 so=0

Time=30 rst=0 sin=1 so=0

Time=40 rst=0 sin=0 so=0

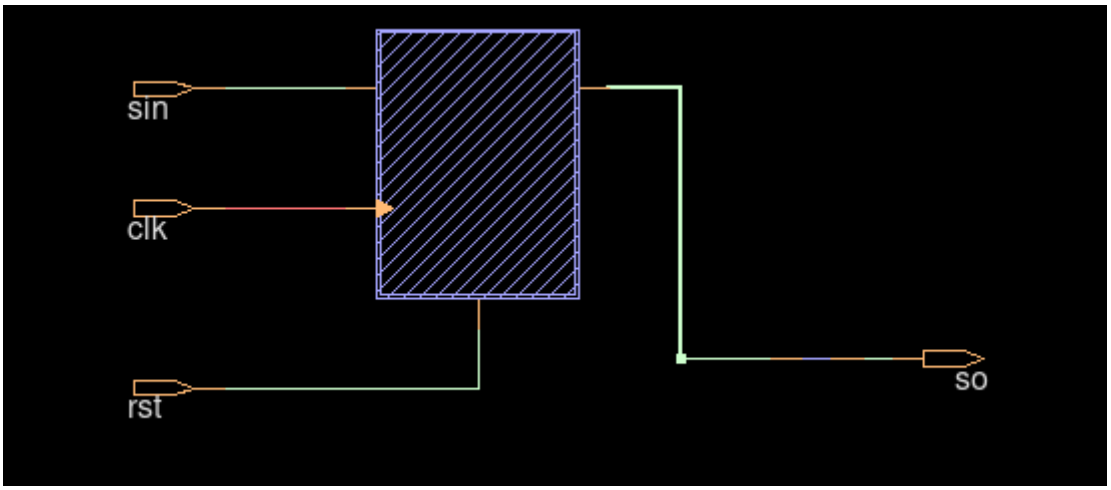
Time=45 rst=0 sin=0 so=1

Time=55 rst=0 sin=0 so=0

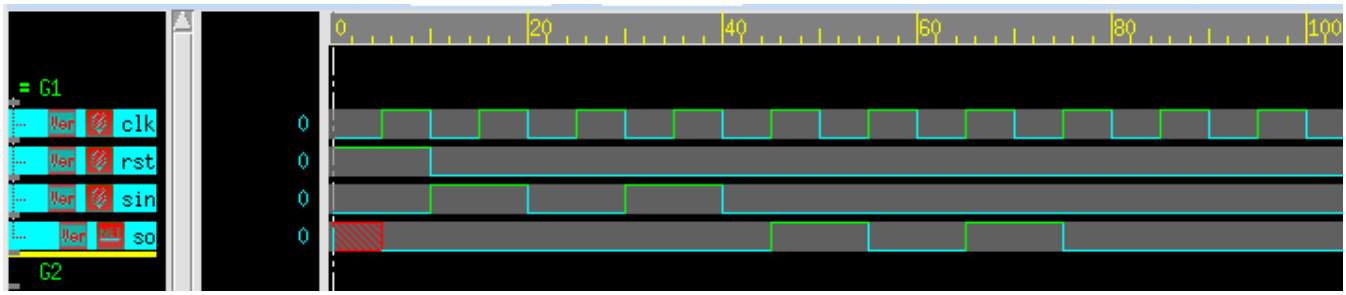
Time=65 rst=0 sin=0 so=1

Time=75 rst=0 sin=0 so=0

Schematic Design



waveform



63.0 2.0 SIPO (SERIAL-IN PARALLEL-OUT)

Design code

```
module shift_R_SIPO(so,sin,clk,rst);
output [3:0]so;
input sin,clk,rst;

reg [3:0]q;

always@(posedge clk)
begin
	if (rst)
		q<=4'b0000;
	else begin
		q[3]<=sin;
		q[2]<=q[3];
		q[1]<=q[2];
		q[0]<=q[1];
	end
end
assign so = q;
endmodule
```

Testbench

```
module SIPO_testbench();
reg clk,rst;
reg sin;
wire [3:0]so;

shift_R_SIPO dut(.so(so),.sin(sin),.clk(clk),.rst(rst));
initial
	clk=0;
always #5 clk=~clk;

initial begin
	$monitor("Time=%0t rst=%b sin=%b so=%b",$time,rst,sin,so);
	rst=1; sin=0;
	#10 rst=0;
	sin=1;#10;
	sin=0;#10;
	sin=1;#10;
	sin=0;#10;
	#85 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,SIPO_testbench);
end
endmodule
```

Simulation Result

Output

Time=0 rst=1 sin=0 so=xxxx

Time=5 rst=1 sin=0 so=0000

Time=10 rst=0 sin=1 so=0000

Time=15 rst=0 sin=1 so=1000

Time=20 rst=0 sin=0 so=1000

Time=25 rst=0 sin=0 so=0100

Time=30 rst=0 sin=1 so=0100

Time=35 rst=0 sin=1 so=1010

Time=40 rst=0 sin=0 so=1010

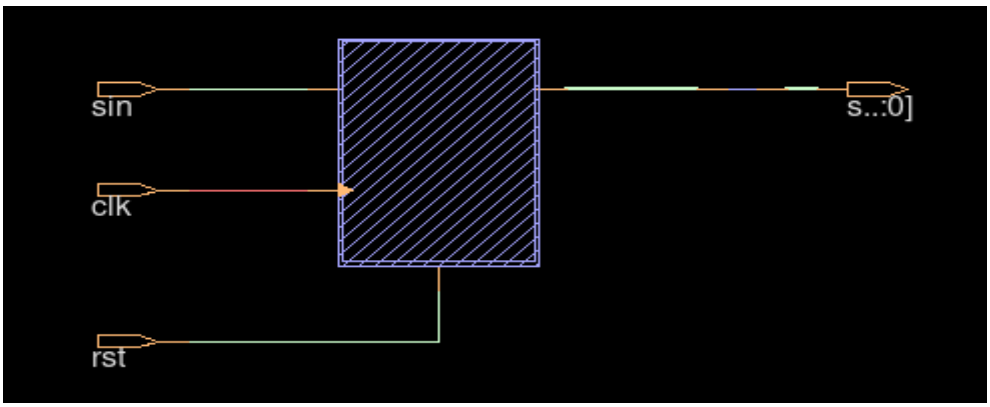
Time=45 rst=0 sin=0 so=0101

Time=55 rst=0 sin=0 so=0010

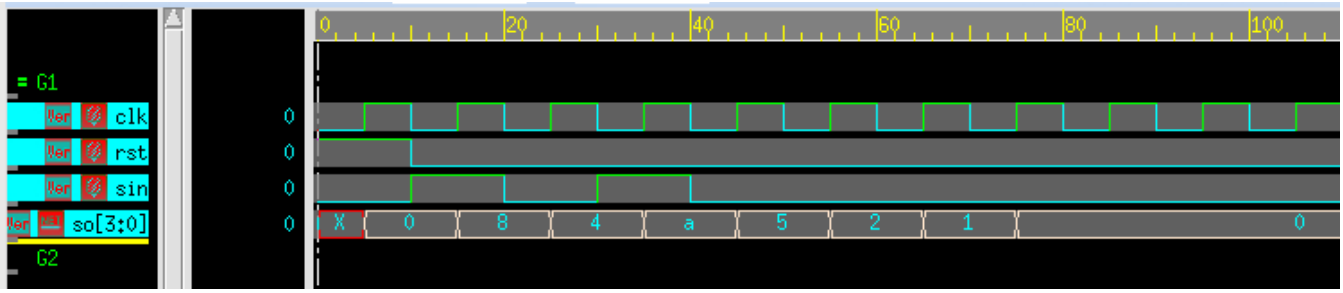
Time=65 rst=0 sin=0 so=0001

Time=75 rst=0 sin=0 so=0000

Schematic Design



waveform



64.0 3.0 PISO (PARALLEL-IN SERIAL-OUT)

Design code

```
module shift_R_PISO(so,din,load,clk,rst);
output so;
input [3:0]din;
input clk,rst,load;

reg [3:0]q;

always@(posedge clk)
begin
	if (rst)
		q<=4'b0000;
	else if(load==1)
		q<=din;
	else begin

		q[3]<=1'bx;
		q[2]<=q[3];
		q[1]<=q[2];
		q[0]<=q[1];
	end
```

```
end

assign so = q[0];
endmodule
```

Testbench

```
module PISO_testbench();
reg clk,rst,load;
reg [3:0]din;
wire so;

shift_R_PISO dut(.so(so),.din(din),.clk(clk),.rst(rst),.load(load));
initial
	clk=0;
always #5 clk=~clk;

initial begin
	$monitor("Time=%0t rst=%b din=%b so=%b",$time,rst,din,so);
	rst=1; din=0;
	#10 rst=0;
	load=0;din=1111;#10;
	#10 rst=1;
	#10 rst=0;
	#10 load=1;din=1010;
	#10;load=0;
	#85 $finish();
end
```

```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,PISO_testbench);
end
endmodule
```

Simulation Result

Output

Time=0 rst=1 din=0000 so=x

Time=5 rst=1 din=0000 so=0

Time=10 rst=0 din=0111 so=0

Time=30 rst=1 din=0111 so=0

Time=40 rst=0 din=0111 so=0

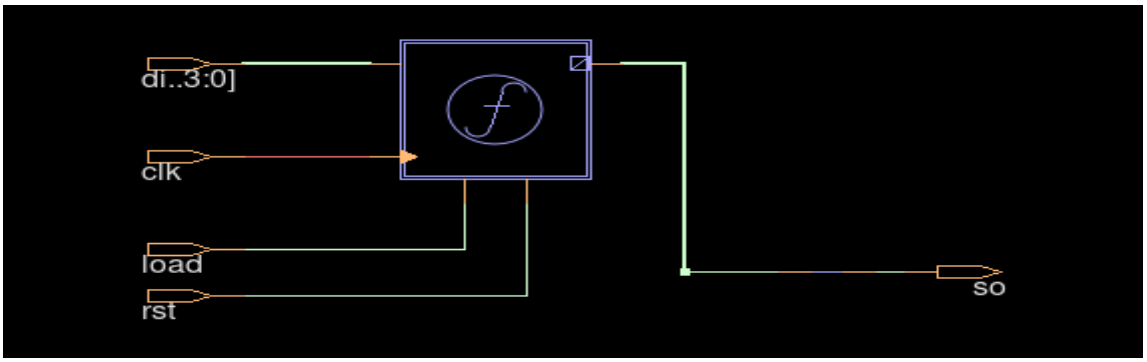
Time=50 rst=0 din=0010 so=0

Time=65 rst=0 din=0010 so=1

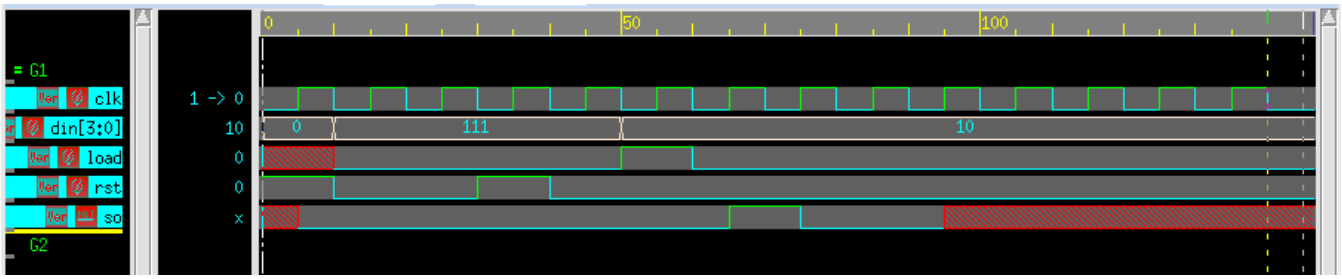
Time=75 rst=0 din=0010 so=0

Time=95 rst=0 din=0010 so=x

Schematic Design



waveform



65.0 4.0 PIPO (PARALLEL-IN PARALLEL-OUT)

Design code

```
module shift_R_PIPO(q,din,clk,rst);
input [3:0]din;
input clk,rst;
output reg [3:0]q;

always@(posedge clk)
begin
	if (rst)
		q<=4'b0000;
	else
		q<=din;
end
endmodule
```

Testbench

```
module PIPO_testbench();
reg clk,rst;
reg [3:0]din;
wire [3:0]q;

shift_R_PIPO dut(.q(q),.din(din),.clk(clk),.rst(rst));
```

```
initial
    clk=0;
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t rst=%b din=%b q=%b",$time,rst,din,q);
    rst=1; din=0;
    #10 rst=0;
    #40 din=4'b1111;
    #10 din=4'b1100;
    #10 din=4'b0110;
    #40 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,PIPO_testbench);
end
endmodule
```

Simulation Result

Output

```
Time=0 rst=1 din=0000 q=xxxx
Time=5 rst=1 din=0000 q=0000
Time=10 rst=0 din=0000 q=0000
Time=50 rst=0 din=1111 q=0000
```

Time=55 rst=0 din=1111 q=1111

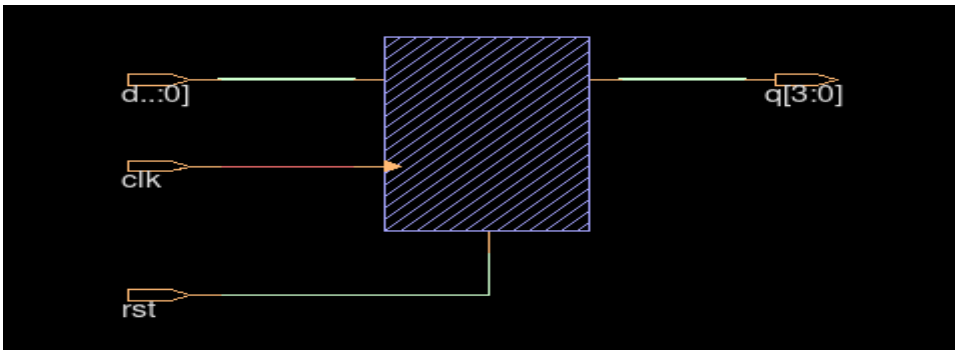
Time=60 rst=0 din=1100 q=1111

Time=65 rst=0 din=1100 q=1100

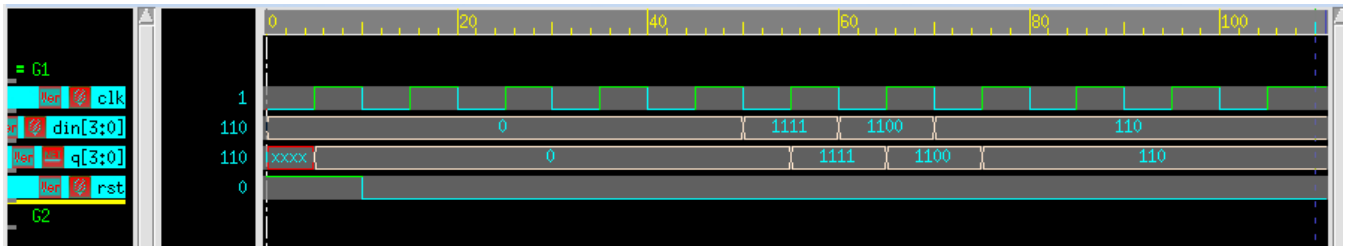
Time=70 rst=0 din=0110 q=1100

Time=75 rst=0 din=0110 q=0110

Schematic Design



waveform



Lab13 66.0 Frequency Divider by 2

Design

Design the output frequency fo=f/2 where f is input frequency and fo is output frequency of flip flop and time period of graph will be increased. (Time period- To=2T)

Design Code

```
module freq_div_2(clk_out,clk,rst);

input clk,rst;

output reg clk_out;
```

```
always @(posedge clk ) begin
    if (rst==1)
        clk_out <=0;
    else
        clk_out <=~clk_out;
end
endmodule
```

Testbench

```
module freq_div2_tb();
reg clk, rst;
wire clk_out;

freq_div_2 dut(.clk(clk),.rst(rst),.clk_out(clk_out));

initial begin
    clk=0;
end
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t, for rst=%d,we get clk_out=%d",$time,rst,clk_out);
    rst=1;clk=0;

#10 rst=0;
#80;
#10 $finish();
```

```
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,freq_div2_tb);
end
endmodule
```

Simulation Result

Output

Time=0,for rst=1,we get clk_out=x

Time=5,for rst=1,we get clk_out=0

Time=10,for rst=0,we get clk_out=0

Time=15,for rst=0,we get clk_out=1

Time=25,for rst=0,we get clk_out=0

Time=35,for rst=0,we get clk_out=1

Time=45,for rst=0,we get clk_out=0

Time=55,for rst=0,we get clk_out=1

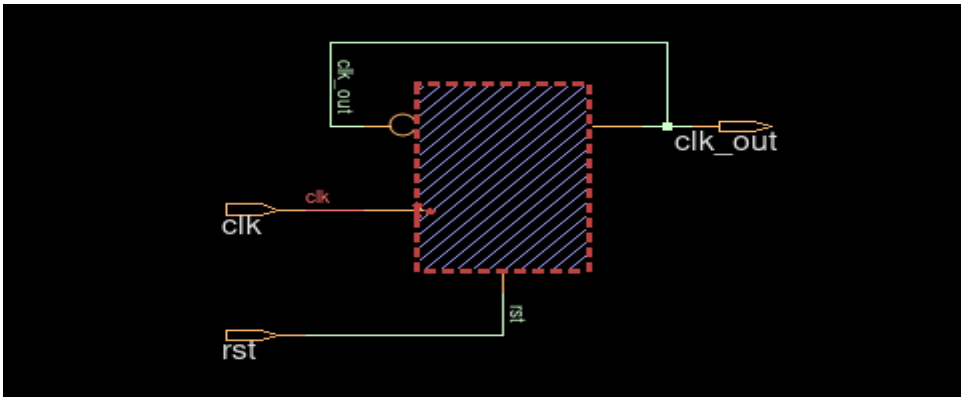
Time=65,for rst=0,we get clk_out=0

Time=75,for rst=0,we get clk_out=1

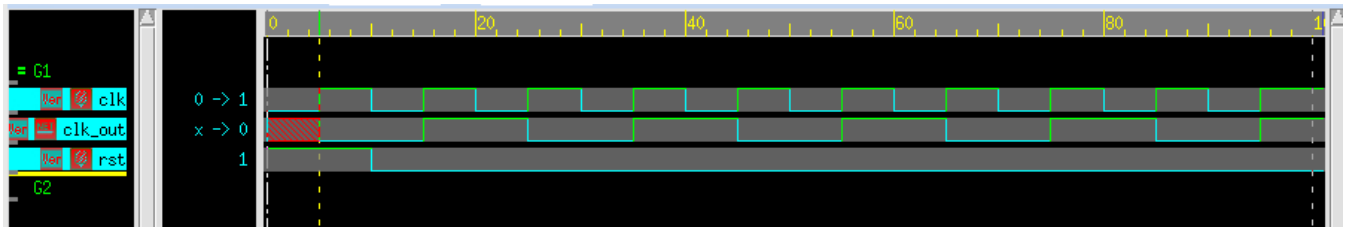
Time=85,for rst=0,we get clk_out=0

Time=95,for rst=0,we get clk_out=1

Schematic Design



waveform



66.0 Frequency Divider by f/4

Design code

```
module freq_div_4(clk_out,clk_div2,clk,rst);
input clk,rst;
output reg clk_div2,clk_out;

always @(posedge clk or posedge rst) begin
    if (rst==1)
        clk_div2 <=0;
    else
        clk_div2 <=~clk_div2;
end

always @(posedge clk_div2 or posedge rst) begin
    if (rst)
        clk_out <=0;
```

```
    else
        clk_out <=~clk_out;
end
endmodule
```

Testbench

```
module freq_div4_tb();
reg clk, rst;
wire clk_div2,clk_out;

freq_div_4 dut(.clk(clk),.rst(rst),.clk_div2(clk_div2),.clk_out(clk_out));

initial begin
    clk=0;
end
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t,for rst=%d,we get clk_div2=%0d clk_out=%d",$time,rst,clk_div2,clk_out);

    rst=1;clk=0;
#10 rst=0;
#80;
#10 $finish();
end

initial begin
```

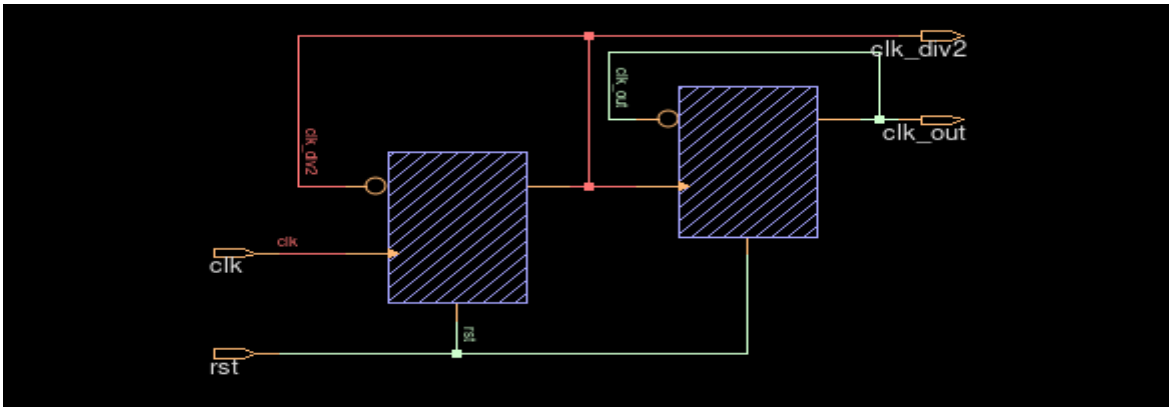
```
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,freq_div4_tb);
end
endmodule
```

Simulation Result

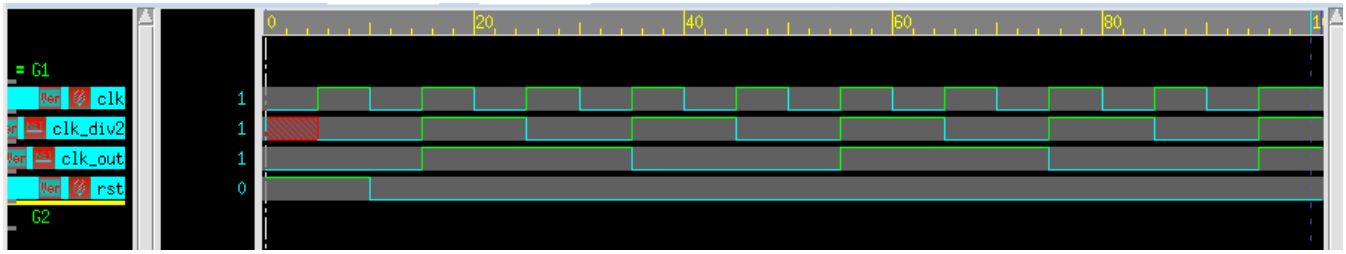
Output

```
Time=0,for rst=1,we get clk_div2=x clk_out=0
Time=5,for rst=1,we get clk_div2=0 clk_out=0
Time=10,for rst=0,we get clk_div2=0 clk_out=0
Time=15,for rst=0,we get clk_div2=1 clk_out=1
Time=25,for rst=0,we get clk_div2=0 clk_out=1
Time=35,for rst=0,we get clk_div2=1 clk_out=0
Time=45,for rst=0,we get clk_div2=0 clk_out=0
Time=55,for rst=0,we get clk_div2=1 clk_out=1
Time=65,for rst=0,we get clk_div2=0 clk_out=1
Time=75,for rst=0,we get clk_div2=1 clk_out=0
Time=85,for rst=0,we get clk_div2=0 clk_out=0
Time=95,for rst=0,we get clk_div2=1 clk_out=1
$finish at simulation time                100
```

Schematic Design



waveform



Method-2 by add initialize condition (synchronous reset)

Design code

```
module freq_div_4(clk_out,clk_div2,clk,rst);
input clk,rst;
output reg clk_div2=0,clk_out=0;

always @(posedge clk) begin
    if (rst==1)
        clk_div2 <=0;
    else
        clk_div2 <=~clk_div2;
end
always @(posedge clk_div2) begin
    if (rst)
        clk_out <=0;
```

```
    else
        clk_out <=~clk_out;
end
endmodule
```

Simulation Result

Output

Time=0,for rst=1,we get clk_div2=0 clk_out=0

Time=10,for rst=0,we get clk_div2=0 clk_out=0

Time=15,for rst=0,we get clk_div2=1 clk_out=1

Time=25,for rst=0,we get clk_div2=0 clk_out=1

Time=35,for rst=0,we get clk_div2=1 clk_out=0

Time=45,for rst=0,we get clk_div2=0 clk_out=0

Time=55,for rst=0,we get clk_div2=1 clk_out=1

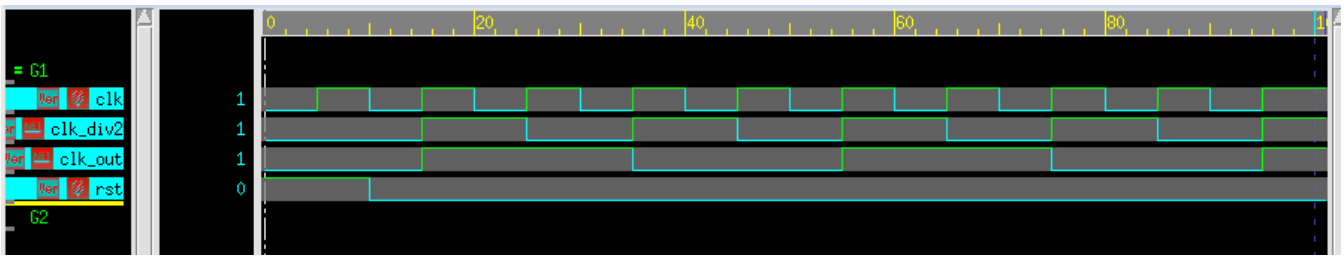
Time=65,for rst=0,we get clk_div2=0 clk_out=1

Time=75,for rst=0,we get clk_div2=1 clk_out=0

Time=85,for rst=0,we get clk_div2=0 clk_out=0

Time=95,for rst=0,we get clk_div2=1 clk_out=1

waveform



67.0 Frequency Divider by f/3

Design code

```
module freq_div_3(clk_out,,clk,rst);
input clk,rst;
```

```
output reg clk_out;

reg [1:0]count;

always @(posedge clk) begin
    if (rst==1)begin
        count <=0;
        clk_out <=0;
    end
    else begin
        count <= count+1;
        if (count==2) begin
            clk_out <=~clk_out;
            count <=0;
        end
    end
end
endmodule
```

Testbench

```
module freq_div3_tb();
reg clk, rst;
wire clk_out;

freq_div_3 dut(.clk(clk),.rst(rst),.clk_out(clk_out));

initial begin
    clk=0;
```

```
end
always #5 clk=~clk;

initial begin
	$monitor("Time=%0t,for rst=%d,we get clk_out=%d",$time,rst,clk_out);

	rst=1;clk=0;
#10 rst=0;

#200;
#10 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,freq_div3_tb);
end
endmodule
```

Simulation Result

Output

```
Time=0,for rst=1,we get clk_out=x
Time=5,for rst=1,we get clk_out=0
Time=10,for rst=0,we get clk_out=0
Time=35,for rst=0,we get clk_out=1
Time=65,for rst=0,we get clk_out=0
Time=95,for rst=0,we get clk_out=1
Time=125,for rst=0,we get clk_out=0
```

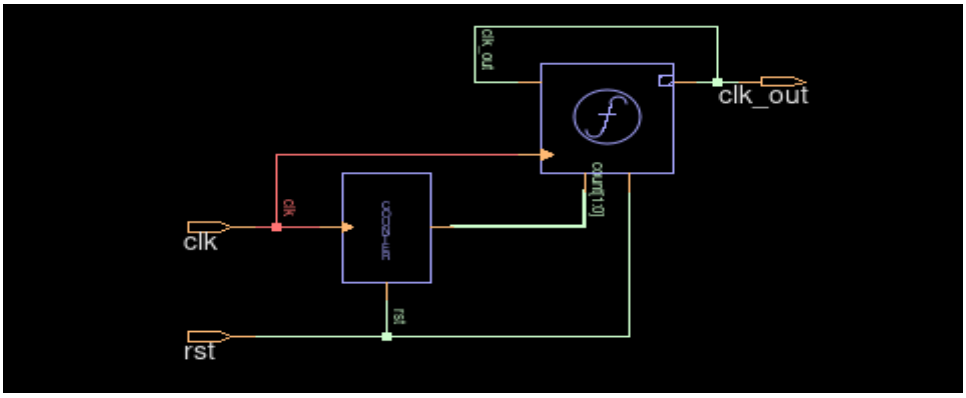
Time=155,for rst=0,we get clk_out=1

Time=185,for rst=0,we get clk_out=0

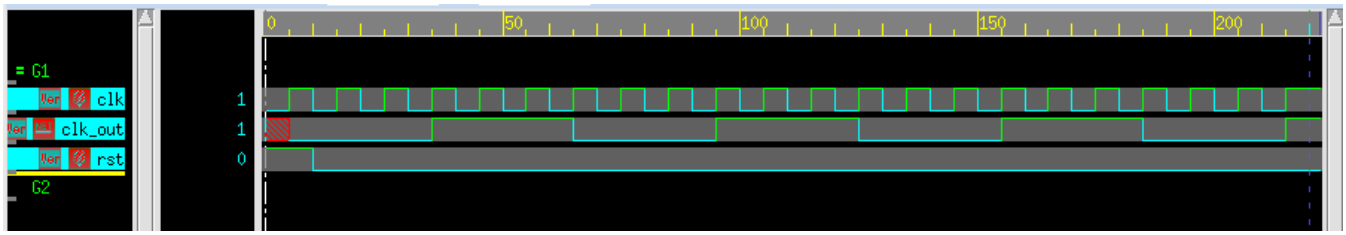
Time=215,for rst=0,we get clk_out=1

$finish at simulation time 220

Schematic Design



waveform



Method-2

Design code

```
module freq_div_3(clk_out,,clk,rst);
input clk,rst;
output reg clk_out;

reg [1:0]count;

always @(posedge clk or posedge rst) begin
    if (rst==1)
```

```
            count <=0;
        else if (count==2)
            count <=0;
        else
            count <= count+1;
end

always @(negedge clk or posedge rst)begin
        if (rst)
            clk_out <=0;
        else if (count==2)
            clk_out <=~clk_out;
end
endmodule
```

Testbench

```
module freq_div3_tb();
reg clk, rst;
wire clk_out;

freq_div_3 dut(.clk(clk),.rst(rst),.clk_out(clk_out));
initial begin
        clk=0;
end
always #5 clk=~clk;

initial begin
        $monitor("Time=%0t,for rst=%d,we get clk_out=%d",$time,rst,clk_out);
```

```
        rst=1;clk=0;
#10 rst=0;

#200;
#10 $finish();
end

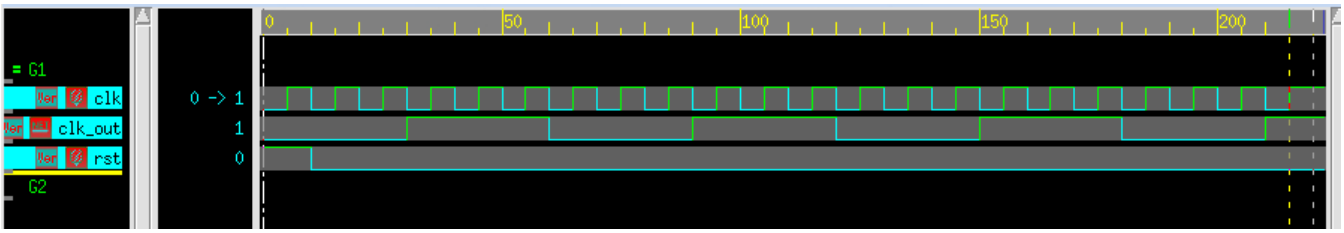
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,freq_div3_tb);
end
endmodule
```

Simulation Result

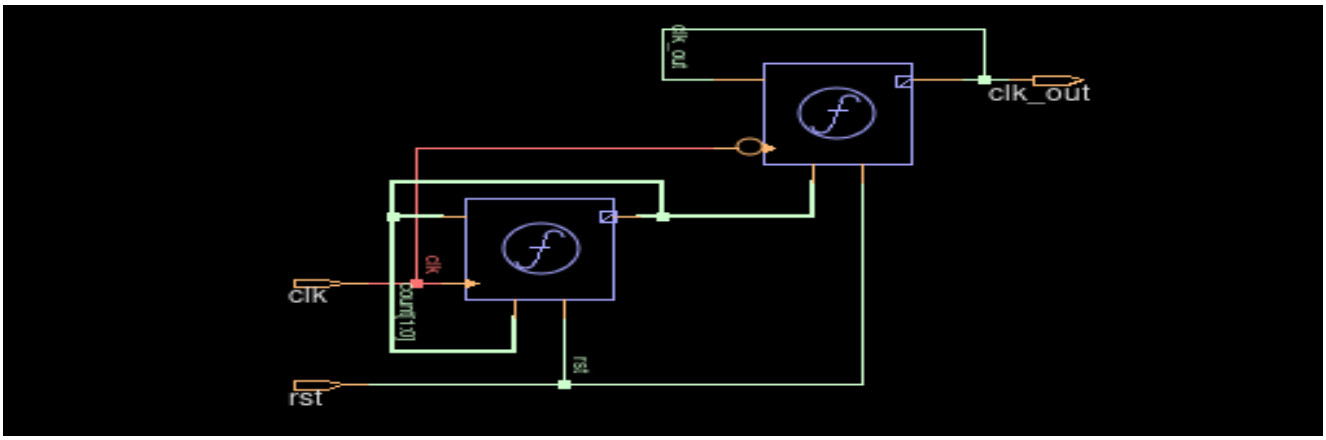
Output

```
Time=0,for rst=1,we get clk_out=0
Time=10,for rst=0,we get clk_out=0
Time=30,for rst=0,we get clk_out=1
Time=60,for rst=0,we get clk_out=0
Time=90,for rst=0,we get clk_out=1
Time=120,for rst=0,we get clk_out=0
Time=150,for rst=0,we get clk_out=1
Time=180,for rst=0,we get clk_out=0
Time=210,for rst=0,we get clk_out=1
$finish at simulation time                220
```

waveform



Schematic Design



68.0 Frequency Divider by f/5

Design code

```
module freq_div_5(clk_out,,clk,rst);
input clk,rst;
output reg clk_out;

reg [2:0]count;

always @(posedge clk) begin
    if (rst==1)begin
        count <=0;
        clk_out <=0;
    end
    else begin
        count <= count+1;
```

```
            if (count==4) begin
                clk_out <=~clk_out;
                count <=0;
            end
        end
end
endmodule
```

Testbench

```
module freq_div5_tb();
reg clk, rst;
wire clk_out;

freq_div_5 dut(.clk(clk),.rst(rst),.clk_out(clk_out));

initial begin
        clk=0;
end
always #5 clk=~clk;

initial begin
        $monitor("Time=%0t,for rst=%d,we get clk_out=%d",$time,rst,clk_out);

        rst=1;clk=0;
#10 rst=0;

#250;
#10 $finish();
```

```
end


initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,freq_div5_tb);
end
endmodule
```

Simulation Result

Output

Time=0,for rst=1,we get clk_out=x

Time=5,for rst=1,we get clk_out=0

Time=10,for rst=0,we get clk_out=0

Time=55,for rst=0,we get clk_out=1

Time=105,for rst=0,we get clk_out=0

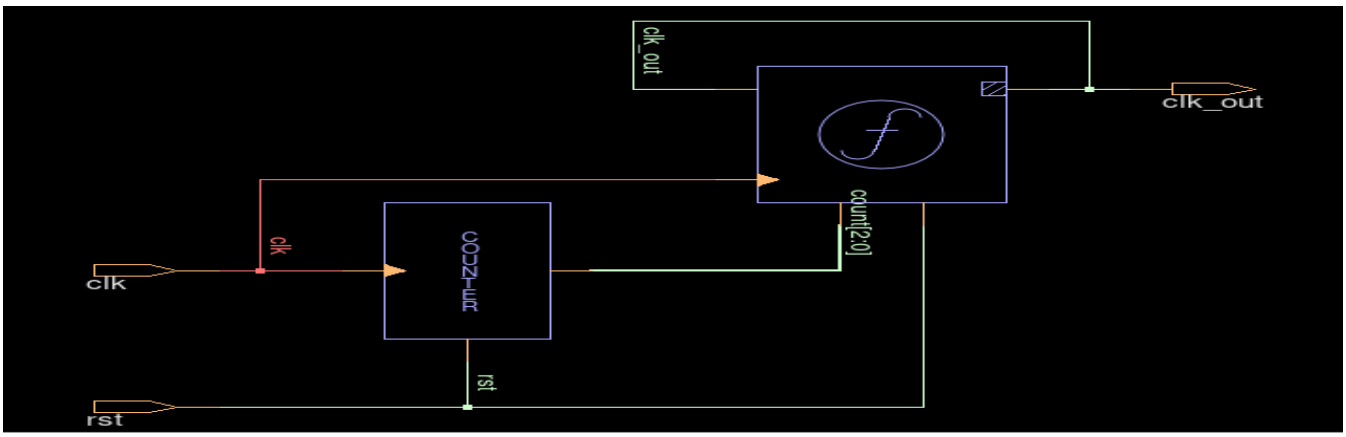
Time=155,for rst=0,we get clk_out=1

Time=205,for rst=0,we get clk_out=0

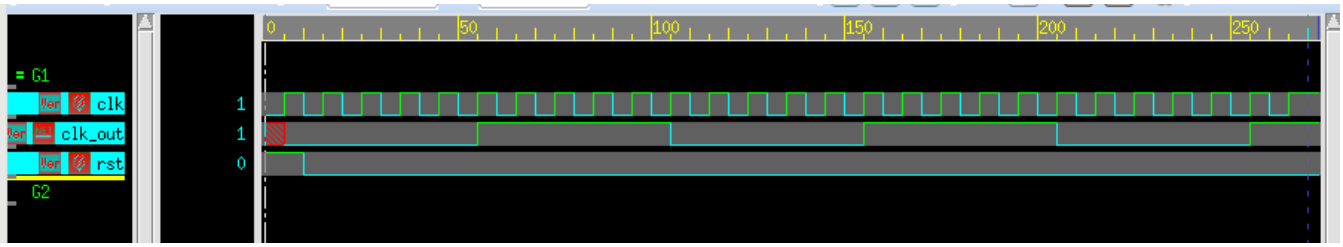
Time=255,for rst=0,we get clk_out=1

$finish at simulation time 270

Schematic Design



waveform



69.0 Frequency Divider by f/3.5

Design code

```
// frequency devider by f/3.5
module freq_div(clk_out,clk,rst);
input clk,rst;
output reg clk_out;

reg [2:0]count;

always @(posedge clk) begin
        if (rst==1)begin
                count <=0;
                clk_out <=0;
        end
        else begin
                if (count==6)
                        count <=0;
                else
                        count <= count+1;
                if (count ==0 || count==3)
                        clk_out <=~clk_out;
```

```
    end
end
endmodule
```

Testbench

//testbench of **frequency divide by 3.5**

```
module freq_div3.5_tb();
reg clk, rst;
wire clk_out;

freq_div dut(.clk(clk),.rst(rst),.clk_out(clk_out));
initial begin
    clk=0;
end
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t,for rst=%d,we get clk_out=%d",$time,rst,clk_out);
    rst=1;clk=0;
#10 rst=0;

#250;
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
```

```
    $fsdbDumpvars(0,freq_div3.5_tb);
end
endmodule
```

Simulation Result

Output

Time=0,for rst=1,we get clk_out=x

Time=5,for rst=1,we get clk_out=0

Time=10,for rst=0,we get clk_out=0

Time=15,for rst=0,we get clk_out=1

Time=45,for rst=0,we get clk_out=0

Time=85,for rst=0,we get clk_out=1

Time=115,for rst=0,we get clk_out=0

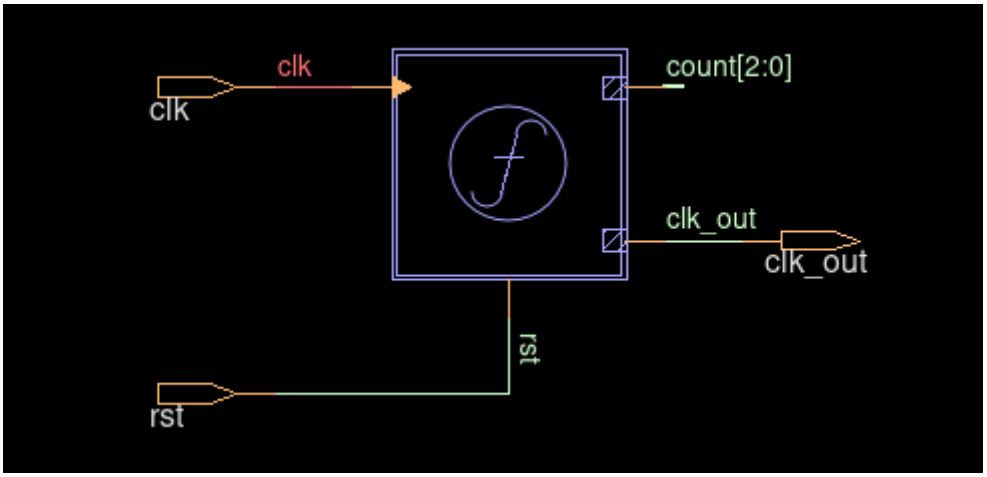
Time=155,for rst=0,we get clk_out=1

Time=185,for rst=0,we get clk_out=0

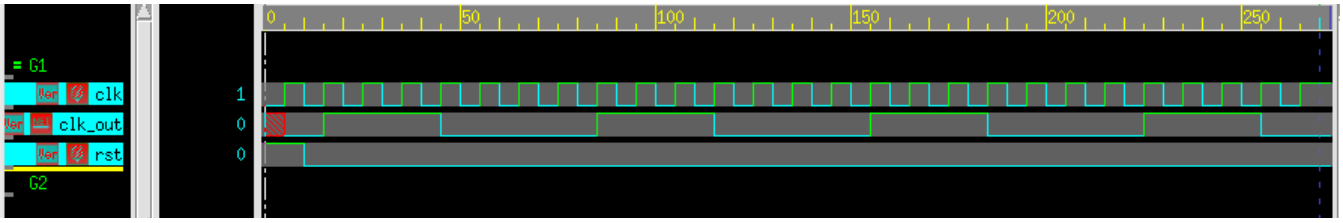
Time=225,for rst=0,we get clk_out=1

Time=255,for rst=0,we get clk_out=0

Schematic Design



waveform



70.0 Design a MOD-10 counter (0-9) / A Decade counter

Design code

```
module mod_10_counter(count,clk,rst);
input clk,rst;
output reg [3:0]count;

always @(posedge clk)
begin
	if (rst==1)
		count <=0;
	else if (count ==9)
		count <=0;
	else
		count <= count+1;
end
endmodule
```

Testbench

```
module mod_10_tb();
reg clk, rst;
wire [3:0] count;

mod_10_counter dut(.clk(clk),.rst(rst),.count(count));
initial begin
```

```
        clk=0;
end
always #5 clk=~clk;

initial begin
        $monitor("Time=%0t,for rst=%d,we get count=%d",$time,rst,count);
        rst=1;clk=0;
#10 rst=0;

#150;
#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,mod_10_tb);
end
endmodule
```

Simulation Result

Output

```
Time=0,for rst=1,we get count= x
Time=5,for rst=1,we get count= 0
Time=10,for rst=0,we get count= 0
Time=15,for rst=0,we get count= 1
Time=25,for rst=0,we get count= 2
Time=35,for rst=0,we get count= 3
```

Time=45,for rst=0,we get count= 4

Time=55,for rst=0,we get count= 5

Time=65,for rst=0,we get count= 6

Time=75,for rst=0,we get count= 7

Time=85,for rst=0,we get count= 8

Time=95,for rst=0,we get count= 9

Time=105,for rst=0,we get count= 0

Time=115,for rst=0,we get count= 1

Time=125,for rst=0,we get count= 2

Time=135,for rst=0,we get count= 3

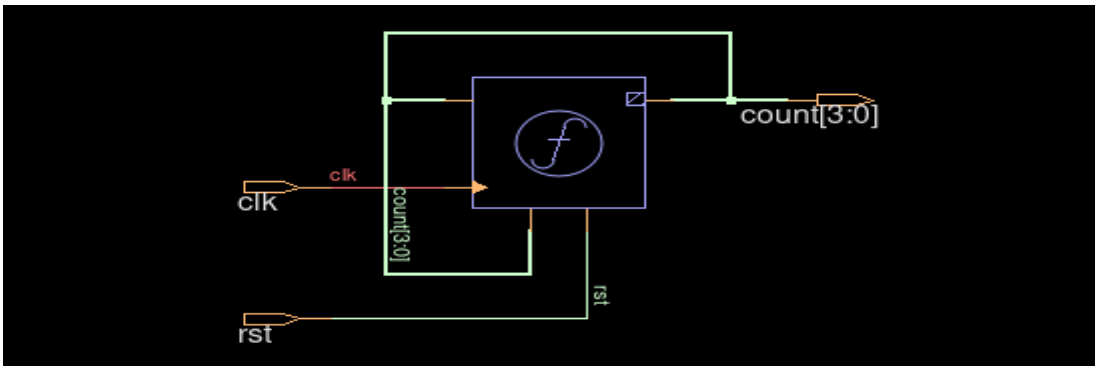
Time=145,for rst=0,we get count= 4

Time=155,for rst=0,we get count= 5

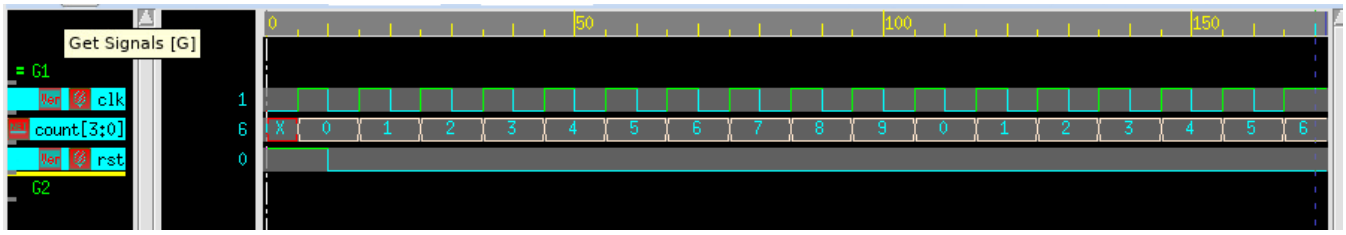
Time=165,for rst=0,we get count= 6

$finish at simulation time 170

Schematic Design



waveform



71.0 Design a MOD-5 up-down counter (0-4)

Design code

```
module mod_5_counter(count,clk,rst,up_down);
input clk,rst;
input up_down;
output reg [2:0]count;

always @(posedge clk)begin
	if (rst) begin
		count <= 0;
	end else begin

		if (up_down) begin
			if (count == 4)
				count <= 0;
			else
				count <= count + 1;
		end else begin
			if (count == 0)
				count <= 4;
			else
				count <= count - 1;
		end
	end
end
endmodule
```

Testbench

```
module mod_5_tb();
reg clk, rst,up_down;
wire [2:0] count;

mod_5_counter dut(.clk(clk),.rst(rst),.up_down(up_down),.count(count));

initial begin
	clk=0;
end
always #5 clk=~clk;

initial begin
	$monitor("Time=%0t,for rst=%b,up_down=%b we get count=%d",$time,rst,up_down,count);
	rst=1;clk=0;up_down=1;

#10 rst=0;
#50 up_down=0;

#100;
#10 $finish();
end
initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,mod_5_tb);
end
endmodule
```

Simulation Result

Output

Time=0,for rst=1,up_down=1 we get count=x

Time=5,for rst=1,up_down=1 we get count=0

Time=10,for rst=0,up_down=1 we get count=0

Time=15,for rst=0,up_down=1 we get count=1

Time=25,for rst=0,up_down=1 we get count=2

Time=35,for rst=0,up_down=1 we get count=3

Time=45,for rst=0,up_down=1 we get count=4

Time=55,for rst=0,up_down=1 we get count=0

Time=60,for rst=0,up_down=0 we get count=0

Time=65,for rst=0,up_down=0 we get count=4

Time=75,for rst=0,up_down=0 we get count=3

Time=85,for rst=0,up_down=0 we get count=2

Time=95,for rst=0,up_down=0 we get count=1

Time=105,for rst=0,up_down=0 we get count=0

Time=115,for rst=0,up_down=0 we get count=4

Time=125,for rst=0,up_down=0 we get count=3

Time=135,for rst=0,up_down=0 we get count=2

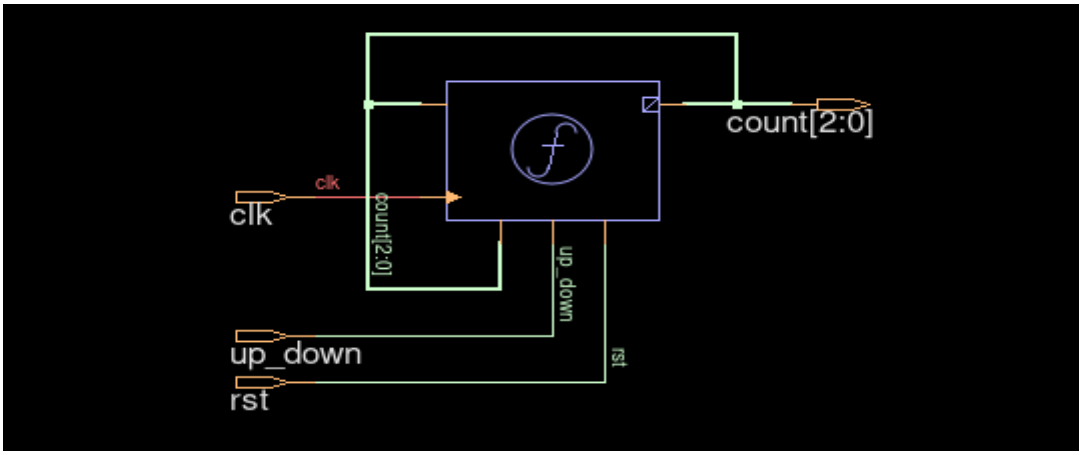
Time=145,for rst=0,up_down=0 we get count=1

Time=155,for rst=0,up_down=0 we get count=0

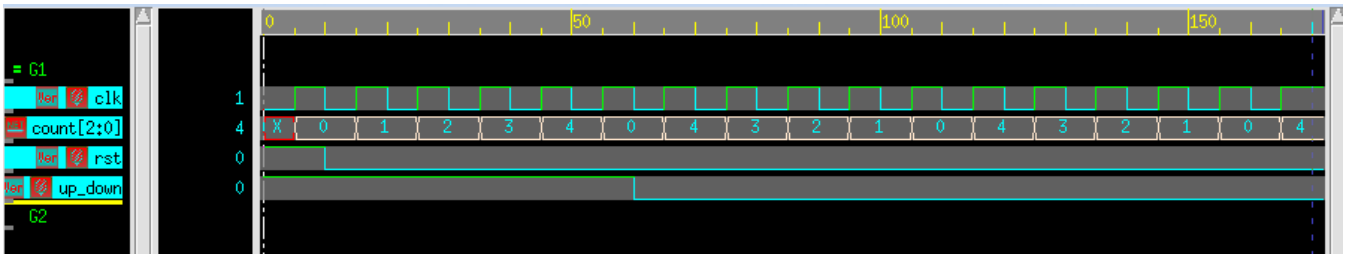
Time=165,for rst=0,up_down=0 we get count=4

$finish at simulation time 170

Schematic Design



waveform



72.0 Verilog code for a mod-8 counter using a parameter

Design code

```
//mod-8 counter by using parameter
module mod_8_param #(parameter WIDTH=3)(
        input clk,rst,
        output reg [WIDTH-1:0]count);


always @(posedge clk)
begin
        if (rst==1)
                count <=0;
        else
                count <=(count ==7) ? 0 : count+1;
end
```

```
endmodule
```

Testbench

```
module mod_8_tb();
reg clk,rst;
wire [2:0] count_mod8;
wire [1:0] count_mod4;

initial begin
    clk=0;
end
always #5 clk=~clk;

mod_8_param #(.WIDTH(3)) dut1(.clk(clk),.rst(rst),.count(count_mod8));
mod_8_param #(.WIDTH(2)) dut2(.clk(clk),.rst(rst),.count(count_mod4));
initial begin
    $monitor("Time=%0t for rst=%b and we get output count_mod8=%0d, count_mod4=%0d ",$time,rst,count_mod8,count_mod4);
    rst=1;
#10 rst=0;

#100 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, mod_8_tb);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1 and we get output count_mod8=x, count_mod4=x

Time=5 for rst=1 and we get output count_mod8=0, count_mod4=0

Time=10 for rst=0 and we get output count_mod8=0, count_mod4=0

Time=15 for rst=0 and we get output count_mod8=1, count_mod4=1

Time=25 for rst=0 and we get output count_mod8=2, count_mod4=2

Time=35 for rst=0 and we get output count_mod8=3, count_mod4=3

Time=45 for rst=0 and we get output count_mod8=4, count_mod4=0

Time=55 for rst=0 and we get output count_mod8=5, count_mod4=1

Time=65 for rst=0 and we get output count_mod8=6, count_mod4=2

Time=75 for rst=0 and we get output count_mod8=7, count_mod4=3

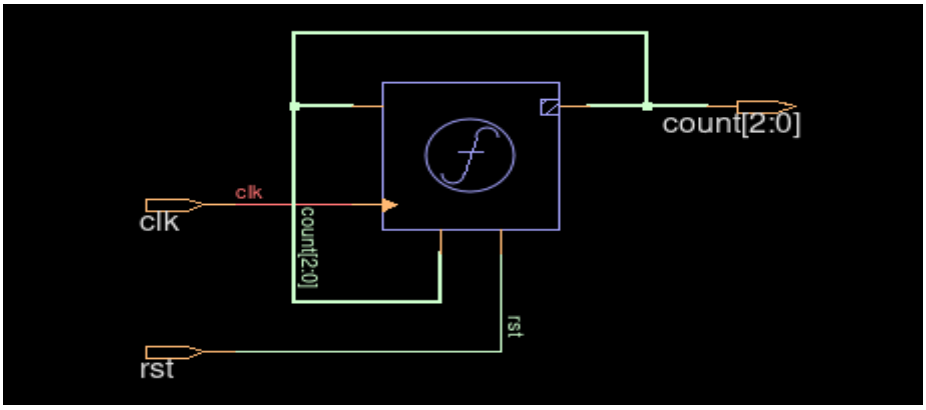
Time=85 for rst=0 and we get output count_mod8=0, count_mod4=0

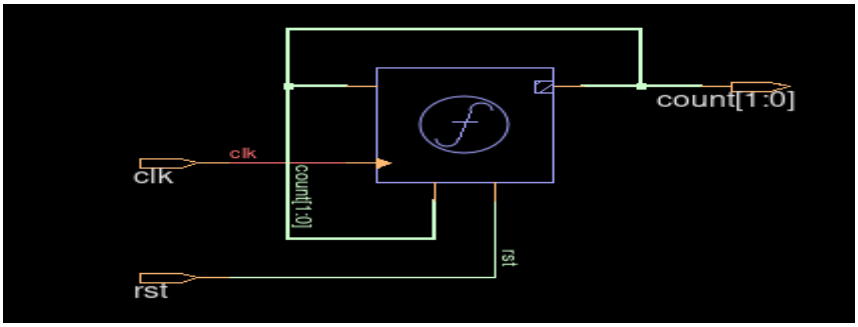
Time=95 for rst=0 and we get output count_mod8=1, count_mod4=1

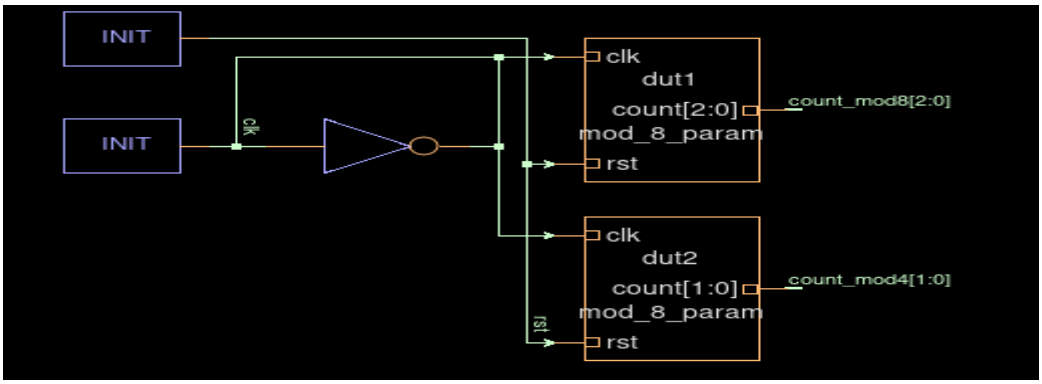
Time=105 for rst=0 and we get output count_mod8=2, count_mod4=2

$finish at simulation time 110

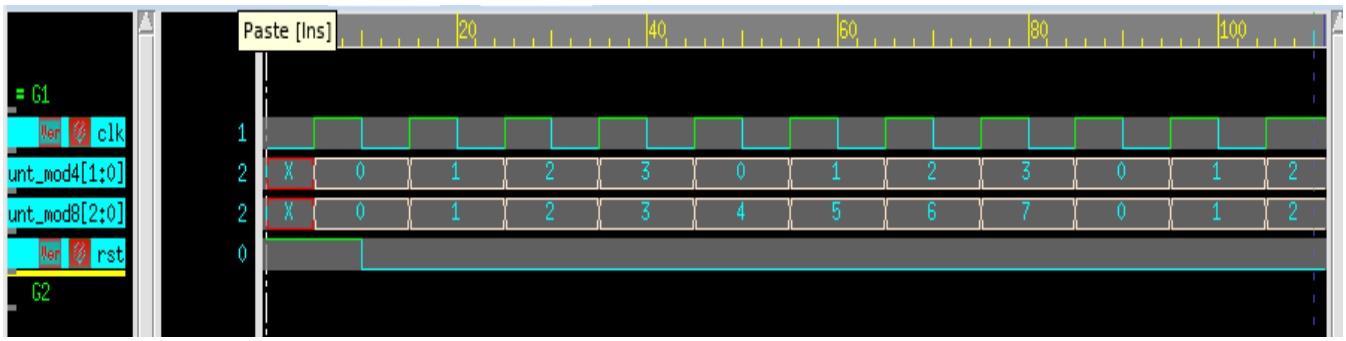
Schematic Design







waveform



73.0 Verilog code for a mod-N counter

Design code

```
//mod-N counter by using parameter
module mod_N_counter #(parameter N=10, parameter WIDTH=4)(
        input clk,rst,
        output reg [WIDTH-1:0] count);


always @(posedge clk)
begin
```

```
    if (rst==1)
        count <=0;
    else
        count <=(count ==N-1) ? 0 : count+1;
end
endmodule
```

Testbench

```
module mod_N_tb();
parameter N=10;
parameter WIDTH=4;

reg clk,rst;
wire [WIDTH-1:0] count;

initial begin
    clk=0;
end
always #5 clk=~clk;

mod_N_counter dut(.clk(clk),.rst(rst),.count(count));

initial begin
    $monitor("Time=%0t for rst=%b and we get output count=%0d ",$time,rst,count);
    rst=1;
#10 rst=0;
repeat(15)@(posedge clk);
```

```
#10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, mod_N_tb);
end
endmodule
```

Simulation Result

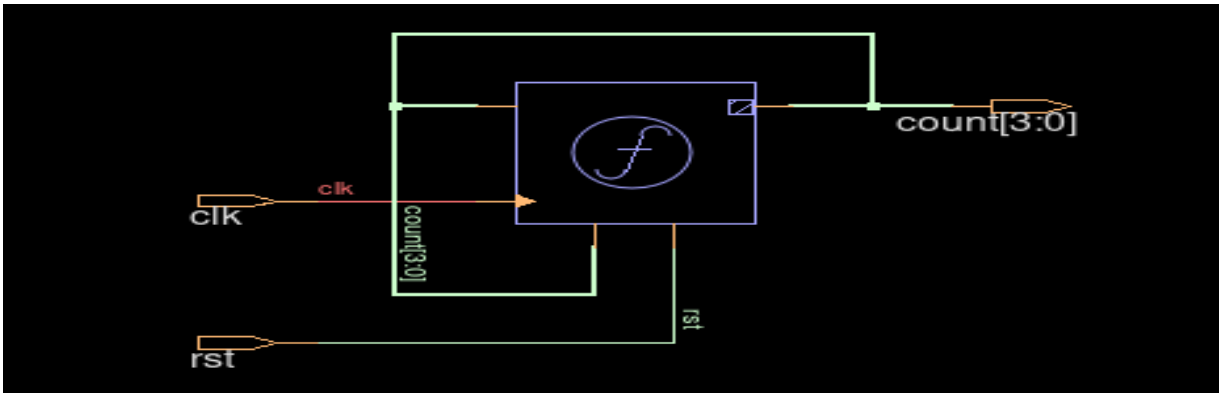
Output

```
Time=0 for rst=1 and we get output count=x
Time=5 for rst=1 and we get output count=0
Time=10 for rst=0 and we get output count=0
Time=15 for rst=0 and we get output count=1
Time=25 for rst=0 and we get output count=2
Time=35 for rst=0 and we get output count=3
Time=45 for rst=0 and we get output count=4
Time=55 for rst=0 and we get output count=5
Time=65 for rst=0 and we get output count=6
Time=75 for rst=0 and we get output count=7
Time=85 for rst=0 and we get output count=8
Time=95 for rst=0 and we get output count=9
Time=105 for rst=0 and we get output count=0
Time=115 for rst=0 and we get output count=1
Time=125 for rst=0 and we get output count=2
Time=135 for rst=0 and we get output count=3
Time=145 for rst=0 and we get output count=4
```

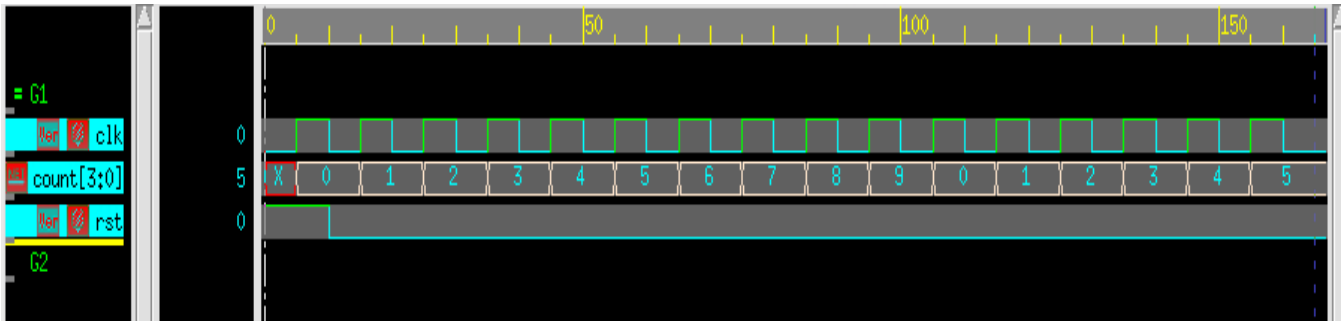
Time=155 for rst=0 and we get output count=5

$finish at simulation time 165

Schematic Design



waveform



74.0 Design a MOD-47 up-counter (0-46)

Design code

```
module mod_47_up_counter(count,clk,rst);
input clk,rst;
output reg [5:0]count;


always @(posedge clk)
begin
	if (rst==1)
		count <=6'd0;
	else if (count ==6'd46)
```

```
            count <=6'd0;
        else
            count <= count+1;
end
endmodule
```

Testbench

```
module TB();
reg clk, rst;
wire [5:0] count;

mod_47_up_counter dut(.clk(clk),.rst(rst),.count(count));
initial begin
        clk=0;
end
always #5 clk=~clk;
initial begin
        $monitor("Time=%0t,for rst=%d,we get count=%d",$time,rst,count);
        rst=1;clk=0;
#10 rst=0;

#250;
#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
```

```
end

endmodule
```

Simulation Result

Output

```
Time=0,for rst=1,we get count= x

Time=5,for rst=1,we get count= 0

Time=10,for rst=0,we get count= 0

Time=15,for rst=0,we get count= 1

Time=25,for rst=0,we get count= 2

Time=35,for rst=0,we get count= 3

Time=45,for rst=0,we get count= 4

Time=55,for rst=0,we get count= 5

Time=65,for rst=0,we get count= 6

Time=75,for rst=0,we get count= 7

Time=85,for rst=0,we get count= 8

Time=95,for rst=0,we get count= 9

Time=105,for rst=0,we get count=10

Time=115,for rst=0,we get count=11

Time=125,for rst=0,we get count=12

Time=135,for rst=0,we get count=13

Time=145,for rst=0,we get count=14

Time=155,for rst=0,we get count=15

Time=165,for rst=0,we get count=16

Time=175,for rst=0,we get count=17

Time=185,for rst=0,we get count=18

Time=195,for rst=0,we get count=19
```

Time=205,for rst=0,we get count=20

Time=215,for rst=0,we get count=21

Time=225,for rst=0,we get count=22

Time=235,for rst=0,we get count=23

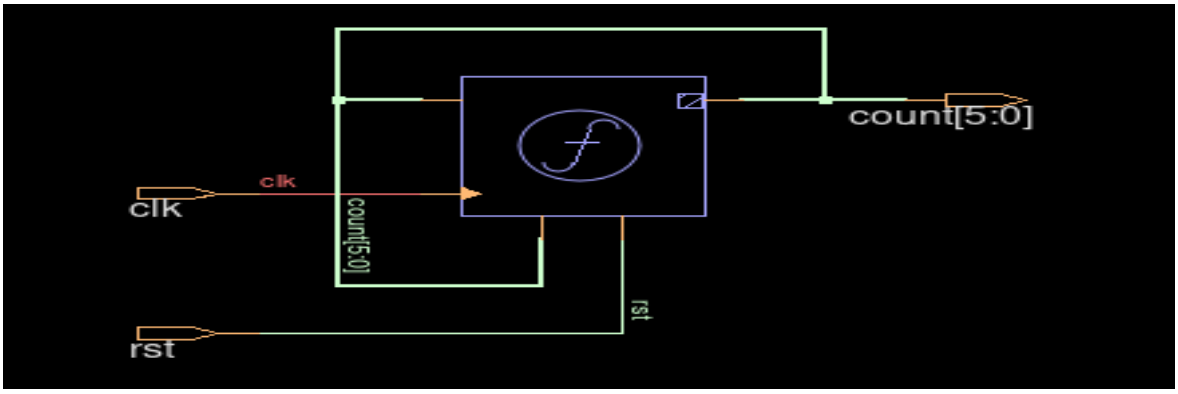
Time=245,for rst=0,we get count=24

Time=255,for rst=0,we get count=25

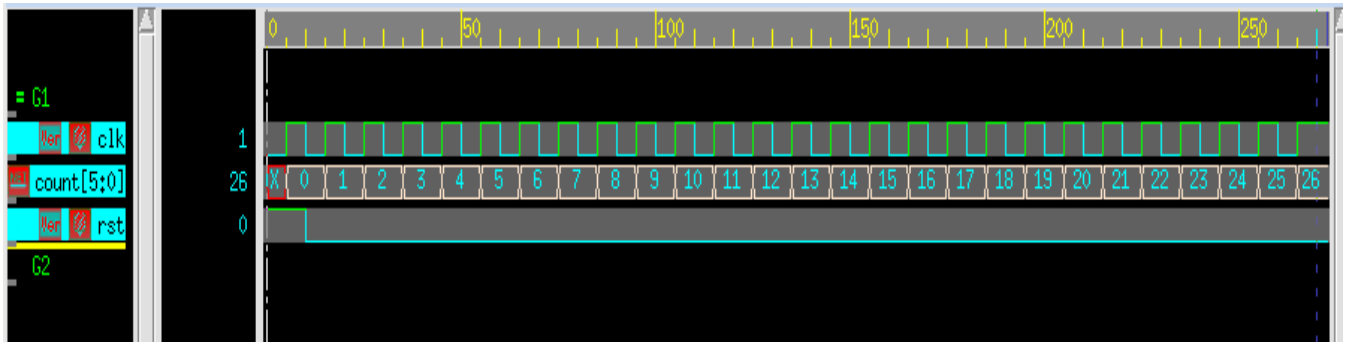
Time=265,for rst=0,we get count=26

……..till upto……=46

Schematic Design



waveform



75.0 Design a Range up counter (10-46)

Design code

module mod_10_to_40_counter(count,clk,rst);

input clk,rst;

```
output reg [5:0]count;

always @(posedge clk)
begin
	if (rst==1)
		count <=6'd10;
	else if (count ==6'd40)
		count <=6'd10;
	else
		count <= count+1;
end
endmodule
```

Testbench

```
module TB();
reg clk, rst;
wire [5:0] count;

mod_10_to_40_counter dut(.clk(clk),.rst(rst),.count(count));
initial begin
	clk=0;
end
always #5 clk=~clk;

initial begin
	$monitor("Time=%0t,for rst=%d,we get count=%d",$time,rst,count);
	rst=1;clk=0;
#10 rst=0;
```

```
#300;
#10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

```
Time=0,for rst=1,we get count= x
Time=5,for rst=1,we get count=10
Time=10,for rst=0,we get count=10
Time=15,for rst=0,we get count=11
Time=25,for rst=0,we get count=12
Time=35,for rst=0,we get count=13
Time=45,for rst=0,we get count=14
Time=55,for rst=0,we get count=15
Time=65,for rst=0,we get count=16
Time=75,for rst=0,we get count=17
Time=85,for rst=0,we get count=18
Time=95,for rst=0,we get count=19
Time=105,for rst=0,we get count=20
Time=115,for rst=0,we get count=21
Time=125,for rst=0,we get count=22
```

Time=135,for rst=0,we get count=23

Time=145,for rst=0,we get count=24

Time=155,for rst=0,we get count=25

Time=165,for rst=0,we get count=26

Time=175,for rst=0,we get count=27

Time=185,for rst=0,we get count=28

Time=195,for rst=0,we get count=29

Time=205,for rst=0,we get count=30

Time=215,for rst=0,we get count=31

Time=225,for rst=0,we get count=32

Time=235,for rst=0,we get count=33

Time=245,for rst=0,we get count=34

Time=255,for rst=0,we get count=35

Time=265,for rst=0,we get count=36

Time=275,for rst=0,we get count=37

Time=285,for rst=0,we get count=38

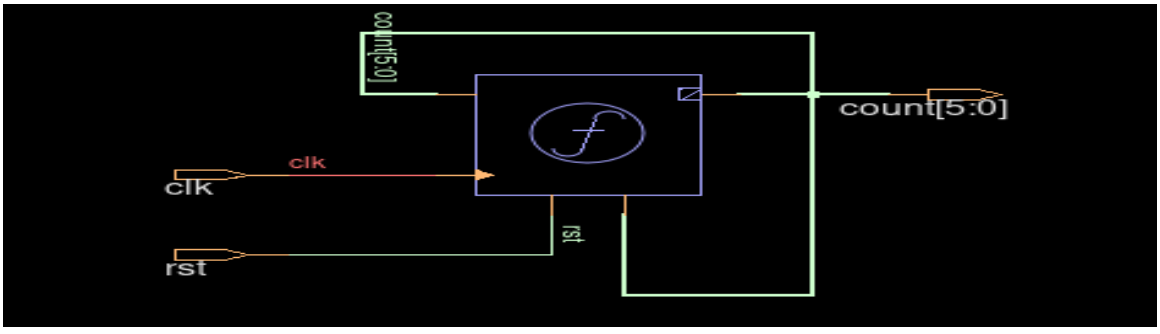
Time=295,for rst=0,we get count=39

Time=305,for rst=0,we get count=40

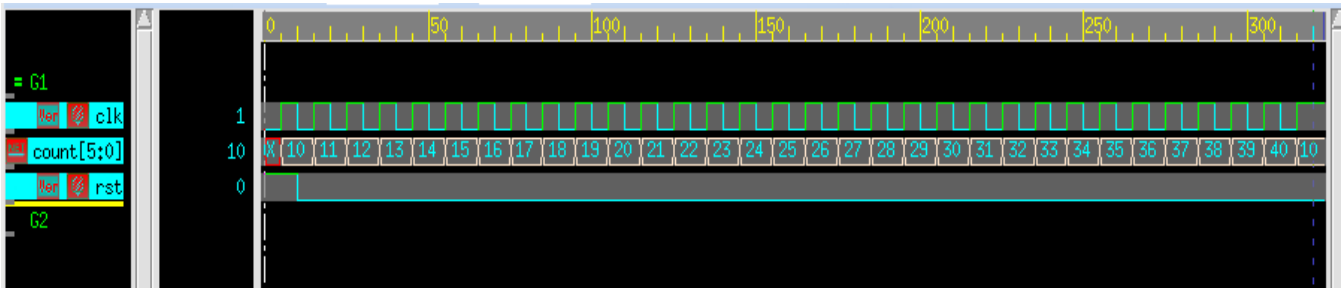
Time=315,for rst=0,we get count=10

$finish at simulation time 320

Schematic Design



waveform



76.0 Design a Range (10-40) up-down counter with load data (6-bit) operation-

Design code

```
module mod_10_to_40_counter_up_down(count,u_d,load,clk,rst,data);
input clk,rst,load,u_d;
input [5:0]data;
output reg [5:0]count;

always @(posedge clk )begin
    if (rst==1)
        count <=6'd10;
    else if (load)
        count <=data;
    else if (u_d)
        count <=(count >=40) ? 6'd10 : count +1;
    else
        count <=(count <=10) ? 6'd40 : count -1;
end
endmodule
```

Testbench

```
module TB();
```

```
reg clk,rst,load,u_d;
reg [5:0]data;
wire [5:0] count;

initial begin
	clk=0;
end
always #5 clk=~clk;

mod_10_to_40_counter_up_down
dut(.count(count),.u_d(u_d),.load(load),.clk(clk),.rst(rst),.data(data));

initial begin
	$monitor("Time=%0d for rst=%0d load=%0d data=%d u_d=%0b and we get output
count=%d ",$time,rst,load,data,u_d,count);
	rst=1;load=0;u_d=0;data=6'd0;

#10 rst=0;
#10 load=1;data=6'd30;

#10 load=0;
u_d=1;
repeat(5) #10;

u_d=0;
repeat(5) #10;

#100 $finish();
```

```
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for rst=1 load=0 data= 0 u_d=0 and we get output count= x
Time=5 for rst=1 load=0 data= 0 u_d=0 and we get output count=10
Time=10 for rst=0 load=0 data= 0 u_d=0 and we get output count=10
Time=15 for rst=0 load=0 data= 0 u_d=0 and we get output count=40
Time=20 for rst=0 load=1 data=30 u_d=0 and we get output count=40
Time=25 for rst=0 load=1 data=30 u_d=0 and we get output count=30
Time=30 for rst=0 load=0 data=30 u_d=1 and we get output count=30
Time=35 for rst=0 load=0 data=30 u_d=1 and we get output count=31
Time=45 for rst=0 load=0 data=30 u_d=1 and we get output count=32
Time=55 for rst=0 load=0 data=30 u_d=1 and we get output count=33
Time=65 for rst=0 load=0 data=30 u_d=1 and we get output count=34
Time=75 for rst=0 load=0 data=30 u_d=1 and we get output count=35
Time=80 for rst=0 load=0 data=30 u_d=0 and we get output count=35
Time=85 for rst=0 load=0 data=30 u_d=0 and we get output count=34
Time=95 for rst=0 load=0 data=30 u_d=0 and we get output count=33
Time=105 for rst=0 load=0 data=30 u_d=0 and we get output count=32
Time=115 for rst=0 load=0 data=30 u_d=0 and we get output count=31
Time=125 for rst=0 load=0 data=30 u_d=0 and we get output count=30
```

Time=135 for rst=0 load=0 data=30 u_d=0 and we get output count=29

Time=145 for rst=0 load=0 data=30 u_d=0 and we get output count=28

Time=155 for rst=0 load=0 data=30 u_d=0 and we get output count=27

Time=165 for rst=0 load=0 data=30 u_d=0 and we get output count=26

Time=175 for rst=0 load=0 data=30 u_d=0 and we get output count=25

Time=185 for rst=0 load=0 data=30 u_d=0 and we get output count=24

Time=195 for rst=0 load=0 data=30 u_d=0 and we get output count=23

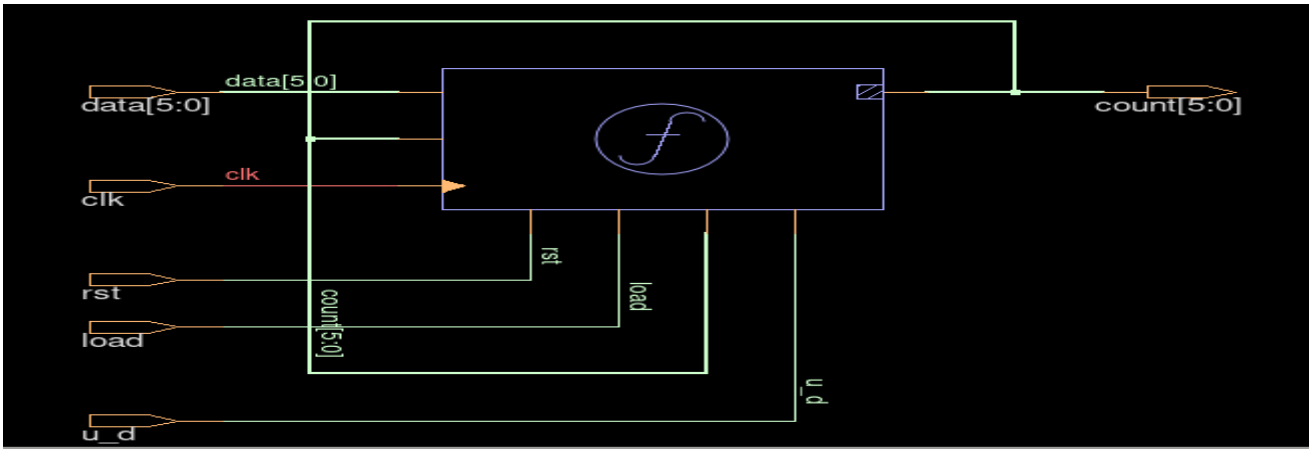
Time=205 for rst=0 load=0 data=30 u_d=0 and we get output count=22

Time=215 for rst=0 load=0 data=30 u_d=0 and we get output count=21

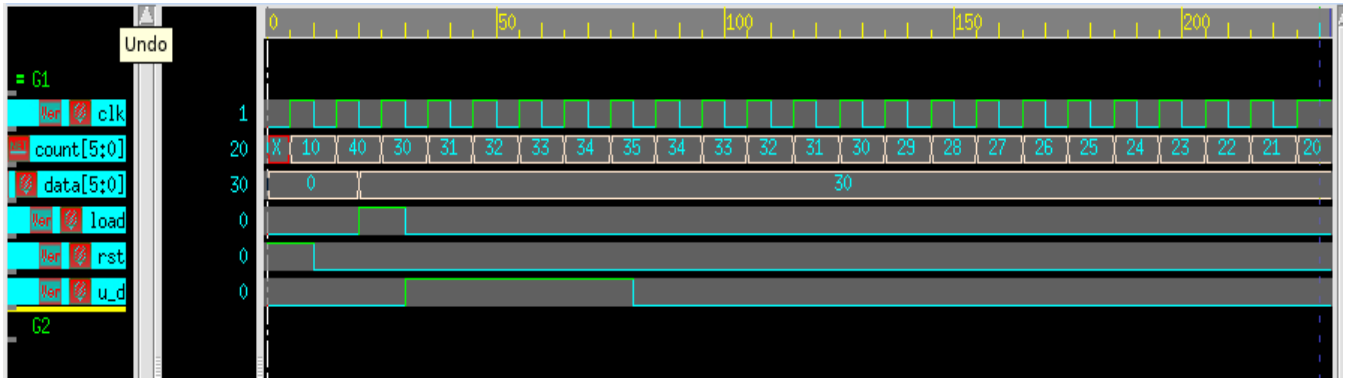
Time=225 for rst=0 load=0 data=30 u_d=0 and we get output count=20

$finish at simulation time 230

Schematic Design



waveform



77.0 8-bit bidirectional barrel shift register (left and right) with a shift control input-

Design code

```
module Barrel_Shifter_8bit(
input [7:0]din,
input [2:0]n,
input Lr,
output reg [7:0]dout
   );

   always @(*)begin
         if(Lr==1)
               dout = din <<n;
         else
               dout = din >>n;
   end
endmodule
```

Testbench

```
module TB();
reg [7:0]din;
reg [2:0]n;
reg Lr;
wire [7:0]dout;

Barrel_Shifter_8bit dut(.din(din),.n(n),.Lr(Lr),.dout(dout));
initial begin
```

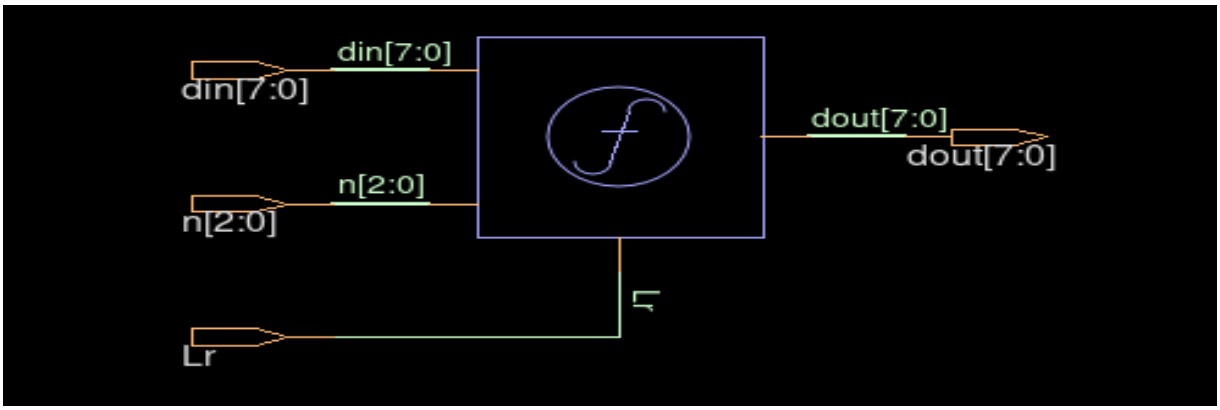
```
        $monitor("Time=%0t for din=%b n=%d Lr=%b and we get output dout=%b
",$time,din,n,Lr,dout);
        din =8'b00001001; n=0;
        #10 Lr=1;  n=1;
        #10 Lr=1;  n=2;
        #10 Lr=1;  n=3;
        #10 Lr=0;  n=1;
        #10 Lr=0;  n=2;
        #10 Lr=0;  n=3;
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

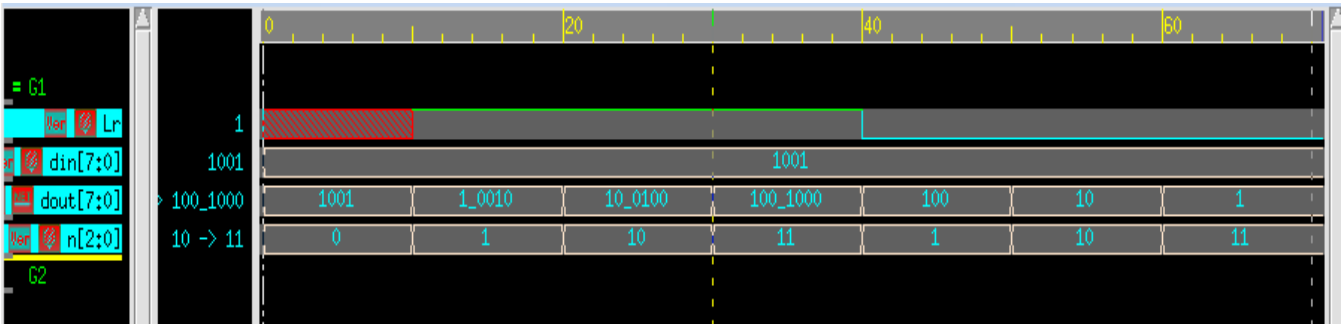
Output

```
Time=0 for din=00001001 n=0 Lr=x and we get output dout=00001001
Time=10 for din=00001001 n=1 Lr=1 and we get output dout=00010010
Time=20 for din=00001001 n=2 Lr=1 and we get output dout=00100100
Time=30 for din=00001001 n=3 Lr=1 and we get output dout=01001000
Time=40 for din=00001001 n=1 Lr=0 and we get output dout=00000100
Time=50 for din=00001001 n=2 Lr=0 and we get output dout=00000010
Time=60 for din=00001001 n=3 Lr=0 and we get output dout=00000001
$finish at simulation time
```

Schematic Design



waveform



78.0 8-bit twin register set with independent storage (parallel loading) -

Design code

```
module twin_8bit_register(q1,q2,clk,rst,d1,d2);
input clk;
input rst;
input [7:0] d1,d2;
output reg [7:0] q1,q2;

always @(posedge clk)begin
    if (rst)begin
        q1 <=8'd0;
        q2 <=8'd0;
```

```
    end else begin
        q1 <=d1;
        q2 <=d2;
    end
end
endmodule
```

Testbench

```
module TB();
reg clk,rst;
reg [7:0]d1,d2;
wire [7:0]q1,q2;

twin_8bit_register dut(.clk(clk),.rst(rst),.d1(d1),.d2(d2),.q1(q1),.q2(q2));

initial clk=0;
always #5 clk=~clk;
initial begin
    $monitor("Time=%0t for rst=%b d1=%h d2=%h and we get q1=%h q2=%h
",$time,rst,d1,d2,q1,q2);
    rst = 1; d1 = 8'h00; d2 = 8'h00;
#10 rst = 0; d1 = 8'hAA; d2 = 8'h55;
#10 d1 = 8'hF0; d2 = 8'h0F;
#10 rst = 1;
#10 rst = 0; d1 = 8'hCC; d2 = 8'h33;
#20 $finish();
end
initial begin
```

```
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1 d1=00 d2=00 and we get q1=xx q2=xx

Time=5 for rst=1 d1=00 d2=00 and we get q1=00 q2=00

Time=10 for rst=0 d1=aa d2=55 and we get q1=00 q2=00

Time=15 for rst=0 d1=aa d2=55 and we get q1=aa q2=55

Time=20 for rst=0 d1=f0 d2=0f and we get q1=aa q2=55

Time=25 for rst=0 d1=f0 d2=0f and we get q1=f0 q2=0f

Time=30 for rst=1 d1=f0 d2=0f and we get q1=f0 q2=0f

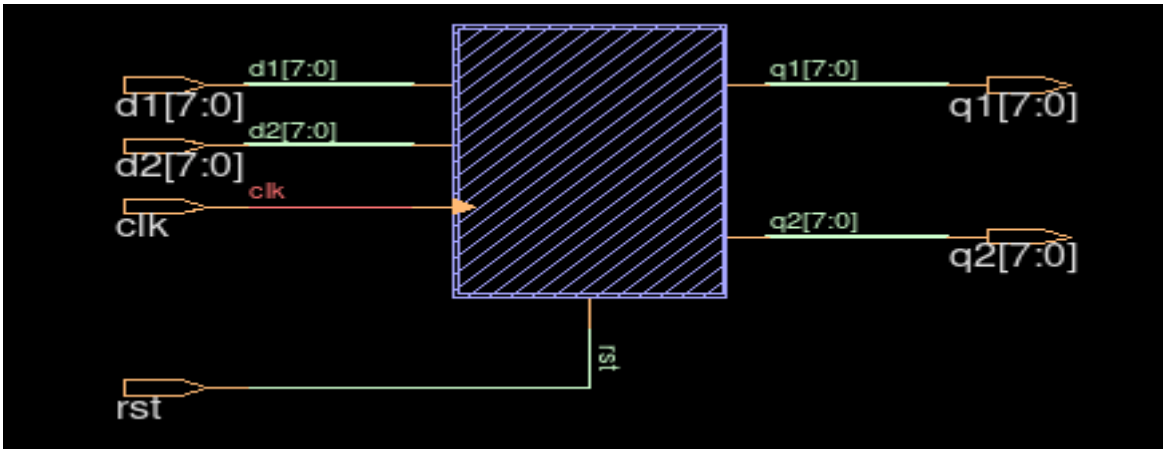
Time=35 for rst=1 d1=f0 d2=0f and we get q1=00 q2=00

Time=40 for rst=0 d1=cc d2=33 and we get q1=00 q2=00

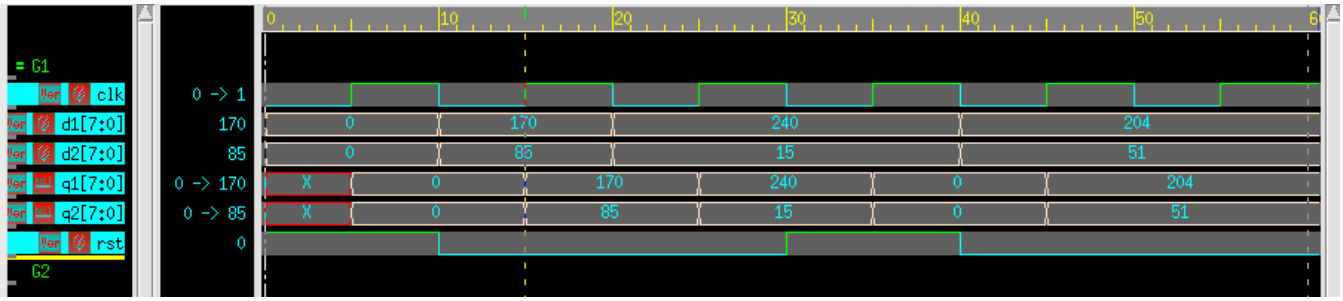
Time=45 for rst=0 d1=cc d2=33 and we get q1=cc q2=33

$finish at simulation time 60

Schematic Design



waveform



79.0 Design an 8-bit Universal shift register

No operation, shift left, shift right, parallel load operation

Design code

```
module Universal_shift_register(data_out,s0,data_in,sel,clk,rst,sin);
input clk,rst,sin;
input [7:0] data_in;
input [1:0]sel;
output reg [7:0]data_out;
output s0;

always @(posedge clk)begin
    if(rst)
        data_out <= 8'd0;
    else begin
        case(sel)
            2'b00: data_out = data_out;
            2'b01: data_out = {sin,data_out[7:1]};
            2'b10: data_out = {data_out[6:0],sin};
            2'b11: data_out = data_in ;
            default: data_out=0;
        endcase
    end
```

```
end
assign s0 =(sel==2'b10) ? data_out[7] : data_out[0];
endmodule
```

Testbench

```
module TB();
reg clk,rst,sin;
reg [7:0] data_in;
reg [1:0]sel;
wire [7:0]data_out;
wire s0;

Universal_shift_register
dut(.clk(clk),.rst(rst),.sin(sin),.data_in(data_in),.sel(sel),.data_out(data_out),.s0(s0));

initial clk =0;
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t for rst=%b sin=%b data_in=%h sel=%b and we get
data_out=%b s0=%b ",$time,rst,sin,data_in,sel,data_out,s0);
    rst=1;sel = 2'b00;data_in = 8'h00;sin=0;

#10 rst = 0;
sel = 2'b11; data_in = 8'b10101010; #10;

sel = 2'b01; sin = 1; #10;
sel = 2'b01; sin = 0; #10;
```

```
sel = 2'b10; sin = 1; #10;
sel = 2'b10; sin = 0; #10;

sel = 2'b00; #10;
sel = 2'b11; data_in = 8'b11110000; #10;

sel = 2'b00; #10;
#10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for rst=1 sin=0 data_in=00 sel=00 and we get data_out=xxxxxxxx s0=x
Time=5 for rst=1 sin=0 data_in=00 sel=00 and we get data_out=00000000 s0=0
Time=10 for rst=0 sin=0 data_in=aa sel=11 and we get data_out=00000000 s0=0
Time=15 for rst=0 sin=0 data_in=aa sel=11 and we get data_out=10101010 s0=0
Time=20 for rst=0 sin=1 data_in=aa sel=01 and we get data_out=10101010 s0=0
Time=25 for rst=0 sin=1 data_in=aa sel=01 and we get data_out=11010101 s0=1
Time=30 for rst=0 sin=0 data_in=aa sel=01 and we get data_out=11010101 s0=1
Time=35 for rst=0 sin=0 data_in=aa sel=01 and we get data_out=01101010 s0=0
Time=40 for rst=0 sin=1 data_in=aa sel=10 and we get data_out=01101010 s0=0
Time=45 for rst=0 sin=1 data_in=aa sel=10 and we get data_out=11010101 s0=1
```

Time=50 for rst=0 sin=0 data_in=aa sel=10 and we get data_out=11010101 s0=1

Time=55 for rst=0 sin=0 data_in=aa sel=10 and we get data_out=10101010 s0=1

Time=60 for rst=0 sin=0 data_in=aa sel=00 and we get data_out=10101010 s0=0

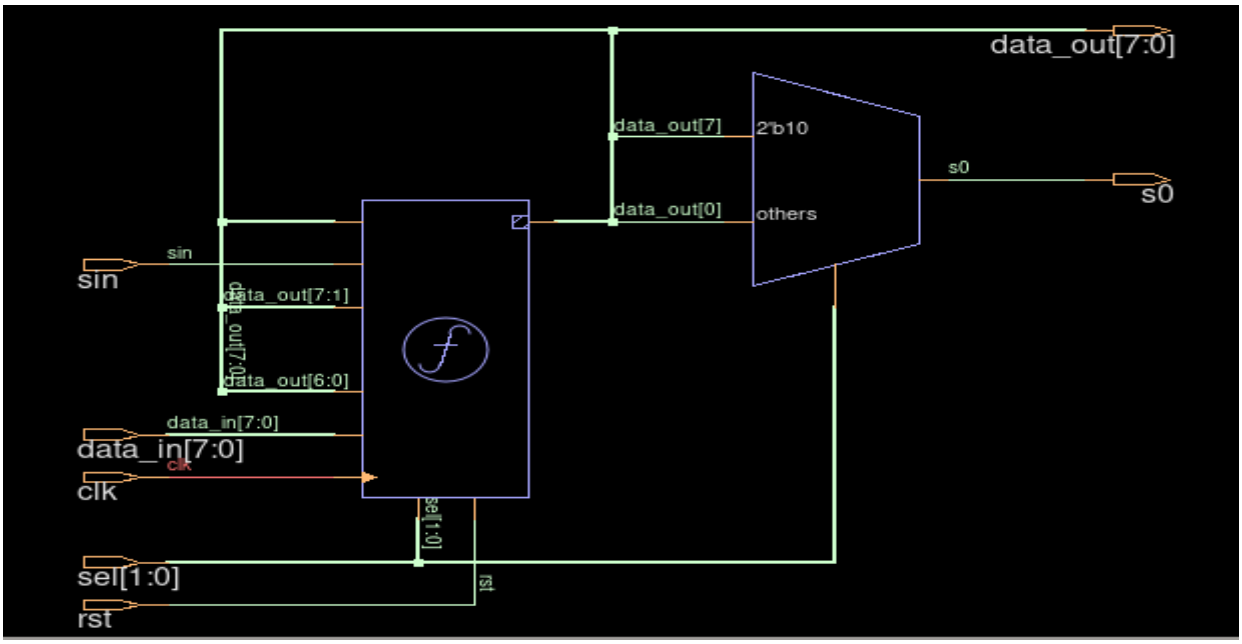
Time=70 for rst=0 sin=0 data_in=f0 sel=11 and we get data_out=10101010 s0=0

Time=75 for rst=0 sin=0 data_in=f0 sel=11 and we get data_out=11110000 s0=0

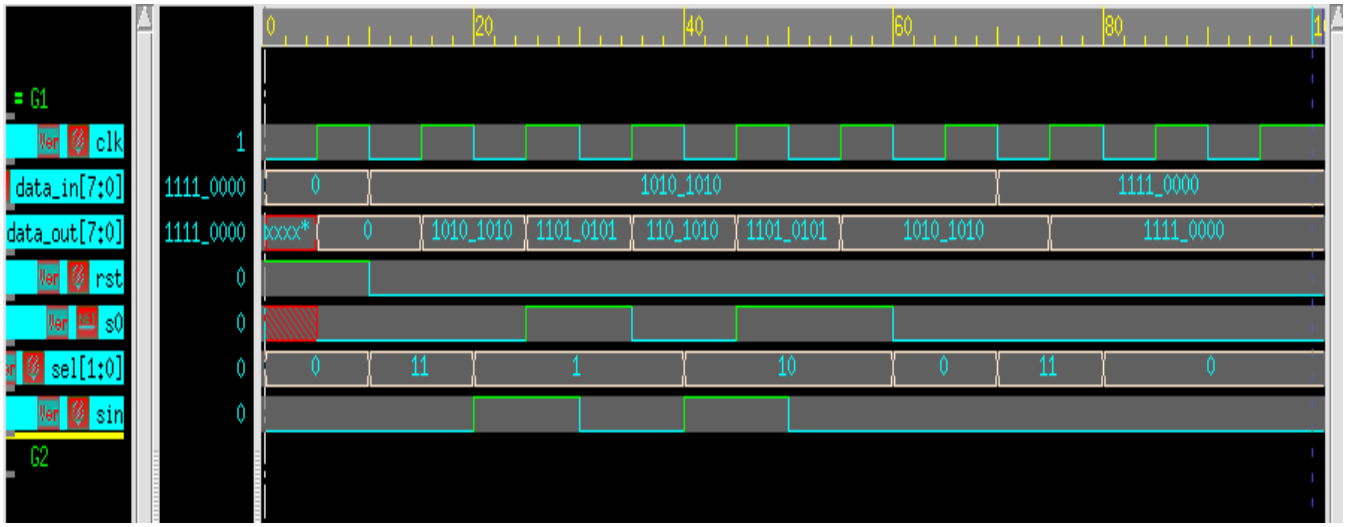
Time=80 for rst=0 sin=0 data_in=f0 sel=00 and we get data_out=11110000 s0=0

$finish at simulation time 100

Schematic Design



waveform



80.0 Design a 4-bit Left shift register (LSR)

Design code

```
module Left_shift_register(q,s0,din,load,clk,rst,sin);
input clk,rst,load,sin;
input [3:0]din;
output reg [3:0]q;
output s0;

always @(posedge clk or posedge rst)begin
	if(rst)
		q<=4'b0000;
	else if(load)
		q<=din;
	else
		q<={q[2:0],sin};
end
assign s0 = q[3];

endmodule
```

Testbench

```
module TB();
reg clk,rst,sin,load;
reg [3:0] din;
wire [3:0]q;
wire s0;
Left_shift_register dut(.clk(clk),.rst(rst),.sin(sin),.load(load),.din(din),.q(q),.s0(s0));
```

```
initial clk =0;
always #5 clk=~clk;
initial begin
	$monitor("Time=%0t for rst=%b sin=%b din=%b load=%b and we get q=%b s0=%b ",$time,rst,sin,din,load,q,s0);
	rst=1;sin=0;
#10 rst = 0;
load=1;
din=4'b1001;

#10
load=0;
din=4'b1010;

#10
load=0;
sin=1;
din=4'b1110;

#10 $finish();
end
initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1 sin=0 din=xxxx load=x and we get q=0000 s0=0

Time=10 for rst=0 sin=0 din=1001 load=1 and we get q=0000 s0=0

Time=15 for rst=0 sin=0 din=1001 load=1 and we get q=1001 s0=1

Time=20 for rst=0 sin=0 din=1010 load=0 and we get q=1001 s0=1

Time=25 for rst=0 sin=0 din=1010 load=0 and we get q=0010 s0=0

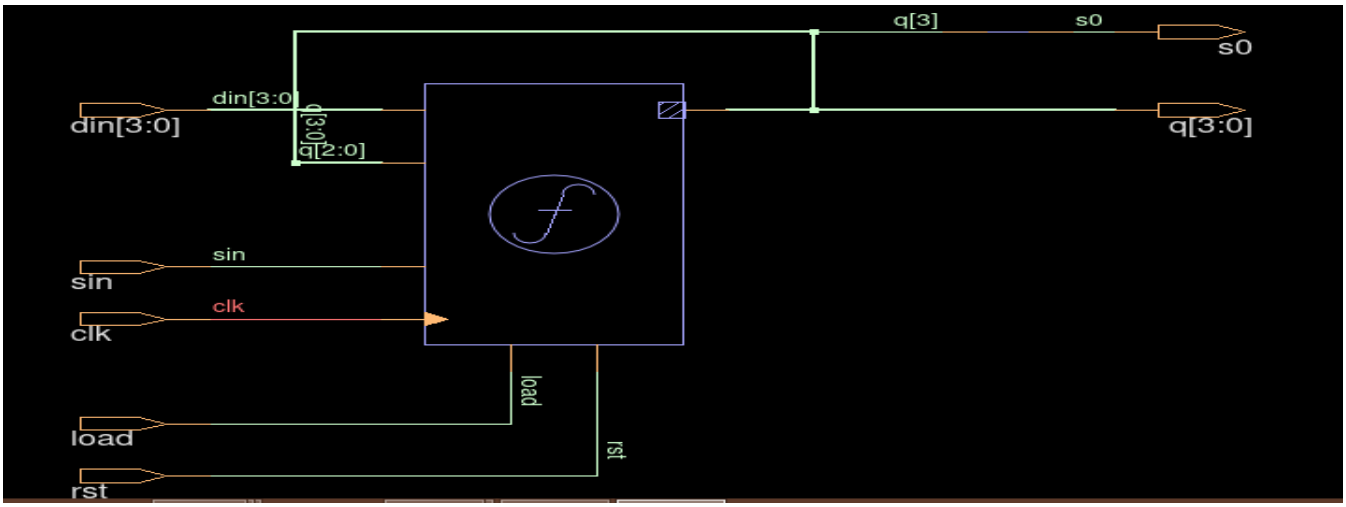
Time=30 for rst=0 sin=1 din=1110 load=0 and we get q=0010 s0=0

Time=35 for rst=0 sin=1 din=1110 load=0 and we get q=0101 s0=0

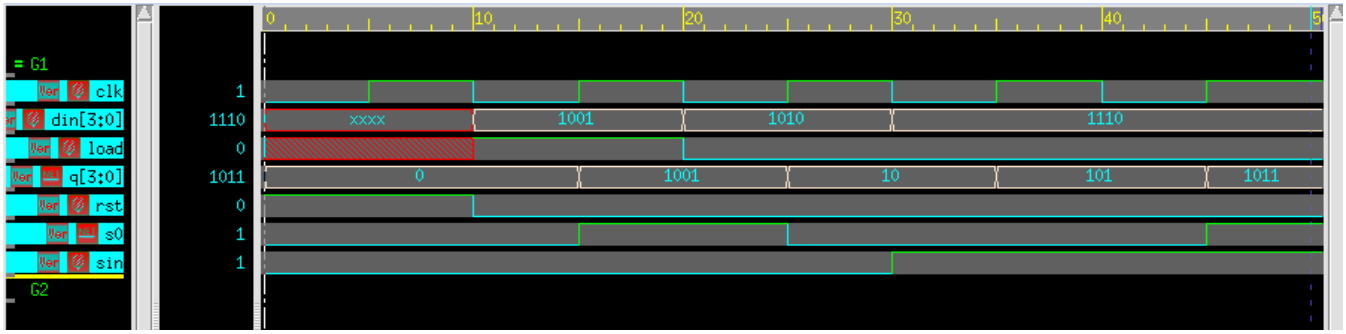
Time=45 for rst=0 sin=1 din=1110 load=0 and we get q=1011 s0=1

$finish at simulation time 50

Schematic Design



waveform



81.0 Design a 4-bit Right shift register (RSR)

Design code

```
module Right_shift_register(q,s0,din,load,clk,rst,sin);
input clk,rst,load,sin;
input [3:0]din;
output reg [3:0]q;
output s0;

always @(posedge clk or posedge rst)begin
	if(rst)
		q<=4'b0000;
	else if(load)
		q<=din;
	else
		q<={sin,q[3:1]};
end
assign s0 = q[0];
endmodule
```

Testbench

```
module TB();
reg clk,rst,sin,load;
reg [3:0] din;
wire [3:0]q;
wire s0;

Right_shift_register dut(.clk(clk),.rst(rst),.sin(sin),.load(load),.din(din),.q(q),.s0(s0));
```

```
initial clk =0;
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t for rst=%b sin=%b din=%b load=%b and we get q=%b s0=%b ",$time,rst,sin,din,load,q,s0);
    rst=1;sin=0;

#10 rst = 0;
load=1;
din=4'b1001;

#10
load=0;
din=4'b1010;

#10
load=0;
sin=1;
din=4'b1110;
#10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1 sin=0 din=xxxx load=x and we get q=0000 s0=0

Time=10 for rst=0 sin=0 din=1001 load=1 and we get q=0000 s0=0

Time=15 for rst=0 sin=0 din=1001 load=1 and we get q=1001 s0=1

Time=20 for rst=0 sin=0 din=1010 load=0 and we get q=1001 s0=1

Time=25 for rst=0 sin=0 din=1010 load=0 and we get q=0100 s0=0

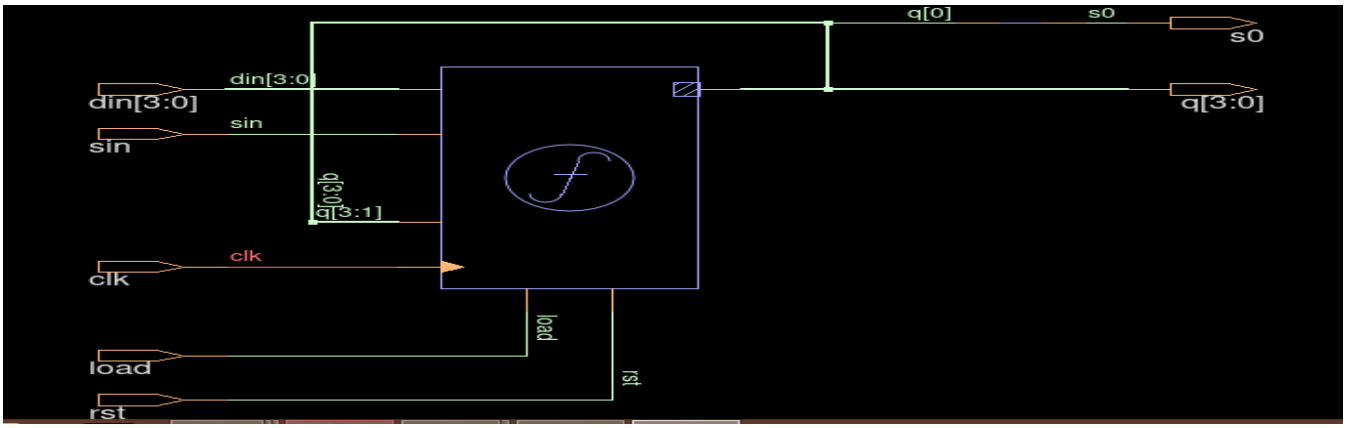
Time=30 for rst=0 sin=1 din=1110 load=0 and we get q=0100 s0=0

Time=35 for rst=0 sin=1 din=1110 load=0 and we get q=1010 s0=0

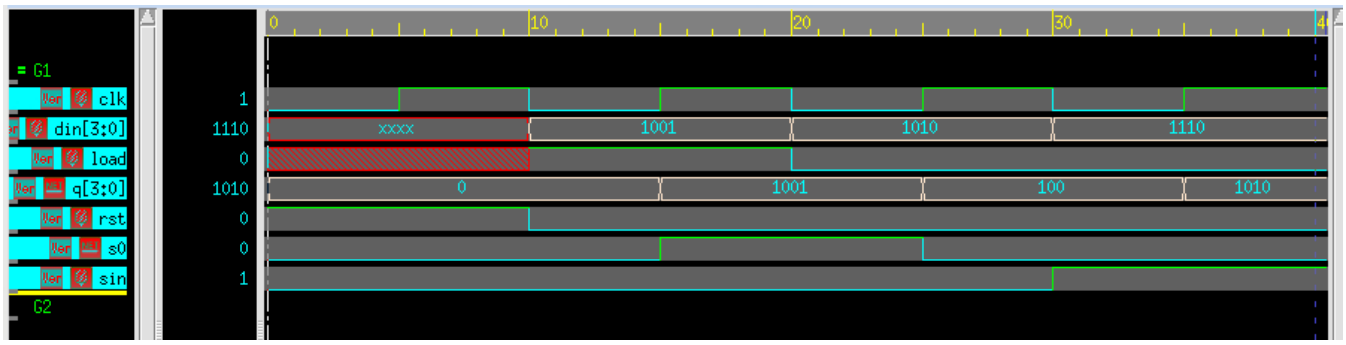
$finish called from file "nettype_tri_tb.v", line 31.

$finish at simulation time 40

Schematic Design



waveform



82.0 Design a universal_shift_register2-

Design code

```
module universal_shift_register_2(q,din,sin,sel,load,clk,rst);
input clk,rst,load,sin;
input [1:0]sel;
input [3:0]din;
output reg [3:0]q=0;

always @(posedge clk or posedge rst)begin
	if(rst)
		q<=4'b0000;
	else if(load)
		q<=din;
	else if(sel==2'b01)
		q<={sin,q[3:1]};
	else if(sel==2'b10)
		q<={q[2:0],sin};
	else if(sel==2'b11)
		q<=din;
end
endmodule
```

Testbench

```
module TB();
reg clk,rst,sin,load;
reg [3:0] din;
reg [1:0] sel;
```

```
wire [3:0]q;


universal_shift_register_2
dut(.clk(clk),.rst(rst),.sin(sin),.load(load),.din(din),.sel(sel),.q(q));


initial clk =0;

always #5 clk=~clk;

initial begin

	$monitor("Time=%0t for rst=%b sin=%b din=%b load=%b sel=%b and we get q=%b",$time,rst,sin,din,load,sel,q);

	rst=1;sin=0;


#10;rst =0;

load=1;

din=4'b1101;

sel = 2'b01;


#10

load=0;

din=4'b1101;

sel = 2'b01;


#10

load=1;

din=4'b1101;

sel = 2'b10;


#10
```

```
load=0;
din=4'b1101;
sel = 2'b10;

#10
load=1;
din=4'b1101;
sel = 2'b11;

#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

Time=0 for rst=1 sin=0 din=xxxx load=x sel=xx and we get q=0000

Time=10 for rst=0 sin=0 din=1101 load=1 sel=01 and we get q=0000

Time=15 for rst=0 sin=0 din=1101 load=1 sel=01 and we get q=1101

Time=20 for rst=0 sin=0 din=1101 load=0 sel=01 and we get q=1101

Time=25 for rst=0 sin=0 din=1101 load=0 sel=01 and we get q=0110

Time=30 for rst=0 sin=0 din=1101 load=1 sel=10 and we get q=0110

Time=35 for rst=0 sin=0 din=1101 load=1 sel=10 and we get q=1101

Time=40 for rst=0 sin=0 din=1101 load=0 sel=10 and we get q=1101

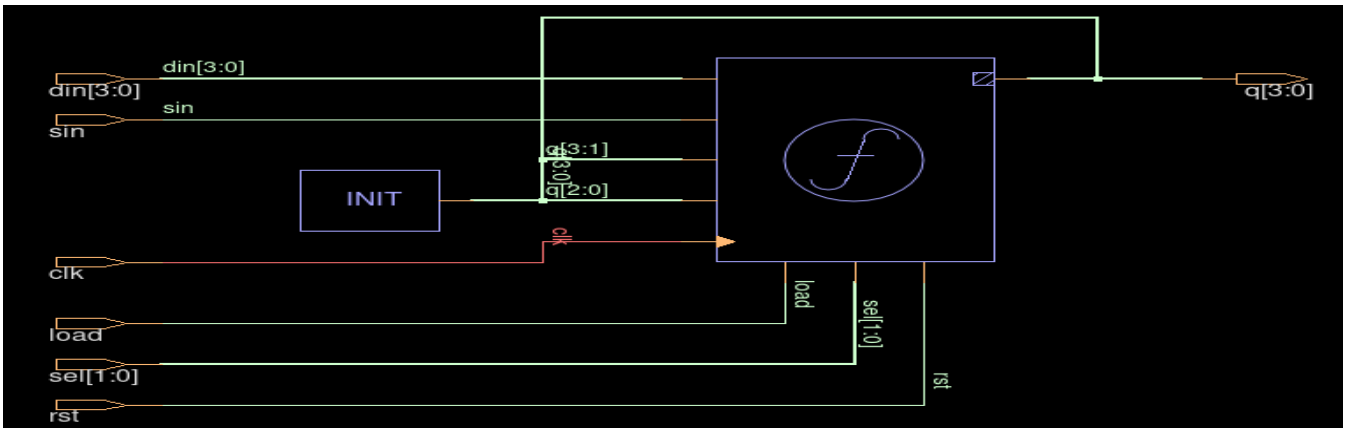
Time=45 for rst=0 sin=0 din=1101 load=0 sel=10 and we get q=1010

Time=50 for rst=0 sin=0 din=1101 load=1 sel=11 and we get q=1010

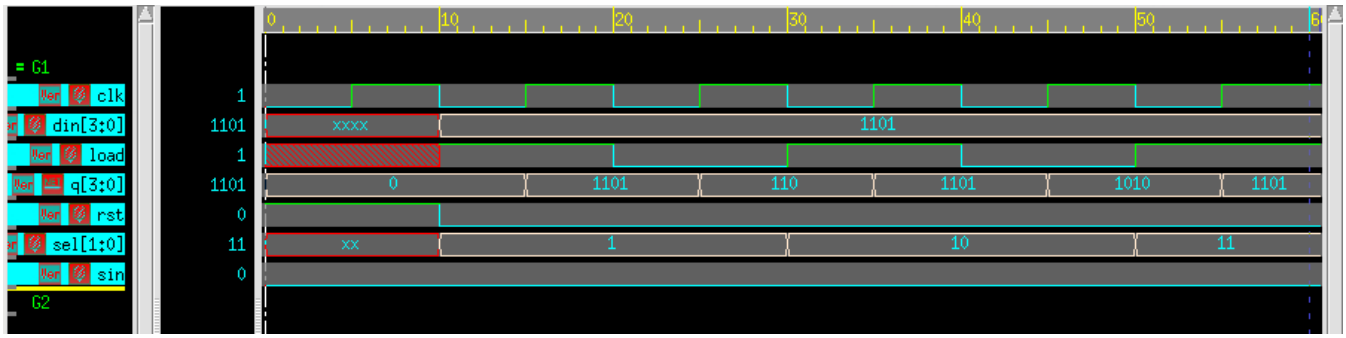
Time=55 for rst=0 sin=0 din=1101 load=1 sel=11 and we get q=1101

$finish at simulation time 60

Schematic Design



waveform



83.0 Left_circular_shift_register_using_loop (load and reset)

Design code

module Left_shift_register_loop(dout,din,load,clk,rst);

input clk,rst,load;

input [7:0]din;

output reg [7:0]dout;

```
integer i;
always @(posedge clk)begin
	if(rst)
		dout <=0;
	else begin
		if(load==1)
			dout <=din;
		else begin
			for(i=0;i<8;i=i+1)
				dout[i+1] <= dout[i];
			dout[0]<= dout[7];
		end
	end
end
endmodule
```

Testbench

```
module TB();
reg clk,rst,load;
reg [7:0] din;
wire [7:0]dout;

Left_shift_register_loop dut(.clk(clk),.rst(rst),.load(load),.din(din),.dout(dout));
initial clk =0;
always #5 clk=~clk;

initial begin
```

```
        $monitor("Time=%0t for rst=%b din=%b load=%b and we get
dout=%b",$time,rst,din,load,dout);
        rst=1;din=8'h00;


#10;rst =0;
din=8'h01;
load=0;

#5 load=1;
#5 load=0;
#100 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for rst=1 din=00000000 load=x and we get dout=xxxxxxxx
Time=5 for rst=1 din=00000000 load=x and we get dout=00000000
Time=10 for rst=0 din=00000001 load=0 and we get dout=00000000
Time=15 for rst=0 din=00000001 load=1 and we get dout=00000001
Time=20 for rst=0 din=00000001 load=0 and we get dout=00000001
Time=25 for rst=0 din=00000001 load=0 and we get dout=00000010
Time=35 for rst=0 din=00000001 load=0 and we get dout=00000100
```

Time=45 for rst=0 din=00000001 load=0 and we get dout=00001000

Time=55 for rst=0 din=00000001 load=0 and we get dout=00010000

Time=65 for rst=0 din=00000001 load=0 and we get dout=00100000

Time=75 for rst=0 din=00000001 load=0 and we get dout=01000000

Time=85 for rst=0 din=00000001 load=0 and we get dout=10000000

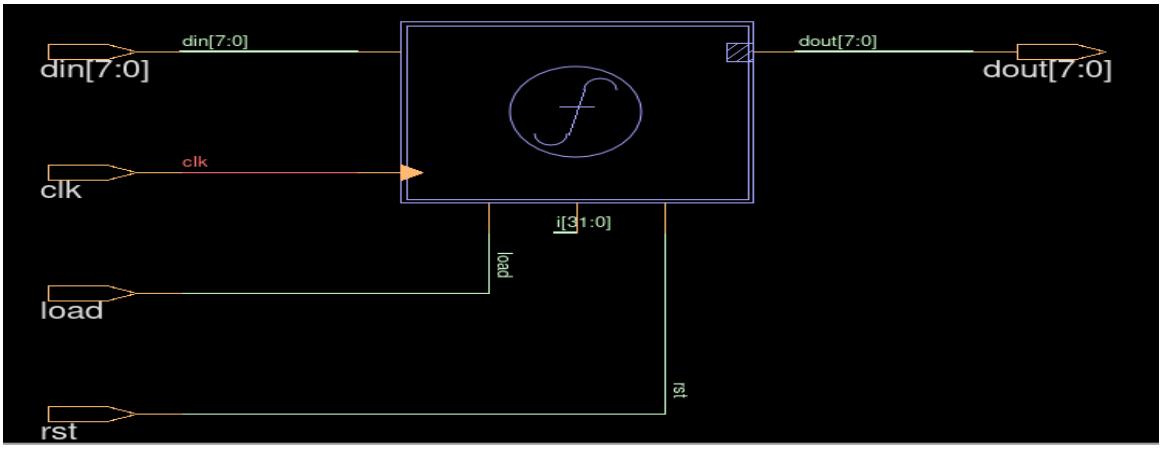
Time=95 for rst=0 din=00000001 load=0 and we get dout=00000001

Time=105 for rst=0 din=00000001 load=0 and we get dout=00000010

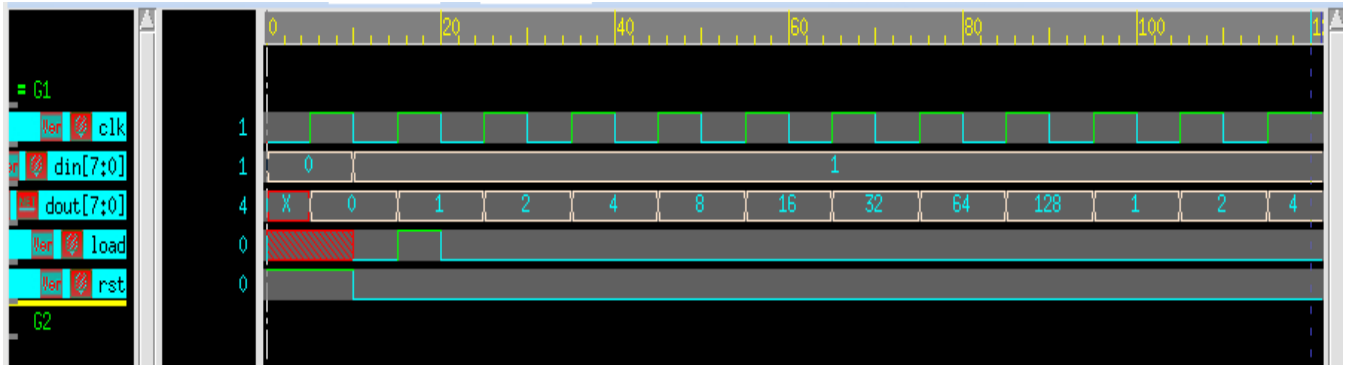
Time=115 for rst=0 din=00000001 load=0 and we get dout=00000100

$finish at simulation time 120

Schematic Design



waveform



84.0 Design a 4-bit Ring Counter (circular left shift)

Design code

```
module Ring_4bit_counter(q,clk,rst);
input clk,rst;
output reg[3:0]q;

always @(posedge clk)begin
        if(rst)
              q <= 4'd1000;
        else begin
             q <= {q[2:0],q[3]};
      end
end
endmodule
```

Testbench

```
module TB();
reg clk,rst;
wire [3:0]q;

Ring_4bit_counter dut(.clk(clk),.rst(rst),.q(q));

initial clk =0;
always #5 clk=~clk;

initial begin
        $monitor("Time=%0t for rst=%b and we get q=%b",$time,rst,q);
        rst=1;

#10;rst =0;
```

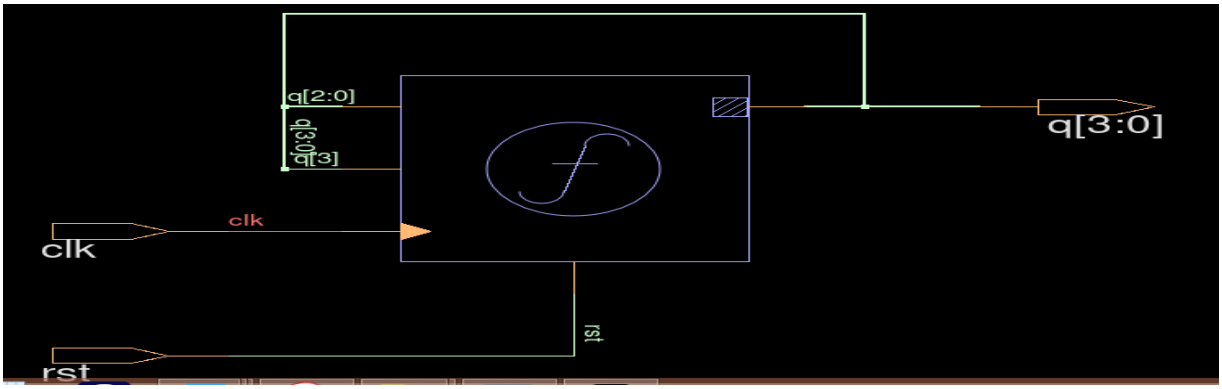
```
#100 $finish();
end
initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

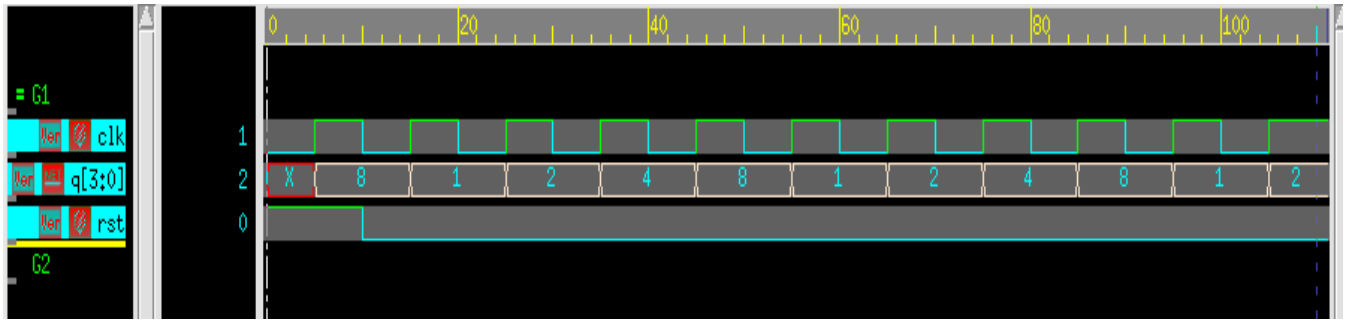
Output

```
Time=0 for rst=1 and we get q=xxxx
Time=5 for rst=1 and we get q=1000
Time=10 for rst=0 and we get q=1000
Time=15 for rst=0 and we get q=0001
Time=25 for rst=0 and we get q=0010
Time=35 for rst=0 and we get q=0100
Time=45 for rst=0 and we get q=1000
Time=55 for rst=0 and we get q=0001
Time=65 for rst=0 and we get q=0010
Time=75 for rst=0 and we get q=0100
Time=85 for rst=0 and we get q=1000
Time=95 for rst=0 and we get q=0001
Time=105 for rst=0 and we get q=0010
$finish called from file "nettype_tri_tb.v", line 19.
$finish at simulation time                  110
```

Schematic Design



waveform



85.0 Design a 4-bit Johnson Counter (shift right)

Design code

```
module Johnshon_4bit_counter(q,clk,rst);
input clk,rst;
output reg[3:0]q;

always @(posedge clk)begin
      if(rst)
            q <= 4'd0000;
      else begin
           q <= {~q[0],q[3:1]};
     end
end
endmodule
```

Testbench

```
module TB();
reg clk,rst;
wire [3:0]q;

Johnshon_4bit_counter dut(.clk(clk),.rst(rst),.q(q));

initial clk =0;
always #5 clk=~clk;
initial begin
        $monitor("Time=%0t for rst=%b and we get q=%b",$time,rst,q);
        rst=1;
#10;rst =0;

#100 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for rst=1 and we get q=xxxx
Time=5 for rst=1 and we get q=0000
Time=10 for rst=0 and we get q=0000
```

Time=15 for rst=0 and we get q=1000

Time=25 for rst=0 and we get q=1100

Time=35 for rst=0 and we get q=1110

Time=45 for rst=0 and we get q=1111

Time=55 for rst=0 and we get q=0111

Time=65 for rst=0 and we get q=0011

Time=75 for rst=0 and we get q=0001

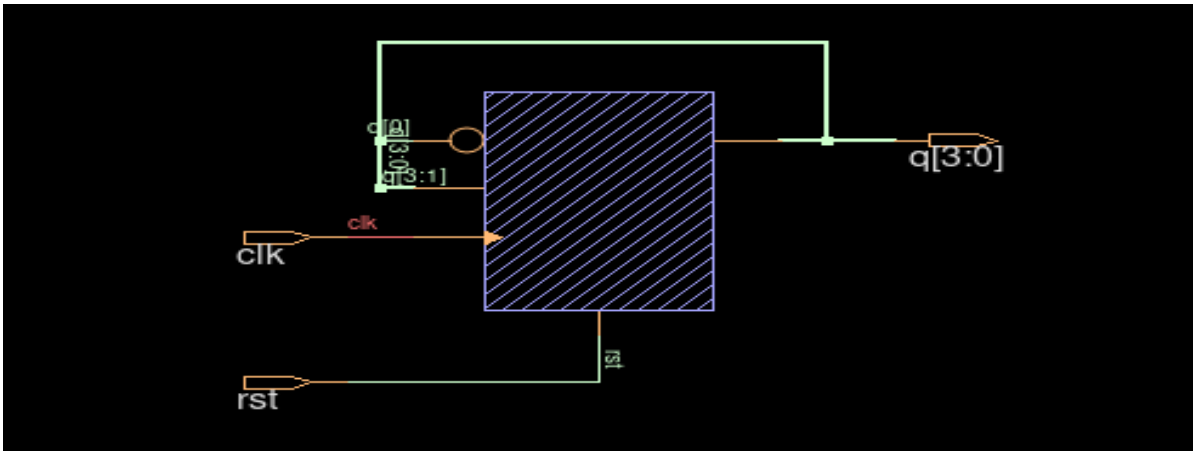
Time=85 for rst=0 and we get q=0000

Time=95 for rst=0 and we get q=1000

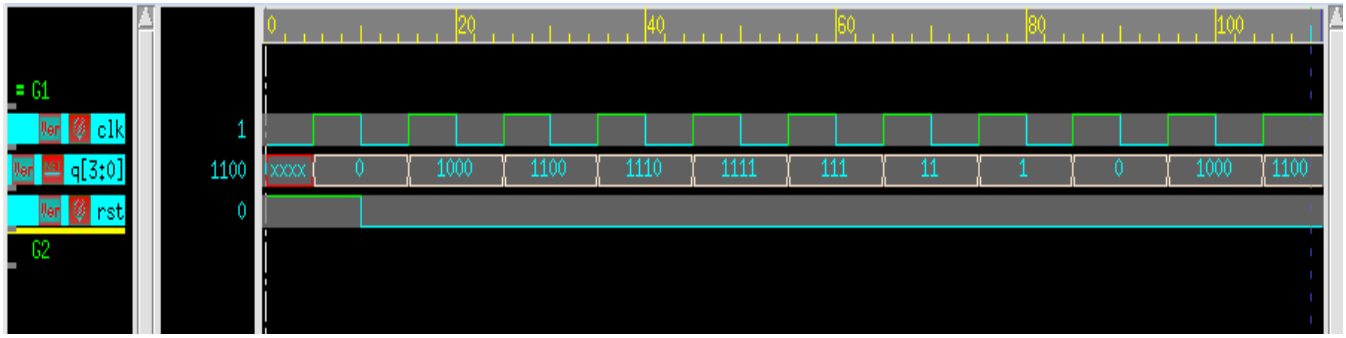
Time=105 for rst=0 and we get q=1100

$finish at simulation time 110

Schematic Design



waveform



86.0 Design a 1x4 D-MUX-

Design code

```
module Demux_1x4 (out,din,sel);
input din;
input [1:0]sel;
output reg [3:0]out;

always @(*) begin
	out = 4'b0000;
	case (sel)
		2'b00: out[0] = din;
		2'b01: out[1] = din;
		2'b10: out[2] = din;
		2'b11: out[3] = din;
		default:out = 4'b0000;
	endcase
end
endmodule
```

Testbench

```
module TB();
reg din;
reg [1:0]sel;
wire [3:0]out;

Demux_1x4 dut(.din(din),.sel(sel),.out(out));
initial begin
	$monitor("Time=%0t for sel=%b din=%b and we get out=%b",$time,sel,din,out);
	din=0;sel=2'b00;
```

```
#10 din=1;sel=2'b00;
#10 din=1;sel=2'b01;
#10 din=1;sel=2'b10;
#10 din=1;sel=2'b11;

#10 din=0;sel=2'b00;
#10 din=0;sel=2'b01;
#10 din=0;sel=2'b10;
#10 din=0;sel=2'b11;
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

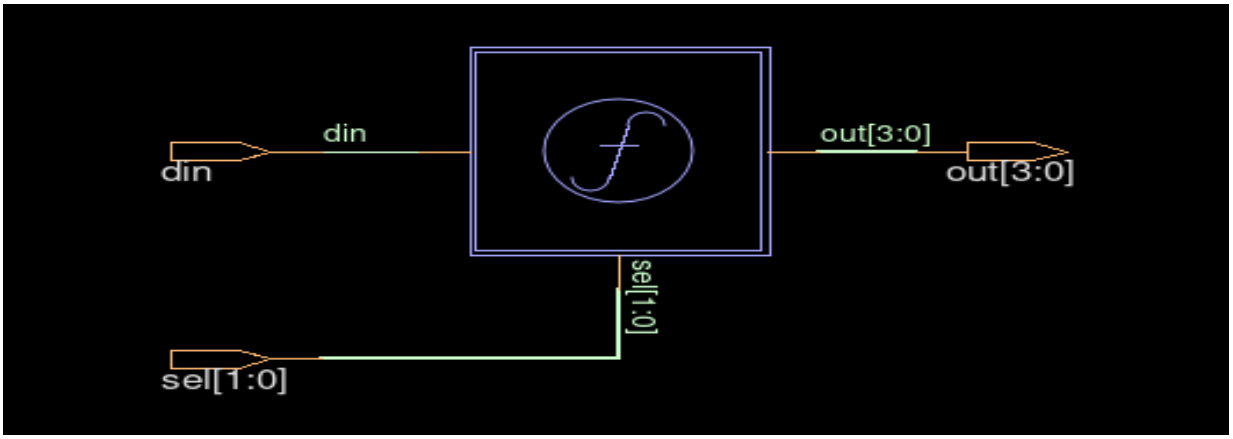
```
Time=0 for sel=00 din=0 and we get out=0000
Time=10 for sel=00 din=1 and we get out=0001
Time=20 for sel=01 din=1 and we get out=0010
Time=30 for sel=10 din=1 and we get out=0100
Time=40 for sel=11 din=1 and we get out=1000
Time=50 for sel=00 din=0 and we get out=0000
Time=60 for sel=01 din=0 and we get out=0000
```

Time=70 for sel=10 din=0 and we get out=0000

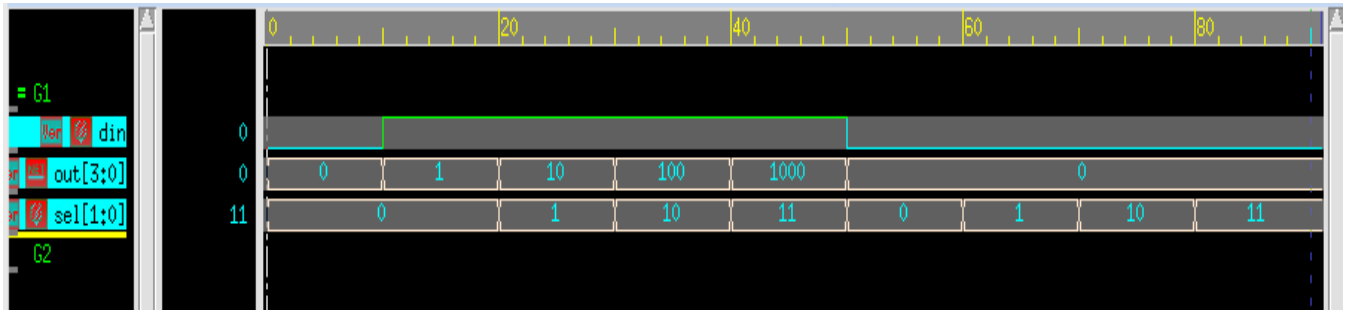
Time=80 for sel=11 din=0 and we get out=0000

$finish at simulation time 90

Schematic Design



waveform



Method-2

Design code

```
module Demux_1x4 (y0,y1,y2,y3,din,sel);
input din;
input [1:0]sel;
output reg y0,y1,y2,y3;

always @(*) begin
        case (sel)
```

```
        2'h0:{y0,y1,y2,y3} = {din,3'b0};
        2'h1:{y0,y1,y2,y3} = {1'b0,din,2'b0};
        2'h2:{y0,y1,y2,y3} = {2'b0,din,1'b0};
        2'h3:{y0,y1,y2,y3} = {3'b0,din};
        default:{y0,y1,y2,y3} = 4'b0000;
    endcase
end
endmodule
```

Testbench

```
module TB();
reg din;
reg [1:0]sel;
wire y0,y1,y2,y3;

Demux_1x4 dut(.din(din),.sel(sel),.y0(y0),.y1(y1),.y2(y2),.y3(y3));
initial begin
    $monitor("Time=%0t for sel=%b din=%b and we get
y0=%b,y1=%b,y2=%b,y3=%b",$time,sel,din,y0,y1,y2,y3);
    din=0;sel=2'b00;

#10 din=1;sel=2'h0;
#10 din=1;sel=2'h1;
#10 din=1;sel=2'h2;
#10 din=1;sel=2'h3;

#10 din=0;sel=2'h0;
#10 din=0;sel=2'h1;
```

```
#10 din=0;sel=2'h2;
#10 din=0;sel=2'h3;
#10 $finish();
end

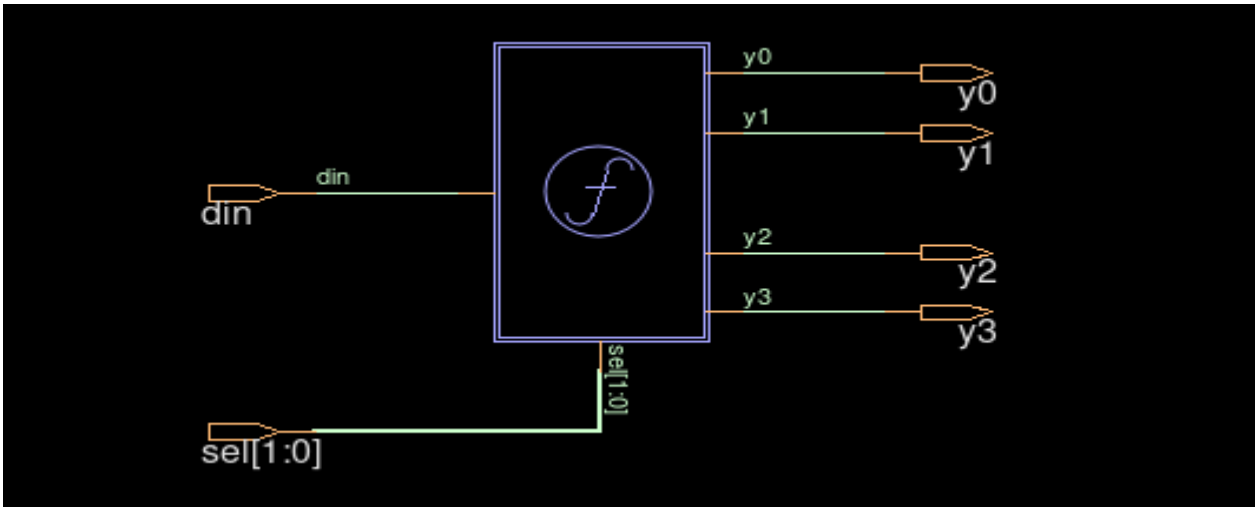
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

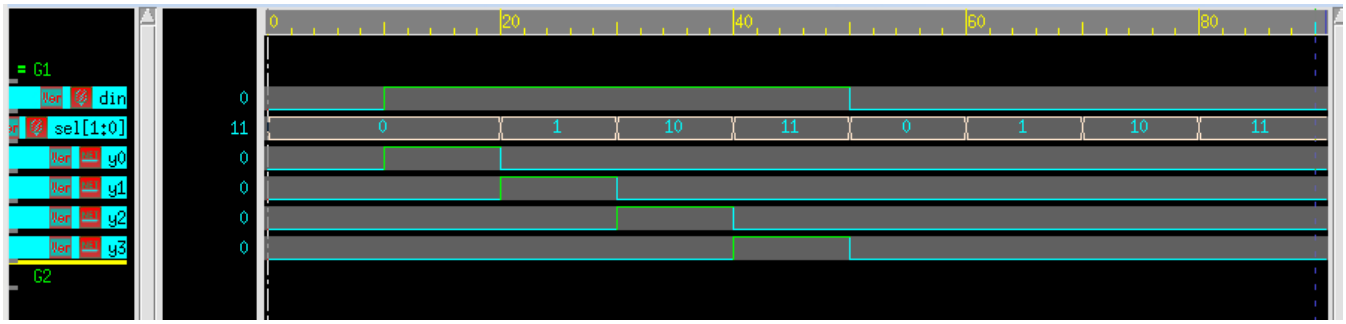
Output

```
Time=0 for sel=00 din=0 and we get y0=0,y1=0,y2=0,y3=0
Time=10 for sel=00 din=1 and we get y0=1,y1=0,y2=0,y3=0
Time=20 for sel=01 din=1 and we get y0=0,y1=1,y2=0,y3=0
Time=30 for sel=10 din=1 and we get y0=0,y1=0,y2=1,y3=0
Time=40 for sel=11 din=1 and we get y0=0,y1=0,y2=0,y3=1
Time=50 for sel=00 din=0 and we get y0=0,y1=0,y2=0,y3=0
Time=60 for sel=01 din=0 and we get y0=0,y1=0,y2=0,y3=0
Time=70 for sel=10 din=0 and we get y0=0,y1=0,y2=0,y3=0
Time=80 for sel=11 din=0 and we get y0=0,y1=0,y2=0,y3=0
$finish at simulation time                90
```

Schematic Design



waveform



87.0 Design a 4-bit Ripple counter (using D-FFs) (Asynchronous counter)

Design code

```
module D_FF(q,d,clk,rst);
input d,clk,rst;
output reg q;


always @(posedge clk or posedge rst)begin
	if(rst)
		q <= 0;
	else
		q <= d;
end
```

```
endmodule
module Ripple_4bit_counter(q,clk,rst);
input clk,rst;
output [3:0]q;

wire q0,q1,q2,q3;
D_FF dff0(.clk(clk),.rst(rst),.d(~q0),.q(q0));
D_FF dff1(.clk(q0),.rst(rst),.d(~q1),.q(q1));
D_FF dff2(.clk(q1),.rst(rst),.d(~q2),.q(q2));
D_FF dff3(.clk(q2),.rst(rst),.d(~q3),.q(q3));

assign q = {q3,q2,q1,q0};
endmodule
```

Testbench

```
module TB();
reg clk,rst;
wire [3:0]q;

Ripple_4bit_counter dut(.clk(clk),.rst(rst),.q(q));

initial clk=0;
always #5 clk=~clk;
initial begin
        $monitor("Time=%0t for rst=%b and we get q=%b",$time,rst,q);
        rst=1;
#10; rst=0;
```

```
#200 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for rst=1 and we get q=0000
Time=10 for rst=0 and we get q=0000
Time=15 for rst=0 and we get q=1111
Time=25 for rst=0 and we get q=1110
Time=35 for rst=0 and we get q=1101
Time=45 for rst=0 and we get q=1100
Time=55 for rst=0 and we get q=1011
Time=65 for rst=0 and we get q=1010
Time=75 for rst=0 and we get q=1001
Time=85 for rst=0 and we get q=1000
Time=95 for rst=0 and we get q=0111
Time=105 for rst=0 and we get q=0110
Time=115 for rst=0 and we get q=0101
Time=125 for rst=0 and we get q=0100
Time=135 for rst=0 and we get q=0011
Time=145 for rst=0 and we get q=0010
Time=155 for rst=0 and we get q=0001
```

Time=165 for rst=0 and we get q=0000

Time=175 for rst=0 and we get q=1111

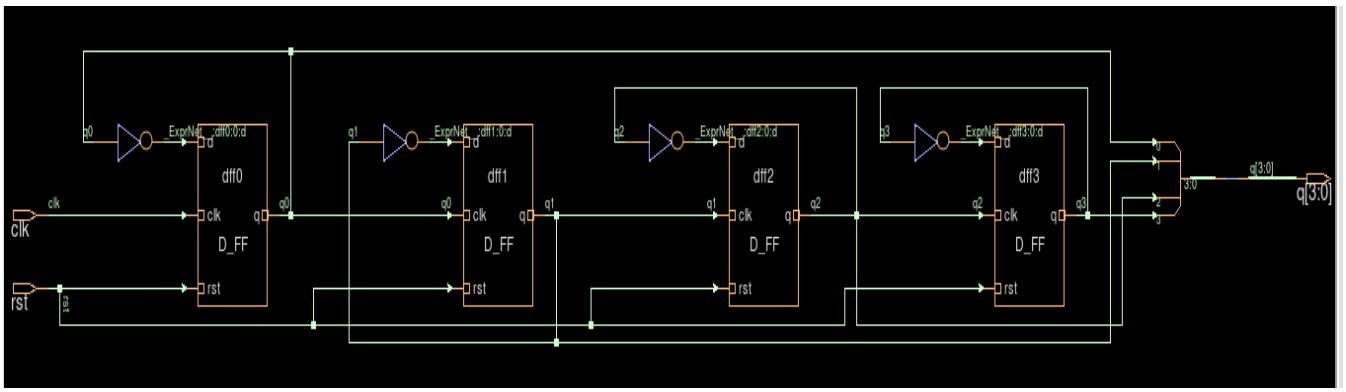
Time=185 for rst=0 and we get q=1110

Time=195 for rst=0 and we get q=1101

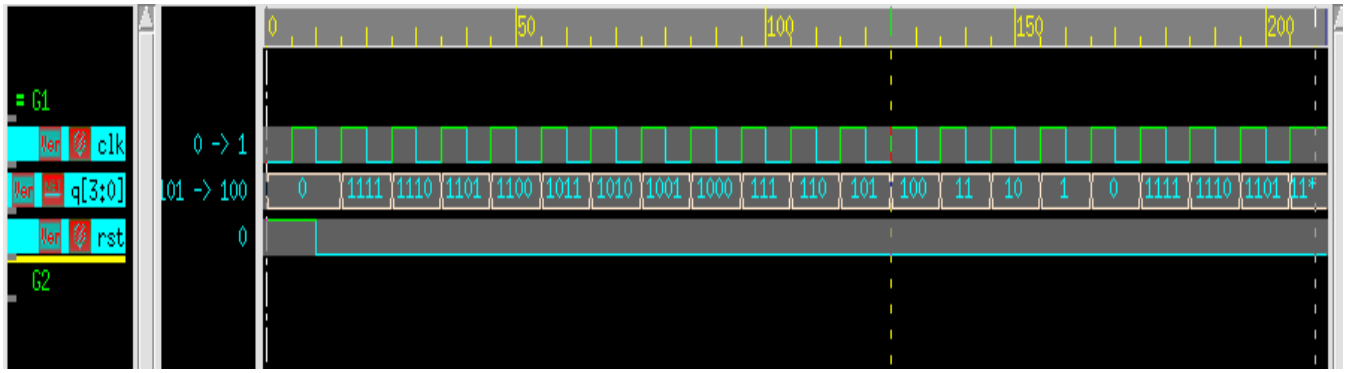
Time=205 for rst=0 and we get q=1100

$finish at simulation time 210

Schematic Design



waveform



88.0 Design a 4-bit Ripple Counter (Asynchronous counter) using T_FF -

Design code

```
module T_FF(q,T,clk,rst);
input T,clk,rst;
output reg q;
```

```
always @(posedge clk or posedge rst)begin
	if(rst)
		q <= 0;
	else if (T)
		q <= ~q;
end
endmodule

module Ripple_4bit_counter(q,clk,rst);
input clk,rst;
output [3:0]q;

wire q0,q1,q2,q3;
T_FF t_ff0(.clk(clk),.rst(rst),.T(1'b1),.q(q0));
T_FF t_ff1(.clk(q0),.rst(rst),.T(1'b1),.q(q1));
T_FF t_ff2(.clk(q1),.rst(rst),.T(1'b1),.q(q2));
T_FF t_ff3(.clk(q2),.rst(rst),.T(1'b1),.q(q3));

assign q = {q3,q2,q1,q0};
endmodule
```

Testbench

```
module TB();
reg clk,rst;
wire [3:0]q;

Ripple_4bit_counter dut(.clk(clk),.rst(rst),.q(q));
```

```
initial clk=0;
always #5 clk=~clk;
initial begin
        $monitor("Time=%0t for rst=%b and we get q=%b",$time,rst,q);
        rst=1;
#10; rst=0;

#200 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for rst=1 and we get q=0000
Time=10 for rst=0 and we get q=0000
Time=15 for rst=0 and we get q=1111
Time=25 for rst=0 and we get q=1110
Time=35 for rst=0 and we get q=1101
Time=45 for rst=0 and we get q=1100
Time=55 for rst=0 and we get q=1011
Time=65 for rst=0 and we get q=1010
Time=75 for rst=0 and we get q=1001
```

Time=85 for rst=0 and we get q=1000

Time=95 for rst=0 and we get q=0111

Time=105 for rst=0 and we get q=0110

Time=115 for rst=0 and we get q=0101

Time=125 for rst=0 and we get q=0100

Time=135 for rst=0 and we get q=0011

Time=145 for rst=0 and we get q=0010

Time=155 for rst=0 and we get q=0001

Time=165 for rst=0 and we get q=0000

Time=175 for rst=0 and we get q=1111

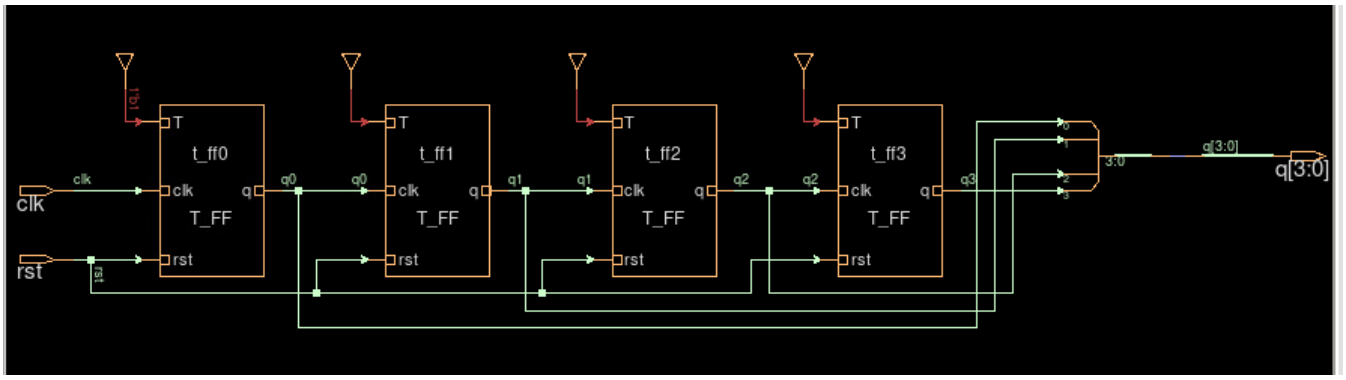
Time=185 for rst=0 and we get q=1110

Time=195 for rst=0 and we get q=1101

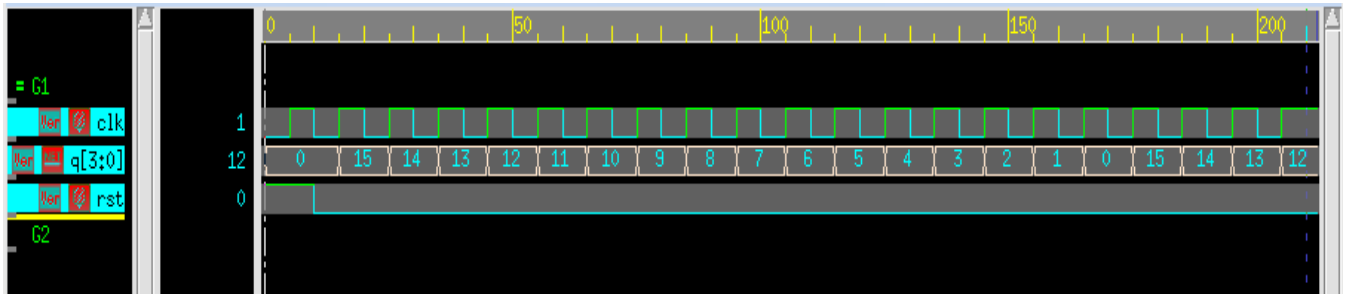
Time=205 for rst=0 and we get q=1100

$finish at simulation time 210

Schematic Design



waveform



89.0 Design a D-latch with enable and reset by using behavioral modelling-

Design code

```
module D_latch(D,en,rst,q);
input D,en,rst;
output reg q;

always @(D,en,rst)begin
	if(rst)
		q <= 0;
	else if (en)begin
		q <= D;
	end
end
endmodule
```

Testbench

```
module TB();
reg D,en,rst;
wire q;

D_latch dut(.D(D),.en(en),.rst(rst),.q(q));
initial begin
	$monitor("Time=%0t for rst=%b en=%b D=%b and we get q=%b",$time,rst,en,D,q);
	rst=1;D=0;
#10; rst=0;en=1;
#10 D=1;
```

```
#10 D=0;
#10 D=1;
#10 D=0;

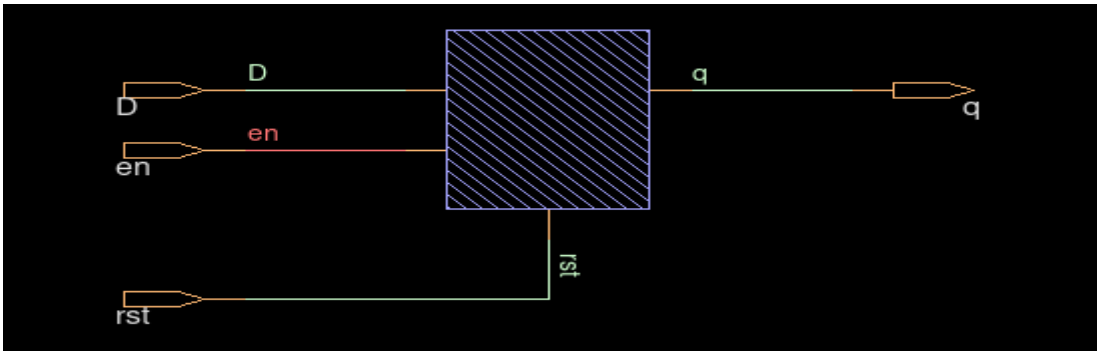
#10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

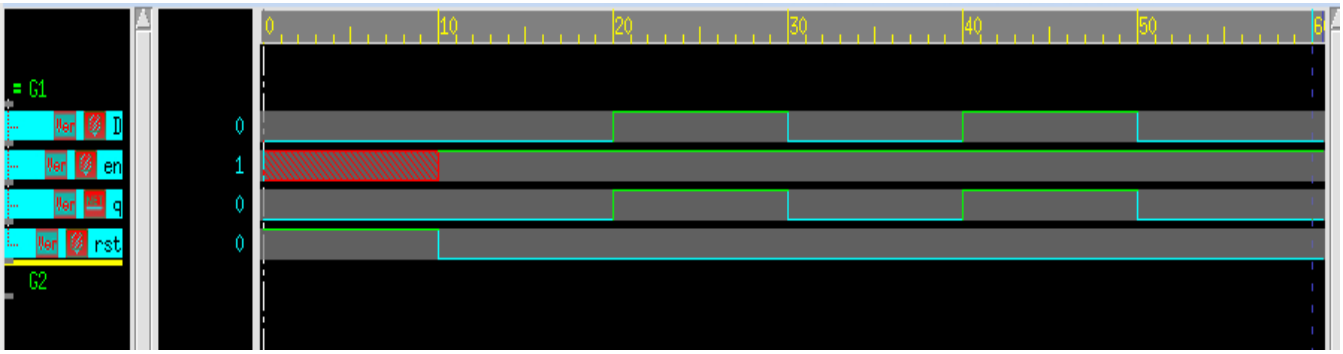
Output

```
Time=0 for rst=1 en=x D=0 and we get q=0
Time=10 for rst=0 en=1 D=0 and we get q=0
Time=20 for rst=0 en=1 D=1 and we get q=1
Time=30 for rst=0 en=1 D=0 and we get q=0
Time=40 for rst=0 en=1 D=1 and we get q=1
Time=50 for rst=0 en=1 D=0 and we get q=0
$finish at simulation time                60
```

Schematic Design



Waveform



90.0 Design a SR-NOR-latch by using gate level modelling

Design code

```
module SR_NOR_latch(Q,Qbar,S,R);
input S,R;
output Q,Qbar;

nor(Q,R,Qbar);
nor(Qbar,Q,S);
endmodule
```

Testbench

```
module TB();
reg S,R;
wire Q,Qbar;

SR_NOR_latch dut(.S(S),.R(R),.Q(Q),.Qbar(Qbar));
initial begin
    $monitor("Time=%0t for S=%b R=%b and we get Q=%b
Qbar=%b",$time,S,R,Q,Qbar);
    S = 0; R = 0;
#10 S = 0; R = 1;
```

```
#10 S = 1; R = 0;
#10 S = 1; R = 1;

#10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

Time=0 for S=0 R=0 and we get Q=x Qbar=x

Time=10 for S=0 R=1 and we get Q=0 Qbar=1

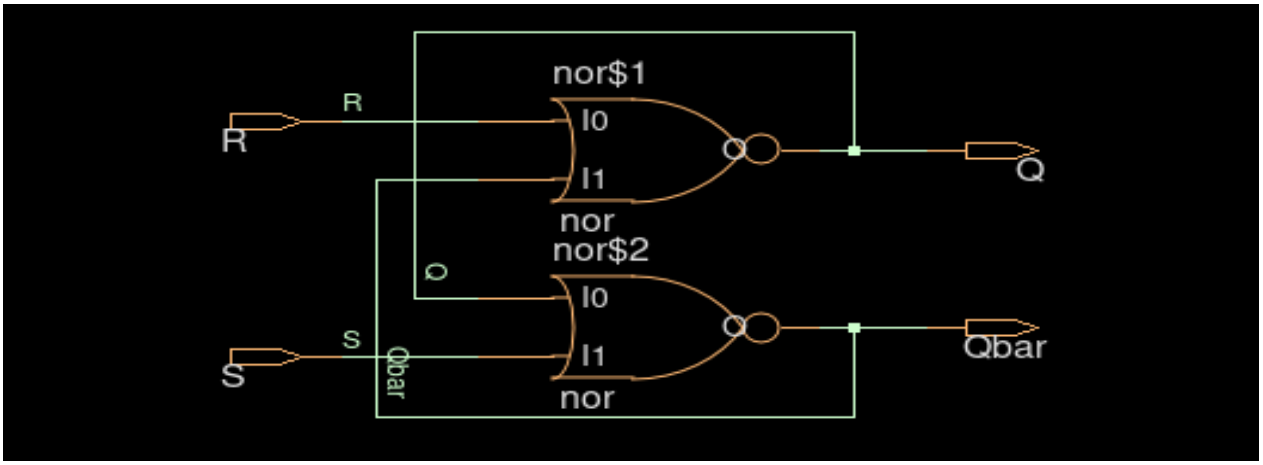
Time=20 for S=1 R=0 and we get Q=1 Qbar=0

Time=30 for S=1 R=1 and we get Q=0 Qbar=0

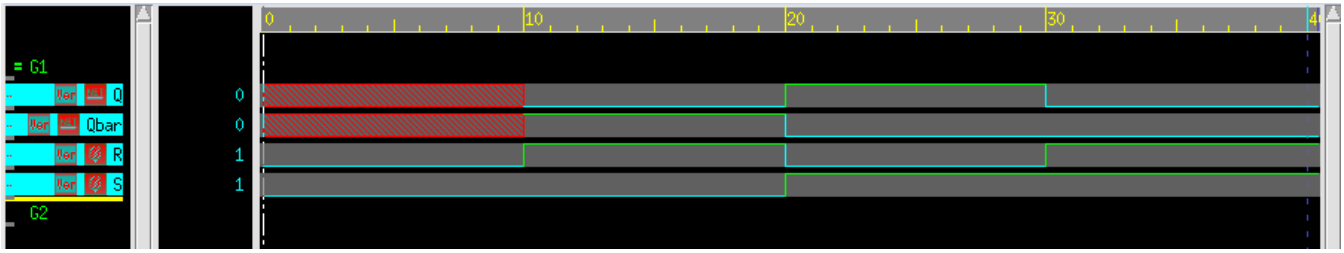
$finish called from file "nettype_tri_tb.v", line 17.

$finish at simulation time 40

Schematic Design



Waveform



91.0 Design a T (Toggle) FLIP_FLOP

Design code

```
module T_Flip_Flop(q,clk,rst,t);
input t,clk,rst;
output reg q;

always @(posedge clk)begin
    if(rst)
        q<=0;
    else if (t==1'b1)
        q<=~q;
    else
        q<=q;
end
endmodule
```

Testbench

```
module TB();
reg t,clk,rst;
wire q;

T_Flip_Flop dut(.t(t),.clk(clk),.rst(rst),.q(q));
```

```
initial clk=0;
always #5 clk=~clk;

initial t=0;
always #10 t=~t;
initial begin
        $monitor("Time=%0t for t=%b rst=%b and we get q=%b",$time,t,rst,q);

        t=0;rst=1;
#10 rst=0;
#100 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for t=0 rst=1 and we get q=x
Time=5 for t=0 rst=1 and we get q=0
Time=10 for t=1 rst=0 and we get q=0
Time=15 for t=1 rst=0 and we get q=1
Time=20 for t=0 rst=0 and we get q=1
Time=30 for t=1 rst=0 and we get q=1
Time=35 for t=1 rst=0 and we get q=0
```

Time=40 for t=0 rst=0 and we get q=0

Time=50 for t=1 rst=0 and we get q=0

Time=55 for t=1 rst=0 and we get q=1

Time=60 for t=0 rst=0 and we get q=1

Time=70 for t=1 rst=0 and we get q=1

Time=75 for t=1 rst=0 and we get q=0

Time=80 for t=0 rst=0 and we get q=0

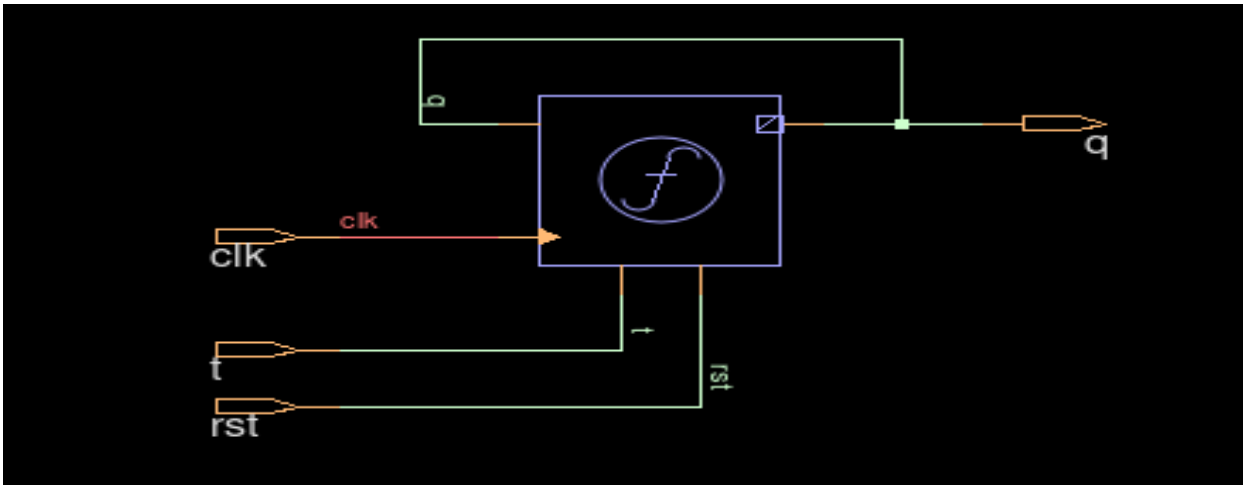
Time=90 for t=1 rst=0 and we get q=0

Time=95 for t=1 rst=0 and we get q=1

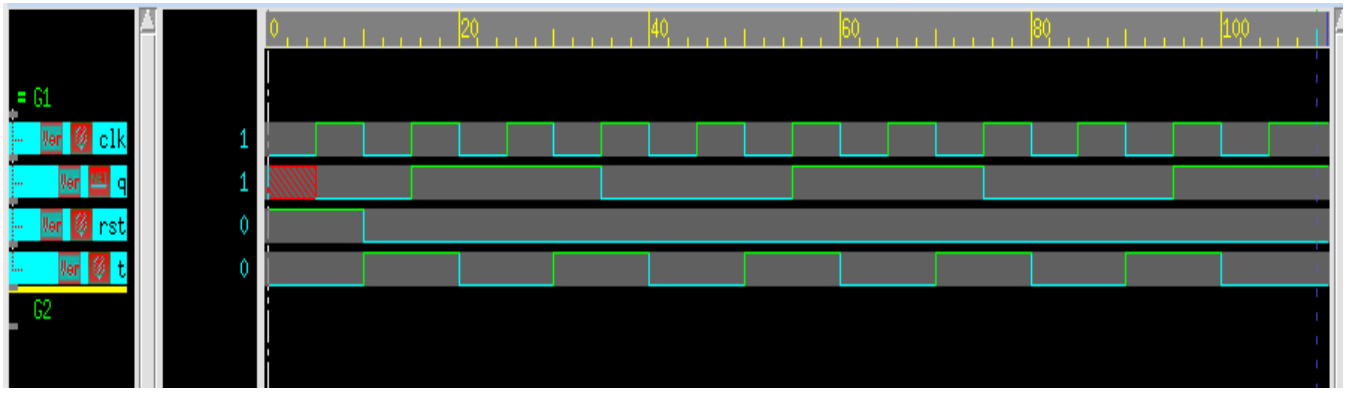
Time=100 for t=0 rst=0 and we get q=1

$finish at simulation time 110

Schematic Design



Waveform



92.0 Design a JK FLIP_FLOP

Design code

```
module JK_FF(Q,Qbar,J,K,clk,rst);
input J,K,clk,rst;
output reg Q;
output Qbar;

assign Qbar = ~Q;
always @(posedge clk)begin
	if (rst)
		Q <=0;
	else begin
		case({J,K})
			2'b00: Q <= Q;
			2'b01: Q <= 0;
			2'b10: Q <= 1;
			2'b11: Q <= ~Q;
		endcase
	end
end
endmodule
```

Testbench

```
module TB();
reg J,K,clk,rst;
wire Q,Qbar;
```

```
JK_FF dut(.J(J),.K(K),.clk(clk),.rst(rst),.Q(Q),.Qbar(Qbar));
initial clk=0;
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t for rst=%b J=%b K=%b and we get Q=%b Qbar=%b",$time,rst,J,K,Q,Qbar);
    rst=1;

#10 rst=0;
#10 J= 0; K= 0;
#10 J= 0; K= 1;
#10 J= 1; K= 0;
#10 J= 1; K=1;
#20 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for rst=1 J=x K=x and we get Q=x Qbar=x
Time=5 for rst=1 J=x K=x and we get Q=0 Qbar=1
Time=10 for rst=0 J=x K=x and we get Q=0 Qbar=1
```

Time=20 for rst=0 J=0 K=0 and we get Q=0 Qbar=1

Time=30 for rst=0 J=0 K=1 and we get Q=0 Qbar=1

Time=40 for rst=0 J=1 K=0 and we get Q=0 Qbar=1

Time=45 for rst=0 J=1 K=0 and we get Q=1 Qbar=0

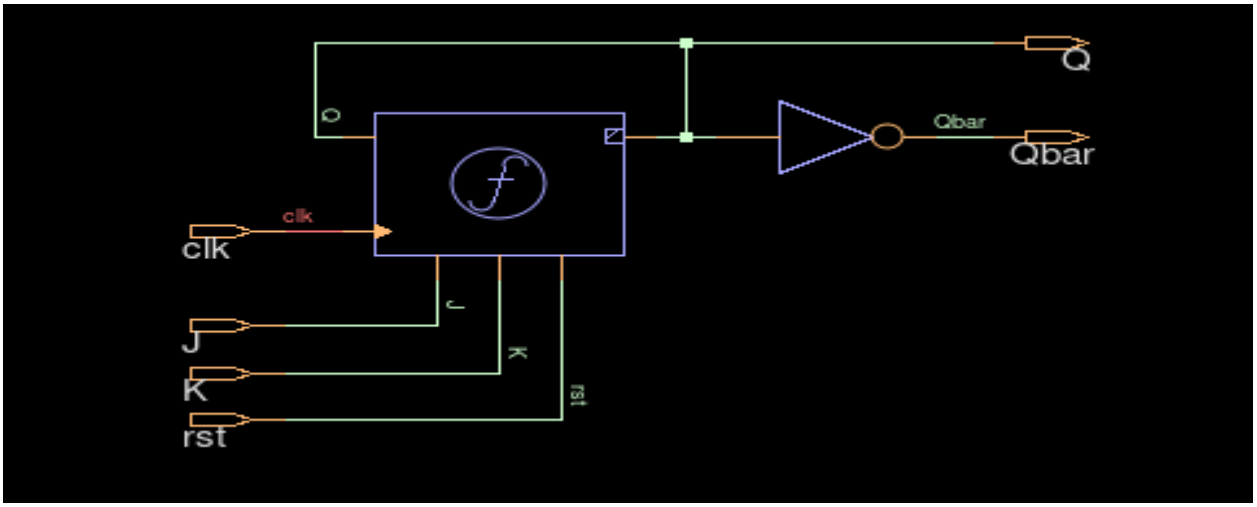
Time=50 for rst=0 J=1 K=1 and we get Q=1 Qbar=0

Time=55 for rst=0 J=1 K=1 and we get Q=0 Qbar=1

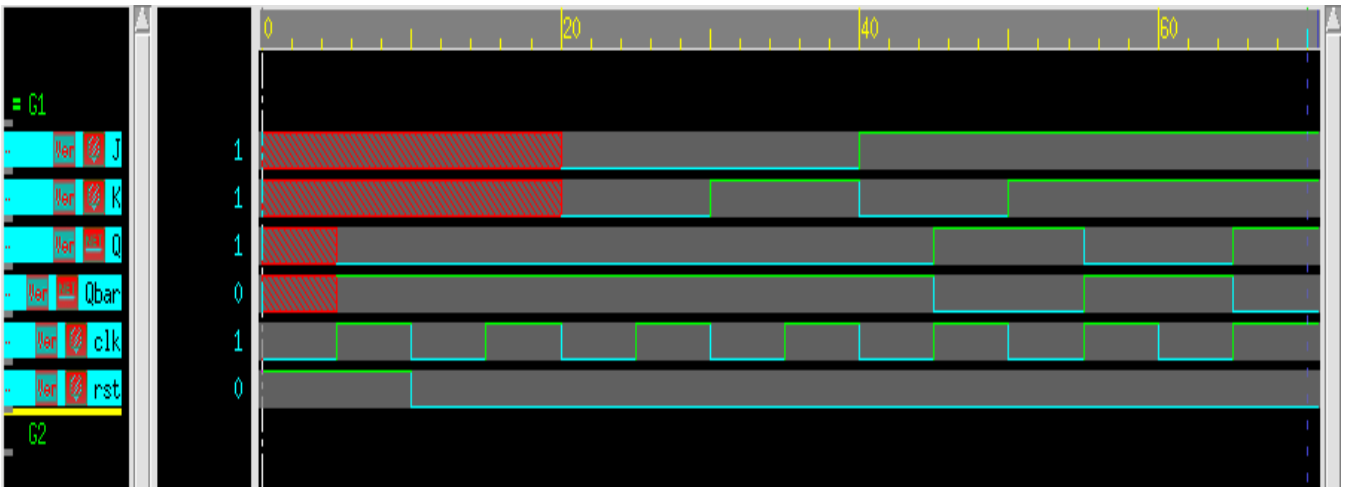
Time=65 for rst=0 J=1 K=1 and we get Q=1 Qbar=0

$finish at simulation time 70

Schematic Design



waveform



93.0 Design a Frequency divided by 2,4, 8,16 Circuit WITH 50% DC By using the Asynchronous D-FF -

Design code

```
module D_FF (q,d,clk,rst);
input clk,rst,d;
output reg q;

always @(posedge clk or posedge rst)begin
	if (rst)
		q <= 0;
	else
		q <= d;
end
endmodule

module freq_div_2_4_8_16(q,q0, q1, q2, q3,clk,rst);
input clk,rst;
output [3:0]q;

output wire q0, q1, q2, q3;
D_FF dff1 (.clk(clk),.rst(rst),.d(~q0),.q(q0));
D_FF dff2 (.clk(q0),.rst(rst),.d(~q1),.q(q1));
D_FF dff3 (.clk(q1),.rst(rst),.d(~q2),.q(q2));
D_FF dff4 (.clk(q2),.rst(rst),.d(~q3),.q(q3));

assign q = {q3, q2, q1, q0};
endmodule
```

Testbench

```
module TB();
reg clk,rst;
wire [3:0]q;
wire q0, q1, q2, q3;

freq_div_2_4_8_16 dut(.clk(clk),.rst(rst),.q(q),.q0(q0),.q1(q1),.q2(q2),.q3(q3));

initial clk=0;
always #5 clk=~clk;

initial begin
    $monitor("Time=%0t for rst=%b and we get q0=%b q1=%b q2=%b q3=%b",$time,rst,q[0],q[1],q[2],q[3]);
    rst=1;
#10; rst=0;

#300 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for rst=1 and we get q0=0 q1=0 q2=0 q3=0
```

Time=10 for rst=0 and we get q0=0 q1=0 q2=0 q3=0

Time=15 for rst=0 and we get q0=1 q1=1 q2=1 q3=1

Time=25 for rst=0 and we get q0=0 q1=1 q2=1 q3=1

Time=35 for rst=0 and we get q0=1 q1=0 q2=1 q3=1

Time=45 for rst=0 and we get q0=0 q1=0 q2=1 q3=1

Time=55 for rst=0 and we get q0=1 q1=1 q2=0 q3=1

Time=65 for rst=0 and we get q0=0 q1=1 q2=0 q3=1

Time=75 for rst=0 and we get q0=1 q1=0 q2=0 q3=1

Time=85 for rst=0 and we get q0=0 q1=0 q2=0 q3=1

Time=95 for rst=0 and we get q0=1 q1=1 q2=1 q3=0

Time=105 for rst=0 and we get q0=0 q1=1 q2=1 q3=0

Time=115 for rst=0 and we get q0=1 q1=0 q2=1 q3=0

Time=125 for rst=0 and we get q0=0 q1=0 q2=1 q3=0

Time=135 for rst=0 and we get q0=1 q1=1 q2=0 q3=0

Time=145 for rst=0 and we get q0=0 q1=1 q2=0 q3=0

Time=155 for rst=0 and we get q0=1 q1=0 q2=0 q3=0

Time=165 for rst=0 and we get q0=0 q1=0 q2=0 q3=0

Time=175 for rst=0 and we get q0=1 q1=1 q2=1 q3=1

Time=185 for rst=0 and we get q0=0 q1=1 q2=1 q3=1

Time=195 for rst=0 and we get q0=1 q1=0 q2=1 q3=1

Time=205 for rst=0 and we get q0=0 q1=0 q2=1 q3=1

Time=215 for rst=0 and we get q0=1 q1=1 q2=0 q3=1

Time=225 for rst=0 and we get q0=0 q1=1 q2=0 q3=1

Time=235 for rst=0 and we get q0=1 q1=0 q2=0 q3=1

Time=245 for rst=0 and we get q0=0 q1=0 q2=0 q3=1

Time=255 for rst=0 and we get q0=1 q1=1 q2=1 q3=0

Time=265 for rst=0 and we get q0=0 q1=1 q2=1 q3=0

Time=275 for rst=0 and we get q0=1 q1=0 q2=1 q3=0

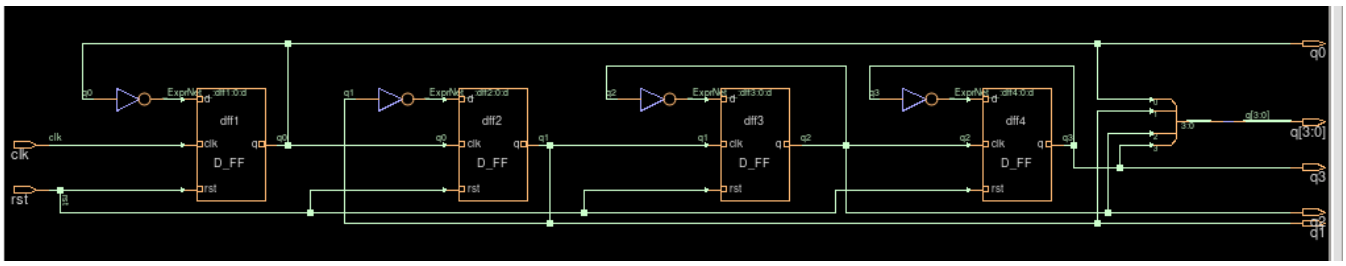
Time=285 for rst=0 and we get q0=0 q1=0 q2=1 q3=0

Time=295 for rst=0 and we get q0=1 q1=1 q2=0 q3=0

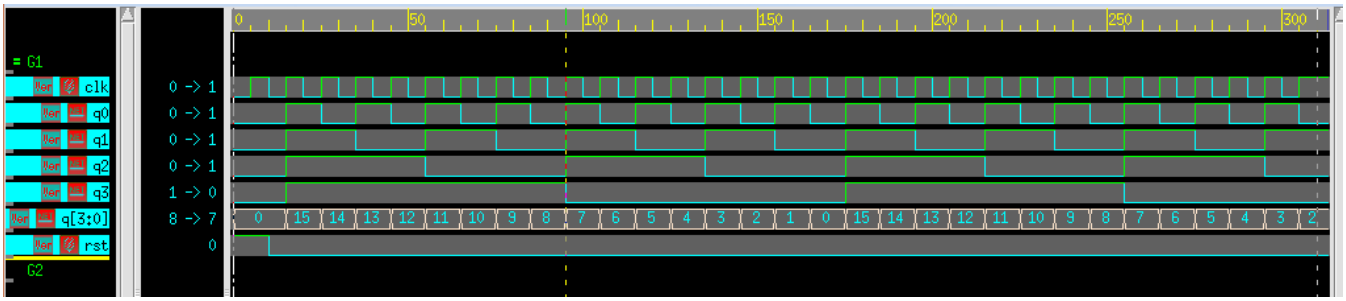
Time=305 for rst=0 and we get q0=0 q1=1 q2=0 q3=0

$finish at simulation time 310

Schematic Design



waveform



94.0 Design a 2x1mux by using the dataflow modelling

Design code

//mux_2x1 by using dataflow style

module MUX_2X1__1(y,in0,in1,s);

input in0,in1,s;

output y;

assign y = s ? in1 : in0;

endmodule

Testbench

```
module MUX_2X1__1tb();
reg in0,in1,s;
wire y;

MUX_2X1__1 mux1(.y(y),.in0(in0),.in1(in1),.s(s));

initial begin
    $monitor("s=%d,in0=%b,in1=%b and we get y=%b",s,in0,in1,y);

    s=0;in0=0; in1=0;
    #5 s=0;in0=1;in1=0;
    #5 s=0;in0=0;in1=1;
    #5 s=1;in0=1;in1=0;
    #5 s=1;in0=1;in1=1;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,MUX_2X1__1tb);
end
endmodule
```

Simulation Result

Output

```
s=0,in0=0,in1=0 and we get y=0
```

s=0,in0=1,in1=0 and we get y=1

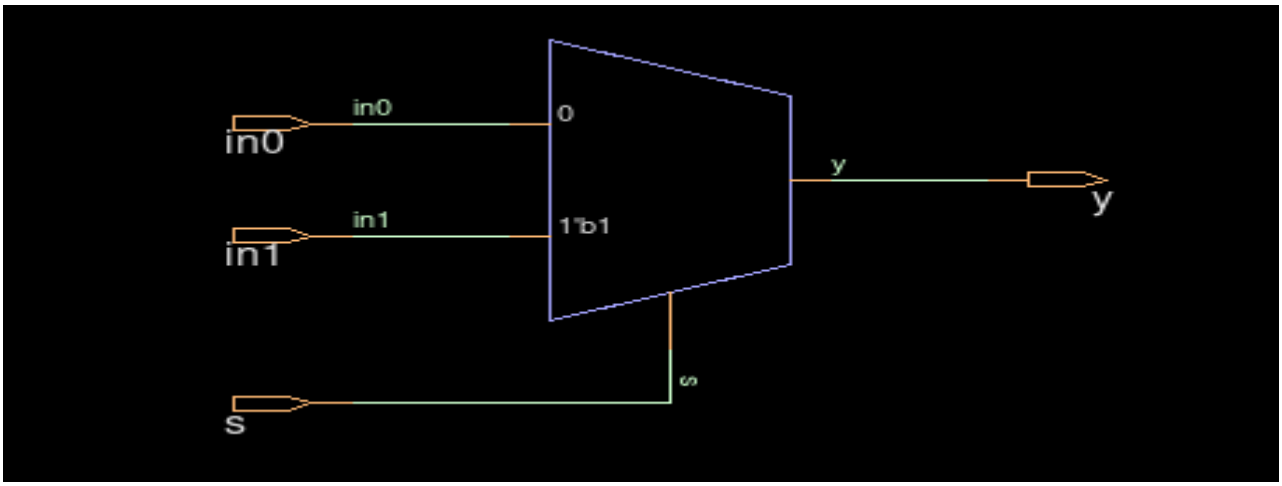
s=0,in0=0,in1=1 and we get y=0

s=1,in0=1,in1=0 and we get y=0

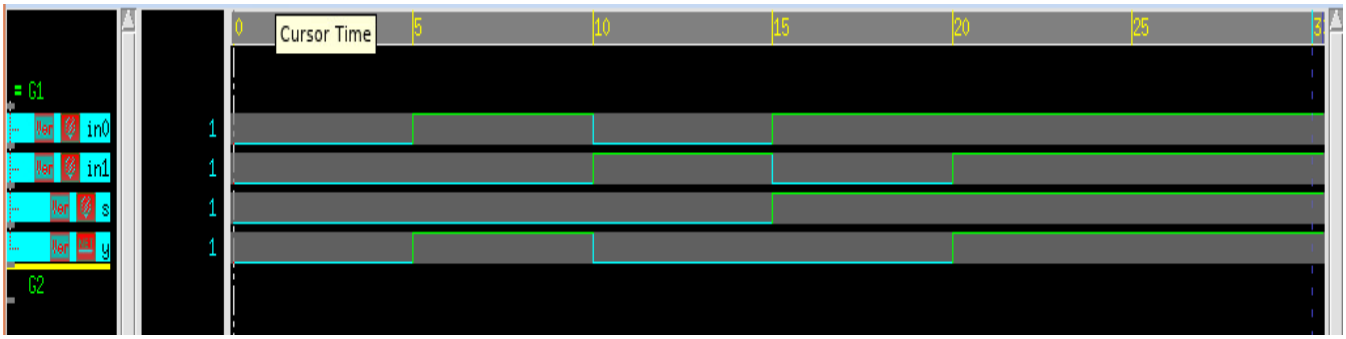
s=1,in0=1,in1=1 and we get y=1

$finish at simulation time 30

Schematic Design



waveform



95.0 Design a 4x1mux by using the dataflow modelling

Design code

module MUX_4X_1(y,in0,in1,in2,in3,s1,s0);

input in0,in1,in2,in3,s1,s0;

output y;

assign y = s1 ? (s0 ? in3:in2) : (s0 ? in1 :in0);

```
endmodule
```

Testbench

```
module MUX_4X1_1tb();
reg in0,in1,in2,in3,s1,s0;
wire y;

MUX_4X_1 mux1(.y(y),.in0(in0),.in1(in1),.in2(in2),.in3(in3),.s1(s1),.s0(s0));

initial begin
    $monitor("s1=%b,s0=%b in0=%b,in1=%b in2=%b in3=%b and we get y=%b",s1,s0,in0,in1,in2,in3,y);

    s1=0;s0=0;in0=1;in1=0;in2=1;in3=0;
    #5 s1=0;s0=1;in0=1;in1=0;in2=1;in3=0;
    #5 s1=1;s0=0;in0=1;in1=0;in2=1;in3=0;
    #5 s1=1;s0=1;in0=1;in1=0;in2=1;in3=0;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,MUX_4X1_1tb);
end
endmodule
```

Simulation Result

Output

```
s1=0,s0=0 in0=1,in1=0 in2=1 in3=0 and we get y=1
```

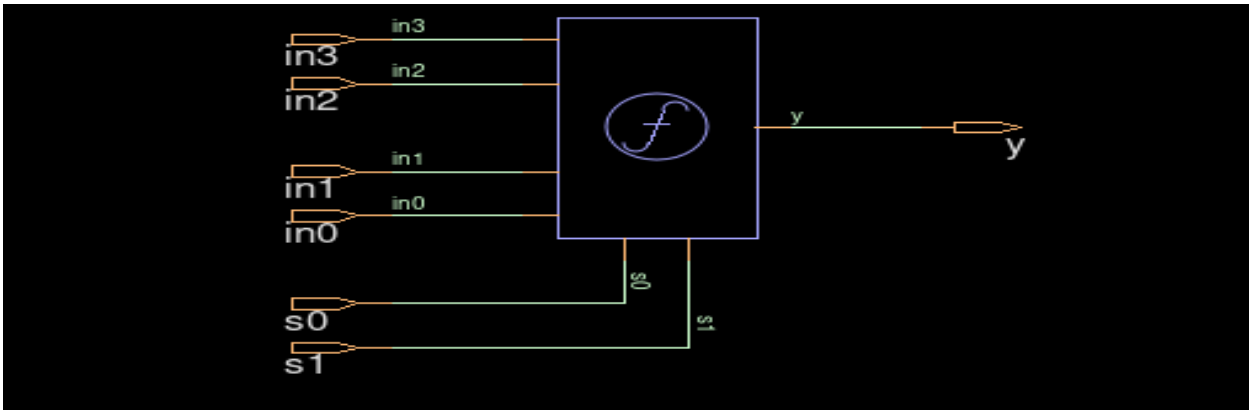
s1=0,s0=1 in0=1,in1=0 in2=1 in3=0 and we get y=0

s1=1,s0=0 in0=1,in1=0 in2=1 in3=0 and we get y=1

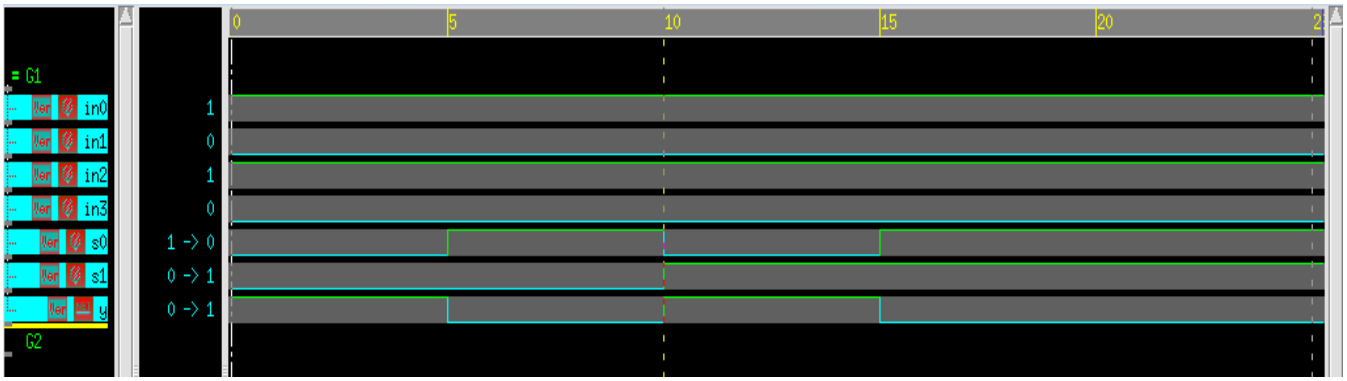
s1=1,s0=1 in0=1,in1=0 in2=1 in3=0 and we get y=0

$finish at simulation time 25

Schematic Design



Waveform



96.0 Design ALL LOGIC GATES by using dataflow modelling

Design code

```
module Logic_gates(y0,y1,y2,y3,y4,y5,y6,a,b);
input a,b;
output y0,y1,y2,y3,y4,y5,y6;


assign y0 = ~a;
```

```
assign y1 = ~(a&b);
assign y2 = ~(a|b);
assign y3 = (a&b);
assign y4 = (a|b);
assign y5 = (a^b);
assign y6 = ~(a^b);

endmodule
```

Testbench

```
module simulation_tb();
reg a,b;
wire y0,y1,y2,y3,y4,y5,y6;

Logic_gates dut(y0,y1,y2,y3,y4,y5,y6,a,b);

initial begin
    $monitor("for a=%b b=%b and we get y0=%b,y1=%b y2=%b,y3=%b y4=%b y5=%b y6=%b",a,b,y0,y1,y2,y3,y4,y5,y6);
    a=0; b=0;
#10 a=0; b=1;
#10 a=1; b=0;
#10 a=1; b=1;
#10 $finish;
 end

initial begin
    $fsdbDumpfile("dump.fsdb");
```

```
    $fsdbDumpvars(0,simulation_tb);
end
endmodule
```

Simulation Result

Output

for a=0 b=0 and we get y0=1,y1=1 y2=1,y3=0 y4=0 y5=0 y6=1

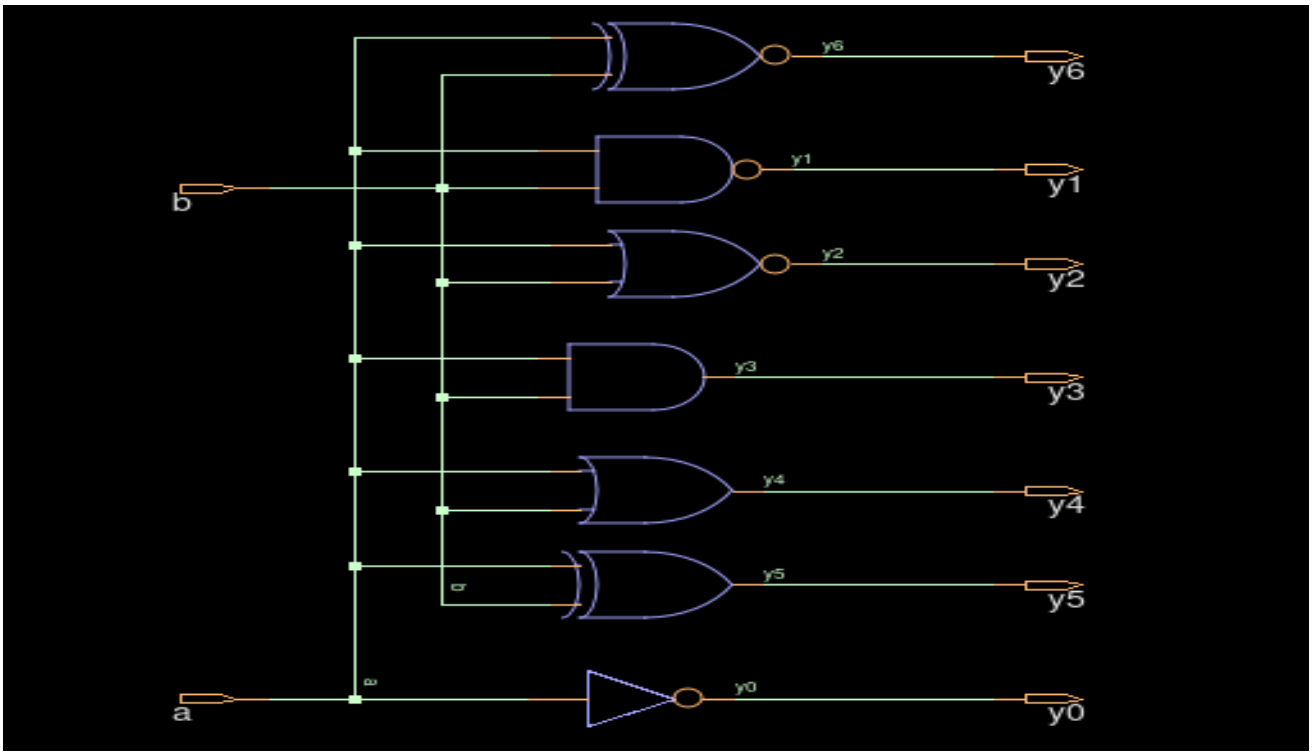
for a=0 b=1 and we get y0=1,y1=1 y2=0,y3=0 y4=1 y5=1 y6=0

for a=1 b=0 and we get y0=0,y1=1 y2=0,y3=0 y4=1 y5=1 y6=0

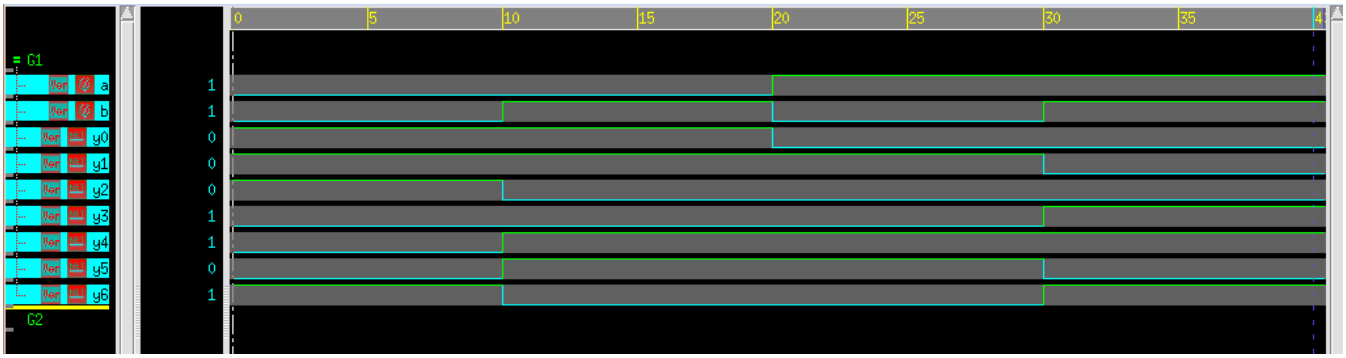
for a=1 b=1 and we get y0=0,y1=0 y2=0,y3=1 y4=1 y5=0 y6=1

$finish at simulation time 40

Schematic Design



Waveform



97.0 Design ALL LOGIC GATES by using gate level modelling

Design code

```
module Logic_gates(y0,y1,y2,y3,y4,y5,y6,y7,a,b);
input a,b;
output y0,y1,y2,y3,y4,y5,y6,y7;

not not_out(y0,a);
nand nand_out(y1,a,b);
nor nor_out(y2,a,b);
and and_out(y3,a,b);
or or_out(y4,a,b);
xor xor_out(y5,a,b);
xnor xnor_out(y6,a,b);
buf buf_out(y7,a);

endmodule
```

Testbench

```
module simulation_tb();
reg a,b;
wire y0,y1,y2,y3,y4,y5,y6,y7;
```

```
Logic_gates dut(y0,y1,y2,y3,y4,y5,y6,y7,a,b);


initial begin
      $monitor("for a=%b b=%b and we get y0=%b y1=%b y2=%b y3=%b y4=%b
y5=%b y6=%b y7=%b",a,b,y0,y1,y2,y3,y4,y5,y6,y7);
      a=0; b=0;
#10 a=0; b=1;
#10 a=1; b=0;
#10 a=1; b=1;
#10 $finish;
 end

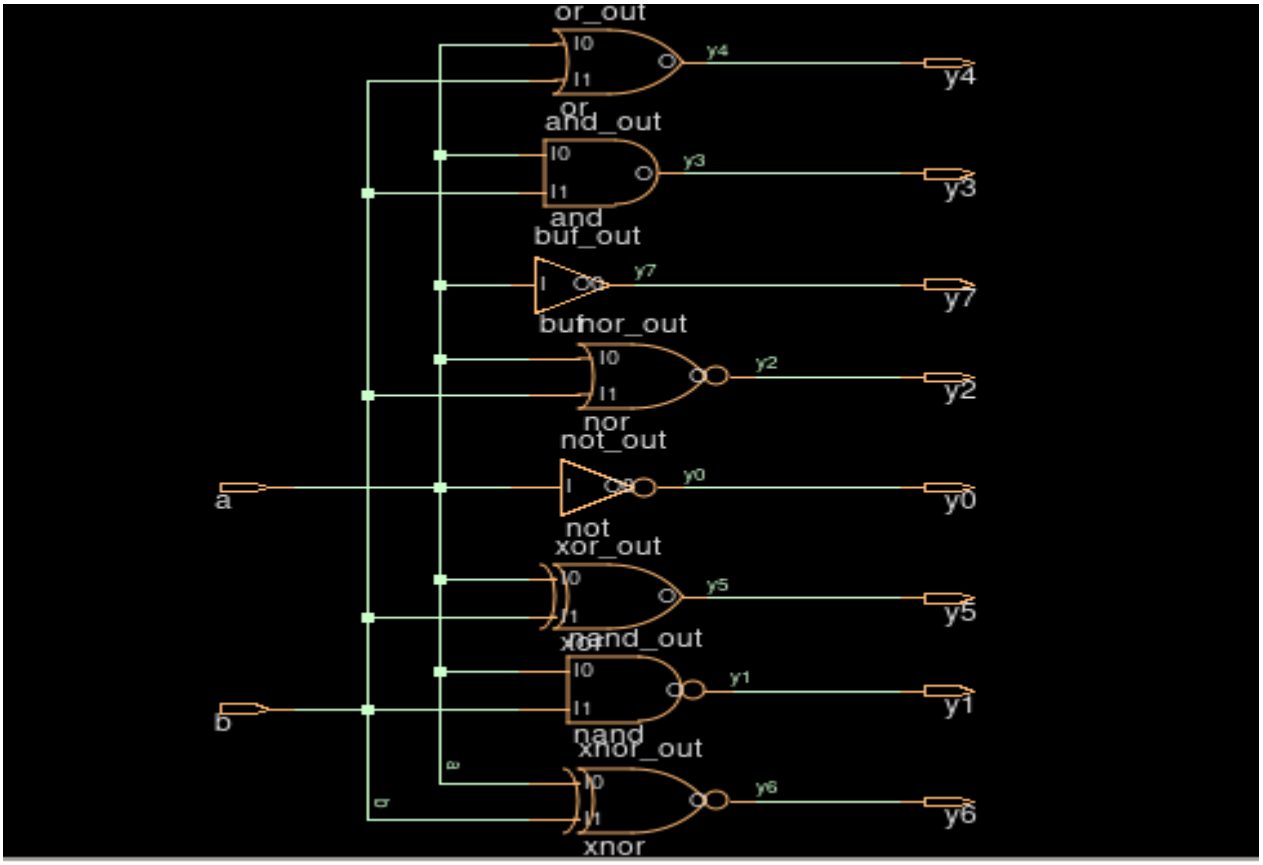

initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,simulation_tb);
end
endmodule
```

Simulation Result

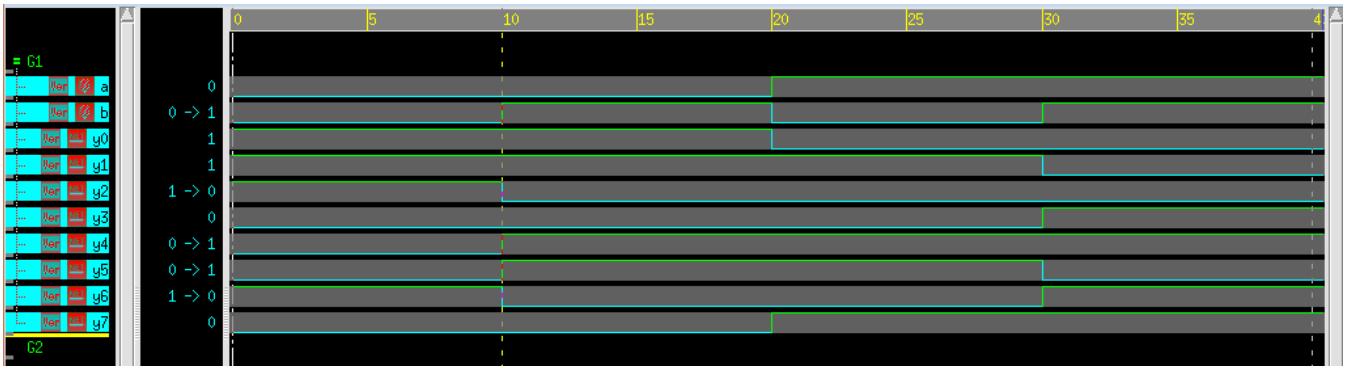
Output

```
for a=0 b=0 and we get y0=1 y1=1 y2=1 y3=0 y4=0 y5=0 y6=1 y7=0
for a=0 b=1 and we get y0=1 y1=1 y2=0 y3=0 y4=1 y5=1 y6=0 y7=0
for a=1 b=0 and we get y0=0 y1=1 y2=0 y3=0 y4=1 y5=1 y6=0 y7=1
for a=1 b=1 and we get y0=0 y1=0 y2=0 y3=1 y4=1 y5=0 y6=1 y7=1
$finish at simulation time                 40
```

Schematic Design



waveform



98.0 Swapping of two variables without using the temporary variable by using the blocking assignment

Design code

//Swapping of two variable without using the third varible using blocking

module swap_blocking;

reg [3:0] a,b;

```
initial begin
    a = 4'd5;
    b = 4'd9;
    $display("Before swap: a = %0d, b = %0d", a, b);
    a = a + b;
    b = a - b;
    a = a - b;
    $display("After swap: a = %0d, b = %0d", a, b);
end
endmodule
```

Simulation Result

Output

Before swap: a = 5, b = 9

After swap: a = 9, b = 5

V C S S i m u l a t i o n R e p o r t

99.0 Swapping of two variables using the third variable by using the blocking assignment

Design code

```
module swap_with_temp;
reg [3:0] a, b, temp;
initial begin
    a = 4'd7;
    b = 4'd12;
    $display("Before swap: a = %0d, b = %0d", a, b);
    temp = a;
    a = b;
```

```
        b = temp;
        $display("After swap:  a = %0d, b = %0d", a, b);
    end
endmodule
```

Simulation Result

Output

```
Before swap: a = 7, b = 12
After swap:  a = 12, b = 7
```

100.0 Swapping of two variables without using the third variable by using the non-blocking assignment

Design code

```
module swap_variables;
reg a, b;
initial begin
        a = 0;
        b = 1;
    $display("Initial values: a = %b, b = %b", a, b);
    a <= b;
    b <= a;
    #1;
    $display("Swapped values: a = %b, b = %b", a, b);
    $finish();
    end
endmodule
```

Simulation Result

Output

Initial values: a = 0, b = 1

Swapped values: a = 1, b = 0

$finish at simulation time 1

101.0 Swapping of two variables using the third variable by using the non-blocking assignment

Design code

```
module swap_variables;
reg a, b, temp;
initial begin
    a = 0;
    b = 1;
    $display("Initial values: a = %b, b = %b", a, b);
    temp = a;
    a <= b;
    b <= temp;
    #1;
    $display("Swapped values: a = %b, b = %b", a, b);
    $finish();
end
endmodule
```

Simulation Result

Output

Initial values: a = 0, b = 1

Swapped values: a = 1, b = 0

$finish at simulation time 1

102.0 Swapping Using a Third Variable (temp) with Blocking Assignment procedural block (always @(posedge clk))

Design code

```
module swap_block_proc;
reg [3:0] a,b,temp;
reg clk;

initial clk = 0;
always #5 clk = ~clk;
initial begin
	a = 4'd3;
	b = 4'd7;
	$display("Before swap: a = %0d, b = %0d", a, b);
end
always @(posedge clk) begin
	temp = a;
	a = b;
	b = temp;
end
initial begin
	#10;
	$display("After swap:  a = %0d, b = %0d", a, b);
	$finish();
end
endmodule
```

Simulation Result

Output

Before swap: a = 3, b = 7

After swap: a = 7, b = 3

$finish at simulation time 10

103.0 Swapping without using a Third Variable with non-Blocking Assignment procedural block (always @(posedge clk))

Design code

```
module swap_nb_proc;
reg [3:0] a,b;
reg clk;

initial clk = 0;
always #5 clk = ~clk;

initial begin
    a = 4'd3;
    b = 4'd7;
    $display("Before swap: a = %0d, b = %0d", a, b);
end
always @(posedge clk) begin
    a <= b;
    b <= a;
end
initial begin
    #10;
```

```
        $display("After swap:  a = %0d, b = %0d", a, b);
        $finish();
end
endmodule
```

Simulation Result

Output

```
Before swap: a = 3, b = 7
After swap:  a = 7, b = 3
$finish at simulation time                 10
```

104.0 Swapping Using a Third Variable (temp) with Non-Blocking Assignment procedural block (always @(posedge clk)) adding delay concept-

Design code

```
module swap_nb_proc;
reg [3:0] a, b, temp;
reg clk;

initial clk = 0;
always #5 clk = ~clk;
initial begin
        a = 4'd3;
        b = 4'd7;
        $display("Before swap: a = %0d, b = %0d", a, b);
end
always @(posedge clk) begin
        temp <= a;
```

```
        a <= b;#1;
        b <= temp;
end
initial begin
        #10;
        #1;
        $display("After swap:  a = %0d, b = %0d", a, b);
        $finish;
end
endmodule
```

Simulation Result

Output

```
Before swap: a = 3, b = 7
After swap:  a = 7, b = 3
$finish at simulation time                 11
```

Design code

Testbench

Run command

Simulation Result

Output

Schematic Design

waveform

1.0 Design a 2x1_mux using bufof1 and bufif0

Mkdir - mux

Design code

```
// Design the mux_2x1 trigates
module mux_2x1 (out,in1,in2,ctrl);
input in1,in2,ctrl;
output out;

bufif1 i1(out,in1,ctrl);
bufif0 i2(out,in2,ctrl);
endmodule
```

Testbench

```
// testbench of 2x1 mux
module TB();
reg in1,in2,ctrl;
wire out;

mux_2x1 dut(.in1(in1),.in2(in2),.ctrl(ctrl),.out(out));
```

```
initial begin
      $monitor("Time=%0t,in1=%0b,in2=%0b,ctrl=%ob,out=%ob",$time,in1,in2,ctrl,out);
        ctrl=0; in1=0; in2=0;
        #10 ctrl=0;in1=1; in2=0;
        #10 ctrl=1;in1=0; in2=1;
        #10 ctrl=1;in1=1;in2=1;
        #10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

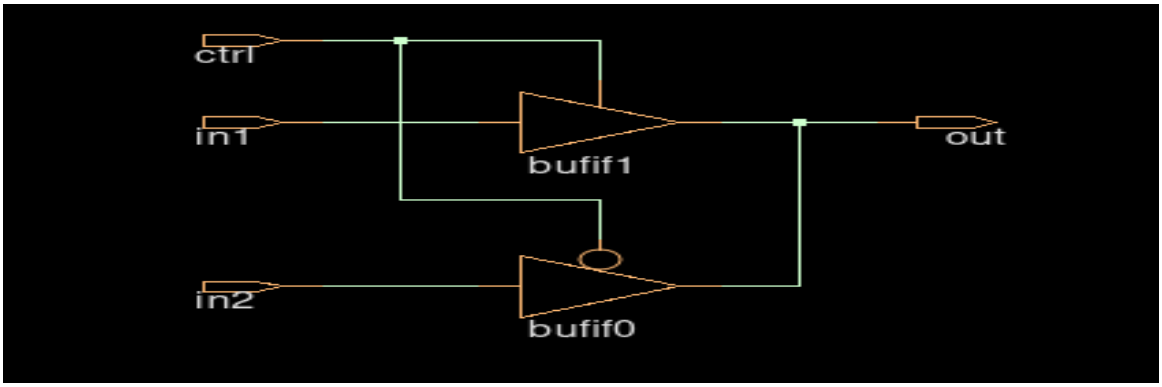
```
vcs mux_2x1_trigates.v mux_2x1_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

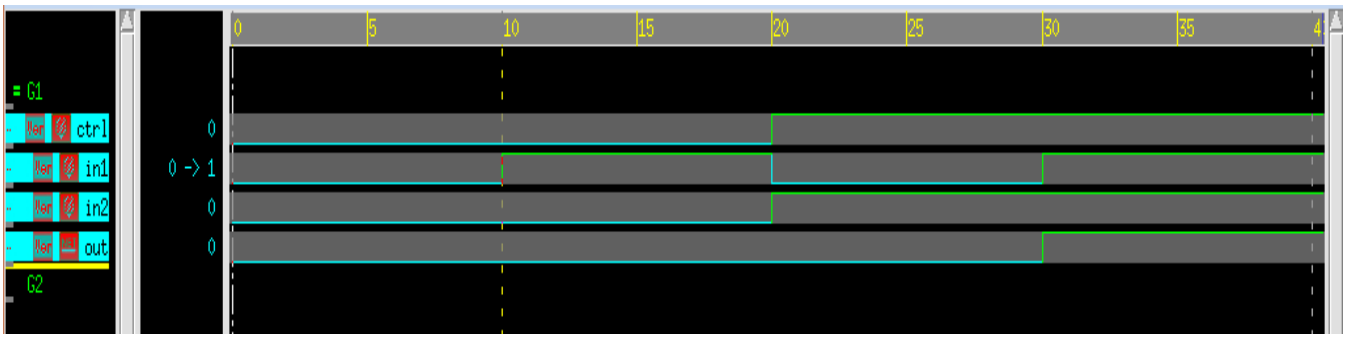
Output

```
Time=0,in1=0,in2=0,ctrl=0b,out=0b
Time=10,in1=1,in2=0,ctrl=0b,out=0b
Time=20,in1=0,in2=1,ctrl=1b,out=0b
Time=30,in1=1,in2=1,ctrl=1b,out=1b
```

Schematic Design



waveform



2.0 Design an 8x1 mux by using data flow

Design code-1

```
// design the 8x1 mux by using the data flow modelling
module mux_8x1(out,in,sel);
input [7:0]in;
input [2:0]sel;
output out;

assign out=in[sel];
endmodule
```

Design code-2 by using behavioural modelling

```
// design the 8x1 mux by using the data flow modelling
module mux_8x1(out,in,sel);
```

```
input [7:0]in;
input [2:0]sel;
output reg out;

always @(*)begin
	case(sel)
		3'b000:out=in[0];
		3'b001:out=in[1];
		3'b010:out=in[2];
		3'b011:out=in[3];
		3'b100:out=in[4];
		3'b101:out=in[5];
		3'b110:out=in[6];
		3'b111:out=in[7];
	endcase
end
endmodule
```

Testbench

```
// testbench of 8x1 mux
module TB();
reg [7:0]in;
reg [2:0]sel;
wire out;

mux_8x1 dut(.in(in),.sel(sel),.out(out));

initial
```

```
begin
        $monitor("for in=%b,sel=%b,out=%b",in,sel,out);


        in=8'b10101010;
sel=3'b000;#10;
sel=3'b001;#10;
sel=3'b010;#10;
sel=3'b011;#10;
sel=3'b100;#10;
sel=3'b101;#10;
sel=3'b110;#10;
sel=3'b111;#10;
#10 $finish();
end


initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

```
vcs mux8x1.v mux8x1_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
for in=10101010,sel=000,out=0
for in=10101010,sel=001,out=1
```

for in=10101010,sel=010,out=0

for in=10101010,sel=011,out=1

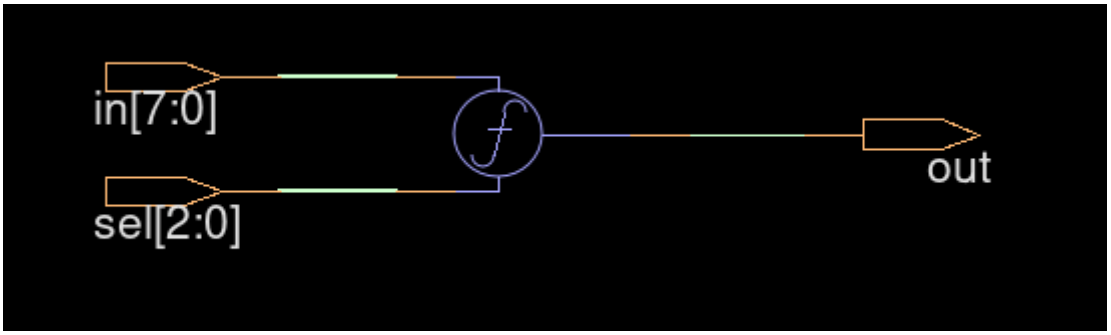
for in=10101010,sel=100,out=0

for in=10101010,sel=101,out=1

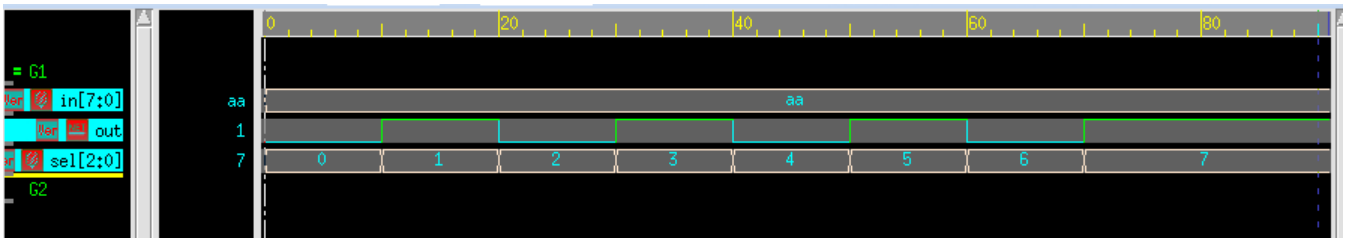
for in=10101010,sel=110,out=0

for in=10101010,sel=111,out=1

Schematic Design



waveform



3.0 Design a 4-bit adder and subtractor by using behavioral modelling

Design code

//Design a verilog code to realize adder and subtractor

module add_sub(a,b,cin,out);

input [3:0]a;

input [3:0]b;

input cin;

output reg [4:0]out;

```
always @(a,b,cin)
begin
	if (cin)
		out=a-b;
	else
		out=a+b;
end
endmodule
```

Testbench

```
// testbech of 4-bit add_sub
module TB();
reg [3:0]a;
reg [3:0]b;
reg cin;
wire [4:0]out;

add_sub dut(.a(a),.b(b),.cin(cin),.out(out));

initial
begin
	$monitor("for a=%0b,b=%0b,cin=%0b,out=%0b",a,b,cin,out);

	a=4'b0111;
b=4'b0010;
cin=0;
```

```
#10;
a=4'b1010;
b=4'b0011;
cin=1;
#10;
a=4'b1100;
b=4'b0111;
cin=0;

#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

```
vcs add_sub.v add_sub_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

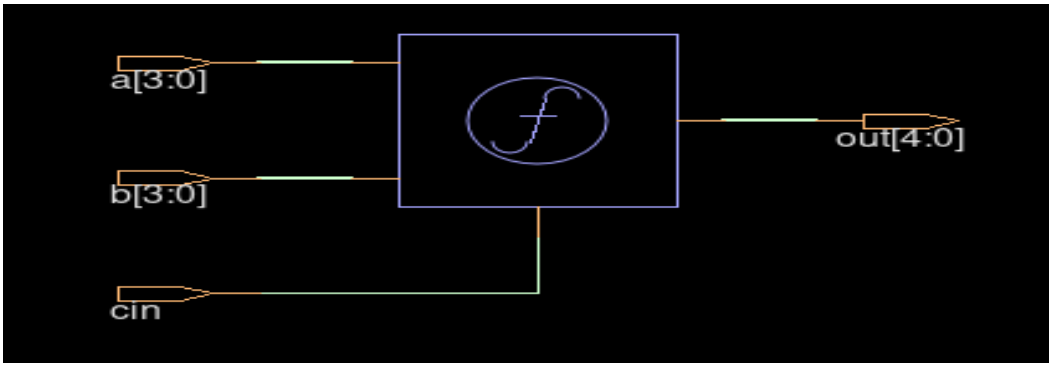
output

for a=111,b=10,cin=0,out=1001

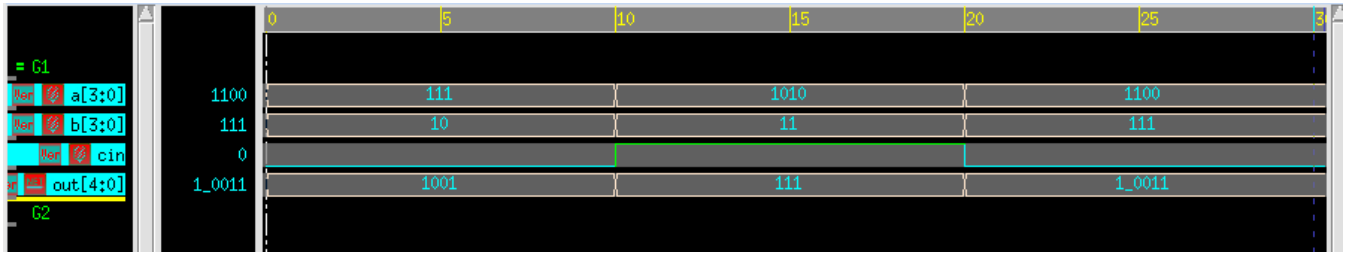
for a=1010,b=11,cin=1,out=111

for a=1100,b=111,cin=0,out=10011

Schematic Design



waveform



4.0 Design a N-bit adder and subtractor by using data flow

Design code

```
//Design a verilog code to realize adder and subtractor
module add_sub #(parameter N=8)(
input [N-1:0]a,
input [N-1:0]b,
input cin,
output [N-1:0]out,
output carry_out,overflow);


wire [N-1:0]b_xor_cin;
wire carry_in;


assign carry_in=cin;
assign b_xor_cin=b ^ {N{cin}};
assign {carry_out,out}=a+b_xor_cin +carry_in;
```

```
assign overflow=(a[N-1] ~^ b_xor_cin[N-1]) & (a[N-1] ^ out[N-1]);
endmodule
```

Testbench

```
// testbech of N-bit add_sub
module TB();
parameter N=8;

reg [N-1:0]a,b;
reg cin;
wire [N-1:0]out;
wire carry_out,overflow;

add_sub #(N)
dut(.a(a),.b(b),.cin(cin),.out(out),.carry_out(carry_out),.overflow(overflow));

initial
begin
      $monitor("for
a=%0d,b=%0d,cin=%0d,out=%0d,carry_out=%0d,overflow=%0d",a,b,cin,out,carry_out,
overflow);

        a=8'd15;
b=8'd10;
cin=0;

#10;
a=8'd100;
```

```
b=8'd200;
cin=0;
#10;
a=8'd25;
b=8'd10;
cin=1;

#10;
a=8'd10;
b=8'd25;
cin=1;

#10;
a=8'd127;
b=8'd1;
cin=0;

#10;
a=8'd128;
b=8'd1;
cin=1;

#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
```

```
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

for a=15,b=10,cin=0,out=25,carry_out=0,overflow=0

for a=100,b=200,cin=0,out=44,carry_out=1,overflow=0

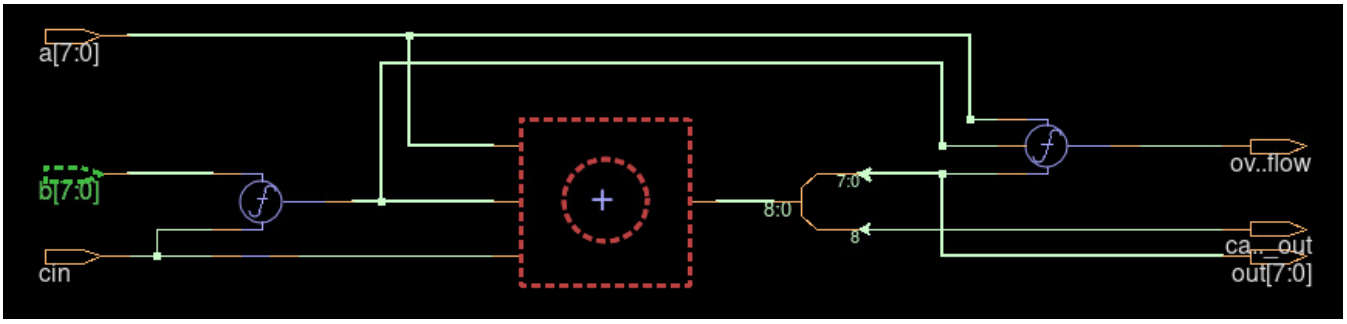
for a=25,b=10,cin=1,out=15,carry_out=1,overflow=0

for a=10,b=25,cin=1,out=241,carry_out=0,overflow=0

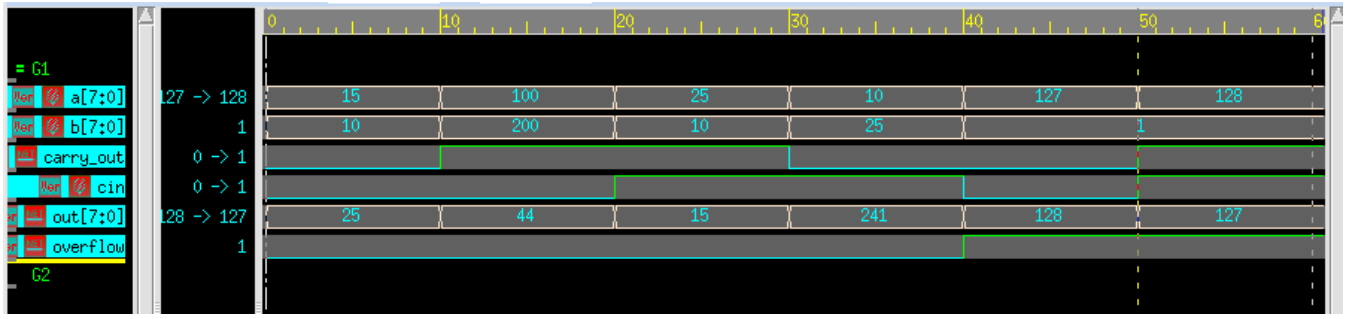
for a=127,b=1,cin=0,out=128,carry_out=0,overflow=1

for a=128,b=1,cin=1,out=127,carry_out=1,overflow=1

Schematic Design



waveform



5.0 BCD to 7-segment code converter

Takes a 4-bit bcd input and generates a 7-bit output

Design code

```
module bcd_to_7seg (bcd_in,leds_out);
input [3:0] bcd_in;
output reg [6:0] leds_out;

always @(*) begin
	case (bcd_in)
		4'd0: leds_out = 7'b1111110;
		4'd1: leds_out = 7'b0110000;
		4'd2: leds_out = 7'b1101101;
		4'd3: leds_out = 7'b1111001;
		4'd4: leds_out = 7'b0110011;
		4'd5: leds_out = 7'b1011011;
		4'd6: leds_out = 7'b1011111;
		4'd7: leds_out = 7'b1110000;
		4'd8: leds_out = 7'b1111111;
		4'd9: leds_out = 7'b1111011;
		default: leds_out = 7'bx;
	endcase
end
endmodule
```

Testbench

```
module TB();
reg [3:0]bcd_in;
wire [6:0]leds_out;
integer i;
```

```
bcd_to_7seg dut(.bcd_in(bcd_in),.leds_out(leds_out));


initial begin
    $monitor("Time=%0t for bcd_in=%0d we get output leds_out=%b
",$time,bcd_in,leds_out);
    for (i=0; i<16;i=i+1)begin
        bcd_in=i;
        #10;
    end
    #10 $finish();
end


initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

Output

```
Time=0 for bcd_in=0 we get output leds_out=1111110
Time=10 for bcd_in=1 we get output leds_out=0110000
Time=20 for bcd_in=2 we get output leds_out=1101101
Time=30 for bcd_in=3 we get output leds_out=1111001
Time=40 for bcd_in=4 we get output leds_out=0110011
Time=50 for bcd_in=5 we get output leds_out=1011011
Time=60 for bcd_in=6 we get output leds_out=1011111
Time=70 for bcd_in=7 we get output leds_out=1110000
```

Time=80 for bcd_in=8 we get output leds_out=1111111

Time=90 for bcd_in=9 we get output leds_out=1111011

Time=100 for bcd_in=10 we get output leds_out=xxxxxxx

Time=110 for bcd_in=11 we get output leds_out=xxxxxxx

Time=120 for bcd_in=12 we get output leds_out=xxxxxxx

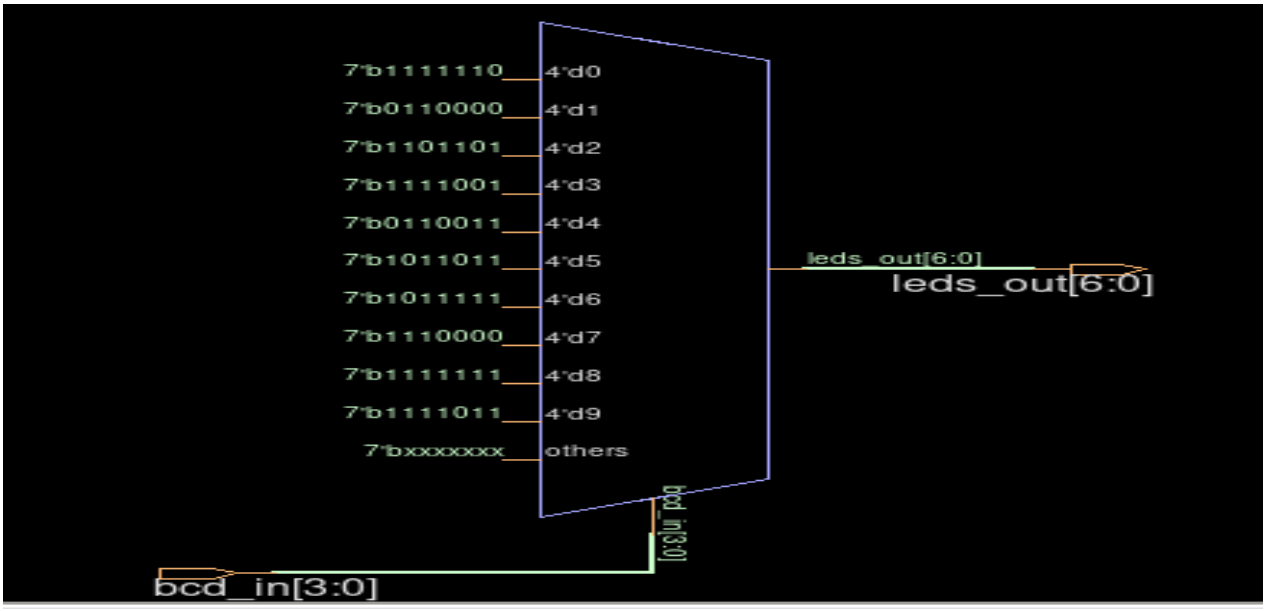
Time=130 for bcd_in=13 we get output leds_out=xxxxxxx

Time=140 for bcd_in=14 we get output leds_out=xxxxxxx

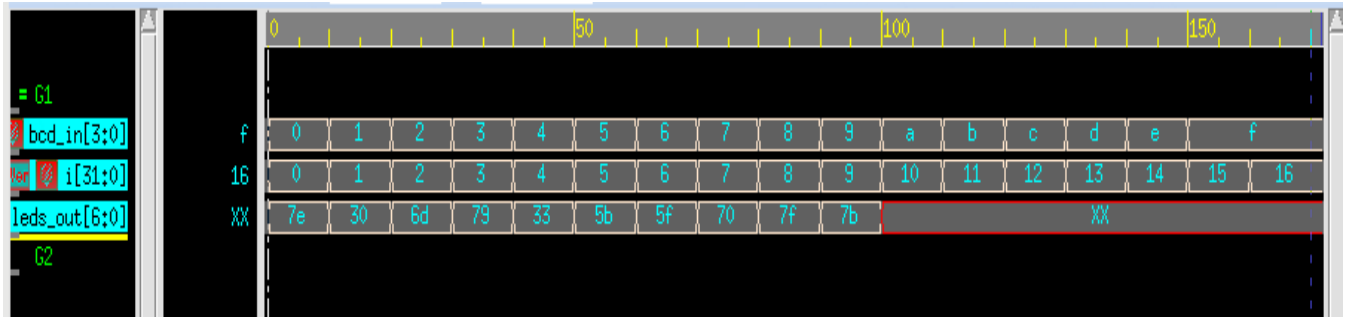
Time=150 for bcd_in=15 we get output leds_out=xxxxxxx

$finish at simulation time 170

Schematic Design



waveform



6.0 Design a Frequency Divider by f/3

Design code

```
module freq_div_by_3 (clk_out,clk,rst);
input clk;
input rst;
output reg clk_out;

reg [1:0] count;

always @(posedge clk) begin
    if (rst) begin
        count <= 2'b00;
        clk_out <= 1'b0;
    end else begin
        if (count == 2'b10) begin
            count <= 1'b0;
            clk_out <= ~clk_out;
        end else begin
            count <= count + 1'b1;
        end
    end
end
endmodule
```

Testbench

```
module TB();
reg clk;
reg rst;
wire clk_out;
```

```
freq_div_by_3 dut (.clk(clk),.rst(rst),.clk_out(clk_out));

initial clk = 0;
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t,for rst=%b,we get clk_out=%b",$time,rst,clk_out);
    rst=1;clk=0;
#10 rst=0;

#200;
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

```
Time=0,for rst=1,we get clk_out=x
Time=5,for rst=1,we get clk_out=0
Time=10,for rst=0,we get clk_out=0
Time=35,for rst=0,we get clk_out=1
Time=65,for rst=0,we get clk_out=0
```

Time=95,for rst=0,we get clk_out=1

Time=125,for rst=0,we get clk_out=0

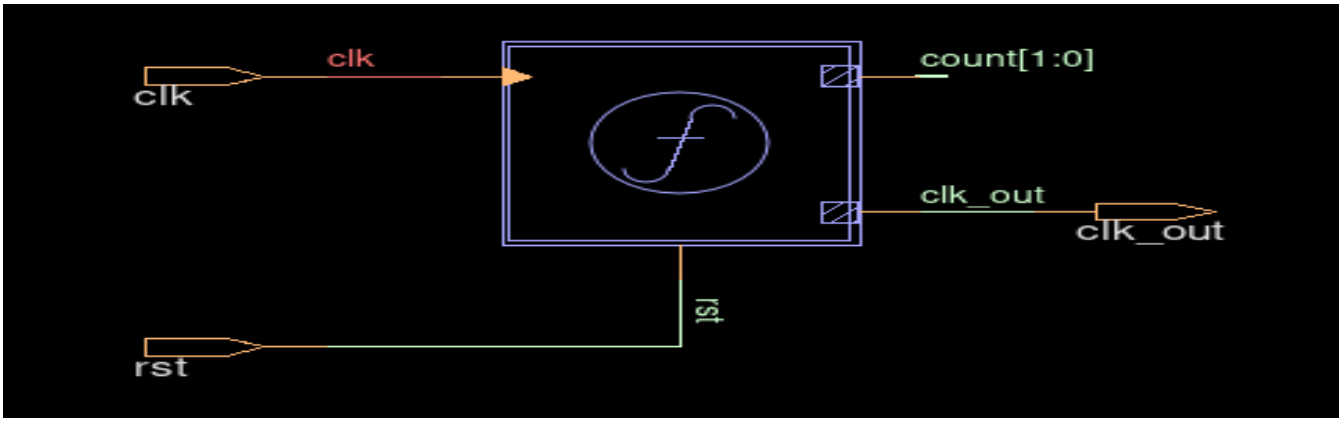
Time=155,for rst=0,we get clk_out=1

Time=185,for rst=0,we get clk_out=0

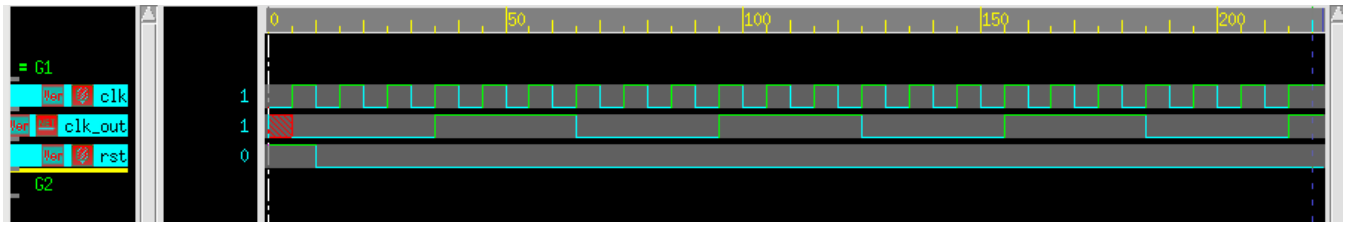
Time=215,for rst=0,we get clk_out=1

$finish at simulation time 220

Schematic Design



waveform



7.0 Design a Frequency Divider by f/1.5

Design code

```
module freq_div_1p5 (clk,rst,clk_out);
input clk;
input rst;
output reg clk_out;

reg [1:0] state;
```

```
reg toggle;

always @(posedge clk or posedge rst) begin
        if (rst) begin
                state <= 2'b00;
                toggle <= 1'b0;
        end else begin
                case (state)
                        2'b00: state <= 2'b01;
                        2'b01: state <= 2'b10;
                        2'b10: state <= 2'b00;
                        default: state <= 2'b00;
                endcase

                if (state == 2'b00 || state == 2'b10)
                        toggle <= ~toggle;
        end
end
always @(negedge clk or posedge rst) begin
    if (rst) begin
        clk_out <= 1'b0;
    end else begin
        if (state == 2'b00 || state == 2'b10)
            clk_out <= toggle;
    end
end
```

```
endmodule
```

Testbench

```
module TB();
reg clk;
reg rst;
wire clk_out;

freq_div_1p5 dut (.clk(clk),.rst(rst),.clk_out(clk_out));

initial clk = 0;
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t,for rst=%b,we get clk_out=%b",$time,rst,clk_out);
    rst=1;clk=0;
#10 rst=0;

#200;
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0,for rst=1,we get clk_out=0

Time=10,for rst=0,we get clk_out=0

Time=30,for rst=0,we get clk_out=1

Time=40,for rst=0,we get clk_out=0

Time=60,for rst=0,we get clk_out=1

Time=70,for rst=0,we get clk_out=0

Time=90,for rst=0,we get clk_out=1

Time=100,for rst=0,we get clk_out=0

Time=120,for rst=0,we get clk_out=1

Time=130,for rst=0,we get clk_out=0

Time=150,for rst=0,we get clk_out=1

Time=160,for rst=0,we get clk_out=0

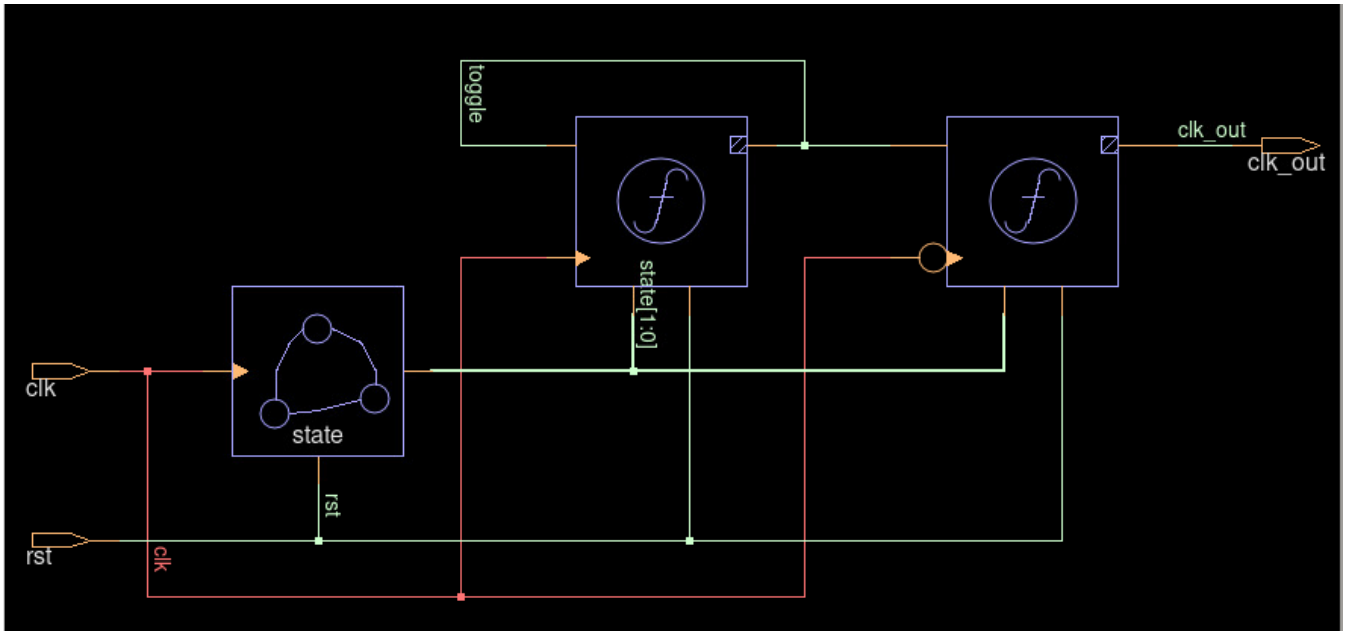
Time=180,for rst=0,we get clk_out=1

Time=190,for rst=0,we get clk_out=0

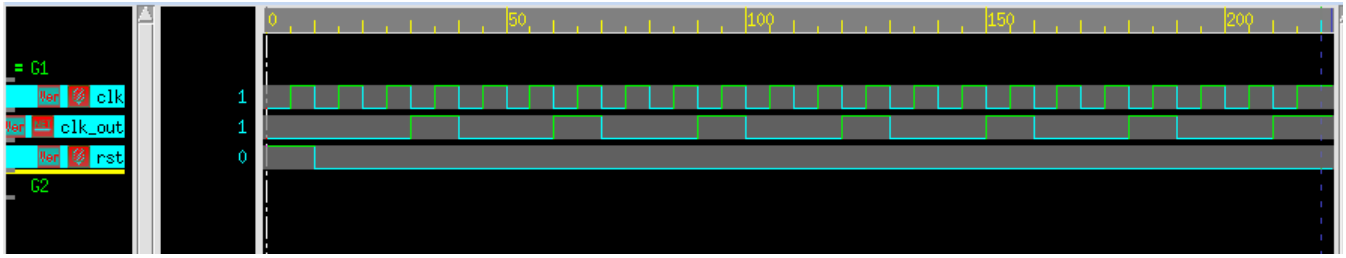
Time=210,for rst=0,we get clk_out=1

$finish at simulation time 220

Schematic Design



waveform



8.0 Design a circuit to rising and falling edge detector

Design code

```
module edge_detector (clk,sig_in,rising_edge,falling_edge);
input clk;
input sig_in;
output reg rising_edge;
output reg falling_edge;

reg prev;
always @(posedge clk)begin
        prev <= sig_in;
        rising_edge <= (~prev) & sig_in; //detect 0 & 1
```

```
    falling_edge <= prev & (~sig_in); //detect 1 & 0
end
endmodule
```

Testbench

```
module TB();
reg clk;
reg sig_in;
wire rising_edge,falling_edge;

edge_detector  dut
(.clk(clk),.sig_in(sig_in),.rising_edge(rising_edge),.falling_edge(falling_edge));

initial clk = 0;
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t,for sig_in=%b,we get rising_edge=%b
falling_edge=%b",$time,sig_in,rising_edge,falling_edge);
    sig_in=0;
#10 sig_in=1;
#10 sig_in=0;
#10 sig_in=1;
#10 sig_in=0;
#10 sig_in=1;
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
```

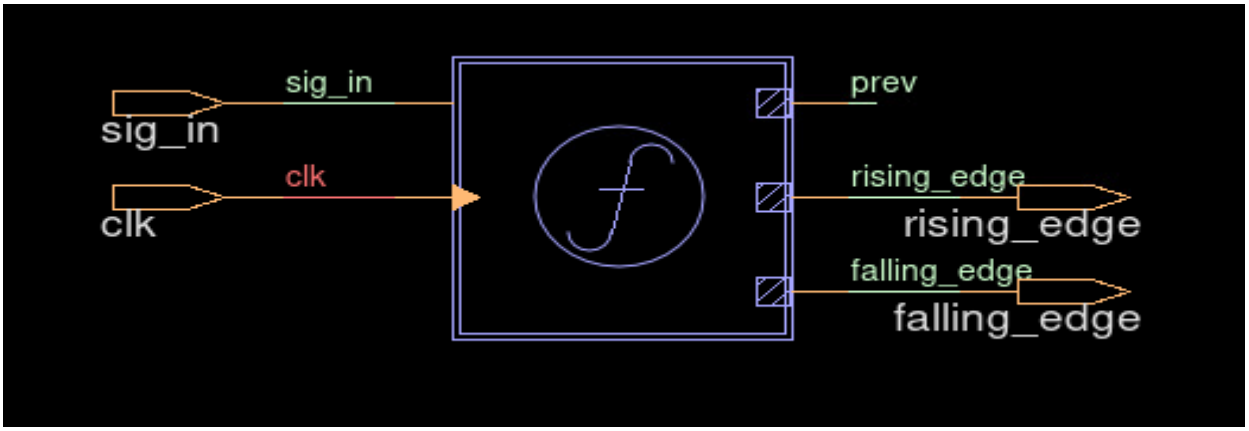
```
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

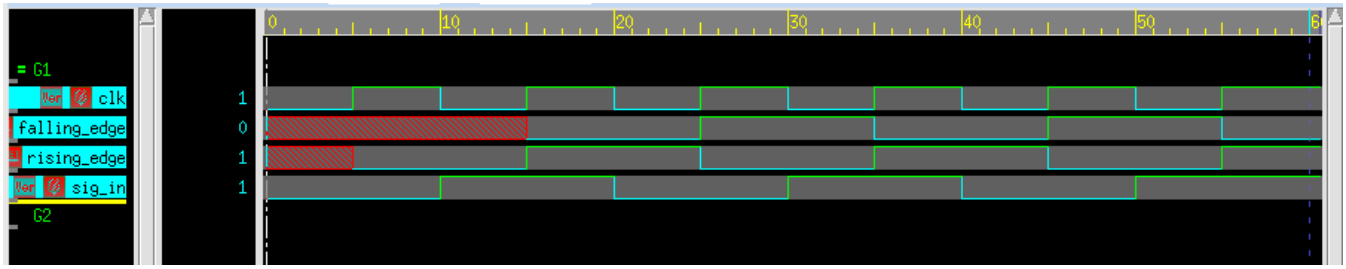
Output

```
Time=0,for sig_in=0,we get rising_edge=x falling_edge=x
Time=5,for sig_in=0,we get rising_edge=0 falling_edge=x
Time=10,for sig_in=1,we get rising_edge=0 falling_edge=x
Time=15,for sig_in=1,we get rising_edge=1 falling_edge=0
Time=20,for sig_in=0,we get rising_edge=1 falling_edge=0
Time=25,for sig_in=0,we get rising_edge=0 falling_edge=1
Time=30,for sig_in=1,we get rising_edge=0 falling_edge=1
Time=35,for sig_in=1,we get rising_edge=1 falling_edge=0
Time=40,for sig_in=0,we get rising_edge=1 falling_edge=0
Time=45,for sig_in=0,we get rising_edge=0 falling_edge=1
Time=50,for sig_in=1,we get rising_edge=0 falling_edge=1
Time=55,for sig_in=1,we get rising_edge=1 falling_edge=0
$finish at simulation time                60
```

Schematic Design



waveform



9.0 Design a 4-bit Carry lookahead adder (CLA) by using dataflow modelling-

Design code

```
module CLA_4bit (a,b,cin,sum,cout);
input [3:0]a,b;
input cin;
output [3:0]sum;
output cout;

wire [3:0]g,p;
wire [3:0]c;

assign g=a & b;//carry generation
assign p=a ^ b;//carry propagation

//carry generation function
assign c[0]=cin;
assign c[1]=g[0] | (p[0] & c[0]);
assign c[2]=g[1] | (p[1] & c[1]);
assign c[3]=g[2] | (p[2] & c[2]);
assign cout=g[3] | (p[3] & c[3]);
```

```
// carry propagation function
assign sum[0]=p[0] ^ c[0];
assign sum[1]=p[1] ^ c[1];
assign sum[2]=p[2] ^ c[2];
assign sum[3]=p[3] ^ c[3];

endmodule
```

Testbench

```
module CLA_4bit_tb();
reg [3:0] a,b;
reg cin;
wire [3:0] sum;
wire cout;

CLA_4bit dut (.a(a),.b(b),.cin(cin),.sum(sum),.cout(cout));

initial begin
    $display("Time |  A     B     Cin | Sum   Cout");
    $monitor("%4t  | %b  %b    %b   | %b     %b",$time,a,b,cin,sum,cout);
    a = 4'b0000; b = 4'b0000; cin = 0; #10;
    a = 4'b0011; b = 4'b0101; cin = 0; #10;
    a = 4'b0011; b = 4'b0101; cin = 1; #10;
    a = 4'b1001; b = 4'b1001; cin = 0; #10;
    a = 4'b1111; b = 4'b0001; cin = 1; #10;
    a = 4'b1111; b = 4'b1111; cin = 0; #10;
    a = 4'b1010; b = 4'b0101; cin = 1; #10;
```

```
        $finish();
end

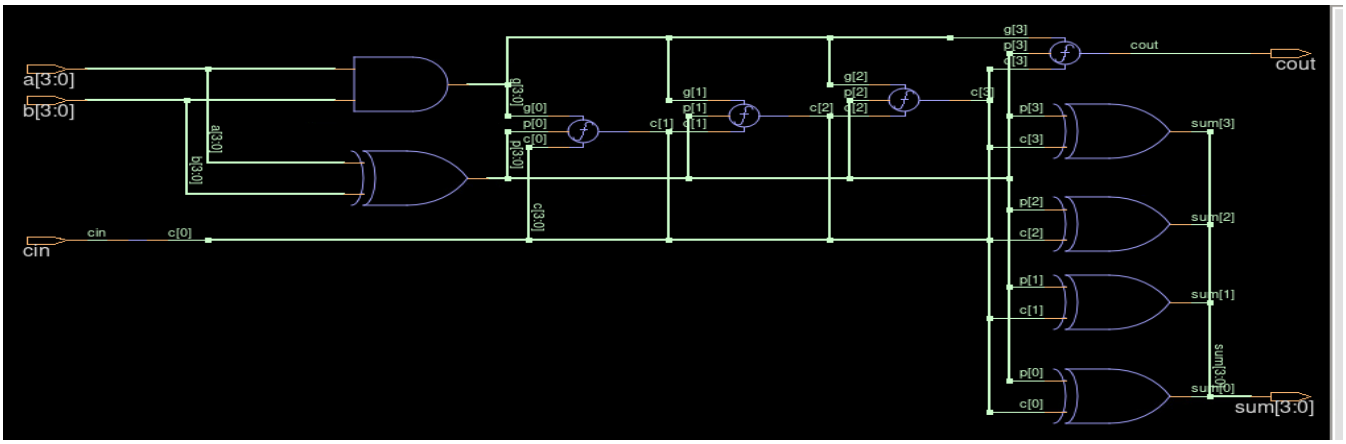
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,CLA_4bit_tb);
end
endmodule
```

Simulation Result

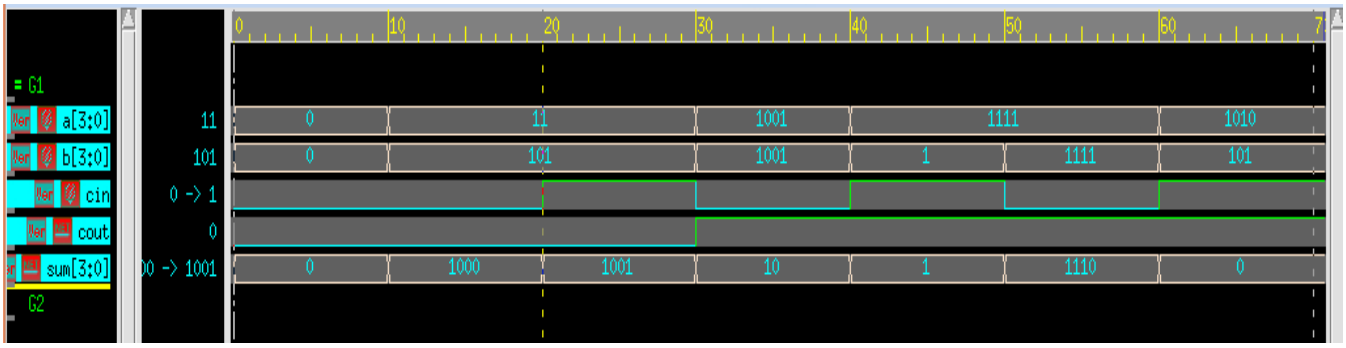
Output

```
  0  | 0000  0000   0   | 0000    0
 10  | 0011  0101   0   | 1000    0
 20  | 0011  0101   1   | 1001    0
 30  | 1001  1001   0   | 0010    1
 40  | 1111  0001   1   | 0001    1
 50  | 1111  1111   0   | 1110    1
 60  | 1010  0101   1   | 0000    1
$finish at simulation time                70
```

Schematic Design



waveform



10.0 Design a, 4-bit Ripple carry adder (RCA) by using data flow

Design code

```
module full_adder (a,b,cin,sum,cout);
input a,b,cin;
output sum,cout;

assign sum  = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (a & cin);
endmodule

module RCA_4bit (a,b,cin,sum,cout);
input [3:0] a,b;
input cin;
output [3:0] sum;
output cout;

wire c1,c2,c3;

full_adder fa0 (.a(a[0]), .b(b[0]),.cin(cin), .sum(sum[0]), .cout(c1));
```

```
full_adder fa1 (.a(a[1]), .b(b[1]),.cin(c1), .sum(sum[1]), .cout(c2));
full_adder fa2 (.a(a[2]), .b(b[2]),.cin(c2), .sum(sum[2]), .cout(c3));
full_adder fa3 (.a(a[3]), .b(b[3]),.cin(c3), .sum(sum[3]), .cout(cout));

endmodule
```

Testbench

```
module RCA_4bit_tb();
reg [3:0] a,b;
reg cin;
wire [3:0] sum;
wire cout;

RCA_4bit dut (.a(a),.b(b),.cin(cin),.sum(sum),.cout(cout));

initial begin
    $display("Time |  A     B     Cin | Sum   Cout");
    $monitor("%4t  | %b  %b    %b   | %b     %b",$time,a,b,cin,sum,cout);

    a = 4'b0000; b = 4'b0000; cin = 0; #10;
    a = 4'b0011; b = 4'b0101; cin = 0; #10;
    a = 4'b0011; b = 4'b0101; cin = 1; #10;
    a = 4'b1001; b = 4'b1001; cin = 0; #10;
    a = 4'b1111; b = 4'b0001; cin = 1; #10;
    a = 4'b1111; b = 4'b1111; cin = 0; #10;
    a = 4'b1010; b = 4'b0101; cin = 1; #10;
    $finish();
end
```

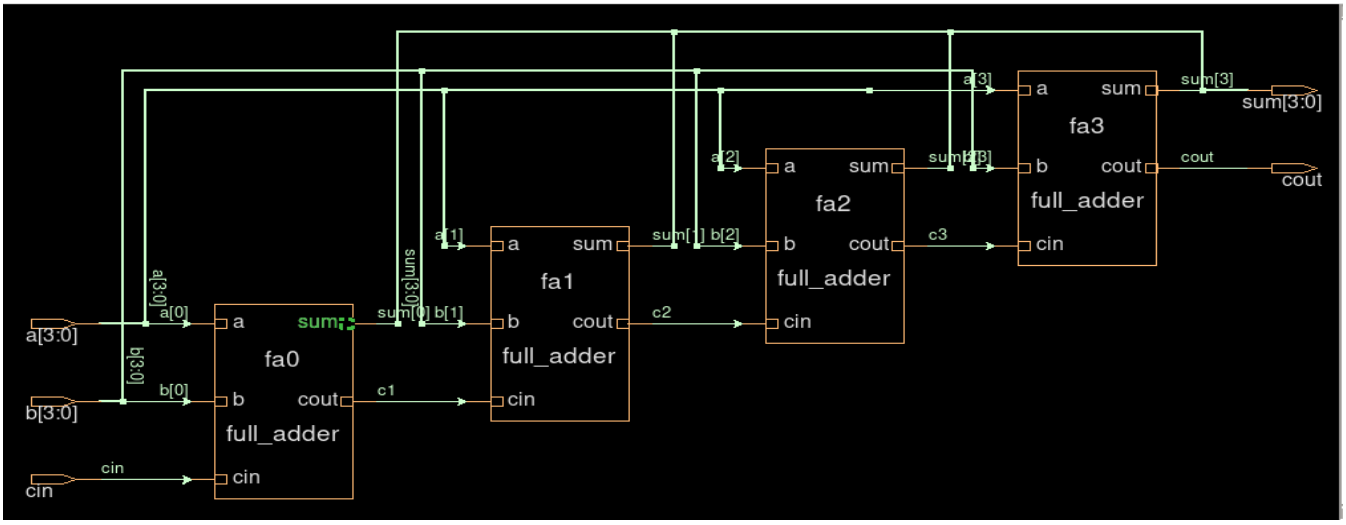
```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,RCA_4bit_tb);
end
endmodule
```

Simulation Result

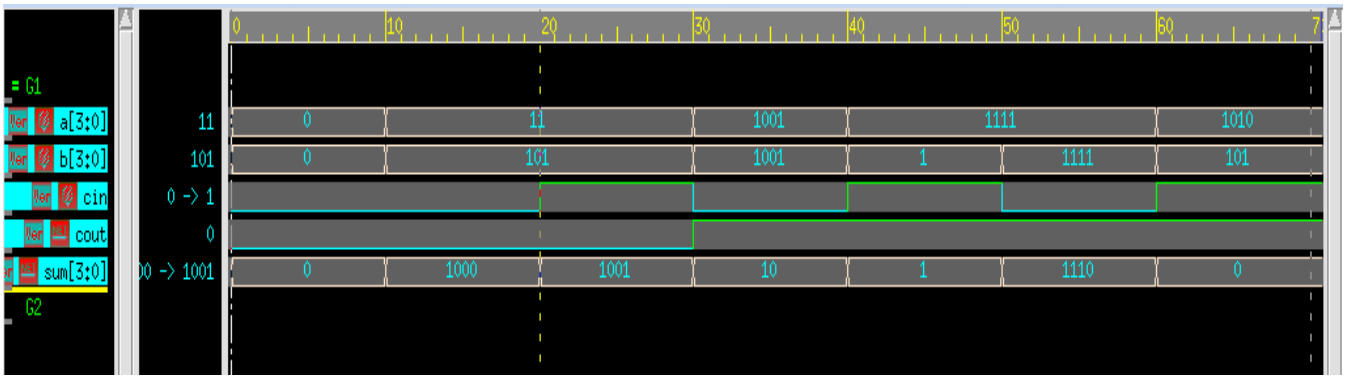
Output

```
  0  | 0000  0000    0   | 0000    0
 10  | 0011  0101    0   | 1000    0
 20  | 0011  0101    1   | 1001    0
 30  | 1001  1001    0   | 0010    1
 40  | 1111  0001    1   | 0001    1
 50  | 1111  1111    0   | 1110    1
 60  | 1010  0101    1   | 0000    1
$finish at simulation time                70
```

Schematic Design



waveform



Method-2 by using the generate block

Design code

```
module full_adder (a,b,cin,sum,cout);
input a,b,cin;
output sum,cout;

assign sum  = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (a & cin);
endmodule

module RCA_4bit (a,b,cin,sum,cout);
input [3:0] a,b;
input cin;
output [3:0] sum;
output cout;

wire [4:0] carry;
assign carry[0] =cin;

genvar i;
generate
```

```
    for (i = 0; i < 4; i = i + 1) begin : FA_LOOP
        full_adder fa (.a(a[i]),.b(b[i]),.cin(carry[i]),.sum(sum[i]),.cout(carry[i+1]));
    end
endgenerate

assign cout = carry[4];
endmodule
```

11.0 Design a Full Adder (FA) by using data flow

Design code

```
// Design a full adder by using the data flow modelling
module Full_adder(sum,cout,a,b,cin);
output sum,cout;
input a,b,cin;

assign sum= a^b^cin;
assign cout=(a & b) |(b & cin) |(cin & a);
endmodule
```

Testbench

```
//Testbench of full adder
module TB( );
reg a,b,cin;
wire sum,cout;

Full_adder dut(.a(a),.b(b),.cin(cin),.sum(sum),.cout(cout));

initial begin
```

```
    $monitor("Time=%0d a=%0b b=%0b cin=%0b sum=%0b
cout=%0b",$time,a,b,cin,sum,cout);
    a=0 ; b=0 ; cin=0 ;
    # 10 a=0 ; b=0 ; cin=1 ;
    # 10 a=0 ; b=1 ; cin=0 ;
    # 10 a=0 ; b=1 ; cin=1 ;
    # 10 a=1 ; b=0 ; cin=0 ;
    # 10 a=1 ; b=0 ; cin=1 ;
    # 10 a=1 ; b=1 ; cin=0 ;
    # 10 a=1 ; b=1 ; cin=1 ;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

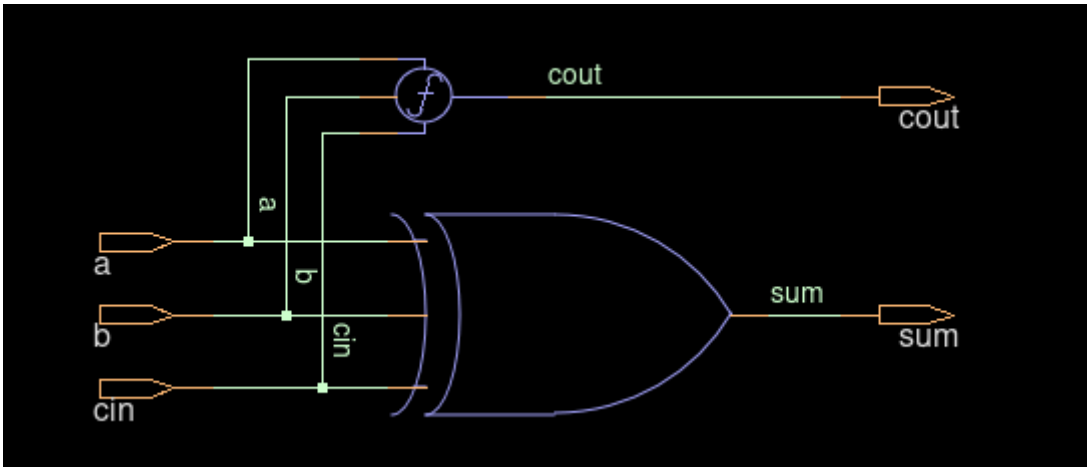
Output

```
Time=0 a=0 b=0 cin=0 sum=0 cout=0
Time=10 a=0 b=0 cin=1 sum=1 cout=0
Time=20 a=0 b=1 cin=0 sum=1 cout=0
Time=30 a=0 b=1 cin=1 sum=0 cout=1
Time=40 a=1 b=0 cin=0 sum=1 cout=0
Time=50 a=1 b=0 cin=1 sum=0 cout=1
Time=60 a=1 b=1 cin=0 sum=0 cout=1
```

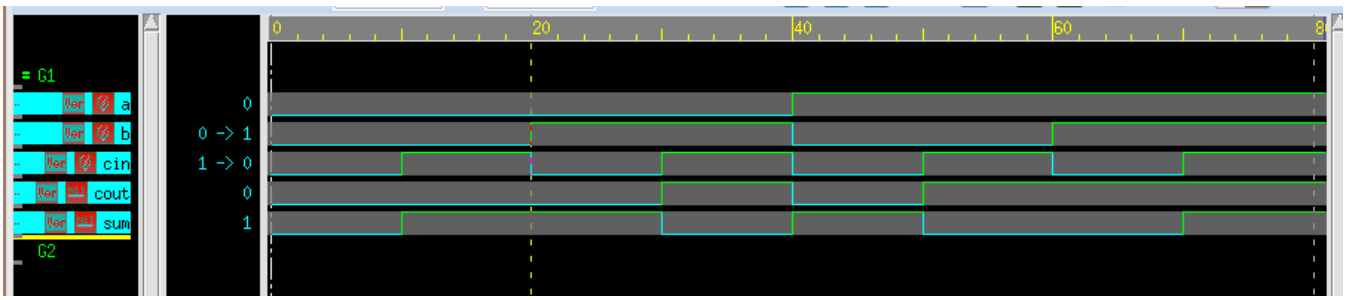
Time=70 a=1 b=1 cin=1 sum=1 cout=1

$finish at simulation time 80

Schematic Design



waveform



12.0 Design a Half Subtractor (HS) by using data flow

Design code

```
module Half_subtractor(diff,borrow,a,b);
input a,b;
output diff,borrow;

assign diff= a^b;
assign borrow= ~a & b;
endmodule
```

Testbench

```
module TB( );
reg a,b;
wire diff,borrow;

Half_subtractor dut(.a(a),.b(b),.diff(diff),.borrow(borrow));

initial begin
    $monitor("Time=%0d a=%0b b=%0b diff=%0b
borrow=%0b",$time,a,b,diff,borrow);
    a=0 ; b=0 ;
    # 10 a=0 ; b=1 ;
    # 10 a=1 ; b=0 ;
    # 10 a=1 ; b=1 ;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

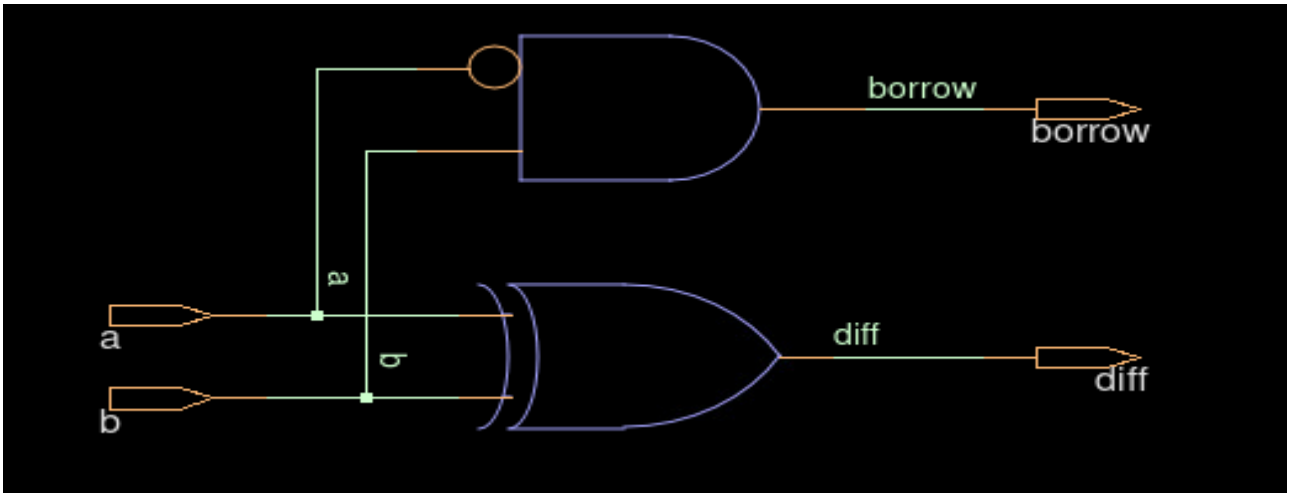
Simulation Result

Output

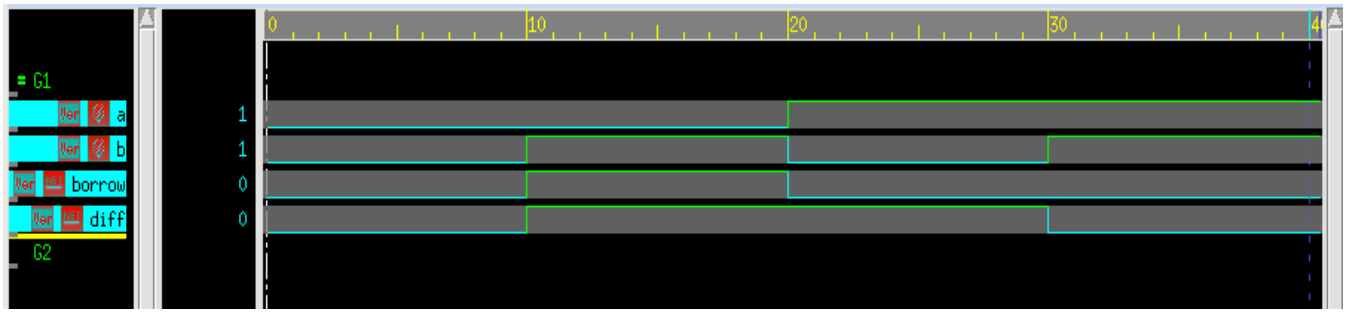
```
Time=0 a=0 b=0 diff=0 borrow=0
Time=10 a=0 b=1 diff=1 borrow=1
Time=20 a=1 b=0 diff=1 borrow=0
Time=30 a=1 b=1 diff=0 borrow=0
```

$finish at simulation time 40

Schematic Design



waveform



13.0 Design a Full Subtractor (FS) by using data flow

Design code

```
module Full_subtractor(diff,borrow,a,b,cin);
input a,b,cin;
output diff,borrow;

assign diff= a^b^cin;
assign borrow=((~a & b) | (b & cin) | (cin & ~a));
endmodule
```

Testbench

```
module TB( );
reg a,b,cin;
wire diff,borrow;

Full_subtractor dut(.a(a),.b(b),.cin(cin),.diff(diff),.borrow(borrow));

initial begin
    $monitor("Time=%0d a=%0b b=%0b cin=%0b diff=%0b
borrow=%0b",$time,a,b,cin,diff,borrow);
    a=0 ; b=0 ; cin=0 ;
    # 10 a=0 ; b=0 ; cin=1 ;
    # 10 a=0 ; b=1 ; cin=0 ;
    # 10 a=0 ; b=1 ; cin=1 ;
    # 10 a=1 ; b=0 ; cin=0 ;
    # 10 a=1 ; b=0 ; cin=1 ;
    # 10 a=1 ; b=1 ; cin=0 ;
    # 10 a=1 ; b=1 ; cin=1 ;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

Output

Time=0 a=0 b=0 cin=0 diff=0 borrow=0

Time=10 a=0 b=0 cin=1 diff=1 borrow=1

Time=20 a=0 b=1 cin=0 diff=1 borrow=1

Time=30 a=0 b=1 cin=1 diff=0 borrow=1

Time=40 a=1 b=0 cin=0 diff=1 borrow=0

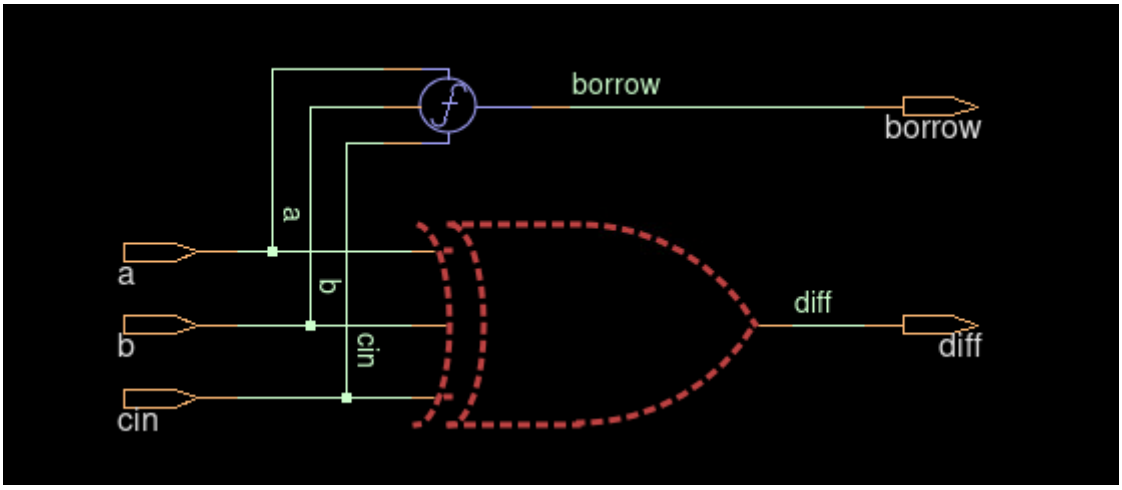
Time=50 a=1 b=0 cin=1 diff=0 borrow=0

Time=60 a=1 b=1 cin=0 diff=0 borrow=0

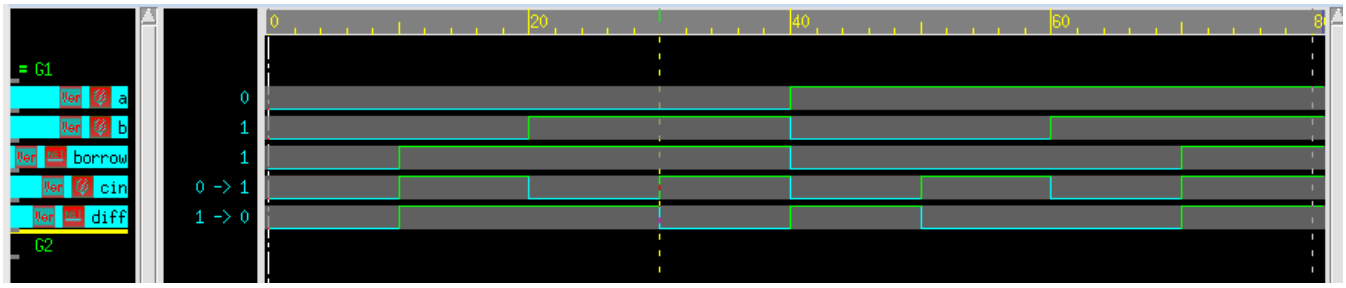
Time=70 a=1 b=1 cin=1 diff=1 borrow=1

$finish at simulation time 80

Schematic Design



waveform



14.0 Design a 16x1 MUX using 4x1 mux

Design code

module mux4x1(out,in,sel);

```
input [3:0]in;
input [1:0]sel;
output reg out;

always @(*)begin
	case(sel)
		2'b00:out=in[0];
		2'b01:out=in[1];
		2'b10:out=in[2];
		2'b11:out=in[3];
		default:out=1'bx;
	endcase
end
endmodule
module mux16x1(out,in,sel);
input [15:0]in;
input [3:0]sel;
output out;

wire [3:0]w;

mux4x1 m0(.in(in[3:0]),.sel(sel[1:0]),.out(w[0]));
mux4x1 m1(.in(in[7:4]),.sel(sel[1:0]),.out(w[1]));
mux4x1 m2(.in(in[11:8]),.sel(sel[1:0]),.out(w[2]));
mux4x1 m3(.in(in[15:12]),.sel(sel[1:0]),.out(w[3]));
mux4x1 m4(.in(w[3:0]),.sel(sel[3:2]),.out(out));
endmodule
```

Testbench

```
module TB( );
reg [15:0]in;
reg [3:0]sel;
wire out;

mux16x1 dut(.in(in),.sel(sel),.out(out));

initial begin
    $monitor("Time=%0d in=%0b sel=%0b out=%0b ",$time,in,sel,out);
    in=16'b1010_1100_0001_1111;
    sel=4'b0000;
    #10 sel=4'b0001;
    #10 sel=4'b0111;
    #10 sel=4'b1111;
    #10 sel=4'b1110;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 in=1010110000011111 sel=0 out=1

Time=10 in=1010110000011111 sel=1 out=1

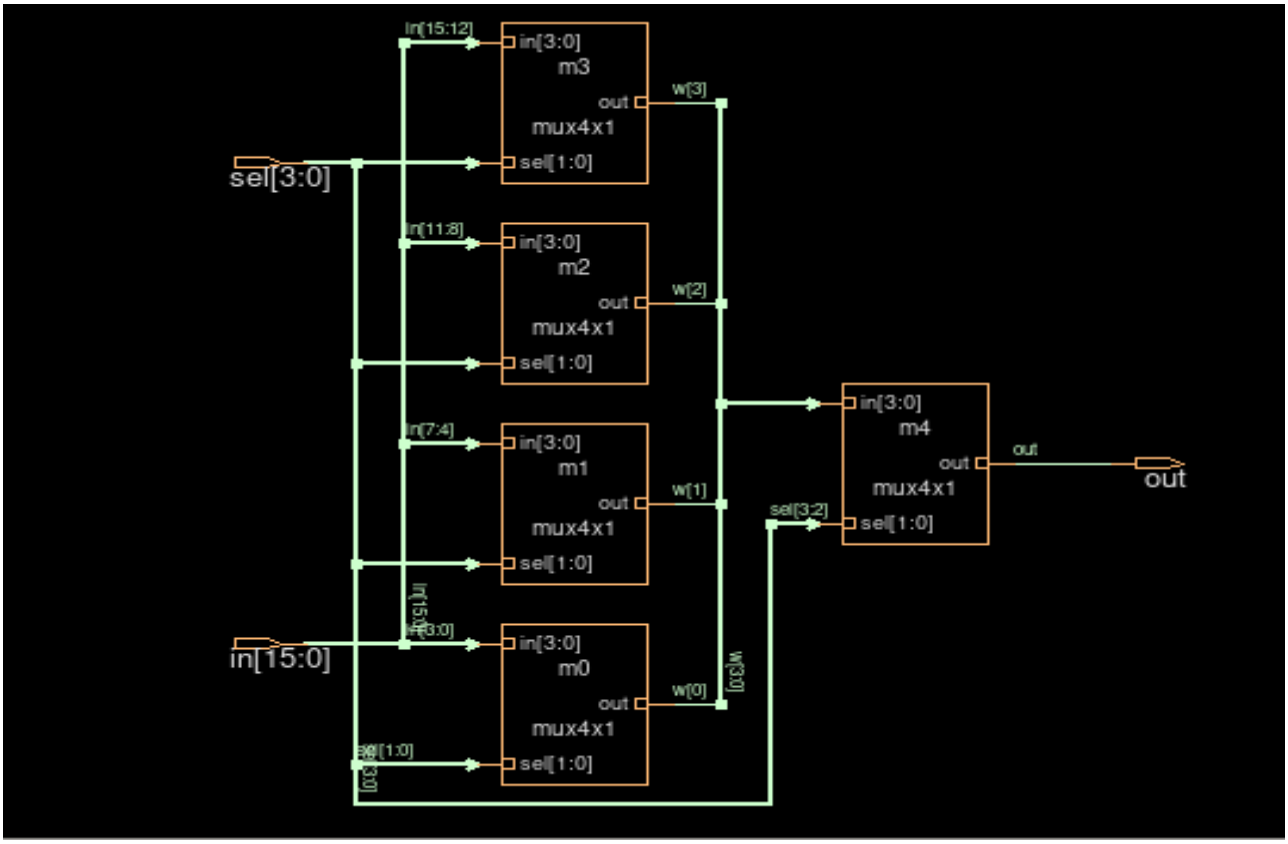
Time=20 in=1010110000011111 sel=111 out=0

Time=30 in=1010110000011111 sel=1111 out=1

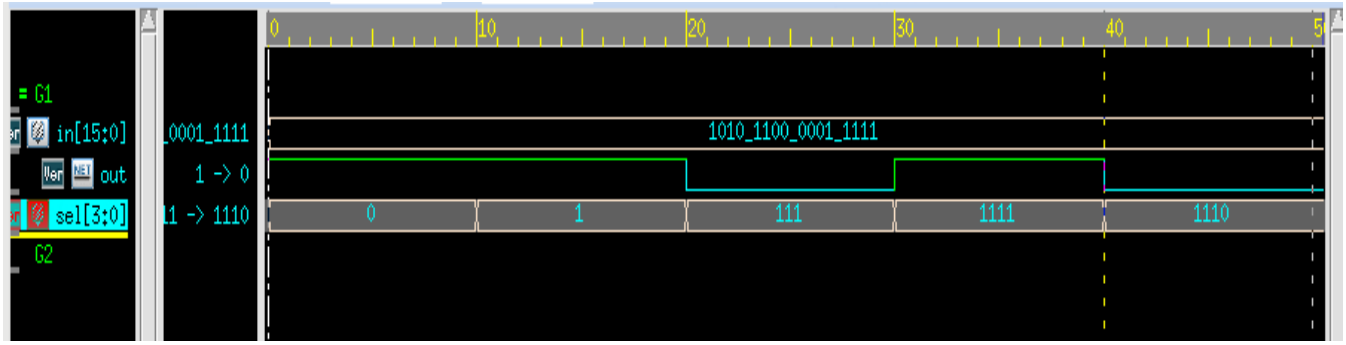
Time=40 in=1010110000011111 sel=1110 out=0

$finish at simulation time 50

Schematic Design



waveform



15.0 Design a 1x2 D-MUX

Design code

```
module Demux_1x2 (y0,y1,din,sel);
input din,sel;
output y0,y1;

assign {y0,y1}= sel? {1'b0,din} :{din,1'b0};
endmodule
```

Testbench

```
module TB( );
reg din,sel;
wire y0,y1;

Demux_1x2 dut(.din(din),.sel(sel),.y0(y0),.y1(y1));

initial begin
    $monitor("Time=%0d din=%0b sel=%0b y0=%0b y1=%0b",$time,din,sel,y0,y1);
    din=0;
    sel=1;
    #10 sel=0;
    #10 din=1;
    #10 sel=1;
    #10 sel=0;
    #10 $finish();
end
initial begin
```

```
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 din=0 sel=1 y0=0 y1=0

Time=10 din=0 sel=0 y0=0 y1=0

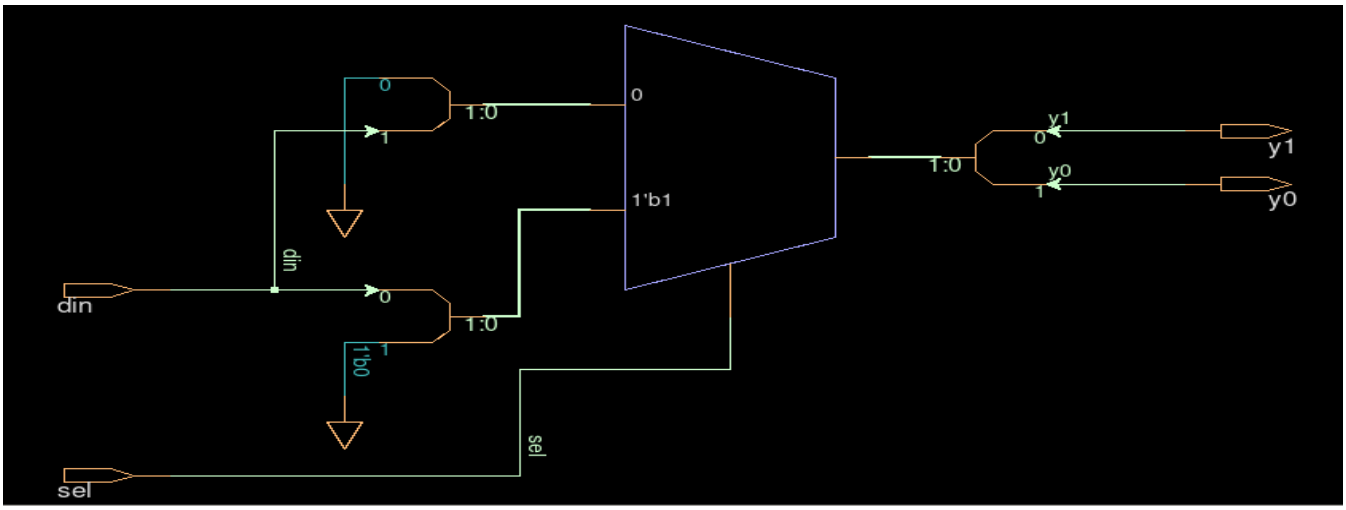
Time=20 din=1 sel=0 y0=1 y1=0

Time=30 din=1 sel=1 y0=0 y1=1

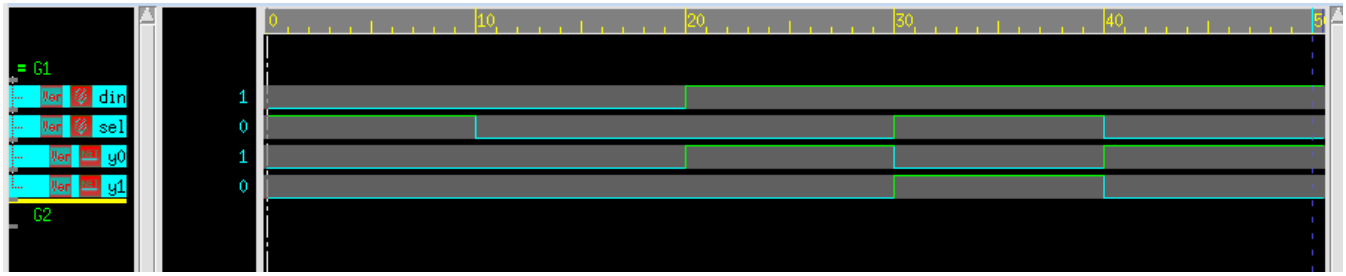
Time=40 din=1 sel=0 y0=1 y1=0

$finish at simulation time 50

Schematic Design



waveform



15.0 Design a 1x4 D-MUX by using behavioural

Design code

```
module Demux_1x4 (y,din,sel);
input din;
input [1:0]sel;
output reg [3:0]y;

always @(*)begin
	y=4'b0000;
	case(sel)
		2'b00:y[0]=din;
		2'b01:y[1]=din;
		2'b10:y[2]=din;
		2'b11:y[3]=din;
	endcase
end
endmodule
```

Testbench

```
module TB();
reg din;
reg [1:0]sel;
wire [3:0]y;

Demux_1x4 dut(.din(din),.sel(sel),.y(y));
initial begin
	$monitor("Time=%0t for sel=%b din=%b and we get y=%b",$time,sel,din,y);
```

```
        din=1;sel=2'b00;
#10 sel=2'b01;
#10 sel=2'b10;
#10 sel=2'b11;

#10 din=0;sel=2'b00;
#10 sel=2'b01;
#10 din=1;sel=2'b10;
#10 sel=2'b11;
#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

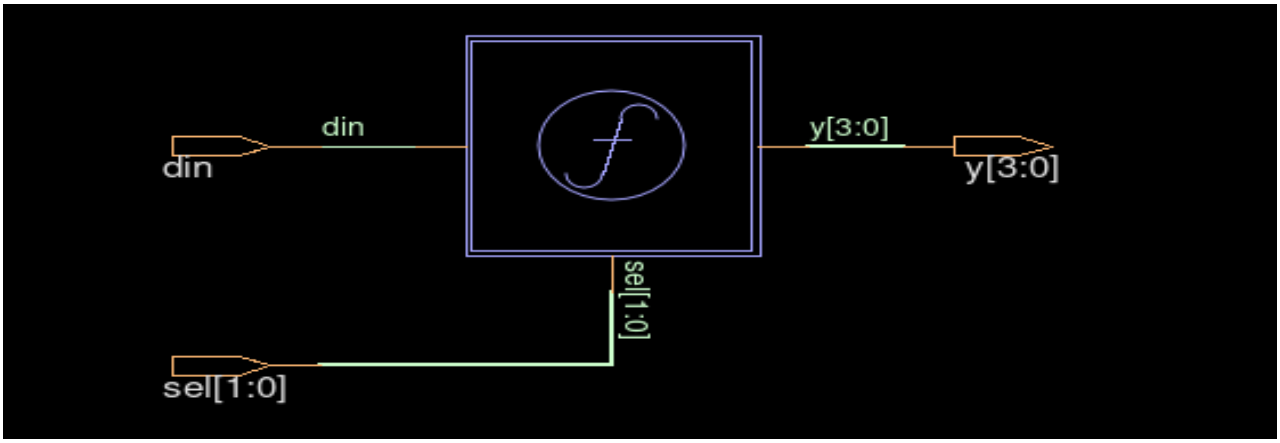
output

```
Time=0 for sel=00 din=1 and we get y=0001
Time=10 for sel=01 din=1 and we get y=0010
Time=20 for sel=10 din=1 and we get y=0100
Time=30 for sel=11 din=1 and we get y=1000
Time=40 for sel=00 din=0 and we get y=0000
Time=50 for sel=01 din=0 and we get y=0000
Time=60 for sel=10 din=1 and we get y=0100
```

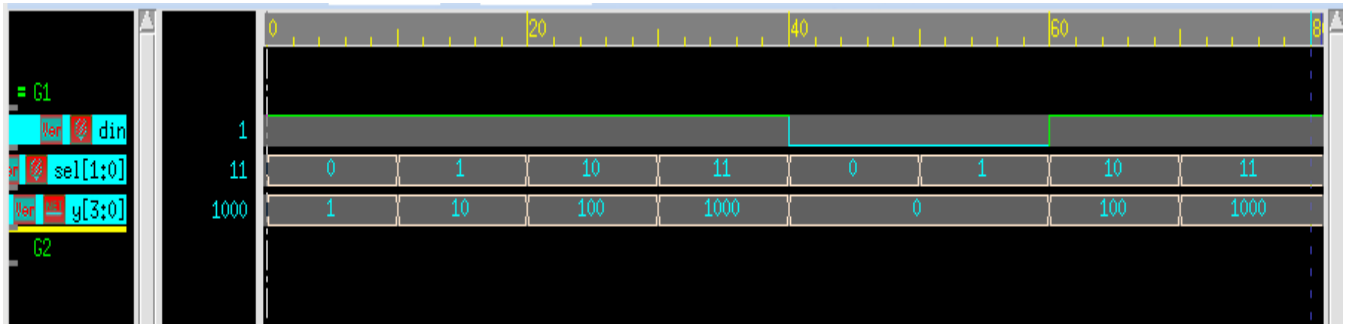
Time=70 for sel=11 din=1 and we get y=1000

$finish at simulation time 80

Schematic Design



waveform



16.0 Design a 4x16 Decoder using 2x4 decoder behavioural

Design code

```
module Decoder_2x4 (y,din,en);
input [1:0]din;
input en;
output reg [3:0]y;

always @(*)begin
    if(en)begin
        case(din)
```

```
                2'b00: y=4'b0001;
                2'b01: y=4'b0010;
                2'b10: y=4'b0100;
                2'b11: y=4'b1000;
                default: y=4'b0000;
            endcase
        end
        else
            y=4'b0000;
end
endmodule

module Decoder_4x16 (y, din, en);
input [3:0]din;
input en;
output [15:0]y;

wire [3:0]w;

Decoder_2x4 d0(.din(din[3:2]),.en(en),.y(w));
Decoder_2x4 d1(.din(din[1:0]),.en(w[0]),.y(y[3:0]));
Decoder_2x4 d2(.din(din[1:0]),.en(w[1]),.y(y[7:4]));
Decoder_2x4 d3(.din(din[1:0]),.en(w[2]),.y(y[11:8]));
Decoder_2x4 d4(.din(din[1:0]),.en(w[3]),.y(y[15:12]));
endmodule
```

Testbench

```
module TB( );
```

```
reg [3:0]din;
reg en;
wire [15:0]y;

Decoder_4x16 dut(.din(din),.en(en),.y(y));

initial begin
    $monitor("Time=%0d din=%b en=%0b y=%b ",$time,din,en,y);
    en=0;din=4'b0000;
    #10 en=1;
    din=4'b0000;
    #10 din=4'b0101;
    #10 din=4'b1010;
    #10 din=4'b1111;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

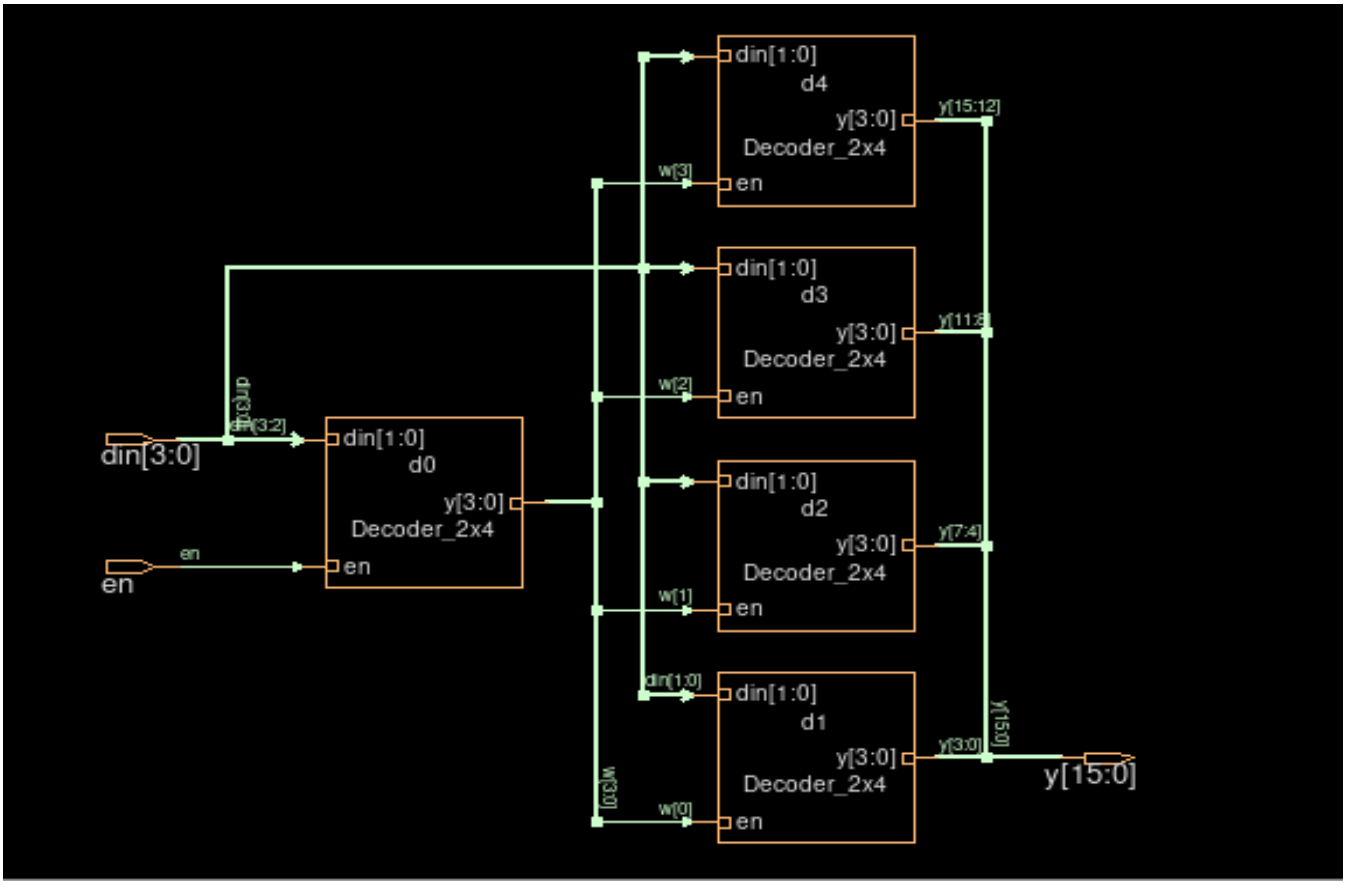
```
Time=0 din=0000 en=0 y=0000000000000000
Time=10 din=0000 en=1 y=0000000000000001
Time=20 din=0101 en=1 y=0000000000100000
```

Time=30 din=1010 en=1 y=0000010000000000

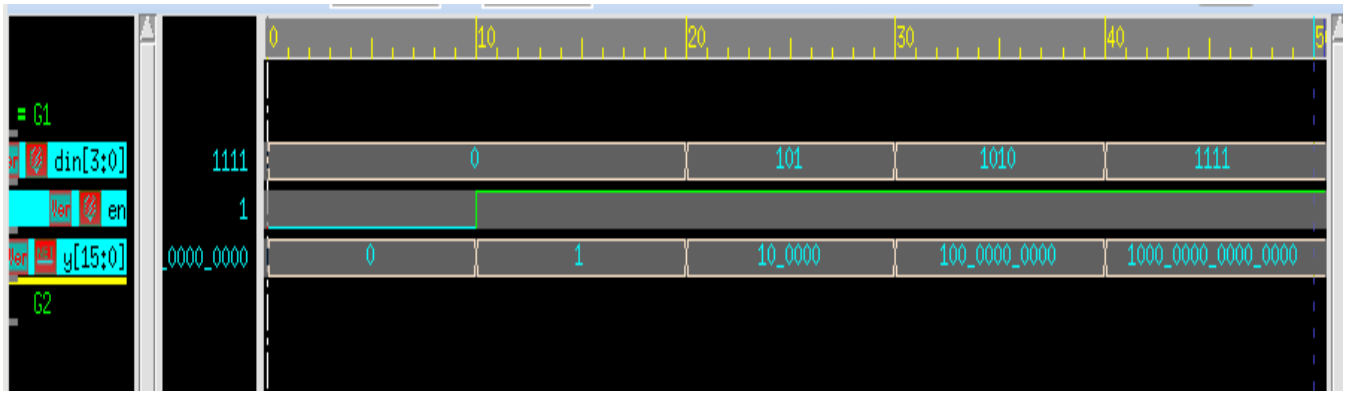
Time=40 din=1111 en=1 y=1000000000000000

$finish at simulation time 50

Schematic Design



waveform



17.0 Design an 8x3 priority encoder by using if else

Design code

```
module priority_encoder_8x3(y,a,en);
input [7:0]a;
input en;
output reg [2:0]y;

always @(a,en)begin
	if (en)begin

		if	(a[7]) y=3'b111;
		else if (a[6]) y=3'b110;
		else if (a[5]) y=3'b101;
		else if (a[4]) y=3'b100;
		else if (a[3]) y=3'b011;
		else if (a[2]) y=3'b010;
		else if (a[1]) y=3'b001;
		else if (a[0]) y=3'b000;
		else		y=3'bxxx;
	end else begin
		y=3'bzzz;
	end
end
endmodule
```

Testbench

```
module TB( );
reg [7:0]a;
reg en;
wire [2:0]y;
```

```
priority_encoder_8x3 dut(.a(a),.en(en),.y(y));

initial begin
    $monitor("Time=%0d a=%b en=%0b y=%b ",$time,a,en,y);
    en=0;a=8'b00000000;
    #10 en=1;
    a=8'b00000000;
    #10 a=8'b00000001;
    #10 a=8'b01010000;
    #10 a=8'b00010000;
    #10 a=8'b10000001;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

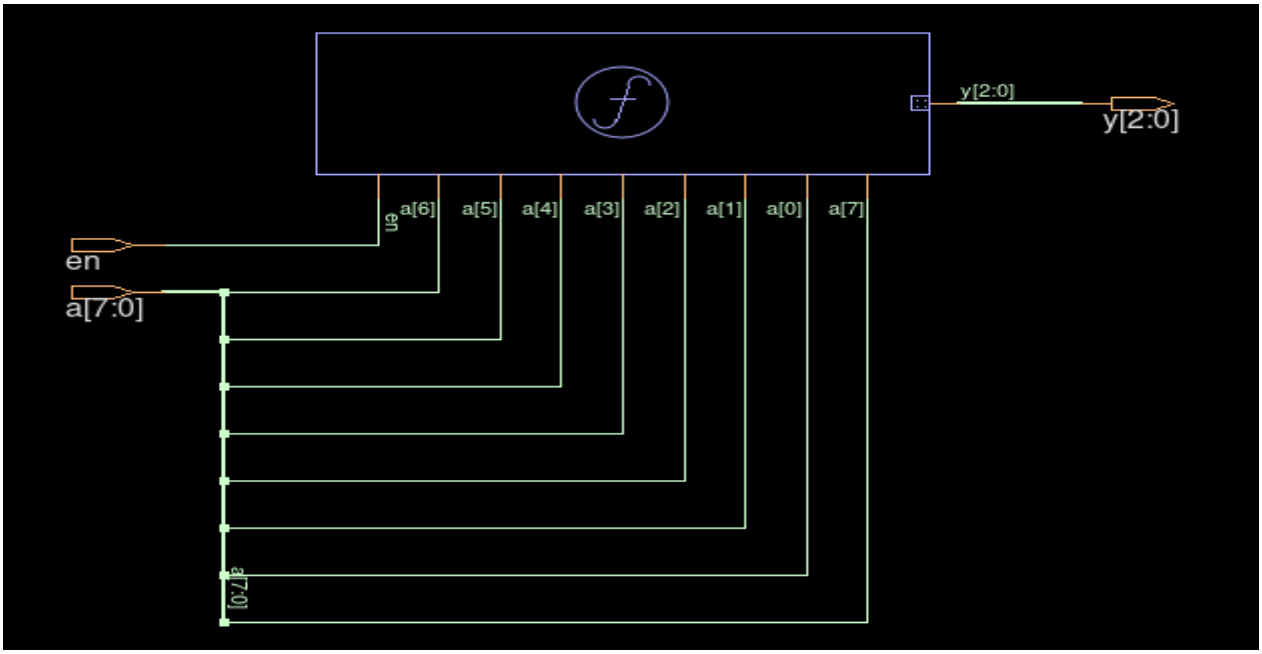
output

```
Time=0 a=00000000 en=0 y=zzz
Time=10 a=00000000 en=1 y=xxx
Time=20 a=00000001 en=1 y=000
Time=30 a=01010000 en=1 y=110
Time=40 a=00010000 en=1 y=100
```

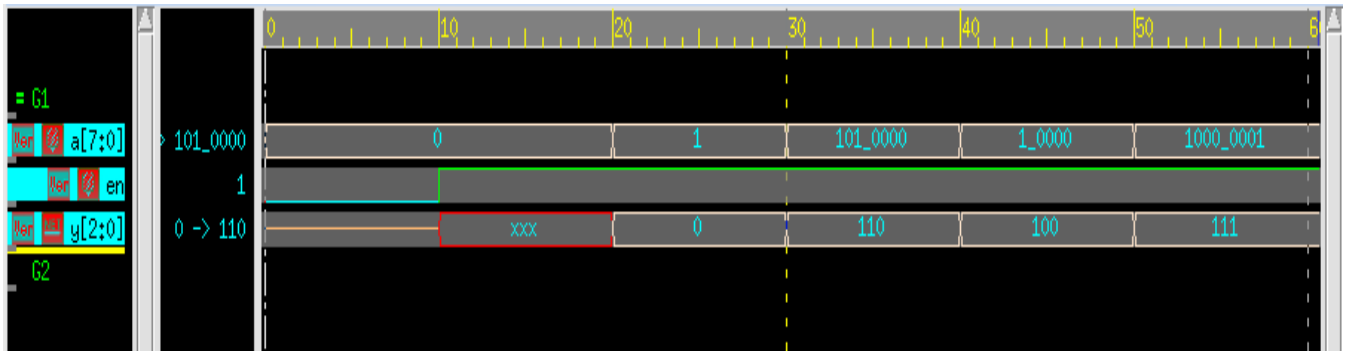
Time=50 a=10000001 en=1 y=111

$finish at simulation time 60

Schematic Design



waveform



18.0 Design an even and odd parity generation

Design code

module even_parity_gen(p, in);

input [3:0]in;

output p;

assign p= in[3]^in[2]^in[1]^in[0];

```
endmodule
```

Testbench

```
module TB( );
reg [3:0]in;
wire p;

even_parity_gen dut(.in(in),.p(p));

initial begin
    $monitor("Time=%0t in=%b p=%b ",$time,in,p);
    in=4'b0000;
    #10 in=4'b1010;
    #10 in=4'b1110;
    #10 in=4'b0101;
    #10 in=4'b1011;
    #10 in=4'b1001;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 in=0000 p=0

Time=10 in=1010 p=0

Time=20 in=1110 p=1

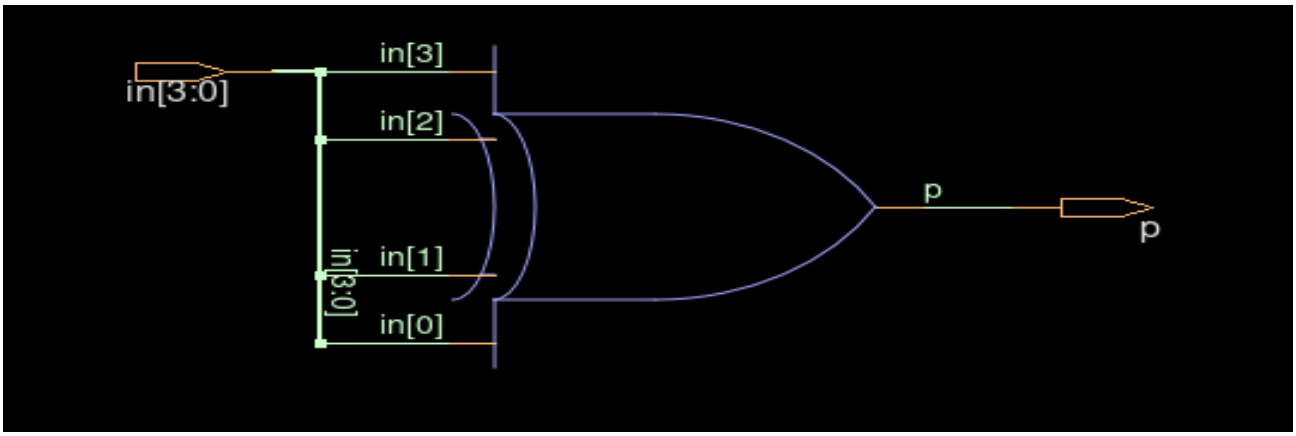
Time=30 in=0101 p=0

Time=40 in=1011 p=1

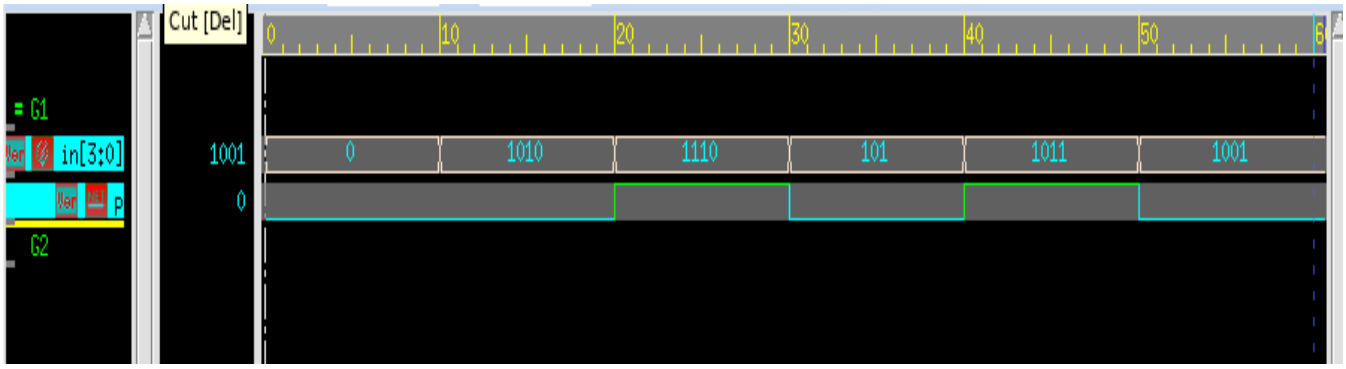
Time=50 in=1001 p=0

$finish at simulation time 60

Schematic Design



waveform



Design code- odd parity generation

```
module odd_parity_gen(p, in);

input [3:0]in;

output p;
```

```
assign p= ~(in[3]^in[2]^in[1]^in[0]);

endmodule
```

Testbench

```
module TB( );
reg [3:0]in;
wire p;

odd_parity_gen dut(.in(in),.p(p));

initial begin
        $monitor("Time=%0t in=%b p=%b ",$time,in,p);
        in=4'b0000;
        #10 in=4'b1010;
        #10 in=4'b1110;
        #10 in=4'b0101;
        #10 in=4'b1011;
        #10 in=4'b1001;
        #10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 in=0000 p=1

Time=10 in=1010 p=1

Time=20 in=1110 p=0

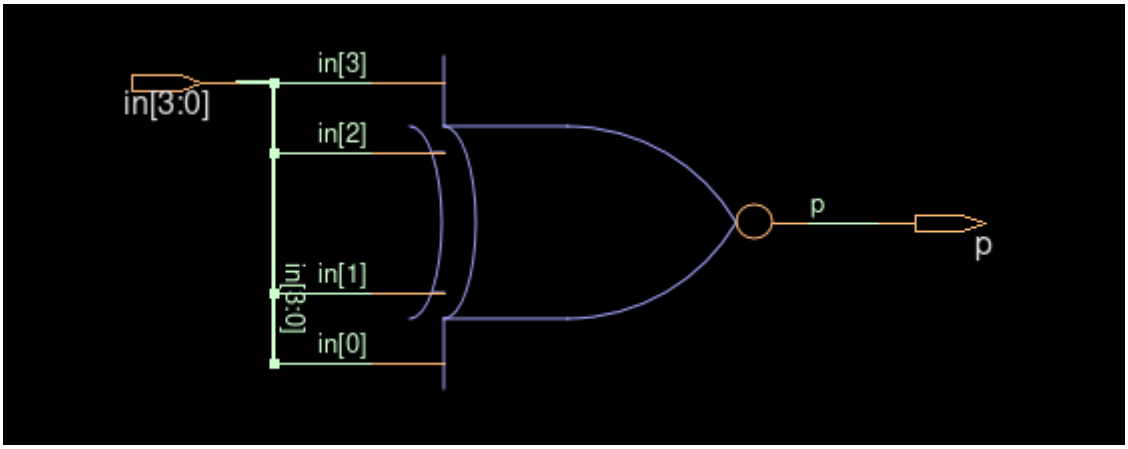
Time=30 in=0101 p=1

Time=40 in=1011 p=0

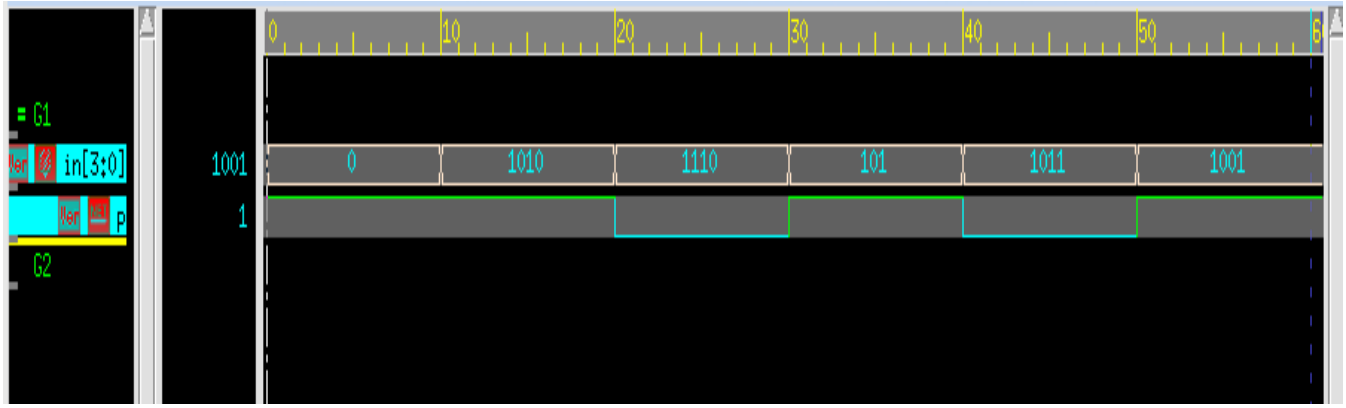
Time=50 in=1001 p=1

$finish at simulation time 60

Schematic Design



waveform



19.0 Design an even and odd parity checker

Design code-even parity checker

```
module even_parity_gen(error,p,in);
input [3:0]in;
input p;
output error;

assign error= in[3]^in[2]^in[1]^in[0]^p;

endmodule
```

Testbench

```
module TB( );
reg [3:0]in;
reg p;
wire error;

even_parity_gen dut(.in(in),.p(p),.error(error));

initial begin
        $monitor("Time=%0t in=%b p=%b error=%b",$time,in,p,error);
        in=4'b0000;p=1;
        #10 in=4'b1010;p=0;
        #10 in=4'b1110;p=1;
        #10 in=4'b0101;p=0;
        #10 in=4'b1011;p=1;
        #10 in=4'b1001;p=0;
        #10 $finish();
end
```

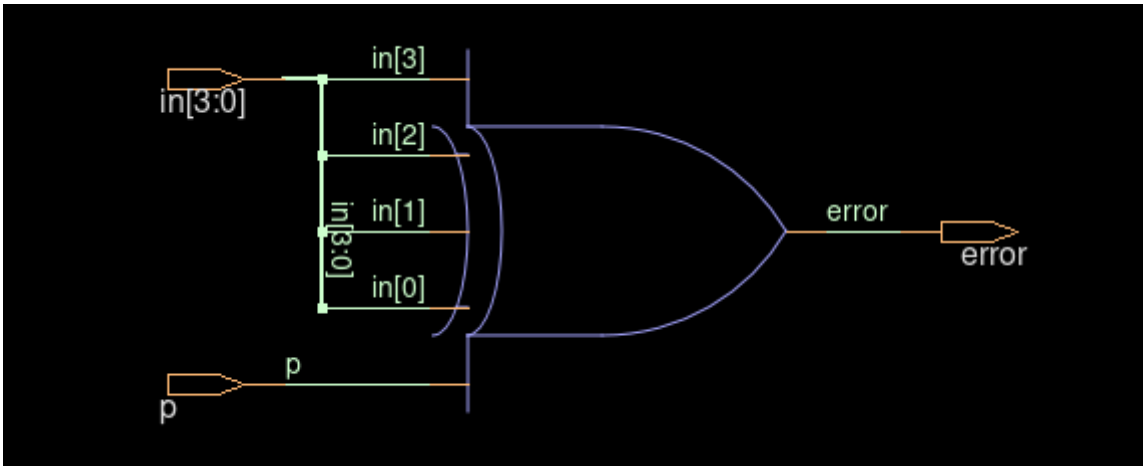
```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

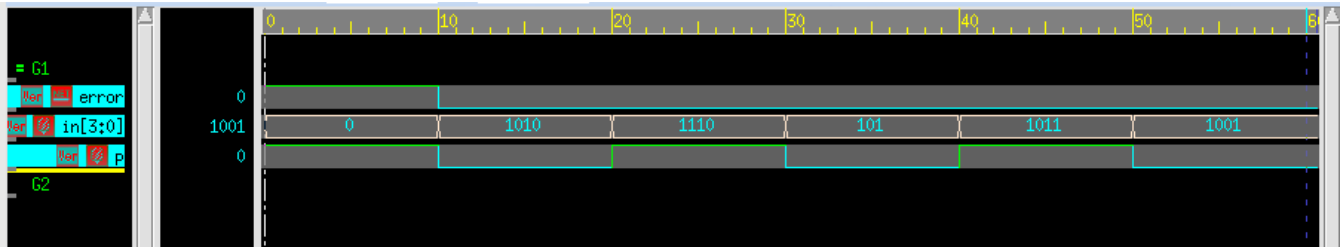
output

```
Time=0 in=0000 p=1 error=1
Time=10 in=1010 p=0 error=0
Time=20 in=1110 p=1 error=0
Time=30 in=0101 p=0 error=0
Time=40 in=1011 p=1 error=0
Time=50 in=1001 p=0 error=0
$finish at simulation time                60
```

Schematic Design



waveform



Design code-odd parity checker

```
module odd_parity_gen(error,p,in);
input [3:0]in;
input p;
output error;

assign error= ~(in[3]^in[2]^in[1]^in[0]^p);

endmodule
```

Testbench

```
module TB( );
reg [3:0]in;
reg p;
wire error;

odd_parity_gen dut(.in(in),.p(p),.error(error));

initial begin
    $monitor("Time=%0t in=%b p=%b error=%b",$time,in,p,error);
    in=4'b0000;p=1;
    #10 in=4'b1010;p=0;
    #10 in=4'b1110;p=0;
    #10 in=4'b0101;p=1;
```

```
        #10 in=4'b1011;p=1;
        #10 in=4'b1001;p=0;
        #10 $finish();
end

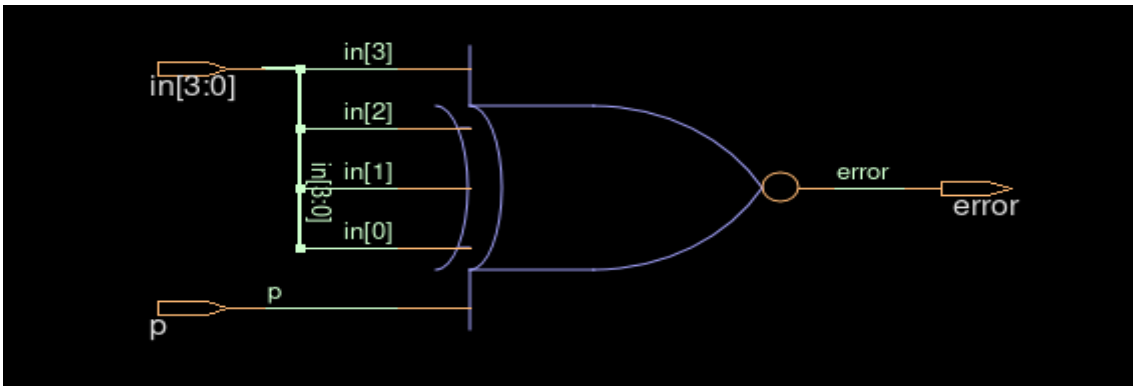
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

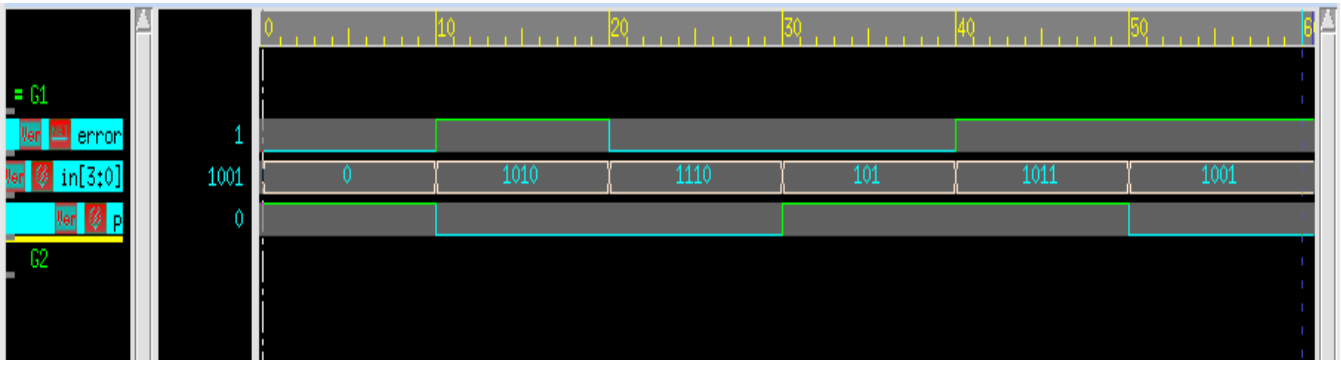
output

```
Time=0 in=0000 p=1 error=0
Time=10 in=1010 p=0 error=1
Time=20 in=1110 p=0 error=0
Time=30 in=0101 p=1 error=0
Time=40 in=1011 p=1 error=1
Time=50 in=1001 p=0 error=1
$finish at simulation time                60
```

Schematic Design



waveform



20.0 Design a 4-bit BCD adder

Design code

```
module BCD_adder (a,b,cin,sum,cout);
input [3:0]a,b;
input cin;
output reg [3:0]sum;
output reg cout;

reg [4:0]sum_temp;

always @(*)begin
        sum_temp=a+b+cin;
        if (sum_temp>9)begin
                sum_temp=sum_temp+6;
                cout=1;
        end
        else begin
                cout=0;
        end
        sum=sum_temp [3:0];
```

```
end
endmodule
```

Testbench

```
module TB( );
reg [3:0]a,b;
reg cin;
wire [3:0]sum;
wire cout;

BCD_adder dut(.a(a),.b(b),.cin(cin),.sum(sum),.cout(cout));

initial begin
    $monitor("Time=%0t a=%b b=%b cin=%b sum=%b
cout=%b",$time,a,b,cin,sum,cout);
    a=4'b0001;b=4'b1001;cin=0;
    #10 a=4'b1001;b=4'b1001;cin=1;
    #10 a=4'b0001;b=4'b0101;cin=0;
    #10 a=4'b0111;b=4'b1001;cin=1;
    #10 a=4'b0111;b=4'b0111;cin=0;
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 a=0001 b=1001 cin=0 sum=0000 cout=1

Time=10 a=1001 b=1001 cin=1 sum=1001 cout=1

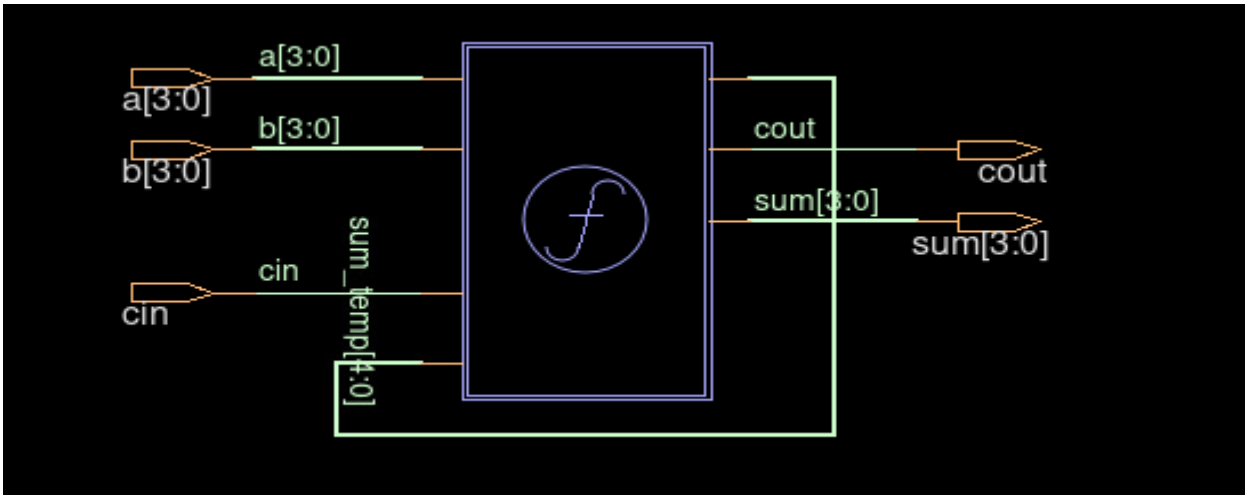
Time=20 a=0001 b=0101 cin=0 sum=0110 cout=0

Time=30 a=0111 b=1001 cin=1 sum=0111 cout=1

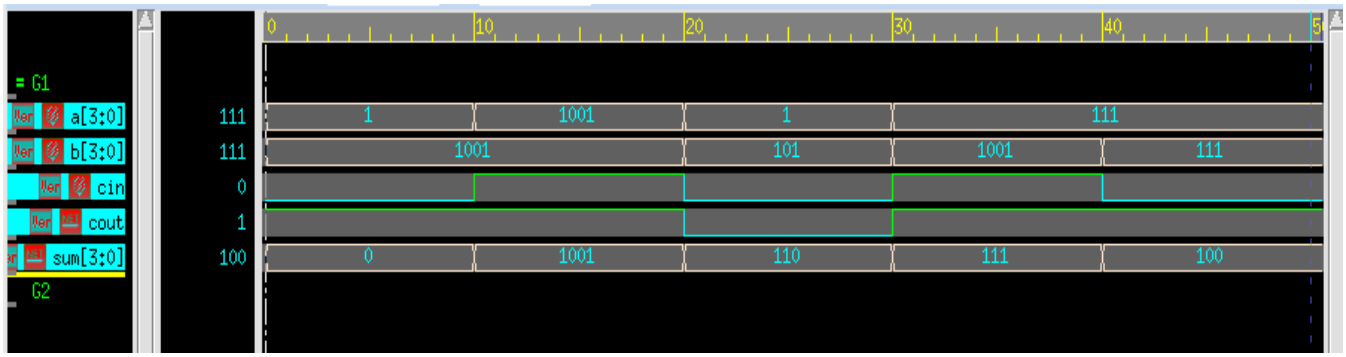
Time=40 a=0111 b=0111 cin=0 sum=0100 cout=1

$finish at simulation time 50

Schematic Design



waveform



21.0 Design a BCD to excess-3 code converter (behavioural modelling)

Design code

```
module BCD_to_excess3 (bcd,excess);
input [3:0]bcd;
output reg [3:0]excess;

always @(*)begin
    if (bcd <=4'd9)
        excess=bcd+4'd3;
    else
        excess=4'bxxxx;
end
endmodule
```

Testbench

```
module TB( );
reg [3:0]bcd;
wire [3:0]excess;

BCD_to_excess3 dut(.bcd(bcd),.excess(excess));

initial begin
    $monitor("Time=%0t bcd=%b excess=%b",$time,bcd,excess);
    bcd=4'd0;
    #10 bcd=4'd1;
    #10 bcd=4'd2;
    #10 bcd=4'd3;
    #10 bcd=4'd4;
    #10 bcd=4'd5;
    #10 bcd=4'd6;
```

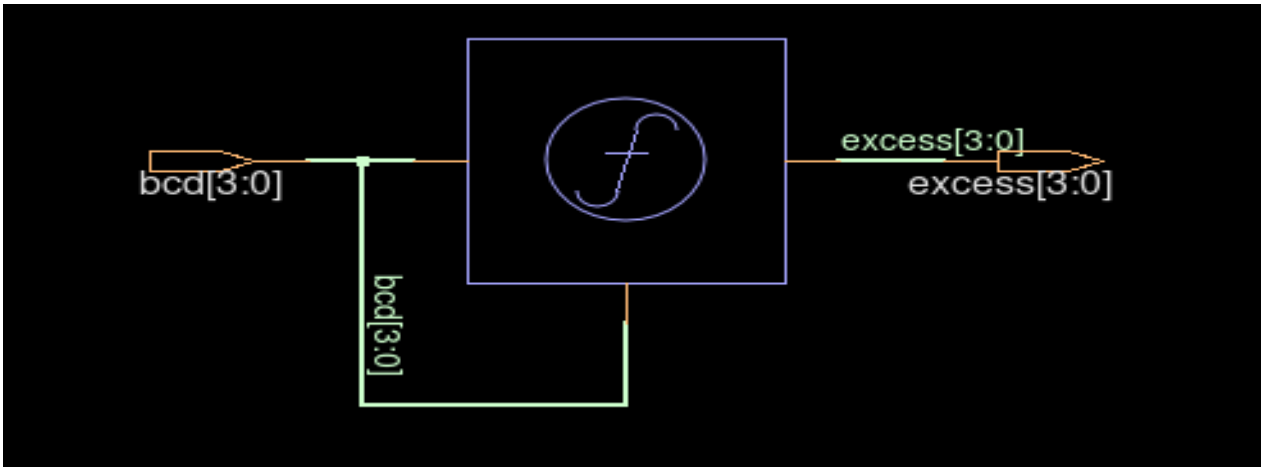
```
        #10 bcd=4'd10;
        #10 bcd=4'd15;
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

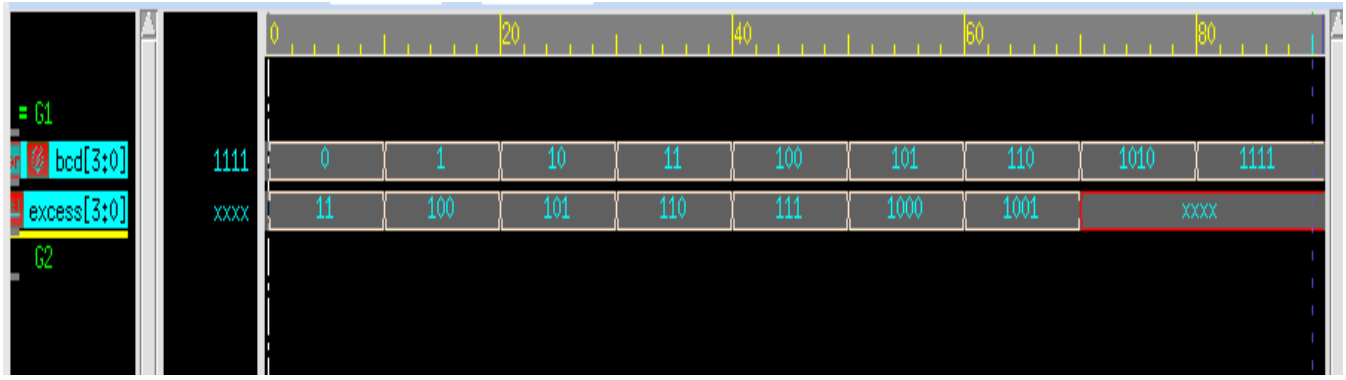
output

```
Time=0 bcd=0000 excess=0011
Time=10 bcd=0001 excess=0100
Time=20 bcd=0010 excess=0101
Time=30 bcd=0011 excess=0110
Time=40 bcd=0100 excess=0111
Time=50 bcd=0101 excess=1000
Time=60 bcd=0110 excess=1001
Time=70 bcd=1010 excess=xxxx
Time=80 bcd=1111 excess=xxxx
$finish at simulation time                90
```

Schematic Design



waveform



22.0 Design a BCD (abcd) to excess-3 (wxyz) code converter (DATAFLOW modelling)-

Design code

```
module BCD_to_excess3 (b,x);
input [3:0]b;
output [3:0]x;

assign x[3]= b[3] | (b[2] & b[1]) | (b[2] & b[0]);
assign x[2]= (b[2] & ~b[1] & ~b[0]) | (~b[2] & b[0]) | (~b[2] & b[1]);
assign x[1]= b[1] ~^ b[0];
assign x[0]= ~b[0];
endmodule
```

Testbench

```
module TB( );
reg [3:0]b;
wire [3:0]x;

BCD_to_excess3 dut(.b(b),.x(x));

initial begin
    $monitor("Time=%0t b=%b x=%b",$time,b,x);
    b=4'd0;
    #10 b=4'd1;
    #10 b=4'd2;
    #10 b=4'd3;
    #10 b=4'd4;
    #10 b=4'd5;
    #10 b=4'd6;
    #10 b=4'd7;
    #10 b=4'd8;
    #10 b=4'd9;
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 b=0000 x=0011

Time=10 b=0001 x=0100

Time=20 b=0010 x=0101

Time=30 b=0011 x=0110

Time=40 b=0100 x=0111

Time=50 b=0101 x=1000

Time=60 b=0110 x=1001

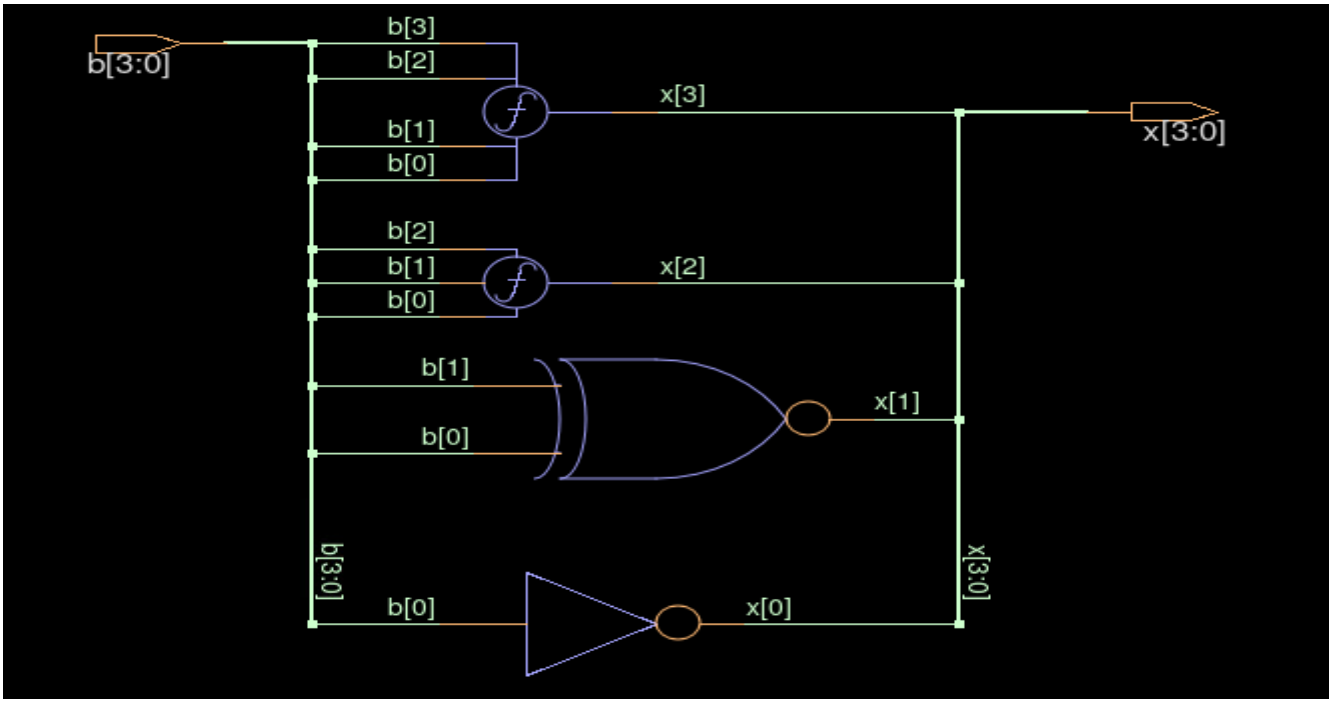
Time=70 b=0111 x=1010

Time=80 b=1000 x=1011

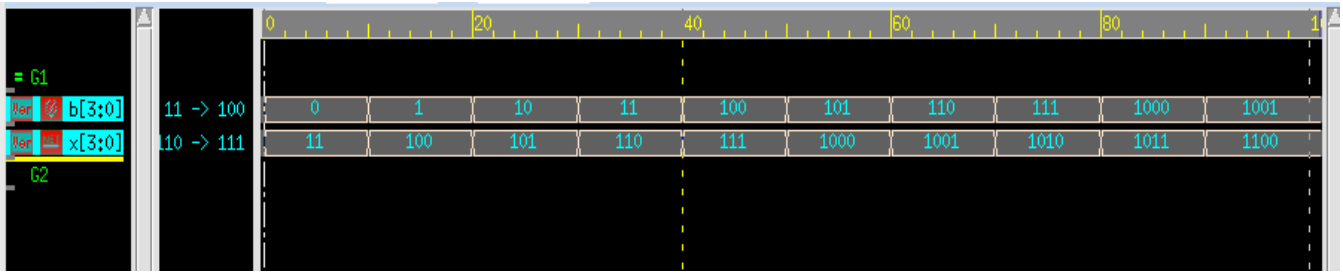
Time=90 b=1001 x=1100

$finish at simulation time 100

Schematic Design



waveform



METHOD-2

Design code

```
module BCD_to_excess3 (b,x);
input [3:0]b;
output reg [3:0]x;

always @(*)begin
    case(b)
        4'b0000 : x=4'b0011;
        4'b0001 : x=4'b0100;
        4'b0010 : x=4'b0101;
        4'b0011 : x=4'b0110;
        4'b0100 : x=4'b0111;
        4'b0101 : x=4'b1000;
        4'b0110 : x=4'b1001;
        4'b0111 : x=4'b1010;
        4'b1000 : x=4'b1011;
        4'b1001 : x=4'b1100;
        default : x=4'bxxxx;
    endcase
end
endmodule
```

Testbench

```
module TB( );
reg [3:0]b;
wire [3:0]x;

BCD_to_excess3 dut(.b(b),.x(x));

initial begin
    $monitor("Time=%0t b=%b x=%b",$time,b,x);
    b=4'd0;
    #10 b=4'd1;
    #10 b=4'd2;
    #10 b=4'd3;
    #10 b=4'd4;
    #10 b=4'd12;
    #10 b=4'd6;
    #10 b=4'd7;
    #10 b=4'd10;
    #10 b=4'd15;
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Run command

Simulation Result

output

Time=0 b=0000 x=0011

Time=10 b=0001 x=0100

Time=20 b=0010 x=0101

Time=30 b=0011 x=0110

Time=40 b=0100 x=0111

Time=50 b=1100 x=xxxx

Time=60 b=0110 x=1001

Time=70 b=0111 x=1010

Time=80 b=1010 x=xxxx

Time=90 b=1111 x=xxxx

$finish at simulation time 100

23.0 Design a Tri-state buffer

Design code

```
module Tristate_buffer (out,in,en);
input in,en;
output out;

assign out=en?in:1'bz;
endmodule
```

Testbench

```
module TB( );
reg in,en;
wire out;
```

```
Tristate_buffer dut(.in(in),.en(en),.out(out));

initial begin
    $monitor("Time=%0t in=%b en=%b out=%b",$time,in,en,out);
    in=0;en=0;
    #10 in=1;en=0;
    #10 in=0;en=1;
    #10 in=1;en=1;
    #10 $finish();
end

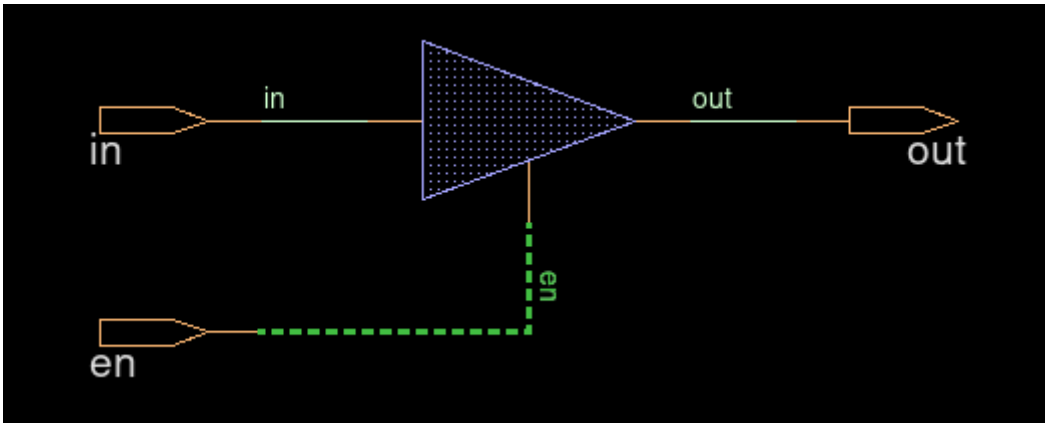
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

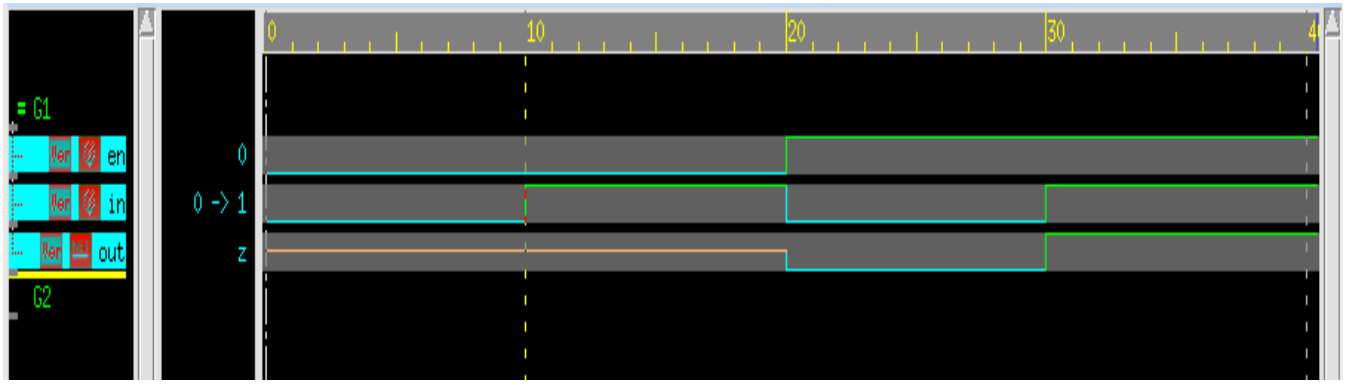
output

```
Time=0 in=0 en=0 out=z
Time=10 in=1 en=0 out=z
Time=20 in=0 en=1 out=0
Time=30 in=1 en=1 out=1
$finish at simulation time                40
```

Schematic Design



waveform



24.0 Design a SR latch using NOR gates-

Design code

```
module SR_NOR_latch(Q,Qbar,S,R);
input S,R;
output Q,Qbar;

assign Q=~(R | Qbar);
assign Qbar=~(S | Q);
endmodule
```

Testbench

```
module TB();
reg S,R;
wire Q,Qbar;
```

```
SR_NOR_latch dut(.S(S),.R(R),.Q(Q),.Qbar(Qbar));
initial begin
    $monitor("Time=%0t for S=%b R=%b and we get Q=%b
Qbar=%b",$time,S,R,Q,Qbar);
    S = 0; R = 0;
#10 S = 0; R = 1;
#10 S = 1; R = 0;
#10 S = 1; R = 1;

#10 $finish();
end

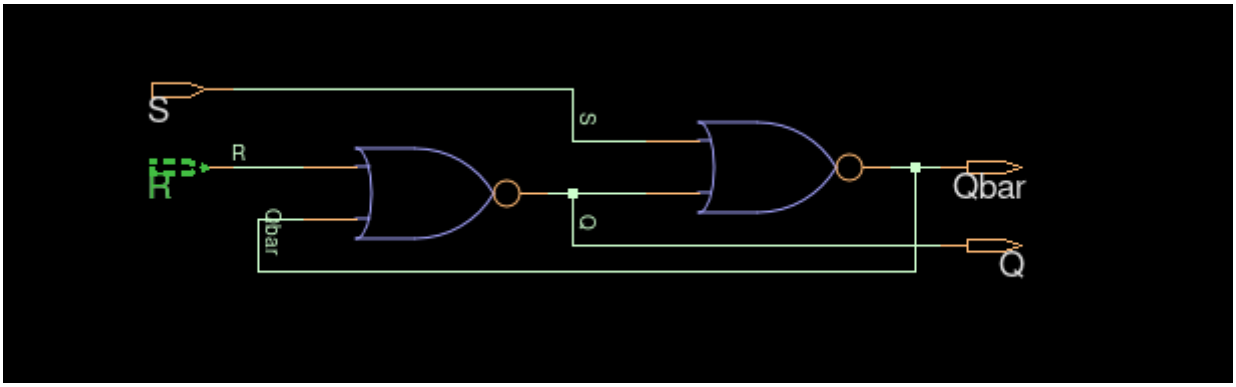
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

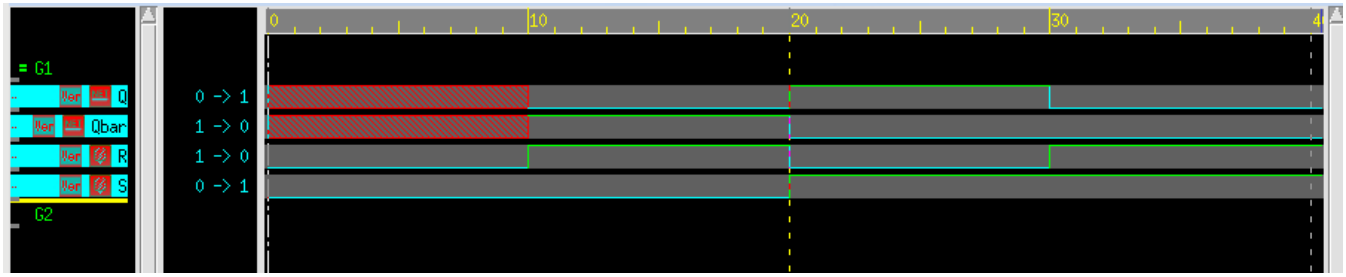
output

```
Time=0 for S=0 R=0 and we get Q=x Qbar=x
Time=10 for S=0 R=1 and we get Q=0 Qbar=1
Time=20 for S=1 R=0 and we get Q=1 Qbar=0
Time=30 for S=1 R=1 and we get Q=0 Qbar=0
$finish at simulation time                40
```

Schematic Design



waveform



25.0 Design a SR latch using NAND gates-

Design code

module SR_NAND_latch(Q,Qbar,S,R);

input S,R;

output Q,Qbar;

assign Q=~(S & Qbar);

assign Qbar=~(R & Q);

endmodule

Testbench

module TB();

reg S,R;

wire Q,Qbar;

SR_NAND_latch dut(.S(S),.R(R),.Q(Q),.Qbar(Qbar));

```
initial begin
    $monitor("Time=%0t for S=%b R=%b and we get Q=%b
Qbar=%b",$time,S,R,Q,Qbar);
    S = 0; R = 0;
#10 S = 0; R = 1;
#10 S = 1; R = 0;
#10 S = 1; R = 1;
#10 $finish();
end

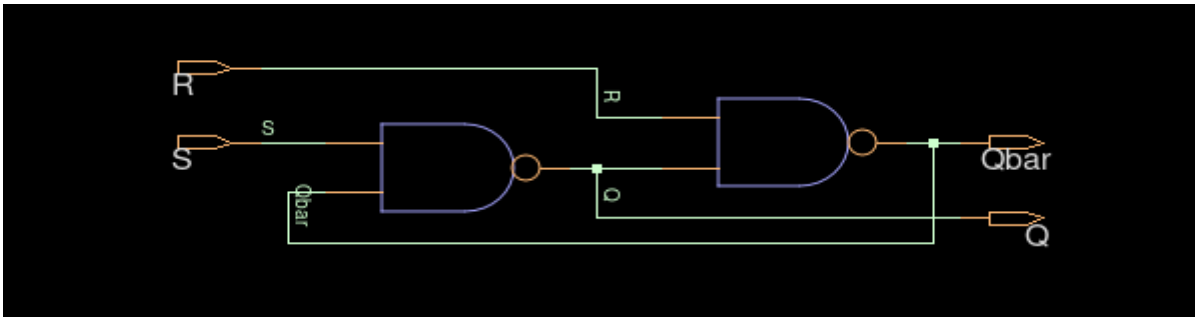
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

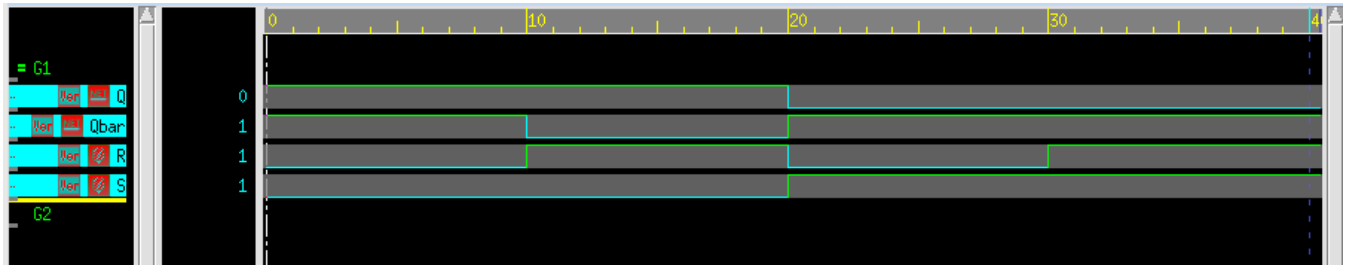
output

```
Time=0 for S=0 R=0 and we get Q=1 Qbar=1
Time=10 for S=0 R=1 and we get Q=1 Qbar=0
Time=20 for S=1 R=0 and we get Q=0 Qbar=1
Time=30 for S=1 R=1 and we get Q=0 Qbar=1
$finish at simulation time                40
```

Schematic Design



waveform



26.0 Design a universal gate by using the behavioral-

Design code

```
module universal_gates(nand_out,nor_out,a,b);
input a,b;
output reg nand_out,nor_out;

always @(*)begin
        nand_out=~(a & b);
        nor_out=~(a | b);
end
endmodule
```

Testbench

```
module TB();
reg a,b;
wire nand_out,nor_out;
```

```
universal_gates dut(.nand_out(nand_out),.nor_out(nor_out),.a(a),.b(b));
initial begin
    $monitor("Time=%0t for a=%b b=%b and we get nand_out=%b
nor_out=%b",$time,a,b,nand_out,nor_out);
    a = 0; b = 0;
#10 a = 0; b = 1;
#10 a = 1; b = 0;
#10 a = 1; b = 1;

#10 $finish();
end

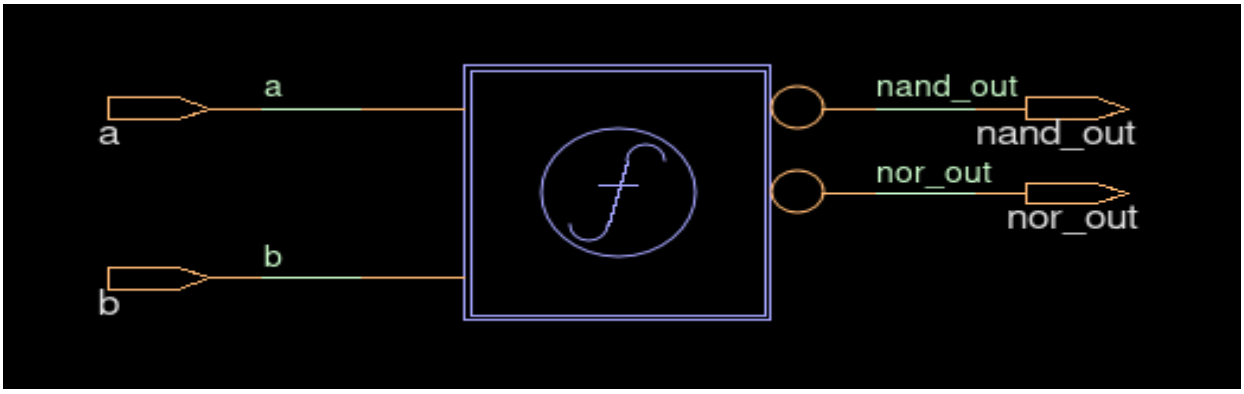
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

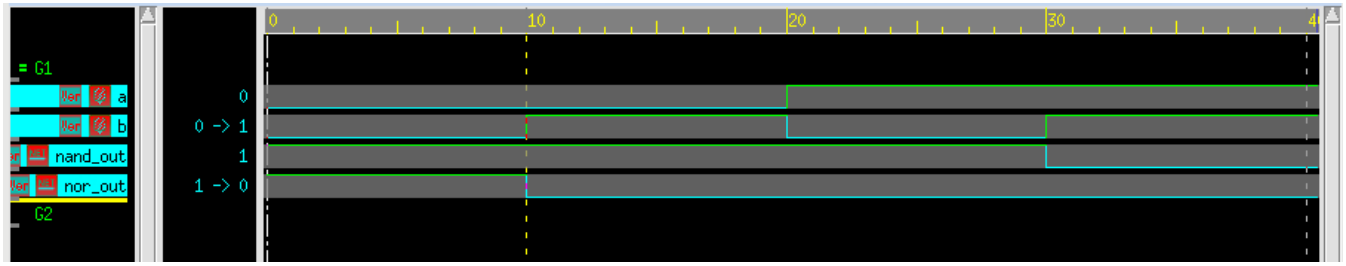
output

```
Time=0 for a=0 b=0 and we get nand_out=1 nor_out=1
Time=10 for a=0 b=1 and we get nand_out=1 nor_out=0
Time=20 for a=1 b=0 and we get nand_out=1 nor_out=0
Time=30 for a=1 b=1 and we get nand_out=0 nor_out=0
$finish at simulation time                40
```

Schematic Design



waveform



27.0 Design a half adder by behavioural level

Design code

```
//module half adder behavioural level
module half_adder(sum,carry,a,b);
input a,b;
output reg carry, sum;

always @(*)begin
        sum= a ^ b;
        carry= a & b;
end
endmodule
```

Testbench

```
module TB();
reg a,b;
```

```
wire carry, sum;


half_adder dut(.a(a),.b(b),.sum(sum),.carry(carry));
initial begin
      $monitor("Time=%0t for a=%b b=%b and we get sum=%b
carry=%b",$time,a,b,sum,carry);
      a = 0; b = 0;
#10 a = 0; b = 1;
#10 a = 1; b = 0;
#10 a = 1; b = 1;


#10 $finish();
end

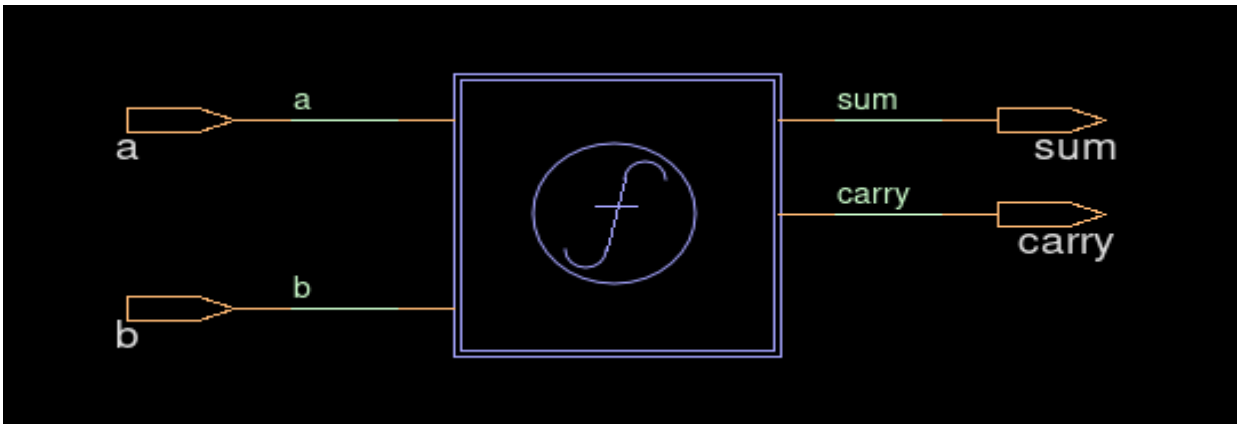

initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0, TB);
end
endmodule
```

Simulation Result

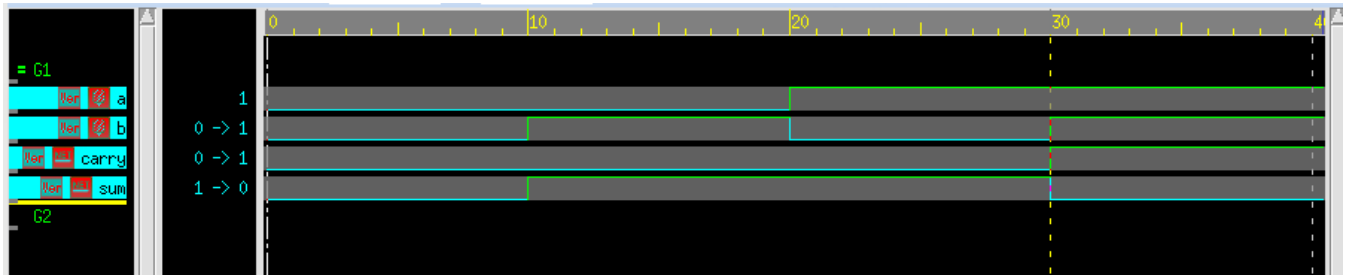
output

```
Time=0 for a=0 b=0 and we get sum=0 carry=0
Time=10 for a=0 b=1 and we get sum=1 carry=0
Time=20 for a=1 b=0 and we get sum=1 carry=0
Time=30 for a=1 b=1 and we get sum=0 carry=1
$finish at simulation time                40
```

Schematic Design



waveform



28.0 Design a Full Subtractor (FS) by using gate level-Design code

```
module Full_subtractor(diff,borrow,a,b,c);
input a,b,c;
output diff,borrow;

wire w1,w2,w3,w4,w5;

xor x1(w1,a,b);
xor x2(diff,w1,c);

not n1(w2,a);
not n2(w3,w1);
```

```
and n3(w4,w2,b);
and n4(w5,w3,c);
or or1(borrow,w4,w5);
endmodule
```

Testbench

```
module TB( );
reg a,b,c;
wire diff,borrow;
integer i;

Full_subtractor dut(.a(a),.b(b),.c(c),.diff(diff),.borrow(borrow));

initial begin
    $monitor("Time=%0t a=%0b b=%0b c=%0b diff=%0b
borrow=%0b",$time,a,b,c,diff,borrow);

    for (i=0;i<9;i=i+1)begin
        {a,b,c}=i[2:0];
        #10;
    end
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
```

end

endmodule

Simulation Result

output

Time=0 a=0 b=0 c=0 diff=0 borrow=0

Time=10 a=0 b=0 c=1 diff=1 borrow=1

Time=20 a=0 b=1 c=0 diff=1 borrow=1

Time=30 a=0 b=1 c=1 diff=0 borrow=1

Time=40 a=1 b=0 c=0 diff=1 borrow=0

Time=50 a=1 b=0 c=1 diff=0 borrow=0

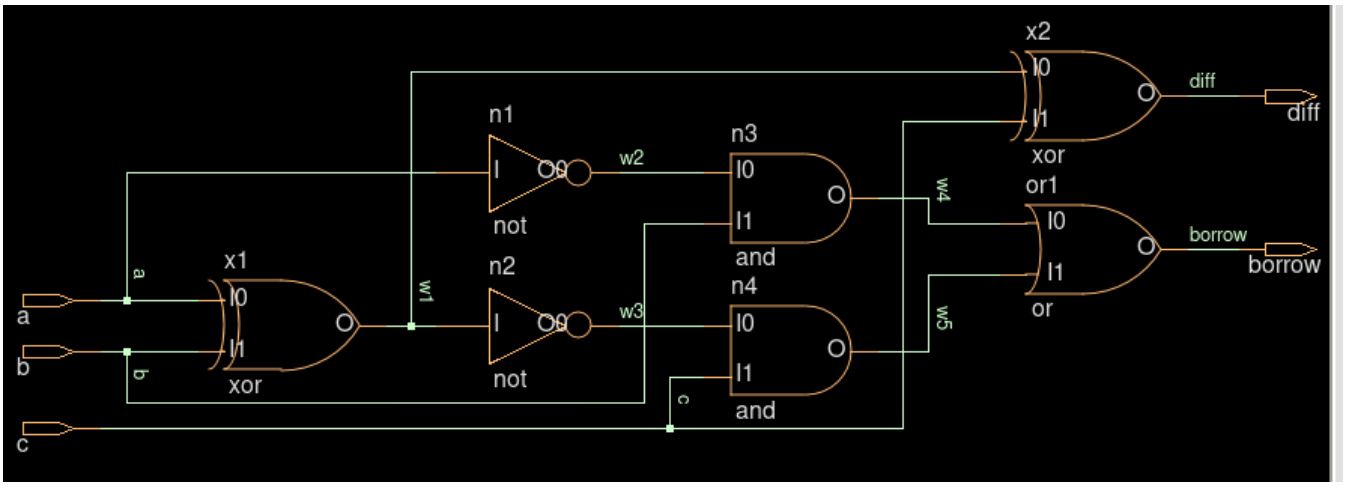
Time=60 a=1 b=1 c=0 diff=0 borrow=0

Time=70 a=1 b=1 c=1 diff=1 borrow=1

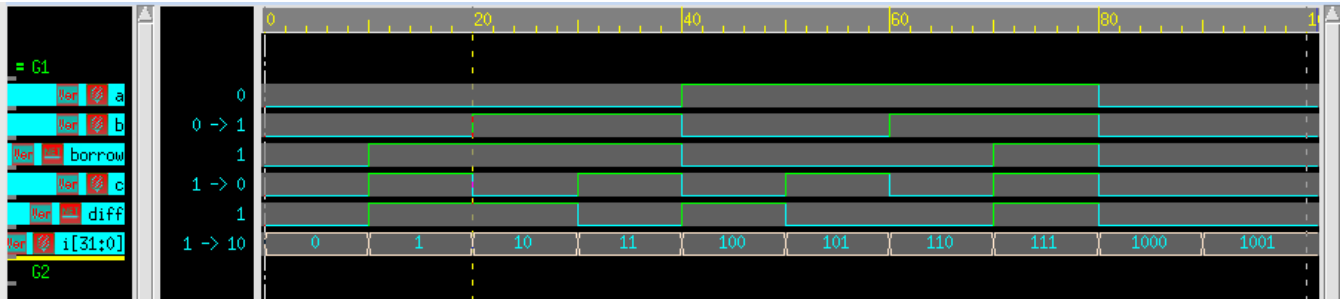
Time=80 a=0 b=0 c=0 diff=0 borrow=0

$finish at simulation time 100

Schematic Design



waveform



29.0 Design a 4-bit Full_adder(FA)_Subtractor (FS) by using gate level-

Design code

```
module full_adder(sum,carry,a,b,c);
input a,b,c;
output sum,carry;

assign sum = a ^ b ^ c;
assign carry = (a & b) | (b & c) | (a & c);

endmodule

module adder_subtract(sum, carry,a, bin, ctrl);
input [3:0] a,bin;
input ctrl;
output [3:0]sum;
output carry;

wire [3:0]c;
wire [3:0]b;
wire cin;
```

```
assign cin = ctrl;

xor x1(b[0], bin[0], ctrl);
xor x2(b[1], bin[1], ctrl);
xor x3(b[2], bin[2], ctrl);
xor x4(b[3], bin[3], ctrl);

full_adder fa0(.a(a[0]), .b(b[0]), .c(cin), .sum(sum[0]),.carry(c[0]));
full_adder fa1(.a(a[1]), .b(b[1]), .c(c[0]),.sum(sum[1]),.carry(c[1]));
full_adder fa2(.a(a[2]), .b(b[2]), .c(c[1]),.sum(sum[2]),.carry(c[2]));
full_adder fa3(.a(a[3]), .b(b[3]), .c(c[2]),.sum(sum[3]),.carry(c[3]));

assign carry = c[3];
endmodule
```

Testbench

```
module TB( );
reg [3:0] a,bin;
reg ctrl;
wire [3:0]sum;
wire carry;

adder_subtract dut(.a(a),.bin(bin),.ctrl(ctrl),.sum(sum),.carry(carry));

initial begin
    $monitor("Time=%0t a=%0b bin=%0b ctrl=%0b sum=%0b
carry=%0b",$time,a,bin,ctrl,sum,carry);
    a=4'b0000;bin=4'b0000;ctrl=1'b0;
```

```
        #10 a=4'b1011;bin=4'b0111;ctrl=1'b1;
        #10 a=4'b0110;bin=4'b0010;ctrl=1'b0;
        #10 a=4'b1001;bin=4'b1010;ctrl=1'b1;
        #10 a=4'b0100;bin=4'b1000;ctrl=1'b0;
        #10 a=4'b1100;bin=4'b1110;ctrl=1'b1;
        #10 a=4'b0101;bin=4'b1011;ctrl=1'b0;
        #10 $finish();
end

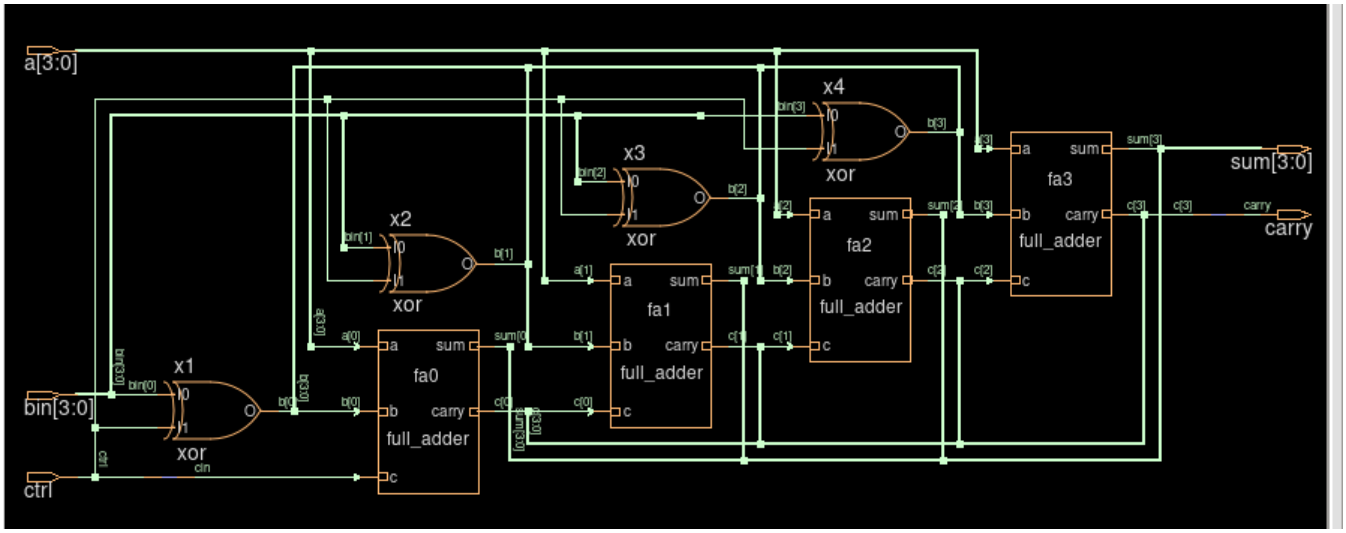
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

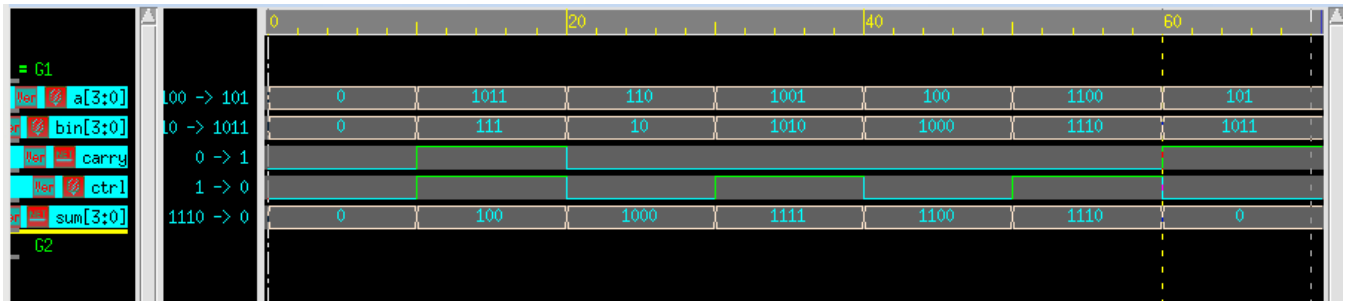
output

```
Time=0 a=0 bin=0 ctrl=0 sum=0 carry=0
Time=10 a=1011 bin=111 ctrl=1 sum=100 carry=1
Time=20 a=110 bin=10 ctrl=0 sum=1000 carry=0
Time=30 a=1001 bin=1010 ctrl=1 sum=1111 carry=0
Time=40 a=100 bin=1000 ctrl=0 sum=1100 carry=0
Time=50 a=1100 bin=1110 ctrl=1 sum=1110 carry=0
Time=60 a=101 bin=1011 ctrl=0 sum=0 carry=1
$finish at simulation time                 70
```

Schematic Design



waveform



30.0 Design a 2x4 decoder by using behavioural

Design code

```
module Decoder_2x4 (y,din,en);
input [1:0]din;
input en;
output reg [3:0]y;

always @(*)begin
    if(en)begin
        case(din)
            2'b00: y=4'b0001;
            2'b01: y=4'b0010;
            2'b10: y=4'b0100;
```

```
            2'b11: y=4'b1000;
            default: y=4'b0000;
        endcase
    end
    else
        y=4'b0000;
end
endmodule
```

Testbench

```
module TB( );
reg [1:0]din;
reg en;
wire [3:0]y;
integer i;

Decoder_2x4 dut(.din(din),.en(en),.y(y));

initial begin
    $monitor("Time=%0d din=%b en=%0b y=%b ",$time,din,en,y);
    en=1;
    for (i=0;i<4;i=i+1)begin
        {din[1:0]}=i[1:0];
        #10;
    end
    #10 $finish();
end
```

```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 din=00 en=1 y=0001

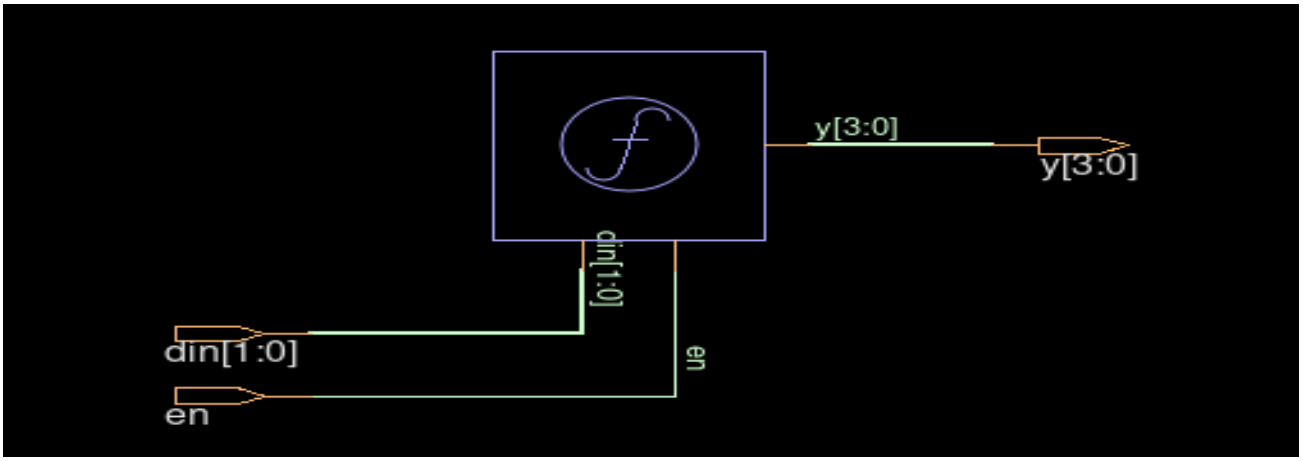
Time=10 din=01 en=1 y=0010

Time=20 din=10 en=1 y=0100

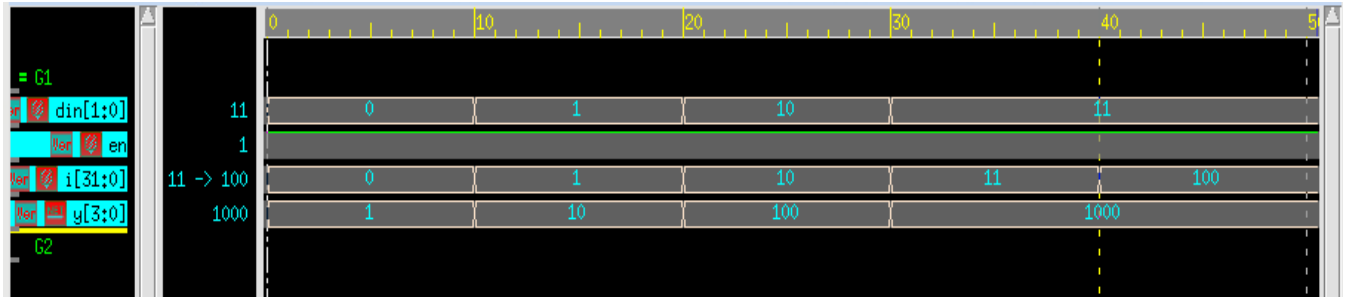
Time=30 din=11 en=1 y=1000

$finish at simulation time 50

Schematic Design



waveform



31.0 Design a 3x8 decoder by using 2x4 decoder-

Design code

```
module Decoder_2x4 (y,din,en);
input [1:0]din;
input en;
output reg [3:0]y;

always @(*)begin
	if(en)begin
		case(din)
			2'b00: y=4'b0001;
			2'b01: y=4'b0010;
			2'b10: y=4'b0100;
			2'b11: y=4'b1000;
			default: y=4'b0000;
		endcase
	end
	else
		y=4'b0000;
end
endmodule
module Decoder_3x8 (y,din);
input [2:0]din;
output [7:0]y;

wire w1;
```

```
Decoder_2x4 d0(.din(din[1:0]),.en(din[2]),.y(y[7:4]));
not n1(w1,din[2]);
Decoder_2x4 d1(.din(din[1:0]),.en(w1),.y(y[3:0]));
endmodule
```

Testbench

```
module TB( );
reg [2:0]din;
wire [7:0]y;
integer i;

Decoder_3x8 dut(.din(din),.y(y));

initial begin
	$monitor("Time=%0d din=%b  y=%b ",$time,din,y);

	for (i=0;i<8;i=i+1)begin
		din=i[2:0];
		#10;
	end
	#10 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 din=000 y=00000001

Time=10 din=001 y=00000010

Time=20 din=010 y=00000100

Time=30 din=011 y=00001000

Time=40 din=100 y=00010000

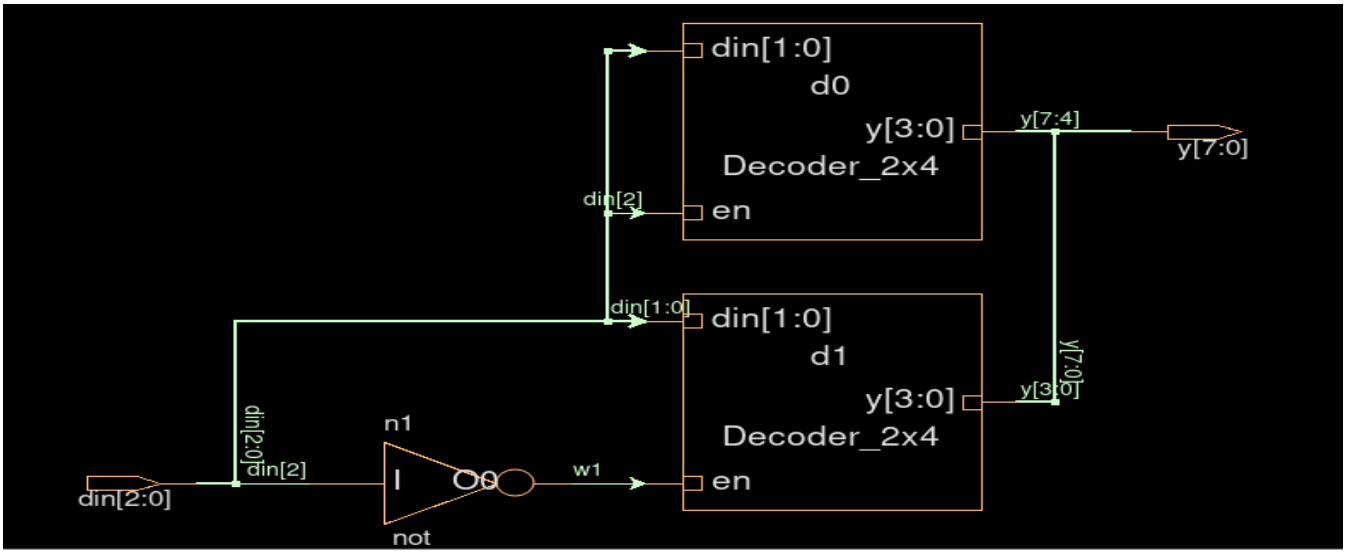
Time=50 din=101 y=00100000

Time=60 din=110 y=01000000

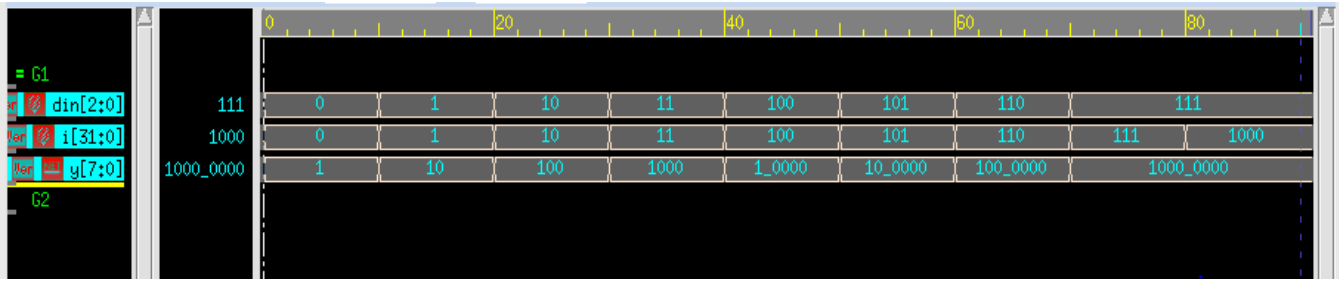
Time=70 din=111 y=10000000

$finish at simulation time 90

Schematic Design



waveform



32.0 Design an 8x3 Priority encoder using valid-

Design code

```
module priority_encoder_8x3(y,a,en,valid);
input [7:0]a;
input en;
output reg [2:0]y;
output reg valid;

always @(a,en) begin
    if (en)begin
        valid=1'b1;
        casex(a)
            8'b00000001 : y=3'b000;
            8'b0000001x : y=3'b001;
            8'b000001xx : y=3'b010;
            8'b00001xxx : y=3'b011;
            8'b0001xxxx : y=3'b100;
            8'b001xxxxx : y=3'b101;
            8'b01xxxxxx : y=3'b110;
            8'b1xxxxxxx : y=3'b111;
            default:begin
                y=3'b000;
                valid=1'b0;
            end
        endcase
    end
    else
```

```
        y=3'bzzz;
end
endmodule
```

Testbench

```
module priority_encoder8x3_tb();
reg [7:0]a;
reg en;
wire [2:0]y;
wire valid;

priority_encoder_8x3 dut(.y(y),.a(a),.en(en),.valid(valid));

initial begin
    $monitor("for a=%b ,en=%b,valid=%b y=%b",a,en,valid,y);
    en=0;a=8'b00000000;
    en=1;
    #10 a=8'b00000001;
    #10 a=8'b00000010;
    #10 a=8'b01010100;
    #10 a=8'b00110000;
    #10 a=8'b00010111;
    #10 a=8'b00100101;
    #10 a=8'b10101010;
    #10 $finish();
end

initial begin
```

```
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, priority_encoder8x3_tb);
end
endmodule
```

Simulation Result

output

for a=00000000 ,en=1,valid=0 y=000

for a=00000001 ,en=1,valid=1 y=000

for a=00000010 ,en=1,valid=1 y=001

for a=01010100 ,en=1,valid=1 y=110

for a=00110000 ,en=1,valid=1 y=101

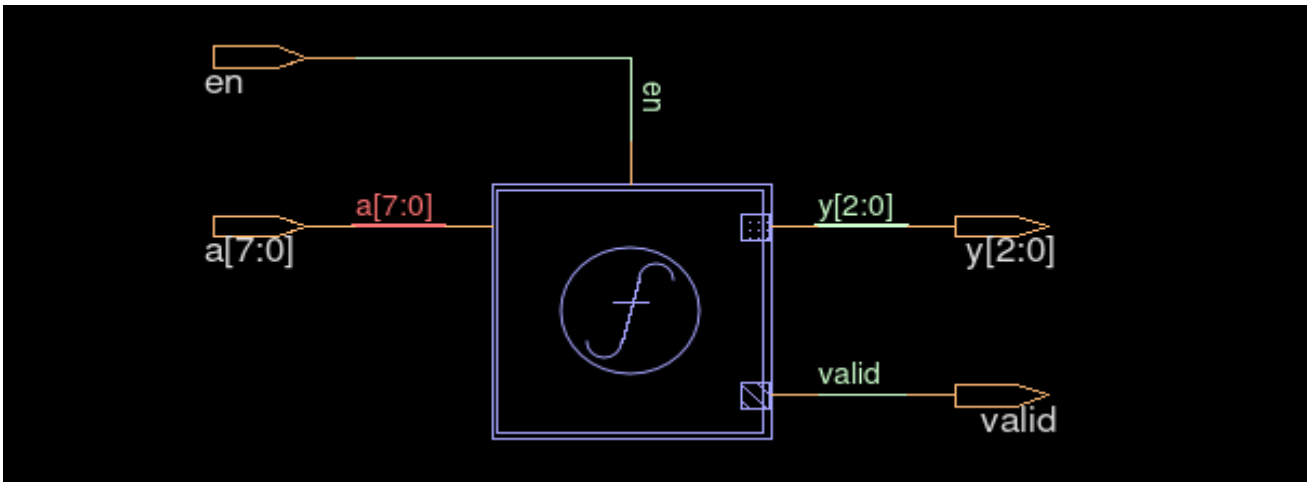
for a=00010111 ,en=1,valid=1 y=100

for a=00100101 ,en=1,valid=1 y=101

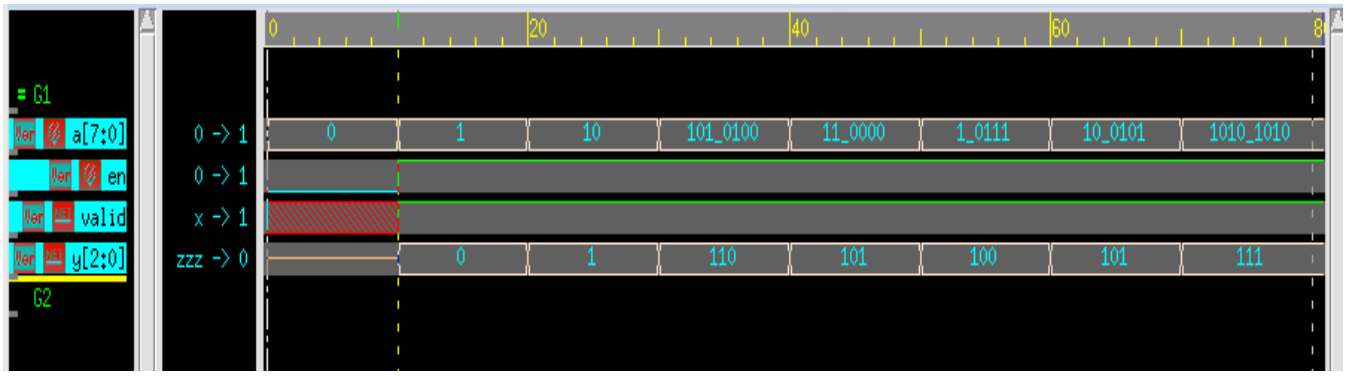
for a=10101010 ,en=1,valid=1 y=111

$finish at simulation time 80

Schematic Design



waveform



33.0 Design a 2x1mux by using case in behavioral

Design code

```
module MUX_2X1__1(y,a,b,s);
input a,b,s;
output reg y;

always @(*)begin
        case(s)
                1'b0 : y=a;
                1'b1 : y=b;
                default:y=0;
        endcase
end
endmodule
```

Testbench

```
module MUX_2X1__1tb();
reg a,b,s;
wire y;

MUX_2X1__1 mux1(.y(y),.a(a),.b(b),.s(s));
```

```
initial begin
        $monitor("s=%b |a=%b |b=%b | we get y=%b",s,a,b,y);

        s=0;a=0; b=0;
        #5 s=0;a=1;b=0;
        #5 s=0;a=0;b=1;
        #5 s=1;a=1;b=0;
        #5 s=1;a=1;b=1;
        #10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,MUX_2X1__1tb);
end
endmodule
```

Testbench-2

```
module MUX_2X1__1tb();
reg a,b,s;
wire y;
integer i;

MUX_2X1__1 mux1(.y(y),.a(a),.b(b),.s(s));

initial begin
        $monitor("s=%b |a=%b |b=%b | we get y=%b",s,a,b,y);
```

```
    s=0;a=$random;b=$random;

    for (i=0;i<2;i=i+1)begin
        s=i;
        #10;
    end
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,MUX_2X1__1tb);
end
endmodule
```

Simulation Result

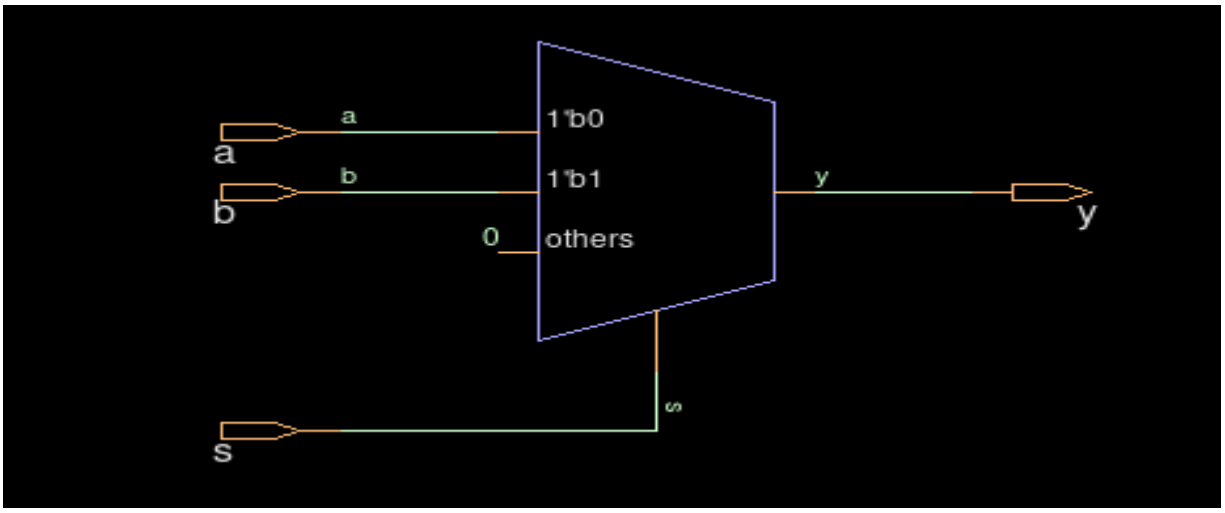
output

```
s=0 |a=0 |b=0 | we get y=0
s=0 |a=1 |b=0 | we get y=1
s=0 |a=0 |b=1 | we get y=0
s=1 |a=1 |b=0 | we get y=0
s=1 |a=1 |b=1 | we get y=1
$finish at simulation time                30
```

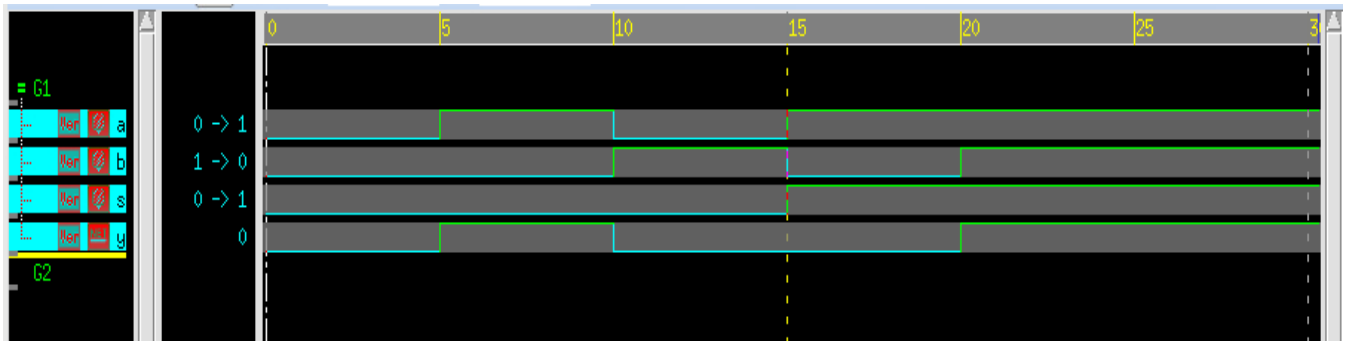
output

```
s=0 |a=0 |b=1 | we get y=0
s=1 |a=0 |b=1 | we get y=1
```

Schematic Design



waveform



34.0 Design a 10x1mux by using 4x1 mux in behavioral

Design code

```
module MUX_4X1 (y,in0,in1,in2,in3,s0,s1);
input in0,in1,in2,in3,s0,s1;
output reg y;

always @(*)begin
        case({s1,s0})
                2'b00: y=in0;
                2'b01: y=in1;
                2'b10: y=in2;
```

```
            2'b11: y=in3;
            default:y=0;
        endcase
end
endmodule

module MUX_10x1 (y,in0,in1,in2,in3,in4,in5,in6,in7,in8,in9,s0,s1,s2,s3);
input in0,in1,in2,in3,in4,in5,in6,in7,in8,in9;
input s0,s1,s2,s3;
output y;

wire w1,w2,w3;

MUX_4X1 m1(.in0(in0),.in1(in1),.in2(in2),.in3(in3),.s0(s0),.s1(s1),.y(w1));
MUX_4X1 m2(.in0(in4),.in1(in5),.in2(in6),.in3(in7),.s0(s0),.s1(s1),.y(w2));
MUX_4X1 m3(.in0(in8),.in1(in9),.in2(1'b0),.in3(1'b0),.s0(s0),.s1(s1),.y(w3));
MUX_4X1 m4(.in0(w1),.in1(w2),.in2(w3),.in3(1'b0),.s0(s2),.s1(s3),.y(y));

endmodule
```

Testbench

```
module MUX_10X1__1tb();
reg in0,in1,in2,in3,in4,in5,in6,in7,in8,in9;
reg s0,s1,s2,s3;
wire y;

integer i;
```

```
MUX_10x1
dut(.y(y),.in0(in0),.in1(in1),.in2(in2),.in3(in3),.in4(in4),.in5(in5),.in6(in6),.in7(in7),.in8(i
n8),.in9(in9),.s0(s0),.s1(s1),.s2(s2),.s3(s3));


initial begin
    $monitor("s0=%b s1=%b s2=%b s3=%b|in0=%b |in1=%b |in2=%b | in3=%b
|in4=%b | in5=%b |in6=%b |in7=%b |in8=%b |in9=%b |we get
y=%b",s3,s2,s1,s1,in0,in1,in2,in3,in4,in5,in6,in7,in8,in9,y);


    in0=1;in1=0;in2=1;in3=0;in4=1;in5=0;in6=1;in7=0;in8=1;in9=0;


    for (i=0;i<10;i=i+1)begin
        {s3,s2,s1,s0}=i;
        #10;
    end
    #10 $finish();
end


initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,MUX_10X1__1tb);
end
endmodule
```

Simulation Result

output

```
s0=0 s1=0 s2=0 s3=0|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1
|in9=0 |we get y=1
```

s0=0 s1=0 s2=0 s3=0|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=0

s0=0 s1=0 s2=1 s3=1|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=1

s0=0 s1=0 s2=1 s3=1|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=0

s0=0 s1=1 s2=0 s3=0|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=1

s0=0 s1=1 s2=0 s3=0|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=0

s0=0 s1=1 s2=1 s3=1|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=1

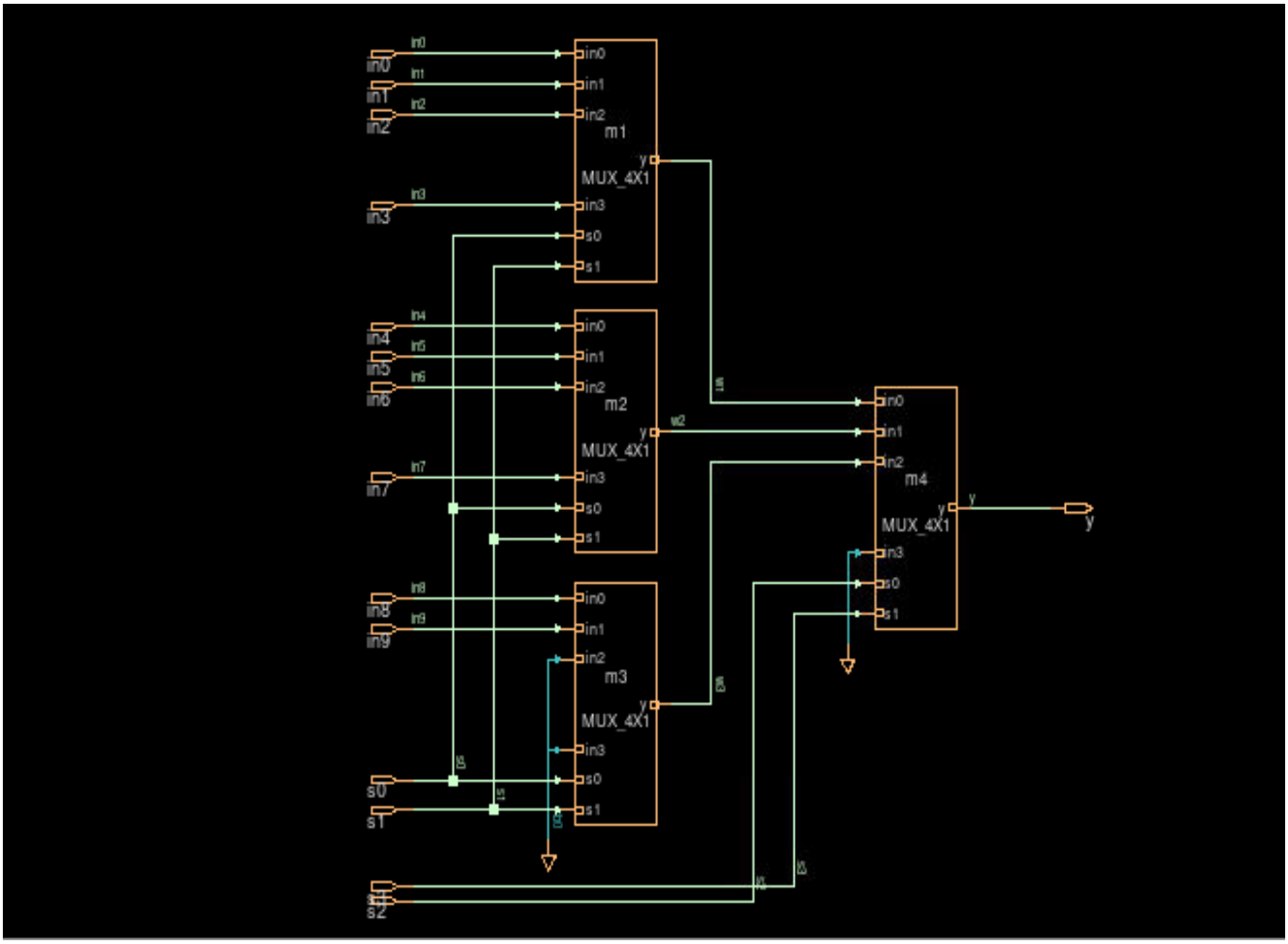
s0=0 s1=1 s2=1 s3=1|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=0

s0=1 s1=0 s2=0 s3=0|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=1

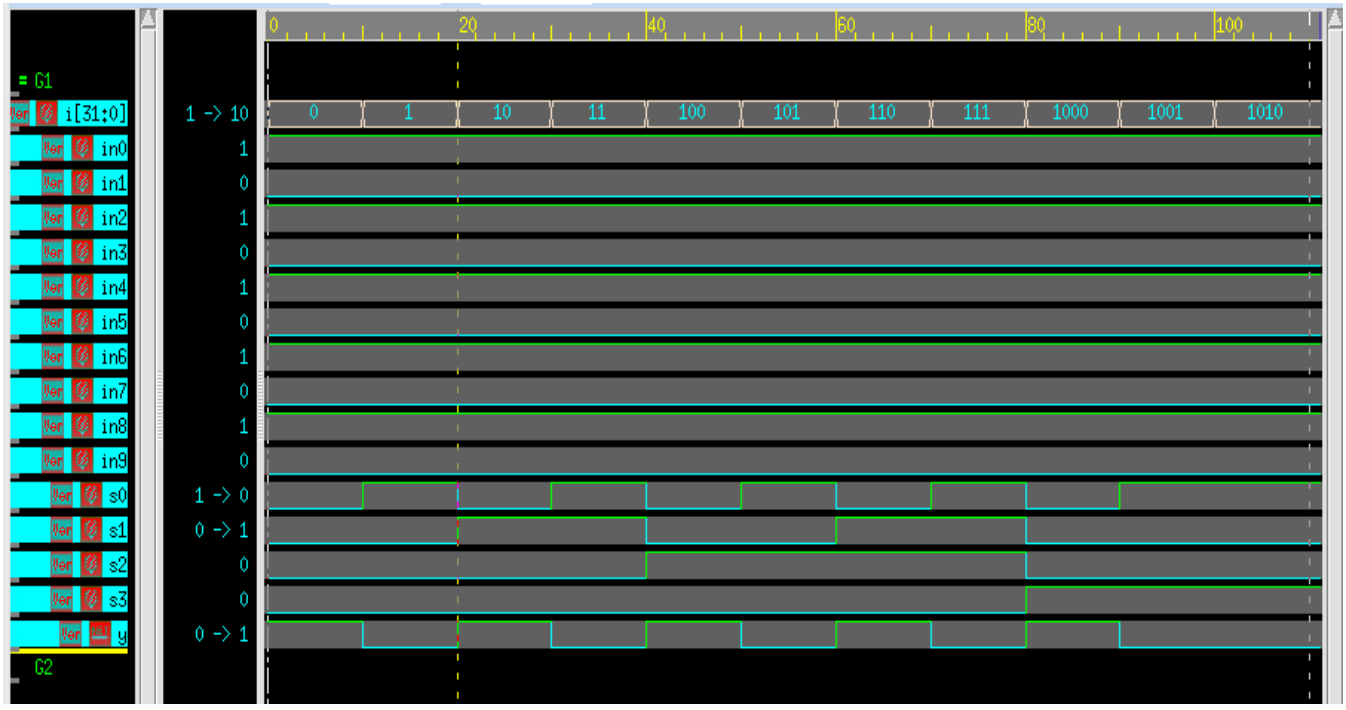
s0=1 s1=0 s2=0 s3=0|in0=1 |in1=0 |in2=1 | in3=0 |in4=1 | in5=0 |in6=1 |in7=0 |in8=1 |in9=0 |we get y=0

$finish at simulation time 110

Schematic Design


in0
in1
in2
in3
in4
in5
in6
in7
in8
in9
s0
s1
s2
m1
m2
m3
m4
MUX_4X1
y
w2

waveform


G1
i[31:0]
in0
in1
in2
in3
in4
in5
in6
in7
in8
in9
s0
s1
s2
s3
y
G2
1 -> 10
1 -> 0
0 -> 1
0 -> 1
0
20
40
60
80
100
0
1
10
11
100
101
110
111
1000
1001
1010

35.0 Design an and gate using 4x1 mux in behavioral

Design code

```
module and_gate_mux_4X1 (y,a,b);
input a,b;
output reg y;

always @(*)begin
	case({a,b})
		2'b00: y=0;
		2'b01: y=0;
		2'b10: y=0;
		2'b11: y=1;
		default:y=1'bx;
	endcase
end
endmodule
```

Testbench

```
module TB();
reg a,b;
wire y;

integer i;

and_gate_mux_4X1 dut(.y(y),.a(a),.b(b));

initial begin
```

```
        $monitor("for a=%b | b=%b |  y=%b",a,b,y);

        a=$random;b=$random;

for (i=0;i<4;i=i+1)begin
        a=i[1];
        b=i[0];
        #10;
end
#10 $finish();

end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Testbench-2

```
module TB();
reg a,b;
wire y;

integer i;

and_gate_mux_4X1 dut(.y(y),.a(a),.b(b));
```

```
initial begin
    $monitor("for a=%b | b=%b |  y=%b",a,b,y);

    a=$random;b=$random;

for (i=0;i<4;i=i+1)begin
    {a,b}=i;
    #10;
end
#10 $finish();

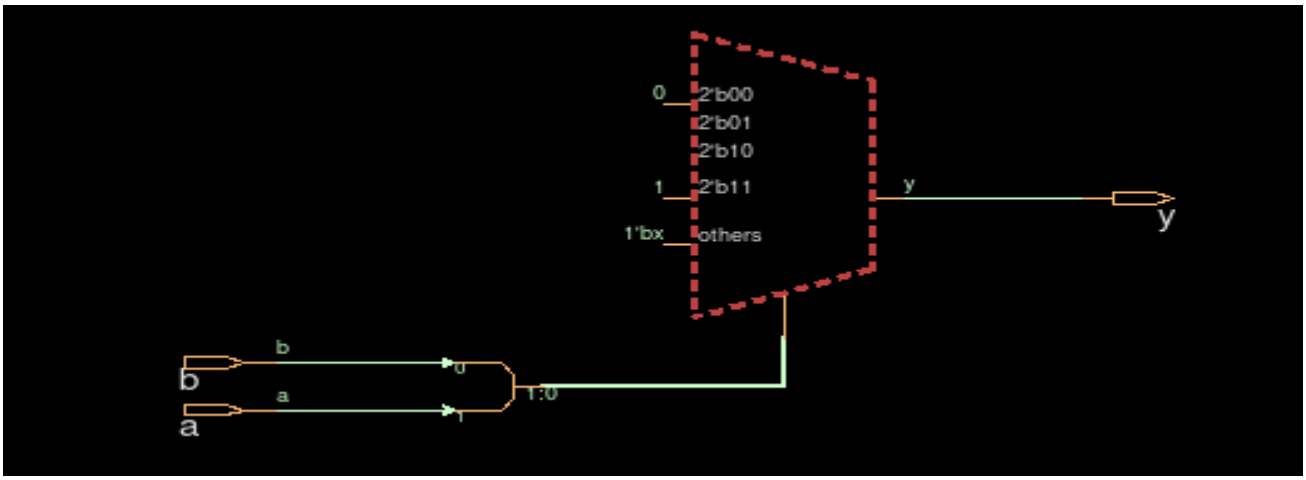
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

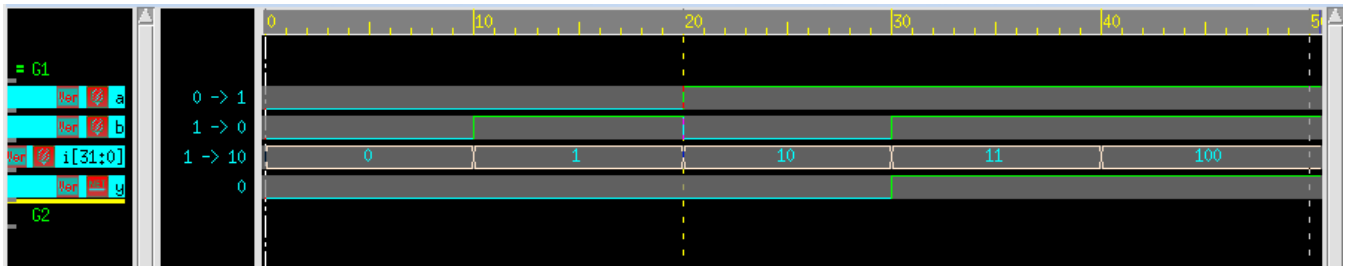
output

```
for a=0 | b=0 |  y=0
for a=0 | b=1 |  y=0
for a=1 | b=0 |  y=0
for a=1 | b=1 |  y=1
$finish at simulation time                50
```

Schematic Design



waveform



36.0 Design an AND gate using data flow modelling-

Design code

module and_gate_using_2X1mux (y,a,b);

input a,b;

output y;

assign y= (b==0) ? 0 : a ;

endmodule

Testbench

module TB();

reg a,b;

```
wire y;
integer i;

and_gate_using_2X1mux dut(.y(y),.a(a),.b(b));

initial begin
    $monitor("Time=%0t for a=%b | b=%b |  y=%b",$time,a,b,y);

    for (i=0;i<4;i=i+1)begin
        a=$random % 2;
        b=$random % 2;
        #10;
    end
    #10 $finish();
end

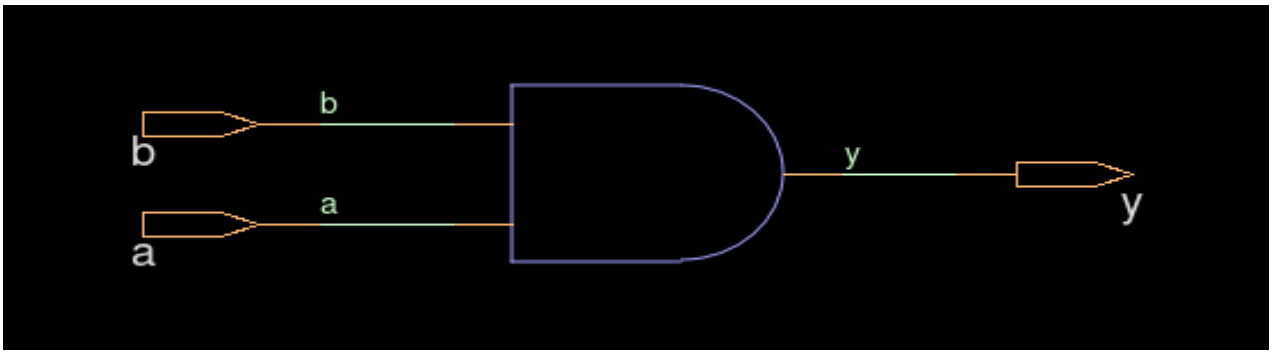
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

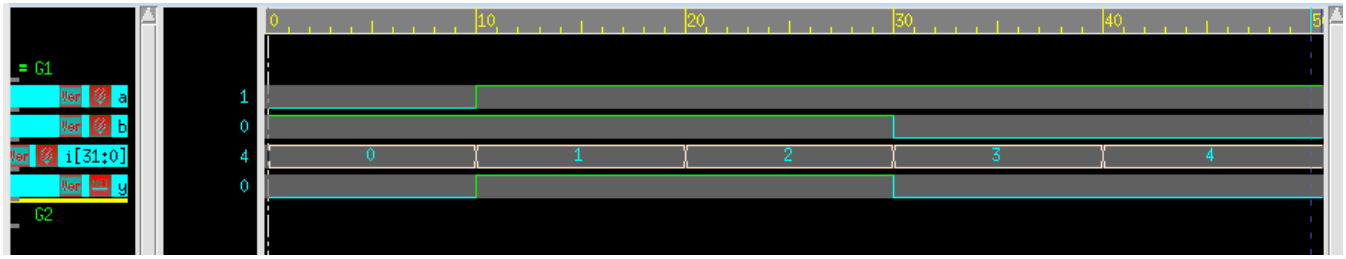
output

```
Time=0 for a=0 | b=1 |  y=0
Time=10 for a=1 | b=1 |  y=1
Time=30 for a=1 | b=0 |  y=0
```

Schematic Design



waveform



37.0 Design an AND gate using <u>case</u> statement-

<u>Design code</u>

```
module and_gate_using_case (y,a,b);
input a,b;
output reg y;

always @(*)begin
        case(b)
                1'b0 : y=0;
                1'b1 : y=a;
                default: y=1'bx;
        endcase
end
endmodule
```

Testbench

```
module TB();
reg a,b;
wire y;

integer i;

and_gate_using_case dut(.y(y),.a(a),.b(b));

initial begin
	$monitor("Time=%0t for a=%b | b=%b |  y=%b",$time,a,b,y);
	a=$random;b=$random;
	for (i=0;i<4;i=i+1)begin
		{a,b}=i;
		#10;
	end
	#10 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 for a=0 | b=0 | y=0

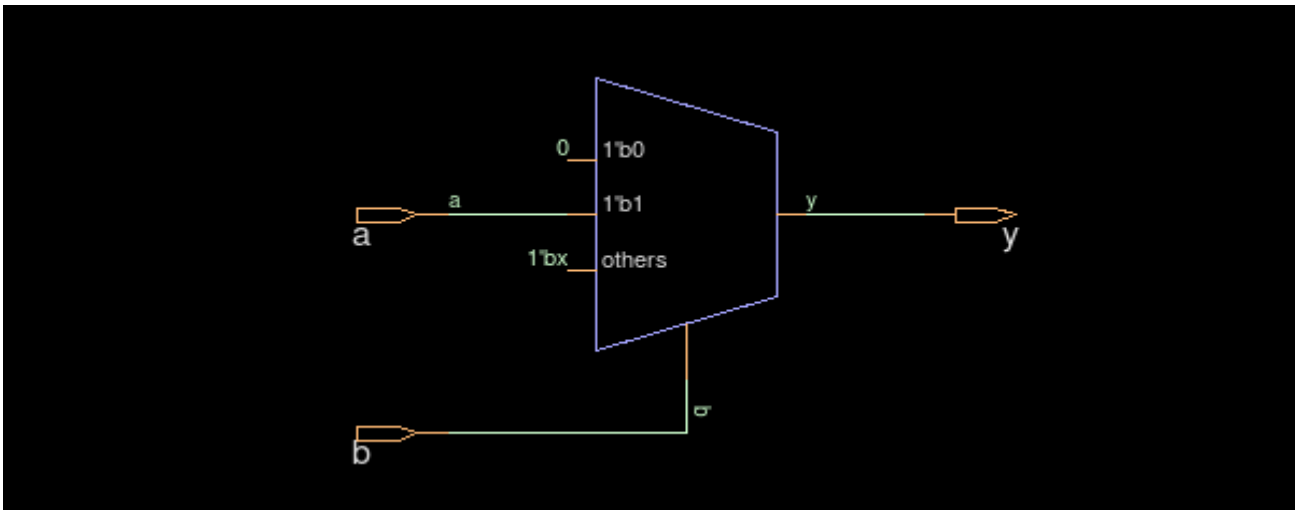
Time=10 for a=0 | b=1 | y=0

Time=20 for a=1 | b=0 | y=0

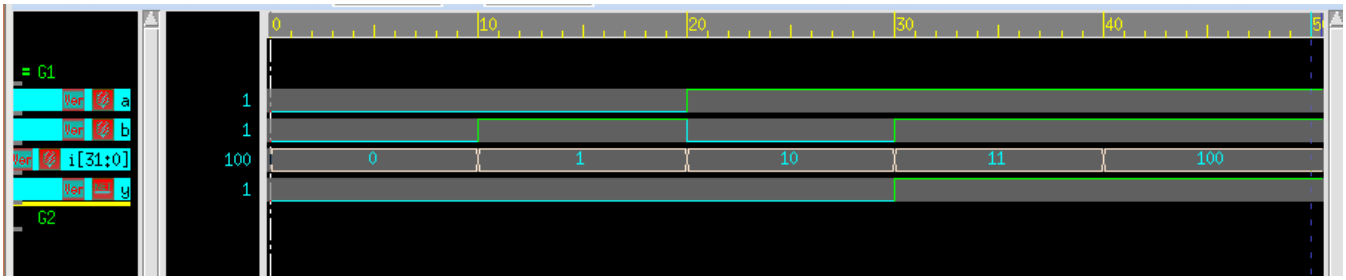
Time=30 for a=1 | b=1 | y=1

$finish at simulation time 50

Schematic Design



waveform



38.0 Design an 8x1 mux by using 2x1 mux-

Design code

module mux2x1 (y,sel,a,b);

input a,b,sel;

output reg y;

always @(*)begin

```
        case(sel)
                1'b0 : y=a;
                1'b1 : y=b;
                default: y=1'b0;
        endcase
end
endmodule

module mux_8x1 (y,s,in);
input [7:0]in;
input [2:0]s;
output y;

wire [5:0]w;

mux2x1 m0(.a(in[0]),.b(in[1]),.sel(s[0]),.y(w[0]));
mux2x1 m1(.a(in[2]),.b(in[3]),.sel(s[0]),.y(w[1]));
mux2x1 m2(.a(in[4]),.b(in[5]),.sel(s[0]),.y(w[2]));
mux2x1 m3(.a(in[6]),.b(in[7]),.sel(s[0]),.y(w[3]));
mux2x1 m4(.a(w[0]),.b(w[1]),.sel(s[1]),.y(w[4]));
mux2x1 m5(.a(w[2]),.b(w[3]),.sel(s[1]),.y(w[5]));
mux2x1 m6(.a(w[4]),.b(w[5]),.sel(s[2]),.y(y));

endmodule
```

Testbench

```
module TB();
reg [7:0]in;
```

```
reg [2:0]s;
wire y;
integer i;

mux_8x1 dut(.y(y),.in(in),.s(s));

initial begin
        $monitor("Time=%0t for in=%b | s=%b | y=%b",$time,in,s,y);
        in=8'b1010_1010;

for (i=0;i<8;i=i+1)begin
        s=i;
        #10;
end
#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

```
Time=0 for in=10101010 | s=000 | y=0
Time=10 for in=10101010 | s=001 | y=1
Time=20 for in=10101010 | s=010 | y=0
```

Time=30 for in=10101010 | s=011 | y=1

Time=40 for in=10101010 | s=100 | y=0

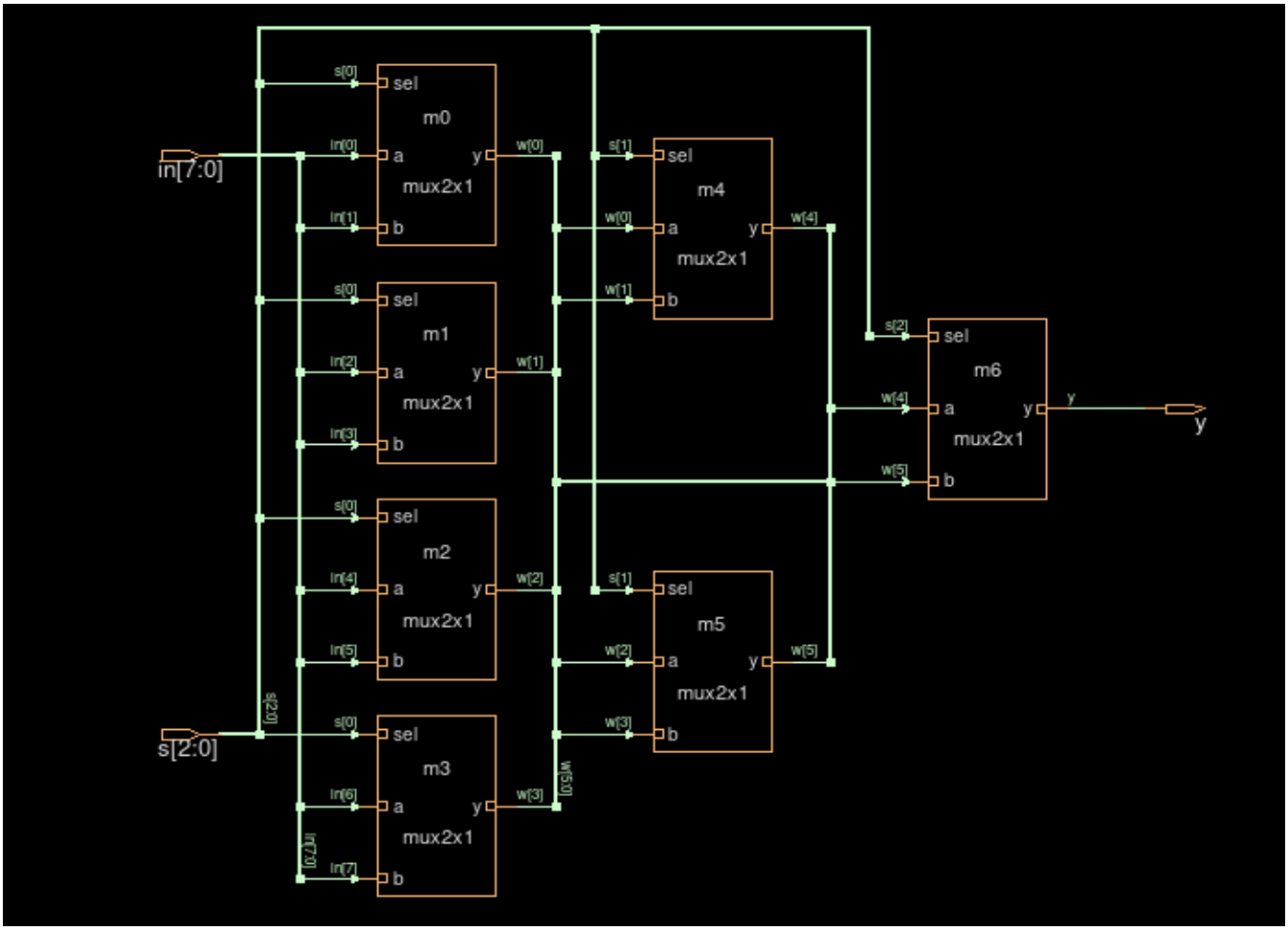
Time=50 for in=10101010 | s=101 | y=1

Time=60 for in=10101010 | s=110 | y=0

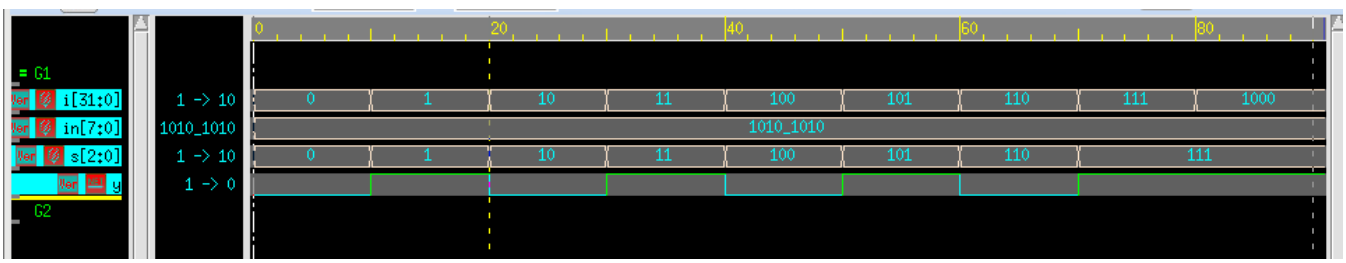
Time=70 for in=10101010 | s=111 | y=1

$finish at simulation time 90

Schematic Design



waveform



39.0 Design an kx1_mux by using parameter -

Design code

```
module mux_kx1 #(parameter k=64)(in,sel,y);
input [k-1:0]in;
input [$clog2(k)-1:0] sel;
output y;

assign y=in[sel];
endmodule
```

Testbench

```
module TB();
reg [63:0]in;
reg [5:0]sel;
wire y;



mux_kx1 dut(.y(y),.in(in),.sel(sel));

initial begin
        $monitor("Time=%0t for in=%b | sel=%b | y=%b",$time,in,sel,y);
        in=64'h0;
#10 in=64'hF0F0_F0F0_F0F0_F0F0;

for (sel=0;sel<7;sel=sel+1)begin
        #10;
end
```

```
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

```
Time=0 for in=00000000000000000000000000000000000000000000000000000000000000000 | sel=xxxxxx | y=x
Time=10 for in=11110000111100001111000011110000111100001111000011110000111100000 | sel=000000 | y=0
Time=20 for in=11110000111100001111000011110000111100001111000011110000111100000 | sel=000001 | y=0
Time=30 for in=11110000111100001111000011110000111100001111000011110000111100000 | sel=000010 | y=0
Time=40 for in=11110000111100001111000011110000111100001111000011110000111100000 | sel=000011 | y=0
Time=50 for in=11110000111100001111000011110000111100001111000011110000111100000 | sel=000100 | y=1
```

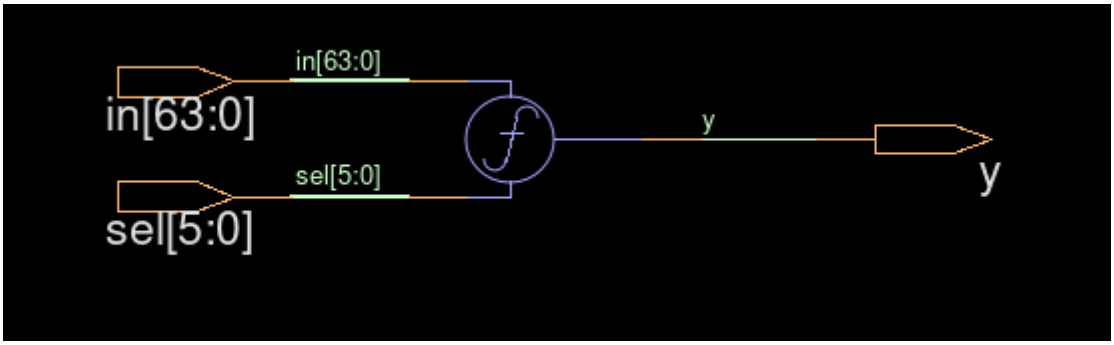
Time=60 for in=1111000011110000111100001111000011110000111100001111000011110000 | sel=000101 | y=1

Time=70 for in=1111000011110000111100001111000011110000111100001111000011110000 | sel=000110 | y=1

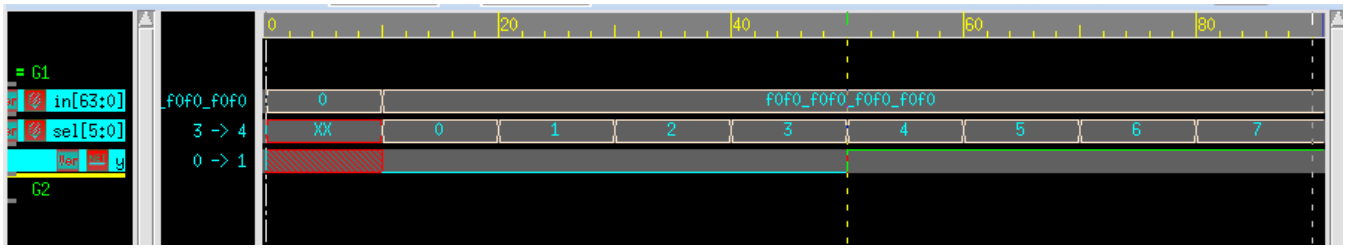
Time=80 for in=1111000011110000111100001111000011110000111100001111000011110000 | sel=000111 | y=1

$finish at simulation time 90

Schematic Design



waveform



40.0 Design a parity generator –

Design code

module parity_generator(p_even,p_odd,d);

input [3:0]d;

output p_even,p_odd;

```
assign p_even= (d[3]^d[2]^d[1]^d[0]);
assign p_odd= ~(d[3]^d[2]^d[1]^d[0]);

endmodule
```

Testbench

```
module TB();
reg [3:0]d;
wire p_even,p_odd;

parity_generator dut(.p_even(p_even),.p_odd(p_odd),.d(d));

initial begin
    $monitor("Time=%0t for d=%b | p_even=%b | p_odd=%b",$time,d,p_even,p_odd);
    d=4'b1010;
#10 d=4'b0011;
#10 d=4'b1111;
#10 d=4'b1001;
#10 d=4'b0101;
#10 d=4'b1011;
#10 d=4'b0100;
#10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

Time=0 for d=1010 | p_even=0 | p_odd=1

Time=10 for d=1011 | p_even=1 | p_odd=0

Time=20 for d=1111 | p_even=0 | p_odd=1

Time=30 for d=1011 | p_even=1 | p_odd=0

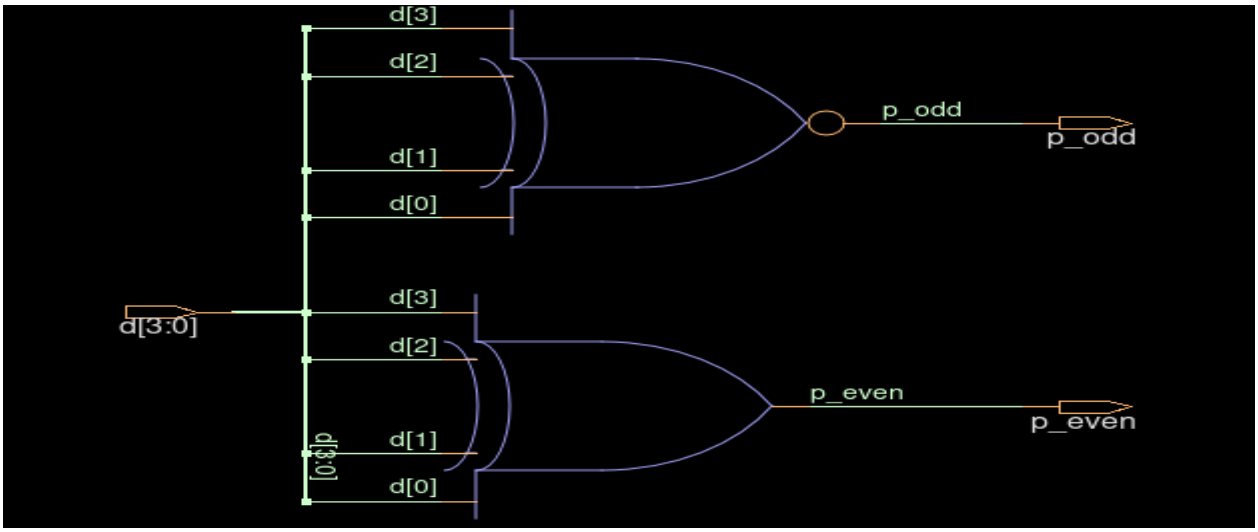
Time=40 for d=0101 | p_even=0 | p_odd=1

Time=50 for d=1011 | p_even=1 | p_odd=0

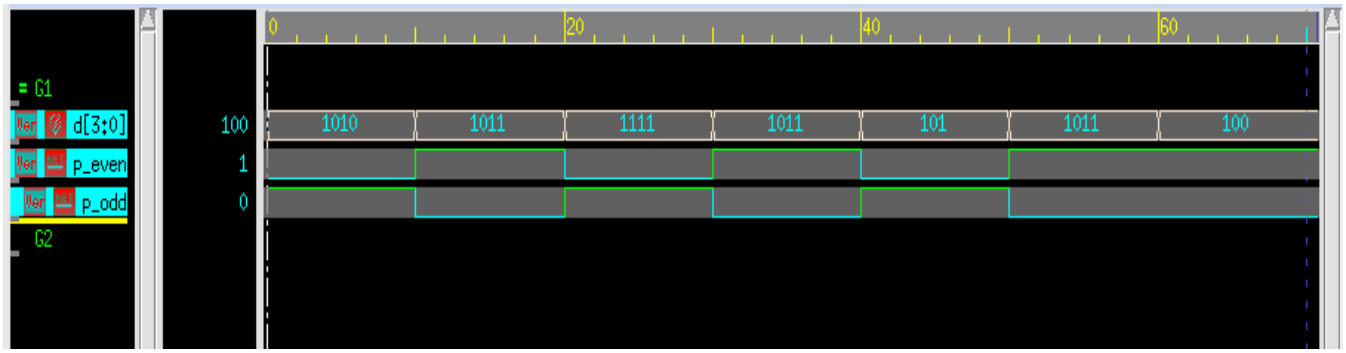
Time=60 for d=0100 | p_even=1 | p_odd=0

$finish at simulation time 70

Schematic Design



waveform



41.0 Design a logical operation by using task –

Design code

```
module logical_operation();
reg [3:0]a,b,c;
reg p,q,r,s,t,u,v,w,x;

task inputs(input [3:0] ain,bin,cin);
    begin
        a=ain;
        b=bin;
        c=cin;
        p=a && b;
        q=a && c;
        r=b && 0;
        s=a || b;
        t=a || c;
        u=b || c;
        v=!a;
        w=!c;
        x=!b;
        #5;

$display("p=%b,q=%b,r=%b,s=%b,t=%b,u=%b,v=%b,w=%b,x=%b",p,q,r,s,t,u,v,w,x);
    end
endtask
initial begin
    inputs(4'b0000,4'b0000,4'b0000);
```

```
        #5;
        inputs(4'bxxxx,4'b1111,4'b0000);
        #5;
        inputs(4'b0011,4'b0100,4'b0101);
        #5;
        inputs(4'b1001,4'b0001,4'b0110);
        #5;
        inputs(4'b1111,4'b0111,4'b1111);
        #10 $finish();
end
endmodule
```

Simulation Result

output

```
p=0,q=0,r=0,s=0,t=0,u=0,v=1,w=1,x=1
p=x,q=0,r=0,s=1,t=x,u=1,v=x,w=1,x=0
p=1,q=1,r=0,s=1,t=1,u=1,v=0,w=0,x=0
p=1,q=1,r=0,s=1,t=1,u=1,v=0,w=0,x=0
p=1,q=1,r=0,s=1,t=1,u=1,v=0,w=0,x=0
$finish at simulation time 55
```

41.0 Design a Bitwise operation by using task –

Design code

```
module Bitwise_operation();
reg [3:0]a,b,c;
reg p,q,r,s,t,u,v,w,x,y,z,m,n,o;

task inputs(input [3:0] ain,bin,cin);
```

```
    begin
        a=ain;
        b=bin;
        c=cin;
        p=a & b;
        q=b & c;
        r=a & c;
        s=a | b;
        t=a | c;
        u=b | c;
        v=a^b;
        w=b^c;
        x=c^a;
        y=~b;
        z=~c;
        m=a~^b;
        n=a~^c;
        o=b~^c;
        #5;

        $display("a=%b ,b=%b, c=%b ",a,b,c);
        $display("p=%b,q=%b,r=%b,s=%b,t=%b,u=%b",p,q,r,s,t,u);
        $display("v=%b,w=%b,x=%b,y=%b,z=%b",v,w,x,y,z);
        $display("m=%b,n=%b, o=%b",m,n,o);
    end
endtask
```

```
initial begin
        inputs(4'b0011,4'b0110,4'b1x01);
        #5;
        inputs(4'bxxxx,4'b1111,4'b0000);
        #5;
        inputs(4'b1010,4'b0101,4'b1100);
        #5;
        inputs(4'b1001,4'b0001,4'b0110);
        #5;
        inputs(4'b1111,4'b0111,4'b1111);
        #10 $finish();
end
endmodule
```

Simulation Result

Output

```
a=0011 ,b=0110, c=1x01
p=0,q=0,r=1,s=1,t=1,u=1
v=1,w=1,x=0,y=1,z=0
m=0,n=1, o=0
a=xxxx ,b=1111, c=0000
p=x,q=0,r=0,s=1,t=x,u=1
v=x,w=1,x=x,y=0,z=1
m=x,n=x, o=0
a=1010 ,b=0101, c=1100
p=0,q=0,r=0,s=1,t=0,u=1
v=1,w=1,x=0,y=0,z=1
m=0,n=1, o=0
```

a=1001 ,b=0001, c=0110

p=1,q=0,r=0,s=1,t=1,u=1

v=0,w=1,x=1,y=0,z=1

m=1,n=0, o=0

a=1111 ,b=0111, c=1111

p=1,q=1,r=1,s=1,t=1,u=1

v=0,w=0,x=0,y=0,z=0

m=1,n=1, o=1

$finish at simulation time 55

42.0 Design a Reduction operation by using task – Design code

```
module Reduction_operator();
reg [3:0]a,b;
reg p,q,r,s,t,u,v,w,x,y,z,m;

task inputs(input [3:0] ain,bin);
    begin
        a=ain;
        b=bin;
        p= &a;
        q= &b;
        r= |a;
        s= |b;
        t= ^a;
        u= ^b;
        v= ~^a;
        w= ~^b;
```

```
            x= ~&a;
            y=~&b;
            z=~|a;
            m= ~|b;
            #5;

            $display("a=%b ,b=%b ",a,b);
            $display("p=%b,q=%b,r=%b,s=%b",p,q,r,s);
            $display("t=%b, u=%b,v=%b,w=%b",t,u,v,w);
            $display("x=%b y=%b z=%b,m=%b",x,y,z,m);
        end
    endtask
    initial begin
        inputs(4'b0100,4'b0111);
        #5;
        inputs(4'b0000,4'b1111);
        #5;
        inputs(4'b1010,4'b1100);
        #5;
        inputs(4'b1111,4'b0001);
        #5 $finish();
    end
    initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,Reduction_operator);
    end
    endmodule
```

Simulation Result

Output

a=0100 ,b=0111

p=0,q=0,r=1,s=1

t=1, u=1,v=0,w=0

x=1 y=1 z=0,m=0

a=0000 ,b=1111

p=0,q=1,r=0,s=1

t=0, u=0,v=1,w=1

x=1 y=0 z=1,m=0

a=1010 ,b=1100

p=0,q=0,r=1,s=1

t=0, u=0,v=1,w=1

x=1 y=1 z=0,m=0

a=1111 ,b=0001

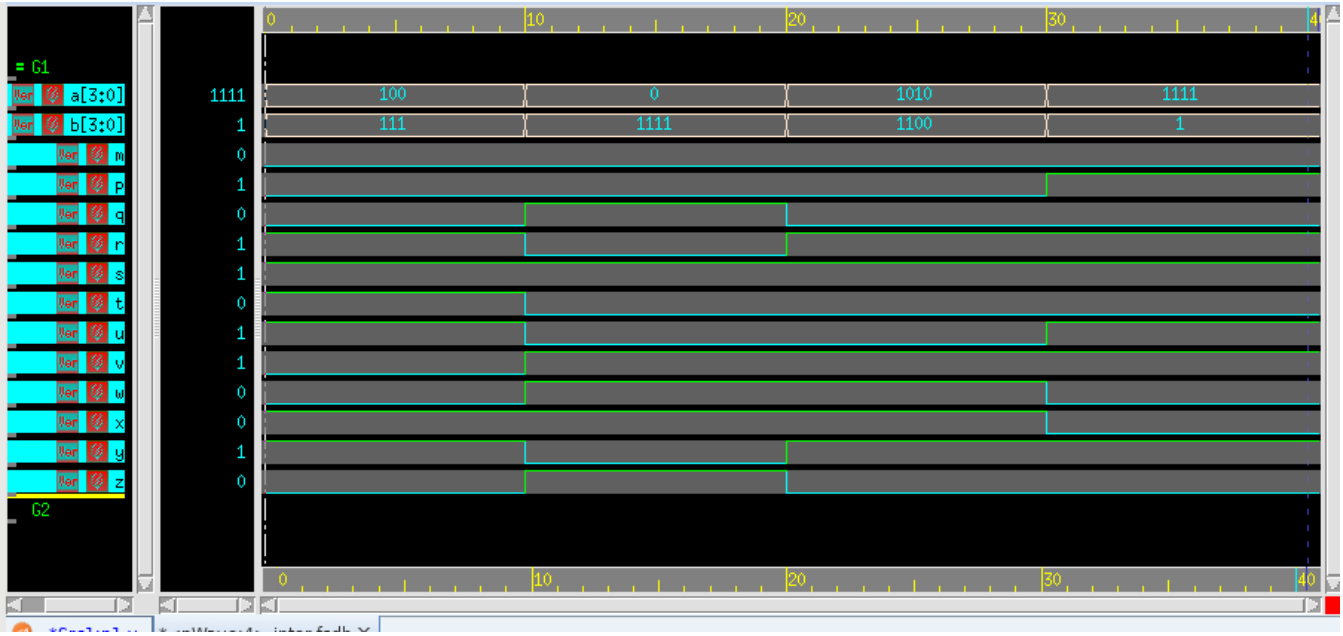
p=1,q=0,r=1,s=1

t=0, u=1,v=1,w=0

x=0 y=1 z=0,m=0

$finish at simulation time 40

waveform



43.0 Design a shift operation by using task –

Design code

```
module Shift_operator();
reg [3:0]a,b;
reg [3:0]w,x,y,z,m;

task inputs(input [3:0] ain,bin);
        begin
                a=ain;
                b=bin;
                w=a>>1;
                x=a>>2;
                y=b<<1;
                z=a>>>2;
                m=a<<<2;
                #5;
```

```
            $display("a=%b ,b=%b w=%b x=%b y=%b z=%b m=%b",a,b,w,x,y,z,m);

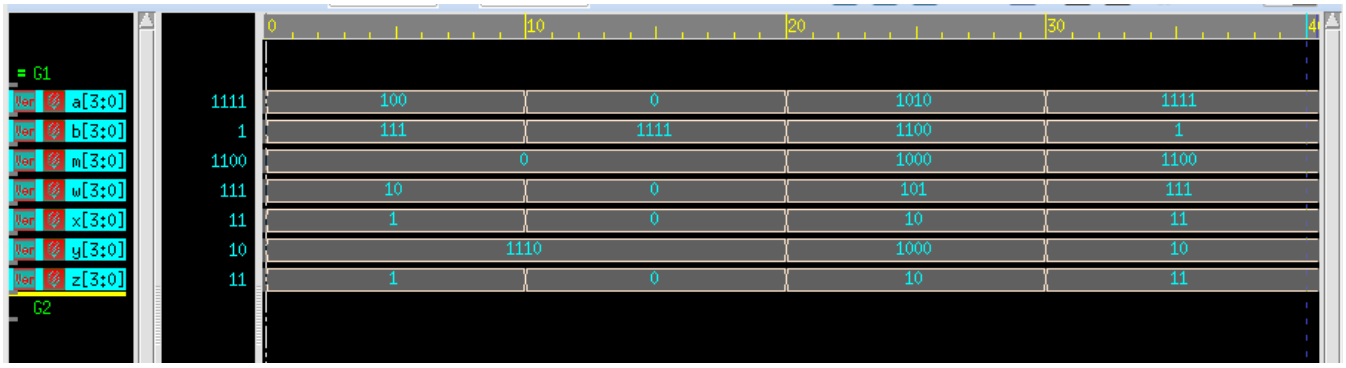
        end
endtask
initial begin
        inputs(4'b0100,4'b0111);
        #5;
        inputs(4'b0000,4'b1111);
        #5;
        inputs(4'b1010,4'b1100);
        #5;
        inputs(4'b1111,4'b0001);
        #5 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,Shift_operator);
end
endmodule
```

Simulation Result

output

```
a=0100 ,b=0111 w=0010 x=0001 y=1110 z=0001 m=0000
a=0000 ,b=1111 w=0000 x=0000 y=1110 z=0000 m=0000
a=1010 ,b=1100 w=0101 x=0010 y=1000 z=0010 m=1000
a=1111 ,b=0001 w=0111 x=0011 y=0010 z=0011 m=1100
$finish at simulation time                 40
```

waveform



44.0 Design an equality operation by using task –

Design code

```
module equality_operator();
reg [3:0]a,b;
reg p,q,r,s;

task inputs(input [3:0] ain,bin);
    begin
        a=ain;
        b=bin;
        p= (a==b);
        q= (a !=b);
        r= (a ===b);
        s= (a !==b);
        #5;

        $display("a=%b ,b=%b p=%b q=%b r=%b s=%b",a,b,p,q,r,s);

    end
endtask
```

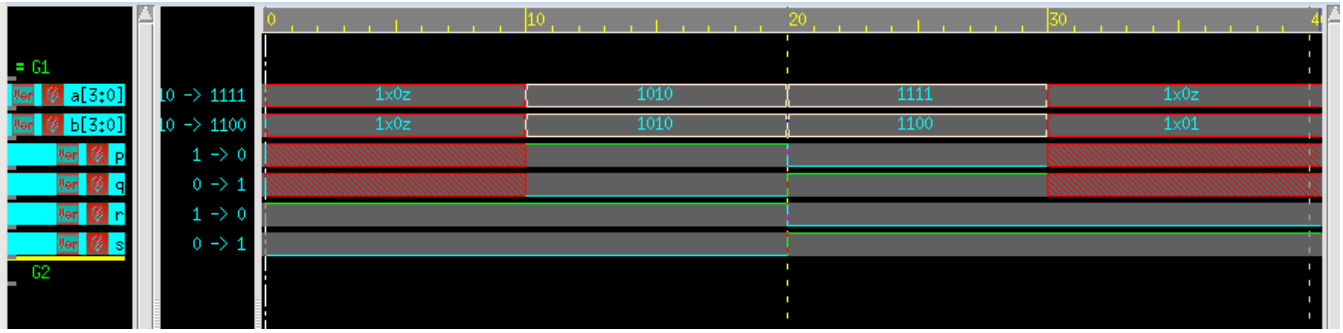
```
initial begin
        inputs(4'b1x0z,4'b1x0z);
        #5;
        inputs(4'b1010,4'b1010);
        #5;
        inputs(4'b1111,4'b1100);
        #5;
        inputs(4'b1x0z,4'b1x01);
        #5 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,equality_operator);
end
endmodule
```

Simulation Result

output

```
a=1x0z ,b=1x0z p=x q=x r=1 s=0
a=1010 ,b=1010 p=1 q=0 r=1 s=0
a=1111 ,b=1100 p=0 q=1 r=0 s=1
a=1x0z ,b=1x01 p=x q=x r=0 s=1
$finish at simulation time                40
```

waveform



45.0 Design a relational operation by using task –

Design code

```
module relational_operator();

reg [3:0]a,b,x,y,z;

reg p,q,r,s,t;


task inputs(input [3:0] ain,bin,xin,yin,zin);

    begin

        a=ain;

        b=bin;

        x=xin;

        y=yin;

        z=zin;

        p= (a < b);

        q= (a > b);

        r= (x >= y);

        s= (x <= y);

        t= (x >= z);

        #5;

        $display("a=%b ,b=%b x=%b y=%b z=%b",a,b,x,y,z);

        $display("p=%b q=%b r=%b s=%b t=%b",p,q,r,s,t);
```

```
        end
endtask

initial begin
        inputs(4'd4,4'd3,4'b0000,4'b1100,4'b1x1x);
        #5;
        inputs(4'd5,4'd5,4'b0111,4'b0111,4'bxxxx);
        #5;
        inputs(4'd2,4'd7,4'b1111,4'b0011,4'b1101);
        #5;
        inputs(4'd0,4'd1,4'b1010,4'b1011,4'b10x0);
        #5 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,relational_operator);
end
endmodule
```

Simulation Result

output

```
a=0100 ,b=0011 x=0000 y=1100 z=1x1x
p=0 q=1 r=0 s=1 t=x
a=0101 ,b=0101 x=0111 y=0111 z=xxxx
p=0 q=0 r=1 s=1 t=x
a=0010 ,b=0111 x=1111 y=0011 z=1101
```

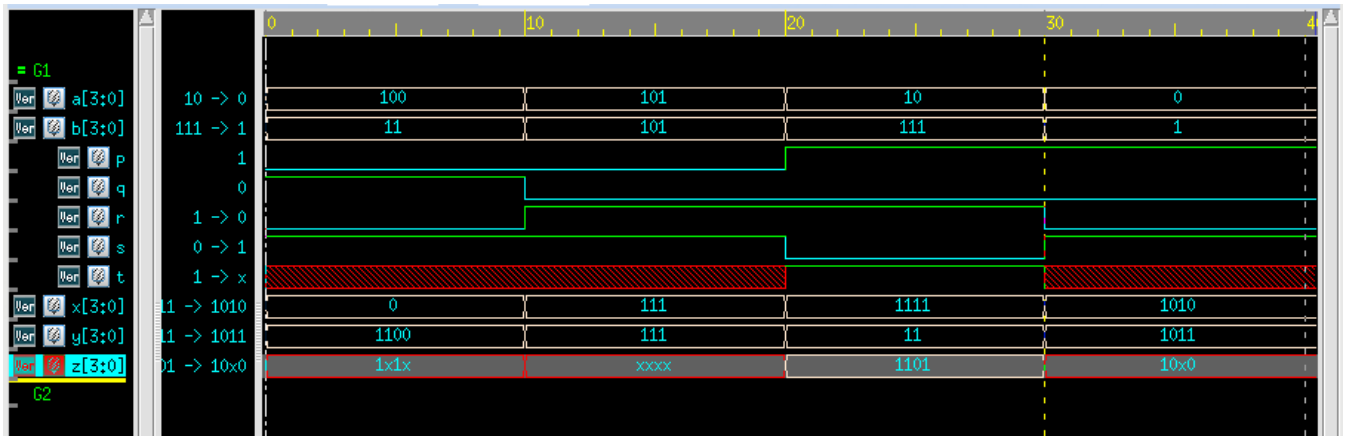
p=1 q=0 r=1 s=0 t=1

a=0000 ,b=0001 x=1010 y=1011 z=10x0

p=1 q=0 r=0 s=1 t=x

$finish at simulation time 40

waveform



46.0 Design a concatenation operation by using task –

Design code

```
module concatenate_operator();
reg a;
reg [2:0]b,c;
reg [7:0]x,y,z,p;

task inputs(input a_in,input [2:0] b_in,c_in);
        begin
                a=a_in;
                b=b_in;
                c=c_in;
                x={a,b,c};
                y={b,2'b01,a};
                z={x[1:0],b[2:1],c};
```

```
        p= {{2{a}},b,{2{c}}};
        #5;

        $display("a=%b ,b=%b c=%b",a,b,c);
        $display("x=%b y=%b z=%b p=%b",x,y,z,p);
    end
endtask
initial begin
    inputs(1'b1,3'b100,3'b110);
    #5;
    inputs(1'b0,3'b001,3'b111);
    #5;
    inputs(1'b1,3'b010,3'b000);
    #5;
    inputs(1'b0,3'b111,3'b101);
    #5 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,concatenate_operator);
end
endmodule
```

Simulation Result

output

```
a=1 ,b=100 c=110
x=01100110 y=00100011 z=01010110 p=00110110
a=0 ,b=001 c=111
```

x=00001111 y=00001010 z=01100111 p=01111111

a=1 ,b=010 c=000

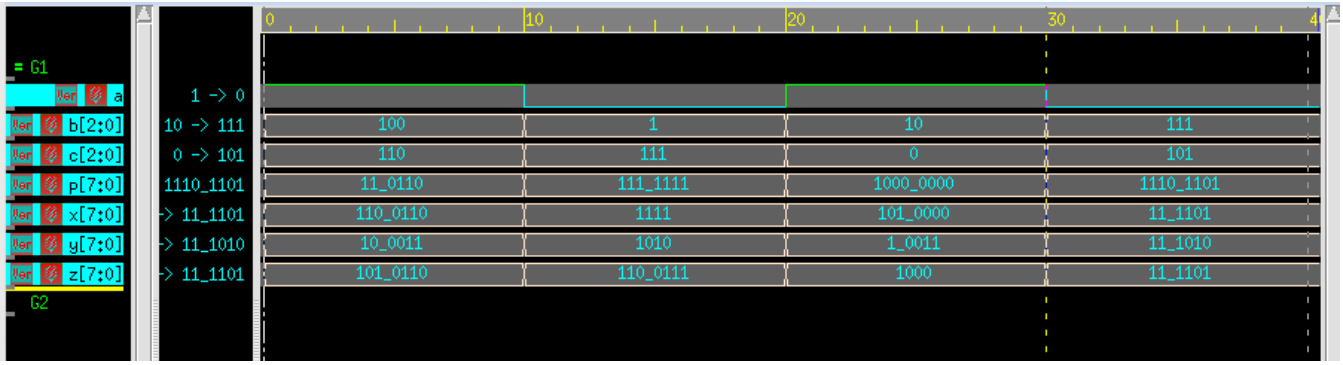
x=01010000 y=00010011 z=00001000 p=10000000

a=0 ,b=111 c=101

x=00111101 y=00111010 z=00111101 p=11101101

$finish at simulation time 40

waveform



47.0 Design an arithmetic operation by using task –

Design code

```
module arithmetic_operator();
reg [3:0]a,b,c;
reg [7:0]p,q,r,u;
integer d,e,s,t;

task inputs(input [3:0] ain,bin,cin,input integer din,ein);
    begin
        a=ain;
        b=bin;
        c=cin;
        d=din;
        e=ein;
```

```
            p= a*b;
            q=a+b;
            r=b-a;
            s=e/d;
            t=e%d;
            u=c*a;
            #5;
            $display("a=%b ,b=%b c=%b d=%0d e=%0d",a,b,c,d,e);
            $display("p=%b q=%b r=%b s=%0d t=%0d u=%b ",p,q,r,s,t,u);
        end
endtask
initial begin
        inputs(4'b0010,4'b0011,4'b1010,3,8);
        #5;
        inputs(4'b0101,4'b0110,4'b01x0,2,9);
        #5;
        inputs(4'b0001,4'b1111,4'b0001,4,16);
        #5;
        inputs(4'b1001,4'b0010,4'b1x1x,5,23);
        #5 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,arithmetic_operator);
end
endmodule
```

Simulation Result

output

a=0010 ,b=0011 c=1010 d=3 e=8

p=00000110 q=00000101 r=00000001 s=2 t=2 u=00010100

a=0101 ,b=0110 c=01x0 d=2 e=9

p=00011110 q=00001011 r=00000001 s=4 t=1 u=xxxxxxxx

a=0001 ,b=1111 c=0001 d=4 e=16

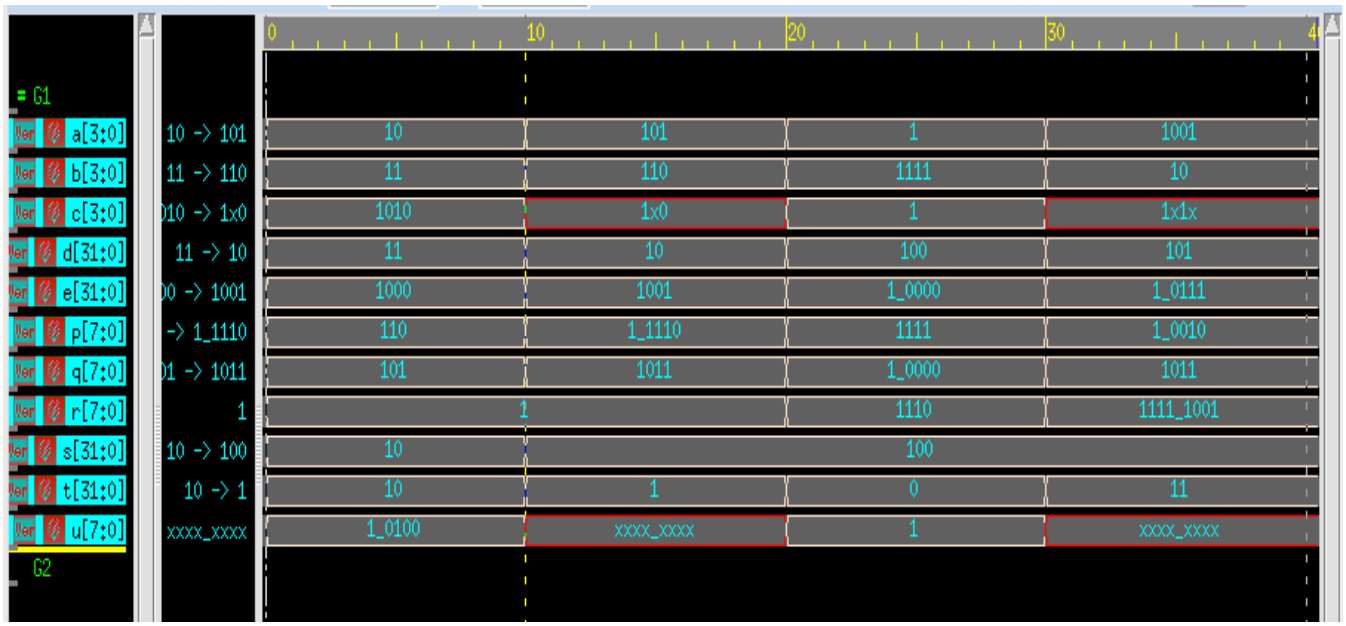
p=00001111 q=00010000 r=00001110 s=4 t=0 u=00000001

a=1001 ,b=0010 c=1x1x d=5 e=23

p=00010010 q=00001011 r=11111001 s=4 t=3 u=xxxxxxxx

$finish at simulation time 40

waveform



48.0 Design a bidirectional shift register-

Design code

module bidir_shift_reg(data_out,data_in,shift,load,clk,rst);

input clk,rst;

input load,shift;

input [7:0]data_in;

```
output reg [7:0]data_out;

always @(posedge clk)begin
    if (rst)
        data_out <=8'd0;
    else if (load)
        data_out <= data_in;
    else begin
        case (shift)
            1'b1:data_out <= {data_out[0],data_out[7:1]};
            1'b0:data_out <= {data_out[6:0],data_out[7]};
        endcase
    end
end
endmodule
```

Testbench

```
module TB();
reg clk,rst;
reg load,shift;
reg [7:0]data_in;
wire [7:0]data_out;

bidir_shift_reg
dut(.clk(clk),.rst(rst),.load(load),.shift(shift),.data_in(data_in),.data_out(data_out));

initial clk=0;
always #5 clk=~clk;
```

```
initial begin
    $monitor("Time=%0t for rst=%b | load=%b | shift=%b | data_in=%b | we get data_out=%b",$time,rst,load,shift,data_in,data_out);
    rst=1;load=0;shift=0;data_in=8'b11001101;
#10 rst=0;
#10 load=1;
#10 load=0;
#10 shift=1;
#10 shift=0;
#10 shift=0;
#10 shift=1;
#10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

```
Time=0 for rst=1 | load=0 | shift=0 | data_in=11001101 | we get data_out=xxxxxxxx
Time=5 for rst=1 | load=0 | shift=0 | data_in=11001101 | we get data_out=00000000
Time=10 for rst=0 | load=0 | shift=0 | data_in=11001101 | we get data_out=00000000
Time=20 for rst=0 | load=1 | shift=0 | data_in=11001101 | we get data_out=00000000
Time=25 for rst=0 | load=1 | shift=0 | data_in=11001101 | we get data_out=11001101
Time=30 for rst=0 | load=0 | shift=0 | data_in=11001101 | we get data_out=11001101
Time=35 for rst=0 | load=0 | shift=0 | data_in=11001101 | we get data_out=10011011
```

Time=40 for rst=0 | load=0 | shift=1 | data_in=11001101 | we get data_out=10011011

Time=45 for rst=0 | load=0 | shift=1 | data_in=11001101 | we get data_out=11001101

Time=50 for rst=0 | load=0 | shift=0 | data_in=11001101 | we get data_out=11001101

Time=55 for rst=0 | load=0 | shift=0 | data_in=11001101 | we get data_out=10011011

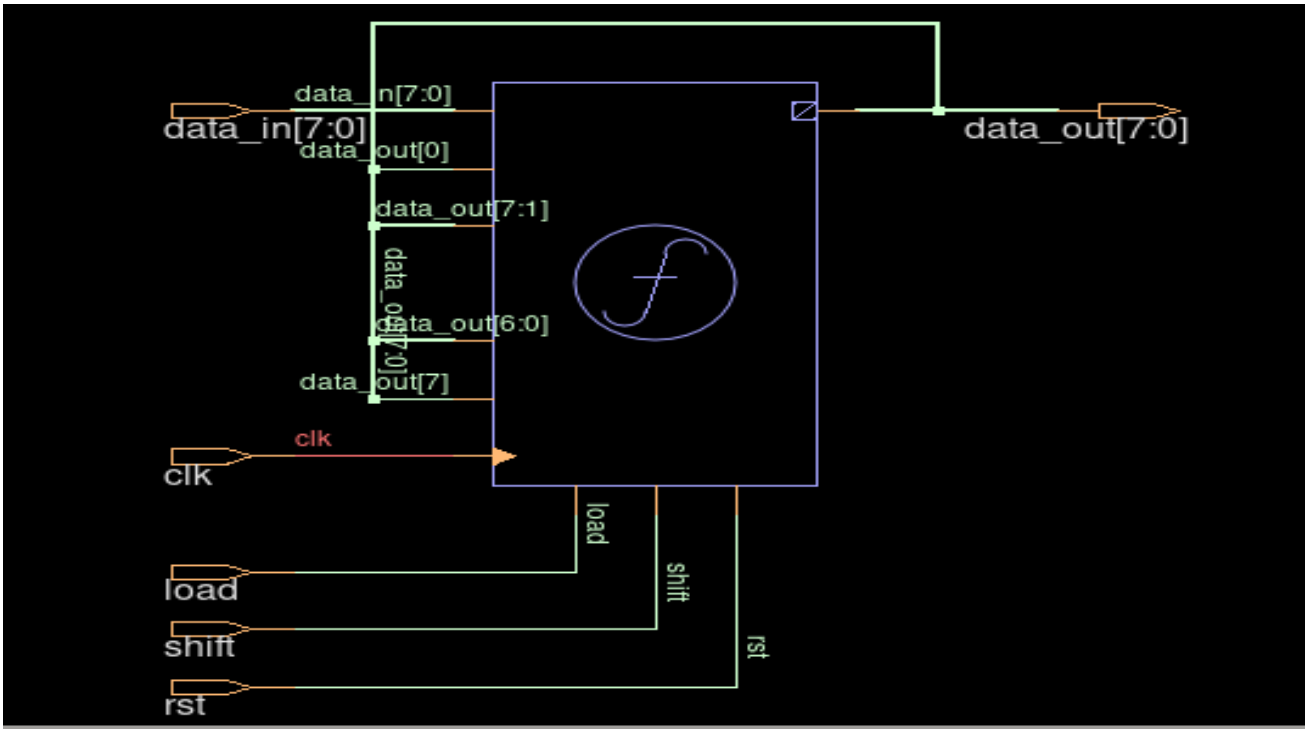
Time=65 for rst=0 | load=0 | shift=0 | data_in=11001101 | we get data_out=00110111

Time=70 for rst=0 | load=0 | shift=1 | data_in=11001101 | we get data_out=00110111

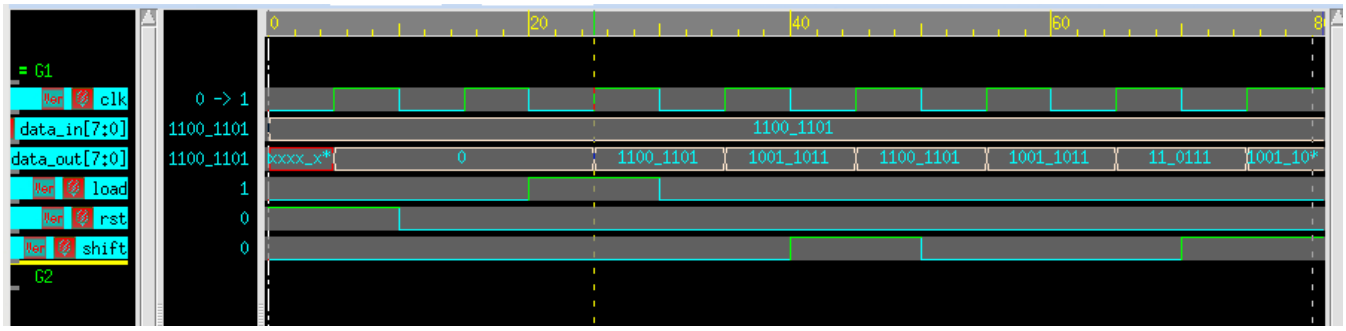
Time=75 for rst=0 | load=0 | shift=1 | data_in=11001101 | we get data_out=10011011

$finish at simulation time 80

Schematic Design



waveform



49.0 Design an +ve edge triggered Asynchronous 4-bit D-FF (Active high reset)

Design code

```
module Async_rst_reg(q,d,clk,rst);
input [3:0]d;
input clk,rst;
output reg [3:0]q;

always @(posedge clk or posedge rst)
begin
	if (rst)
		q<=4'b0000;
	else
		q<=d;
end
endmodule
```

Testbench

```
module d_ff_tb();
reg [3:0]d;
reg clk,rst;
wire [3:0]q;

Async_rst_reg dut(.q(q),.clk(clk),.d(d),.rst(rst));

initial clk=0;
always #5 clk=~clk;
```

```
initial begin
    $monitor("Time=%0t for rst=%b|d=%b|q=%b",$time,rst,d,q);
    rst=1; d=4'b1010;
    #10;rst=0; d=4'b1001;
    #10; d=4'b1101;
    #10; rst=1;d=4'b1100;
    #10; rst=0;d=4'b0111;
    #10; d=4'b0001;
    #10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,d_ff_tb);
end
endmodule
```

Simulation Result

output

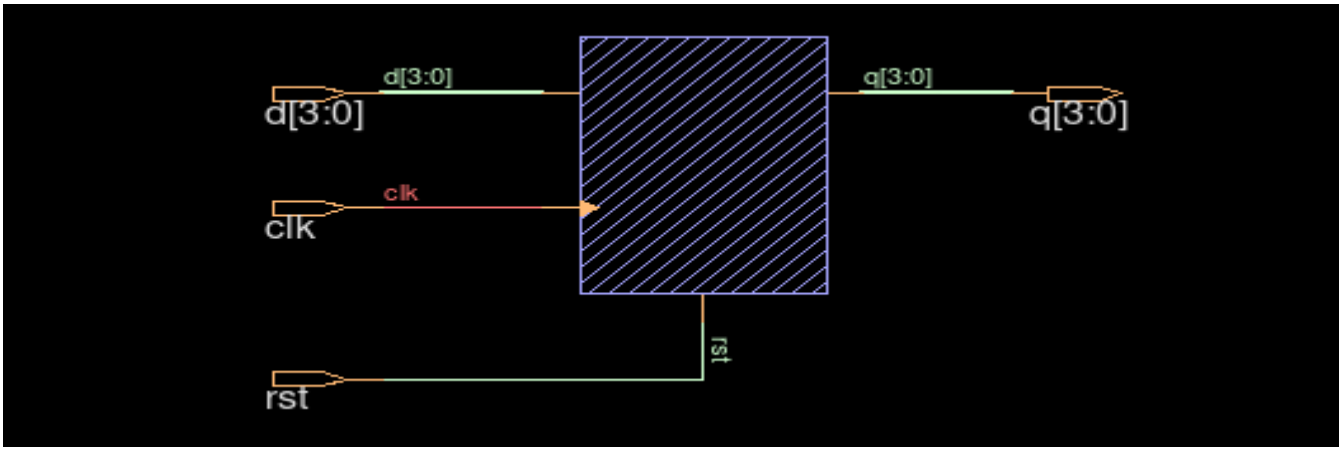
```
Time=0 for rst=1|d=1010|q=0000
Time=10 for rst=0|d=1001|q=0000
Time=15 for rst=0|d=1001|q=1001
Time=20 for rst=0|d=1101|q=1001
Time=25 for rst=0|d=1101|q=1101
Time=30 for rst=1|d=1100|q=0000
Time=40 for rst=0|d=0111|q=0000
Time=45 for rst=0|d=0111|q=0111
```

Time=50 for rst=0|d=0001|q=0111

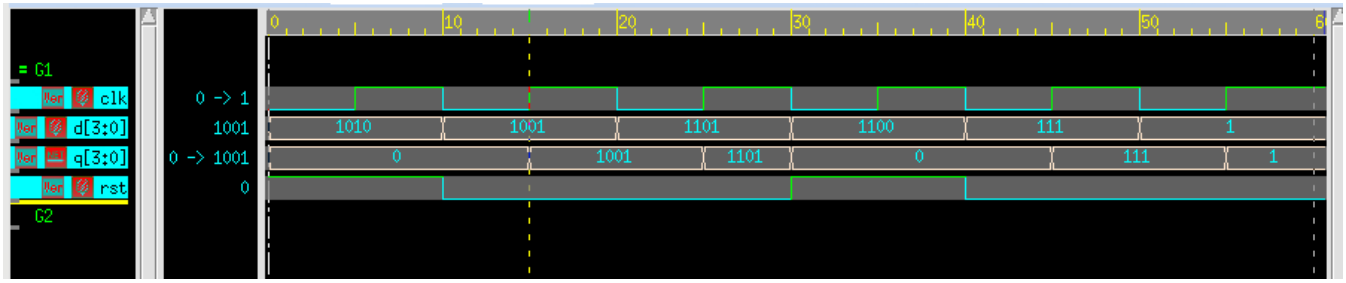
Time=55 for rst=0|d=0001|q=0001

$finish at simulation time 60

Schematic Design



waveform



50.0 Design an up-counter with load data (8-bit) using task

Design code

```
module up_counter(count,load,clk,rst,data);
input clk,rst,load;
input [8:0]data;
output reg [7:0]count;

always @(posedge clk)begin
    if (rst==1)
        count <=8'd0;
```

```
    else if (load)
        count <=data;
    else
        count <= count +1;
end
endmodule
```

Testbench

```
module TB();
reg [7:0]data;
reg clk,rst,load;
wire [7:0]count;

up_counter dut(.data(data),.clk(clk),.rst(rst),.load(load),.count(count));

initial clk=0;
always #5 clk=~clk;

task t1;begin
    rst=1'b1;
    #5;
    rst=1'b0;
end
endtask

task t2(input [7:0]i);begin
    load=1'b1;
    data=i;
```

```
        #10;
        load=1'b0;
end
endtask
task t3(input [7:0]i);begin
        data=i;
end
endtask

initial begin
        clk=0;rst=0;load=0;data=8'd0;
        $monitor("Time=%0t rst=%b|
load=%b|data=%b|count=%b",$time,rst,load,data,count);
        t1;
        t2(8'b0000_1010);
        #150;
        #10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

```
Time=0 rst=1| load=0|data=00000000|count=xxxxxxxx
```

Time=5 rst=0| load=1|data=00001010|count=00001010

Time=15 rst=0| load=0|data=00001010|count=00001011

Time=25 rst=0| load=0|data=00001010|count=00001100

Time=35 rst=0| load=0|data=00001010|count=00001101

Time=45 rst=0| load=0|data=00001010|count=00001110

Time=55 rst=0| load=0|data=00001010|count=00001111

Time=65 rst=0| load=0|data=00001010|count=00010000

Time=75 rst=0| load=0|data=00001010|count=00010001

Time=85 rst=0| load=0|data=00001010|count=00010010

Time=95 rst=0| load=0|data=00001010|count=00010011

Time=105 rst=0| load=0|data=00001010|count=00010100

Time=115 rst=0| load=0|data=00001010|count=00010101

Time=125 rst=0| load=0|data=00001010|count=00010110

Time=135 rst=0| load=0|data=00001010|count=00010111

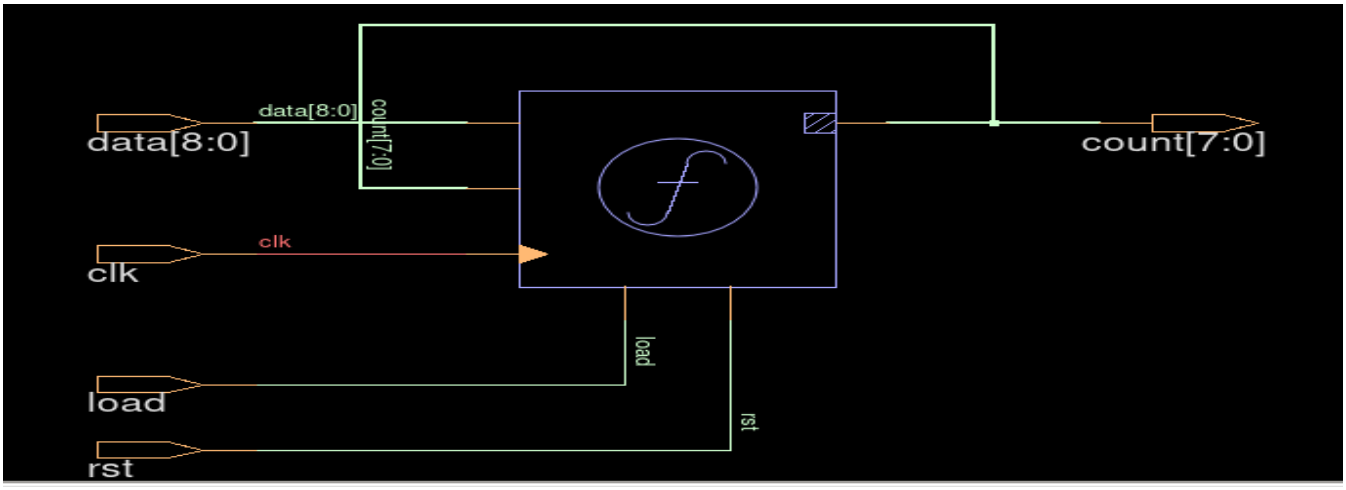
Time=145 rst=0| load=0|data=00001010|count=00011000

Time=155 rst=0| load=0|data=00001010|count=00011001

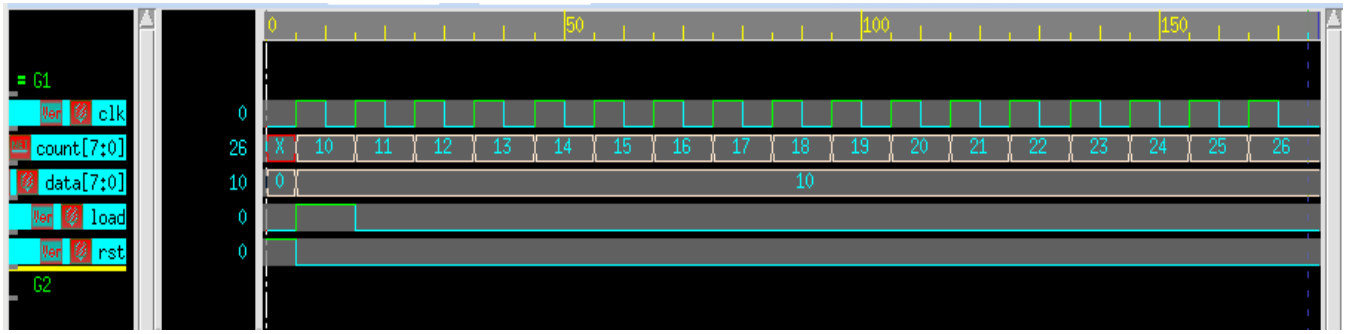
Time=165 rst=0| load=0|data=00001010|count=00011010

$finish at simulation time 175

Schematic Design



waveform



51.0 Design a synchronous 3-bit up-down counter using JK FF

Design code

```
module jk_ff (Q,J,K,clk,rst);
input J,K,clk,rst;
output reg Q;

always @(posedge clk or posedge rst)begin
    if (rst)
        Q <= 0;
    else begin
        case ({J, K})
            2'b00: Q <= Q;
            2'b01: Q <= 0;
            2'b10: Q <= 1;
            2'b11: Q <= ~Q;
        endcase
    end
end
endmodule
```

```
module up_down_counter_3bit (count,clk,rst,up_down);
input clk,rst,up_down;
output [2:0]count;

wire [2:0] J,K;
wire [2:0] Q;

assign count = Q;

assign J[0] = 1'b1;
assign K[0] = 1'b1;

assign J[1] = up_down ? Q[0] : ~Q[0];
assign K[1] = up_down ? Q[0] : ~Q[0];

assign J[2] = up_down ? (Q[1] & Q[0]) : (~Q[1] & ~Q[0]);
assign K[2] = J[2];

jk_ff ff0(.J(J[0]),.K(K[0]),.clk(clk),.rst(rst),.Q(Q[0]));
jk_ff ff1(.J(J[1]),.K(K[1]),.clk(clk),.rst(rst),.Q(Q[1]));
jk_ff ff2(.J(J[2]),.K(K[2]),.clk(clk),.rst(rst),.Q(Q[2]));

endmodule
```

Testbench

```
module TB();
reg clk,rst,up_down;
wire [2:0] count;
```

```
up_down_counter_3bit dut(.clk(clk),.rst(rst),.up_down(up_down),.count(count));


always #5 clk = ~clk;


initial begin
    $monitor("Time=%0t | rst=%b | up_down=%b | count=%b", $time, rst, up_down, count);
    clk =0;rst =1;up_down =1;
    #10 rst = 0;
    up_down = 1;
    #100;
    up_down = 0;
    #100;$finish();
end


initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

```
Time=0 | rst=1 | up_down=1 | count=000
Time=10 | rst=0 | up_down=1 | count=000
Time=15 | rst=0 | up_down=1 | count=001
Time=25 | rst=0 | up_down=1 | count=010
```

Time=35 | rst=0 | up_down=1 | count=011

Time=45 | rst=0 | up_down=1 | count=100

Time=55 | rst=0 | up_down=1 | count=101

Time=65 | rst=0 | up_down=1 | count=110

Time=75 | rst=0 | up_down=1 | count=111

Time=85 | rst=0 | up_down=1 | count=000

Time=95 | rst=0 | up_down=1 | count=001

Time=105 | rst=0 | up_down=1 | count=010

Time=110 | rst=0 | up_down=0 | count=010

Time=115 | rst=0 | up_down=0 | count=001

Time=125 | rst=0 | up_down=0 | count=000

Time=135 | rst=0 | up_down=0 | count=111

Time=145 | rst=0 | up_down=0 | count=110

Time=155 | rst=0 | up_down=0 | count=101

Time=165 | rst=0 | up_down=0 | count=100

Time=175 | rst=0 | up_down=0 | count=011

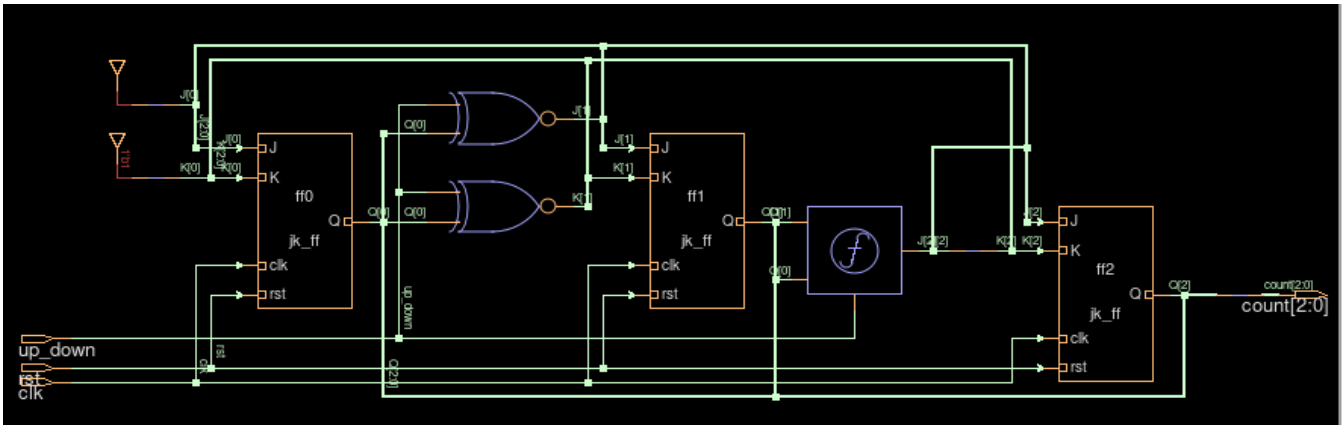
Time=185 | rst=0 | up_down=0 | count=010

Time=195 | rst=0 | up_down=0 | count=001

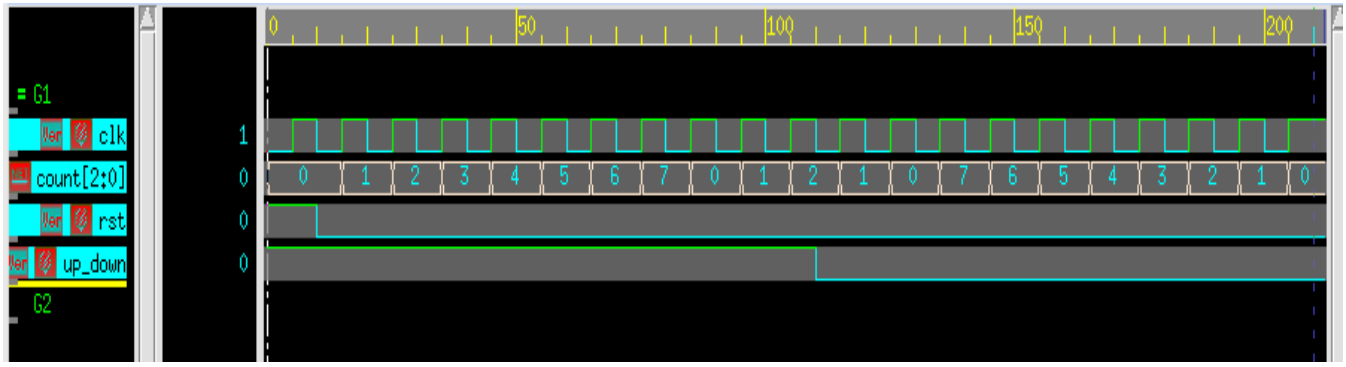
Time=205 | rst=0 | up_down=0 | count=000

$finish at simulation time 210

Schematic Design



waveform



52.0 FSM to detect 1010 sequence detector-

Design code

```
module detect_1010 (clk,rst,in,out);
input clk,rst,in;
output out;

reg [2:0]PST,NST;
parameter s0=3'b000,s1=3'b001,s2=3'b010,s3=3'b011,s4=3'b100;

always @(posedge clk)begin
        if (rst)
                PST<=s0;
        else
```

```
            PST<=NST;
end

always @(*)begin
        case(PST)
            s0:begin
                if(in)
                    NST<=s1;
                else
                    NST<=s0;
            end

            s1:begin
              if (in)
                  NST<=s1;
              else
                  NST<=s2;
          end

            s2:begin
              if (in)
                  NST<=s3;
              else
                  NST<=s0;
          end

            s3:begin
```

```
                if (in)
                    NST<=s1;
                else
                        NST<=s4;
            end

            s4:begin
                if (in)
                        NST<=s1;
                else
                        NST<=s0;
            end
            default:NST<=s0;
        endcase
end
assign out=(PST==s4) ? 1'b1 : 1'b0;
endmodule
```

Testbench

```
module TB();
reg clk,rst,in;
wire out;

detect_1010 dut(.clk(clk),.rst(rst),.in(in),.out(out));

initial clk=0;
always #5 clk=~clk;
```

```
task reset;
	begin
		#10 rst=1;
		#10 rst=0;
	end
endtask

task inputs(input data);
	begin
		in=data;
	end
endtask

initial begin
	$monitor("Time=%0t |in=%0b | out=%0b",$time,in,out);
	rst=0;
	reset;
	#10 inputs(0);
	#10 inputs(0);
	#10 inputs(1);
	#10 inputs(0);
	#10 inputs(1);
	#10 inputs(0);
	#10 inputs(1);
	#10 inputs(1);
	#10 inputs(0);
	#10 inputs(1);
```

```
        #10 inputs(0);
        #10 inputs(1);
        #10 inputs(1);
        #10 inputs(0);
        #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);
end
endmodule
```

Simulation Result

output

```
Time=0 |in=x | out=x
Time=15 |in=x | out=0
Time=30 |in=0 | out=0
Time=50 |in=1 | out=0
Time=60 |in=0 | out=0
Time=70 |in=1 | out=0
Time=80 |in=0 | out=0
Time=85 |in=0 | out=1
Time=90 |in=1 | out=1
Time=95 |in=1 | out=0
Time=110 |in=0 | out=0
Time=120 |in=1 | out=0
Time=130 |in=0 | out=0
Time=135 |in=0 | out=1
```

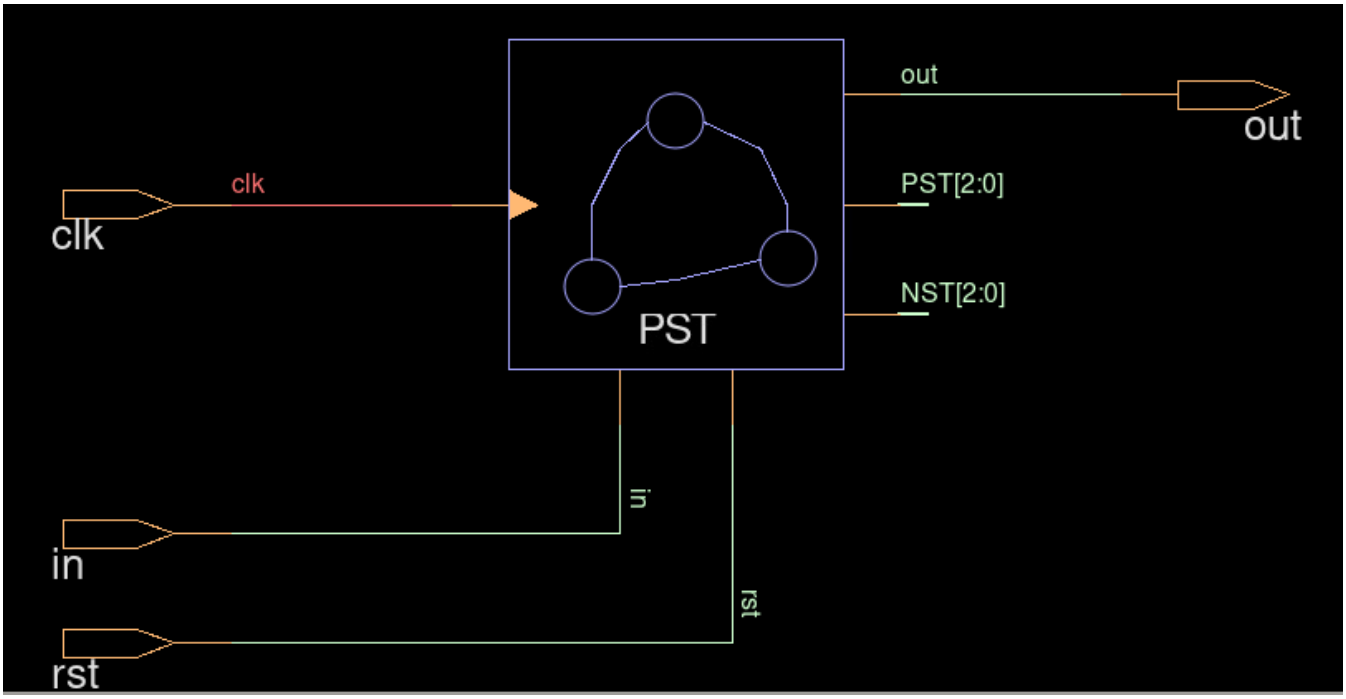
Time=140 |in=1 | out=1

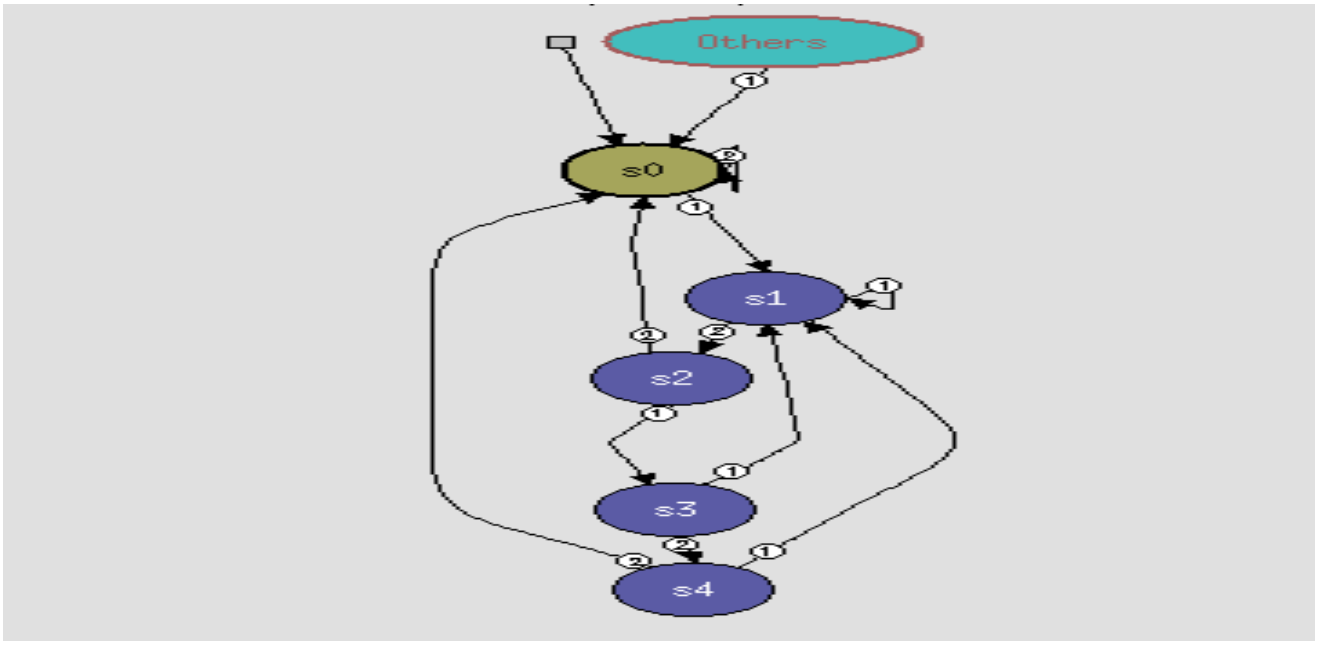
Time=145 |in=1 | out=0

Time=160 |in=0 | out=0

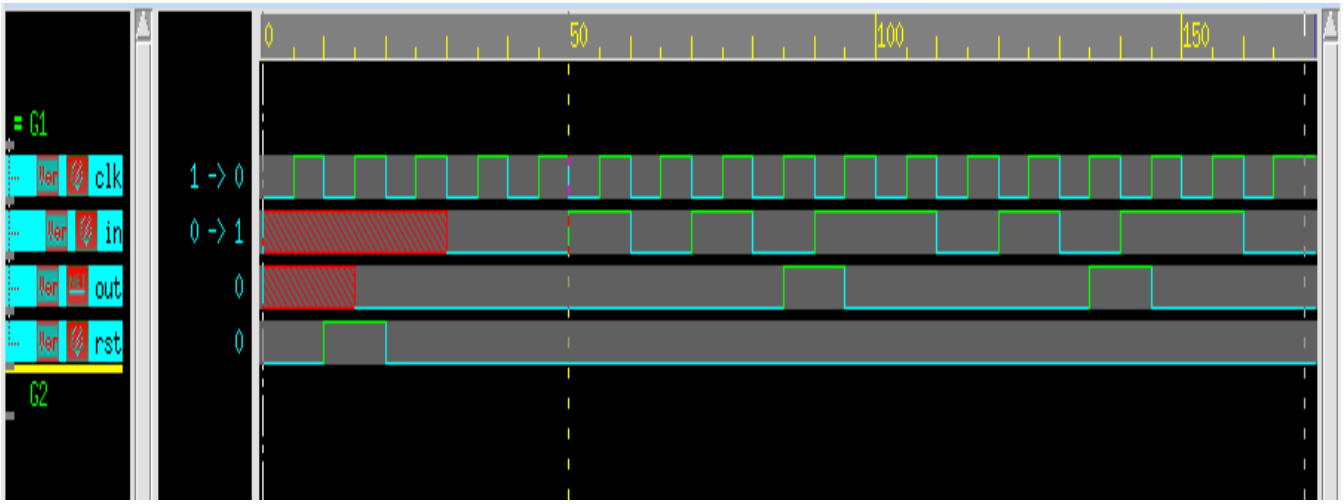
$finish at simulation time 170

Schematic Design





waveform



Design a Generic n-bit full adder using 8-bit full adder by using data flow

Design code

Testbench

Run command

Simulation Result

output

Schematic Design

waveform

Testbench

Run command

Simulation Result

Output

Schematic Design

waveform

Testbench

Lab2

Design code

Testbench

Run command

Simulation Result

Output

Schematic Design

waveform

Lab9

Lab10

Internship

RTL2GDSII FLOW WITH SYNOPSIS TOOLS

1.0 //find the default data size and its value.

Design code

```
module data_size();
integer a;
reg b;
wire c;
real d;
time e;
```

```
initial begin
    $display("size of integer data type is %0d ,value is :%0d", $bits(a),a);
    $display("size of reg  data type is %0d ,value is :%0d", $bits(b),b);
    $display("size of wire data type is %0d ,value is :%0d", $bits(c),c);
    $display("size of real data type is 64 bits ,value is:%0f",d);
    $display("size of time data type is %0t ,value is :%0d", $bits(e),e);
end
endmodule
```

Run command

```
vcs -sverilog default_data_size.v -kdb -lca -debug_access+all -full64
```

output

```
size of integer data type is 32 ,value is :x
size of reg  data type is 1 ,value is :x
size of wire data type is 1 ,value is :z
size of real data type is 64 bits ,value is:0.000000
size of time data type is 64 ,value is :x
```

2.0 Design a RTL of half adder by using data flow

Design code

```
// RTL code of half adder
module half_adder(sum,carry,a,b);
input a,b;
output sum,carry;

assign sum= a^b;
assign carry= a & b;
```

```
endmodule
```

Testbench

```
// testbench of half adder
module half_adder_tb();
reg a,b;
wire sum,carry;

half_adder dut (sum ,carry, a ,b);
initial begin
    $monitor("Time=%0t,sum=%b,carry=%b,a=%b,b=%b",$time,sum,carry,a,b);

    a=0;b=0; #10
    a=0;b=1; #10
    a=1;b=0; #10
    a=1;b=1; #10
    $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,half_adder_tb);
end
endmodule
```

Run command

```
vcs rtl_half_adder.v half_adder_tb.v -kdb -lca -debug_access+all -full64
```

output

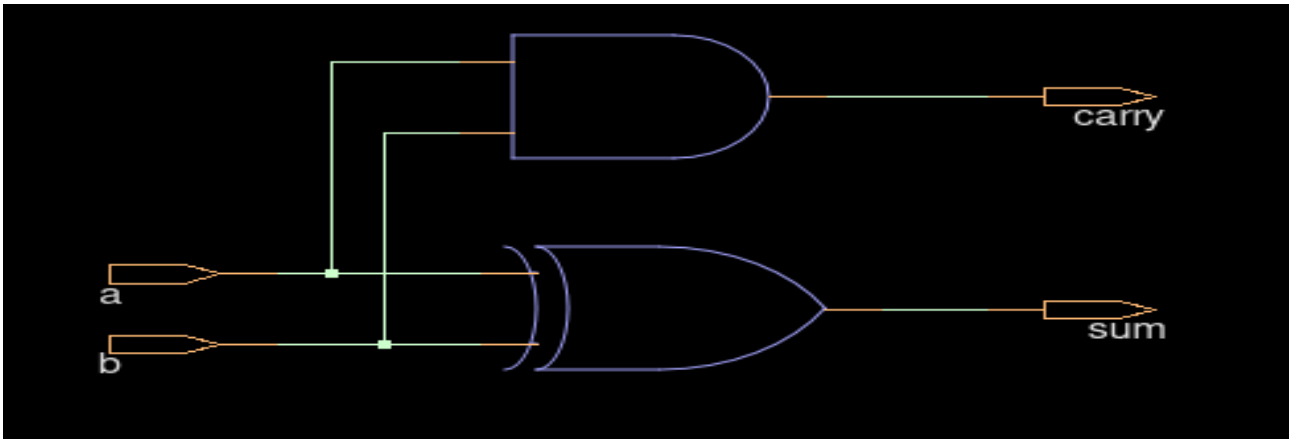
```
Time=0,sum=0,carry=0,a=0,b=0
```

Time=10,sum=1,carry=0,a=0,b=1

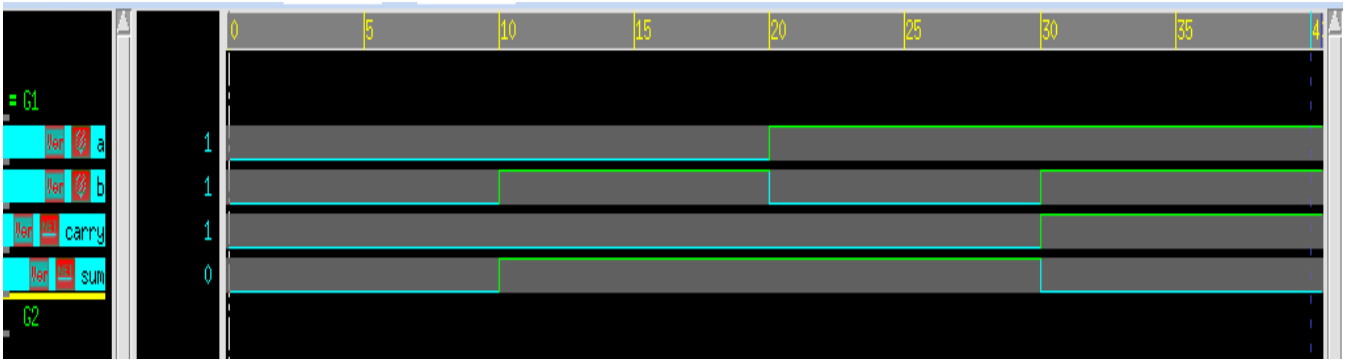
Time=20,sum=1,carry=0,a=1,b=0

Time=30,sum=0,carry=1,a=1,b=1

Schematic Design



waveform



3.0 Design a RTL of half adder by using behavioral modelling

Design code

//Design a RTL of a half adder

module HA(A,B,Clock,Sum,Carry);

input A,B,Clock;

output reg Sum,Carry;

reg A_reg,B_reg,Sum_reg,Carry_reg;

```
always @ (posedge Clock)
begin
	A_reg<=A;
	B_reg <=B;
end

always @ (*)
begin
{Carry_reg, Sum_reg} = A_reg + B_reg;
end

always @ (posedge Clock)
begin
	Sum<=Sum_reg;
	Carry<=Carry_reg;
end
endmodule
```

Testbench

```
//design a testbench of half adder
module TB();
reg A, B, Clock;
wire Sum, Carry;

HA dut(.A(A), .B(B), .Clock(Clock), .Sum(Sum), .Carry(Carry));

initial Clock = 0;
```

```
always #5 Clock = ~Clock;  // posedge at 5, 15, 25, etc.


initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,TB);


    A = 0; B = 0;
    $display("Applying Inputs A=0, B=0");
    @(posedge Clock);
    @(posedge Clock);
    #1;

$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,Carry);
    A = 0; B = 1;
    $display("Applying Inputs A=0, B=1");
    @(posedge Clock);
    @(posedge Clock);

$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,Carry);
    A = 1; B = 0;
$display("Applying Inputs A=1, B=0");
    @(posedge Clock);
    @(posedge Clock);

$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,Carry);
    A = 1; B = 1;
```

```
        $display("Applying Inputs A=1, B=1");
        @(posedge Clock);
        @(posedge Clock);

$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,Carry);
        #40 $finish;
end
endmodule
```

Run command

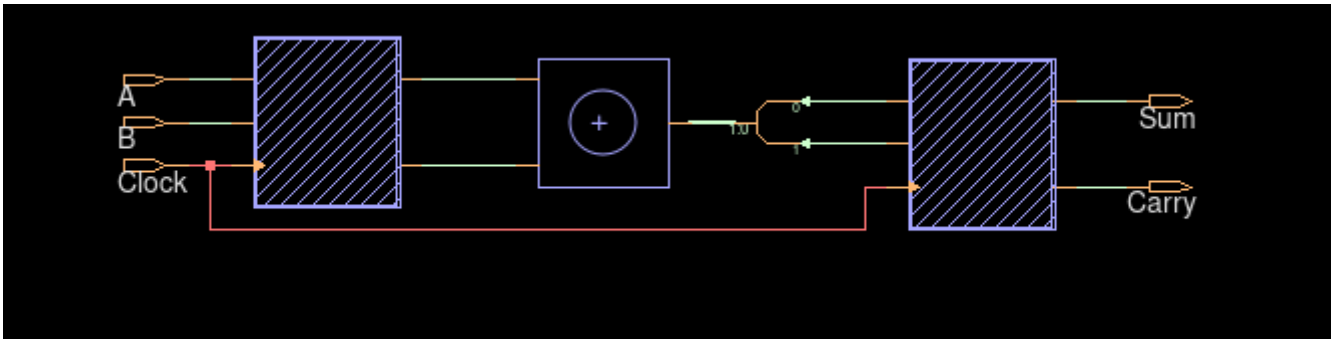
```
vcs half_adder.v half_adder_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

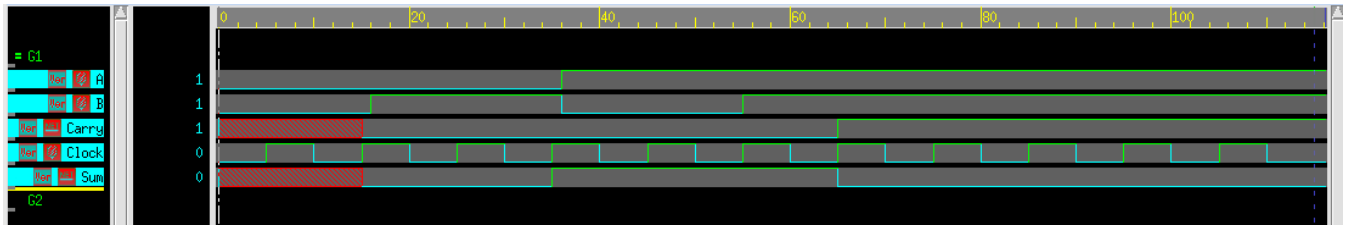
Output

```
Applying Inputs A=0, B=0
Time=16,Clock=1,A=0,B=0,Sum=0,Carry=0
Applying Inputs A=0, B=1
Time=36,Clock=1,A=0,B=1,Sum=1,Carry=0
Applying Inputs A=1, B=0
Time=55,Clock=1,A=1,B=0,Sum=1,Carry=0
Applying Inputs A=1, B=1
Time=75,Clock=1,A=1,B=1,Sum=0,Carry=1
```

Schematic Design



waveform



4.0 Design a 32-bit ALU of RTL code

File name- RTL_ALU

Design code

```
//Design a 32_bit ALU of RTL code
module alu_32 (alu_out,a,b,clk,rst,alu_sel);
input logic clk;
input logic rst;
input logic [31:0]a,b;
input logic [3:0]alu_sel;
output logic [31:0]alu_out;

logic [31:0] reg_a,reg_b, res_reg;
logic [3:0] sel_reg;

//Register inputs and alu_sel
```

```
always_ff @(posedge clk or posedge rst) begin
    if(rst)begin
        reg_a <= 32'd0;
        reg_b <=32'd0;
        sel_reg <= 4'd0;
    end else begin
        reg_a <= a;
        reg_b <= b;
        sel_reg <= alu_sel;
    end
end

//alu operation
always_comb begin
    case (sel_reg)
        4'd0:res_reg= reg_a + reg_b;      //ADD
        4'd1:res_reg= reg_a - reg_b;      //SUB
        4'd2:res_reg= reg_a & reg_b;      //AND
        4'd3:res_reg= reg_a | reg_b;      //OR
        4'd4:res_reg= reg_a ^ reg_b;      //XOR
        4'd5:res_reg= ~reg_a;             //NOT A
        4'd6:res_reg= ~(reg_a & reg_b);   //NAND
        4'd7:res_reg= ~(reg_a | reg_b);   //NOR
        4'd8:res_reg= reg_a <<1;          //SL
        4'd9:res_reg= reg_a >>1;          //SR
        default:res_reg=32'hDEADBEEF;     //DEFAULT
    endcase
```

```
end

//Register output

always_ff @(posedge clk or posedge rst)begin
    if(rst)
        alu_out <=32'd0;
    else
        alu_out <=res_reg;
end

endmodule
```

Testbench

```
/Testbench for 32bit_alu design
module alu_32_tb();
logic clk,rst;
logic [31:0]a,b;
logic [3:0]alu_sel;
logic [31:0]alu_out;

alu_32 dut(.clk(clk),.rst(rst),.a(a),.b(b),.alu_sel(alu_sel),.alu_out(alu_out));

always #5 clk=~clk;

initial begin
    $display("starting ALU test...");
    clk=0;rst=1;
```

```
        a=0;b=0;alu_sel=0;
        #10 rst=0;

        repeat (10) begin
            @ (negedge clk);
            a =$urandom;
            b =$urandom;
            alu_sel =$urandom % 10;
            @(posedge clk);
            #1;
            $display("alu_sel: %0d | a:%0h |b:%0h |Result:%oh",alu_sel,a,b,alu_out);
        end
        #100 $finish();
end

initial begin
        $fsdbDumpfile("myfile.fsdb");
        $fsdbDumpvars(0,alu_32_tb);
end

endmodule
```

Run command

```
vcs -sverilog alu_32bit.v alu_32bit_tb.sv -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
alu_sel: 9 | a:3b29e213 |b:69262e70 |Result:00000000000h
```

alu_sel: 9 | a:3d5d03e2 |b:20adb297 |Result:03545170411h

alu_sel: 4 | a:7a0900c5 |b:cea5c6ec |Result:03653500761h

alu_sel: 3 | a:947d40c |b:214b2b2c |Result:26453143051h

alu_sel: 0 | a:51ab091b |b:73a72e45 |Result:05123777454h

alu_sel: 4 | a:c333cd6c |b:edca9767 |Result:30524433540h

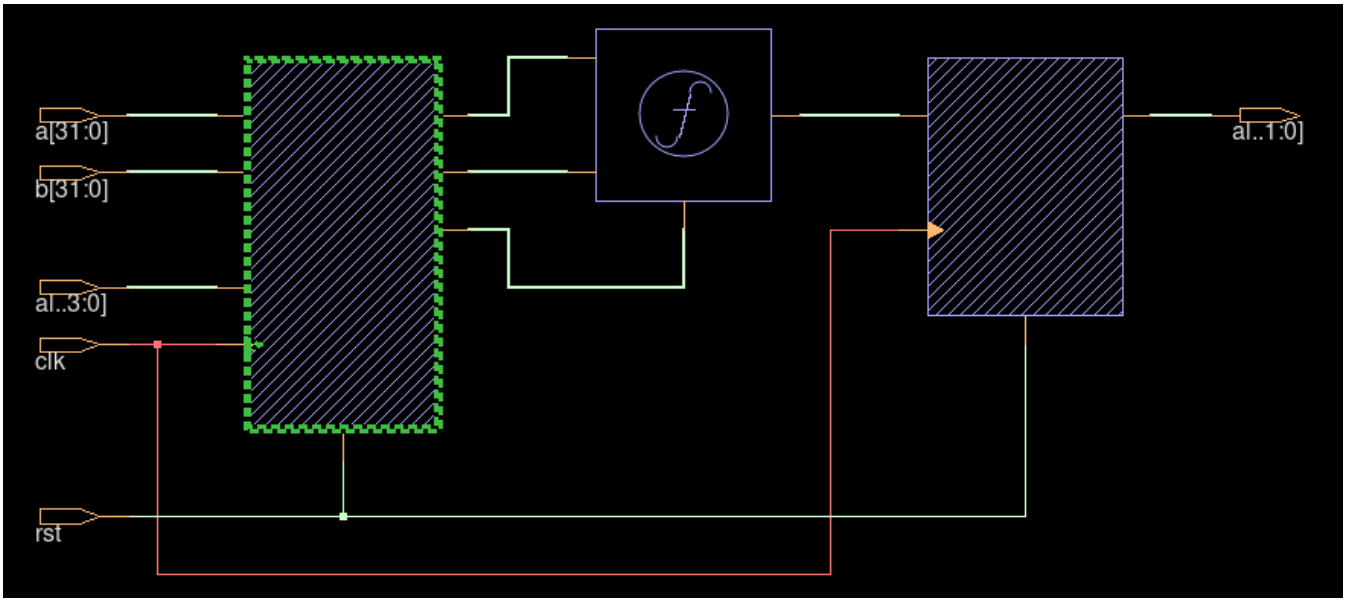
alu_sel: 1 | a:f1305a4a |b:d4e2dca6 |Result:05676255013h

alu_sel: 8 | a:6bfaa09d |b:74a9477c |Result:03423276644h

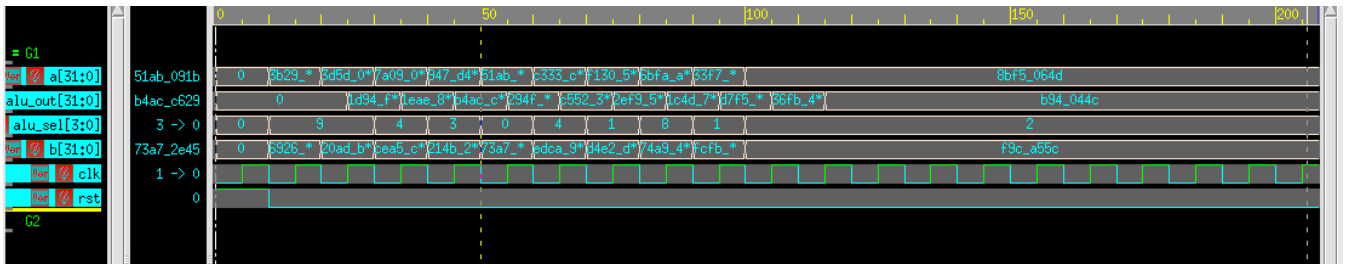
alu_sel: 1 | a:33f729eb |b:fcfbe75b |Result:32775240472h

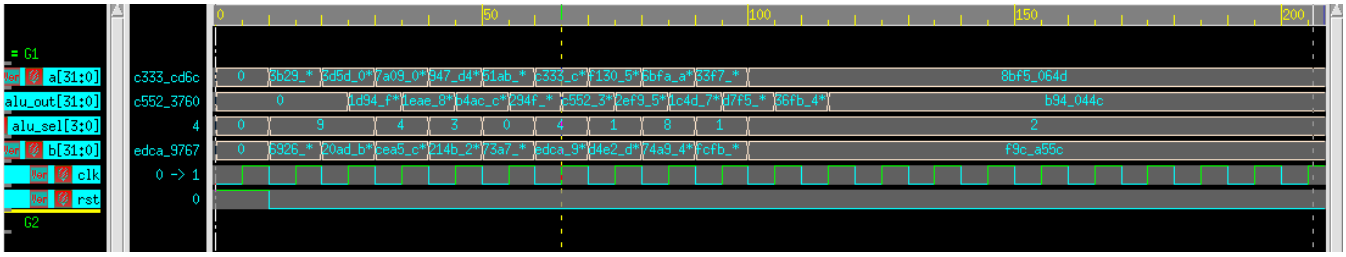
alu_sel: 2 | a:8bf5064d |b:f9ca55c |Result:06676641220h

Schematic Design



waveform





5.0 Design a 32-bit ALU overflow condition of RTL code

File name- RTL_ALU_overflow

Design code

```
//Design a 32_bit ALU_overflow of RTL code


module alu_32 (alu_out,ov_reg,a,b,clk,rst,alu_sel);
input logic clk;
input logic rst;
input logic [31:0]a,b;
input logic [3:0]alu_sel;
output logic [31:0]alu_out;
output logic ov_reg;

logic [31:0] reg_a,reg_b, res_reg;
logic [3:0] sel_reg;
logic overtemp;

//Register inputs and alu_sel

always_ff @(posedge clk or posedge rst) begin
        if(rst)begin
                reg_a <= 32'd0;
                reg_b <=32'd0;
```

```
            sel_reg <= 4'd0;
        end else begin
            reg_a <= a;
            reg_b <= b;
            sel_reg <= alu_sel;
        end
end

//alu operation and overflow
always_comb begin
        ov_reg=1'b0;  //default
        res_reg=32'd0;
        overtemp=1'b0;
        unique case (sel_reg)
        4'd0:begin   //ADD
        res_reg=reg_a +reg_b;
        overtemp=((~reg_a[31] & ~reg_b[31] & res_reg[31]) | (reg_a[31] & reg_b[31] & ~res_reg[31]));
        if (overtemp)
            ov_reg=1;

end

4'd1:res_reg= reg_a - reg_b;      //SUB
4'd2:res_reg= reg_a & reg_b;      //AND
4'd3:res_reg= reg_a | reg_b;      //OR
4'd4:res_reg= reg_a ^ reg_b;      //XOR
4'd5:res_reg= ~reg_a;             //NOT A
```

```
4'd6:res_reg= ~(reg_a & reg_b);   //NAND
4'd7:res_reg= ~(reg_a | reg_b);   //NOR
4'd8:res_reg= reg_a <<1;          //SL
4'd9:res_reg= reg_a >>1;          //SR
default:res_reg=32'hDEADBEEF;     //DEFAULT
endcase
end
//Register output

always_ff @(posedge clk or posedge rst)begin
      if(rst)
            alu_out <=32'd0;
      else
            alu_out <=res_reg;
end
endmodule
```

Testbench

```
//Testbench for 32bit_alu overflow  design
module alu_32overflow_tb();
logic clk,rst;
logic [31:0]a,b;
logic [3:0]alu_sel;
logic [31:0]alu_out;
logic ov_reg;

alu_32
dut(.clk(clk),.rst(rst),.a(a),.b(b),.alu_sel(alu_sel),.alu_out(alu_out),.ov_reg(ov_reg));
```

```
always #5 clk=~clk;

initial begin
	$display("starting ALU test...");
	clk=0;rst=1;
	a=0;b=0;alu_sel=0;
	#10 rst=0;

	repeat (10) begin
		@ (negedge clk);
		a =$urandom;
		b =$urandom;
		alu_sel =$urandom % 10;
		@(posedge clk);
		#1;
		$display("alu_sel: %0d | a:%0h |b:%0h |Result:%oh|
ov_reg:%b",alu_sel,a,b,alu_out,ov_reg);
	end
	#200 $finish();
end

initial begin
	$fsdbDumpfile("myfile.fsdb");
	$fsdbDumpvars(0,alu_32overflow_tb);
end
endmodule
```

Run command

vcs -sverilog alu_32bit_overflow.v alu_32bit_ovfl_tb.sv -kdb -lca -debug_access+all -full64

Simulation Result

Output

alu_sel: 9 | a:3b29e213 |b:69262e70 |Result:00000000000h| ov_reg:0

alu_sel: 9 | a:3d5d03e2 |b:20adb297 |Result:03545170411h| ov_reg:0

alu_sel: 4 | a:7a0900c5 |b:cea5c6ec |Result:03653500761h| ov_reg:0

alu_sel: 3 | a:947d40c |b:214b2b2c |Result:26453143051h| ov_reg:0

alu_sel: 0 | a:51ab091b |b:73a72e45 |Result:05123777454h| ov_reg:1

alu_sel: 4 | a:c333cd6c |b:edca9767 |Result:30524433540h| ov_reg:0

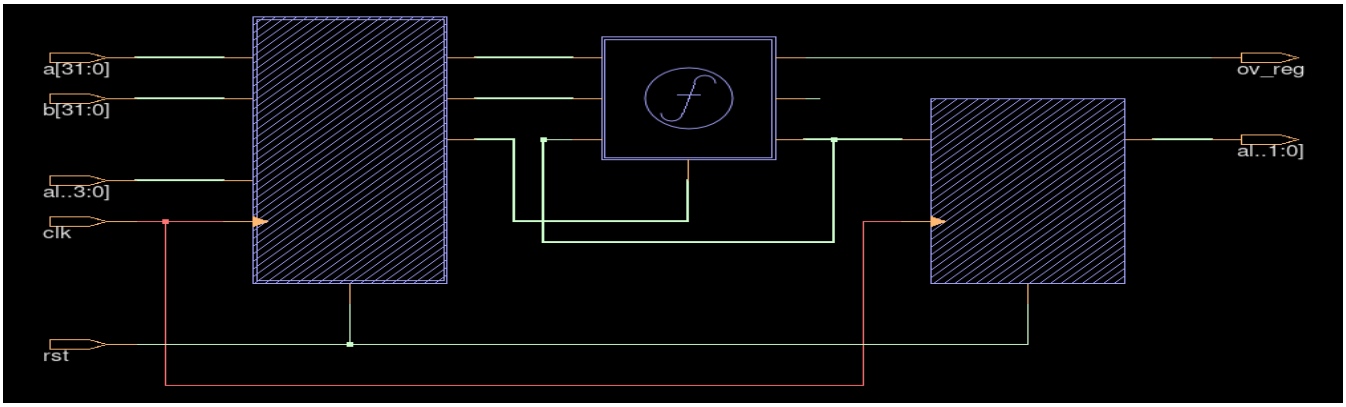
alu_sel: 1 | a:f1305a4a |b:d4e2dca6 |Result:05676255013h| ov_reg:0

alu_sel: 8 | a:6bfaa09d |b:74a9477c |Result:03423276644h| ov_reg:0

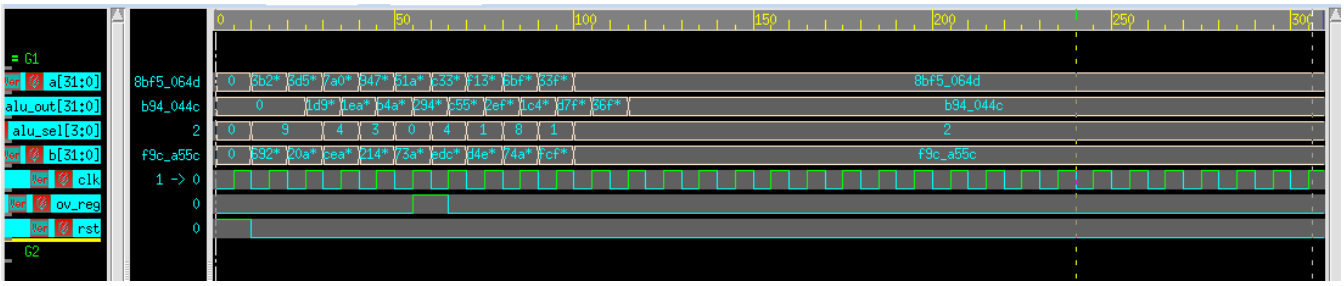
alu_sel: 1 | a:33f729eb |b:fcfbe75b |Result:32775240472h| ov_reg:0

alu_sel: 2 | a:8bf5064d |b:f9ca55c |Result:06676641220h| ov_reg:0

Schematic Design



waveform



6.0 Design a 1-bit full adder by using the gate level modelling

Design code

// Design a 1-bit full adder by using the gate level modelling

module fa(sum,cout,a,b,cin);

input a,b,cin;

output sum,cout;

wire w1,w2,w3;

xor x1(w1,a,b);

xor x2(sum,w1,cin);

and x3(w2,a,b);

and x4(w3,w1,cin);

or x5(cout,w3,w2);

endmodule

Testbench

//testbench of 1-bit full adder

module fa_tb();

reg a,b,cin;

wire sum,cout;

```
fa dut(.sum(sum),.cout(cout),.a(a),.b(b),.cin(cin));
initial
begin
	$monitor("for a=%d,b=%d,cin=%d,and we get sum=%d,cout=%d",a,b,cin,sum,cout);

	a=0;b=0;cin=0;
#5 a=0;b=0;cin=1;
#5 a=0;b=1;cin=0;
#5 a=0;b=1;cin=1;
#5 a=1;b=0;cin=0;
#5 a=1;b=0;cin=1;
#5 a=1;b=1;cin=0;
#5 a=1;b=1;cin=1;
#10 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,fa_tb);
end
endmodule
```

Run command

```
vcs full_1bit_adder.v full_adder_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

```
for a=0,b=0,cin=0,and we get sum=0,cout=0
```

for a=0,b=0,cin=1,and we get sum=1,cout=0

for a=0,b=1,cin=0,and we get sum=1,cout=0

for a=0,b=1,cin=1,and we get sum=0,cout=1

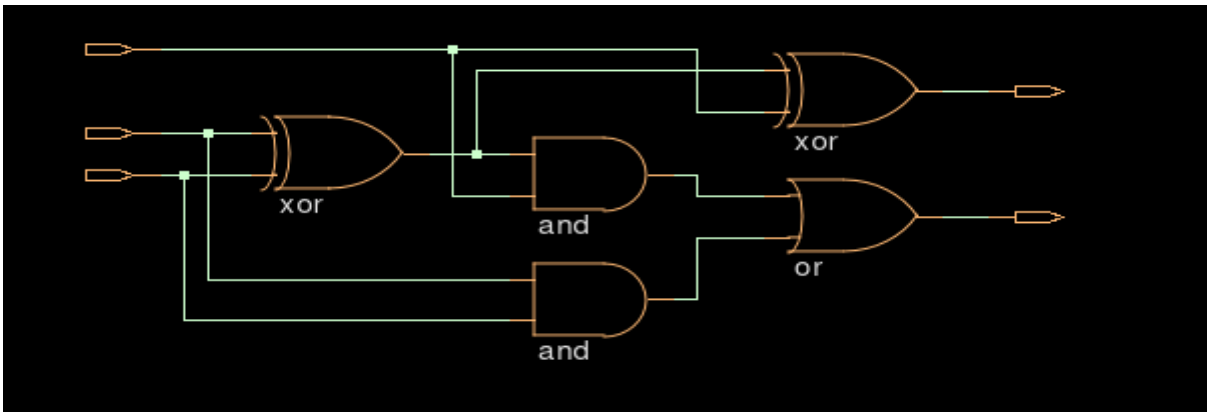
for a=1,b=0,cin=0,and we get sum=1,cout=0

for a=1,b=0,cin=1,and we get sum=0,cout=1

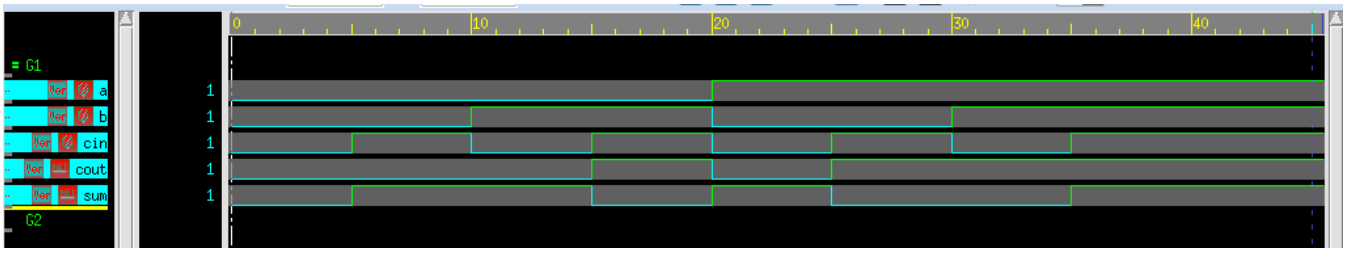
for a=1,b=1,cin=0,and we get sum=0,cout=1

for a=1,b=1,cin=1,and we get sum=1,cout=1

Schematic Design



waveform



7.0 Design a 32-bit full adder using 8-bit full adder by using data flow

File name- FA_32bit_8bit.v

Design code

// Design a 32-bit full adder using 8-bit full adder

module fulladder_8bit(sum,cout,a,b,cin);

```
input [7:0]a;
input [7:0]b;
input cin;
output [7:0]sum;
output cout;

assign {cout,sum}= a+b+cin;

endmodule

module fulladder_32bit(sum,cout,a,b,cin);
input [31:0]a,b;
input cin;
output [31:0]sum;
output cout;

wire [3:0]carry;

fulladder_8bit fa1(.a(a[7:0]),.b(b[7:0]),.cin(cin),.sum(sum[7:0]),.cout(carry[0]));
fulladder_8bit fa2(.a(a[15:8]),.b(b[15:8]),.cin(carry[0]),.sum(sum[15:8]),.cout(carry[1]));
fulladder_8bit
fa3(.a(a[23:16]),.b(b[23:16]),.cin(carry[1]),.sum(sum[23:16]),.cout(carry[2]));
fulladder_8bit fa4(.a(a[31:24]),.b(b[31:24]),.cin(carry[2]),.sum(sum[31:24]),.cout(cout));

endmodule
```

Testbench

File name- FA_32bit_8bit_tb.v

```
// testbench for 32-bit full adder using 8-bit full adder

module fulladder_32bit_tb();
reg [31:0] a,b;
reg cin;
wire [31:0]sum;
wire cout;

fulladder_32bit dut(.a(a),.b(b),.cin(cin),.sum(sum),.cout(cout));

initial
begin
	$monitor("for Time=%0t,a=%h,b=%h,cin=%b,and we get sum=%h,cout=%b",$time,a,b,cin,sum,cout);

	a=0;b=0;cin=0;
#5 a=32'h00000000;b=32'h00000000;cin=0;
#5 a=32'hFFFFFFFF;b=32'hFFFFFFFF;cin=0;
#5 a=32'h00000001;b=32'hFFFFFFFF;cin=0;
#5 a=32'h00000000;b=32'h00000000;cin=1;
#5 a=32'h1212;b=32'h3541;cin=1;

repeat(10)begin
	a=$urandom;
	b=$urandom;
	cin=$urandom_range(0,1);
	#5;
```

```
end
#10 $finish();
end

initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,fulladder_32bit_tb);
end

endmodule
```

Run command

```
vcs -sverilog FA_32bit_8bit.v FA_32bit_8bit_tb.v -kdb -lca -debug_access+all -full64
source env_file
./simv
./simv -verdi
```

Simulation Result

Output

```
for Time=0,a=00000000,b=00000000,cin=0,and we get sum=00000000,cout=0
for Time=10,a=ffffffff,b=ffffffff,cin=0,and we get sum=fffffffe,cout=1
for Time=15,a=00000001,b=ffffffff,cin=0,and we get sum=00000000,cout=1
for Time=20,a=00000000,b=00000000,cin=1,and we get sum=00000001,cout=0
for Time=25,a=05e23536,b=57dea13c,cin=1,and we get sum=5dc0d673,cout=0
for Time=30,a=30aa33e2,b=0279440b,cin=1,and we get sum=332377ee,cout=0
for Time=35,a=f350a940,b=d51778f7,cin=0,and we get sum=c8682237,cout=1
for Time=40,a=30d4b41b,b=025b9db5,cin=0,and we get sum=333051d0,cout=0
for Time=45,a=19bc7a4e,b=5a98a215,cin=1,and we get sum=74551c64,cout=0
```

for Time=50,a=e43a5f72,b=ffbd4196,cin=1,and we get sum=e3f7a109,cout=1

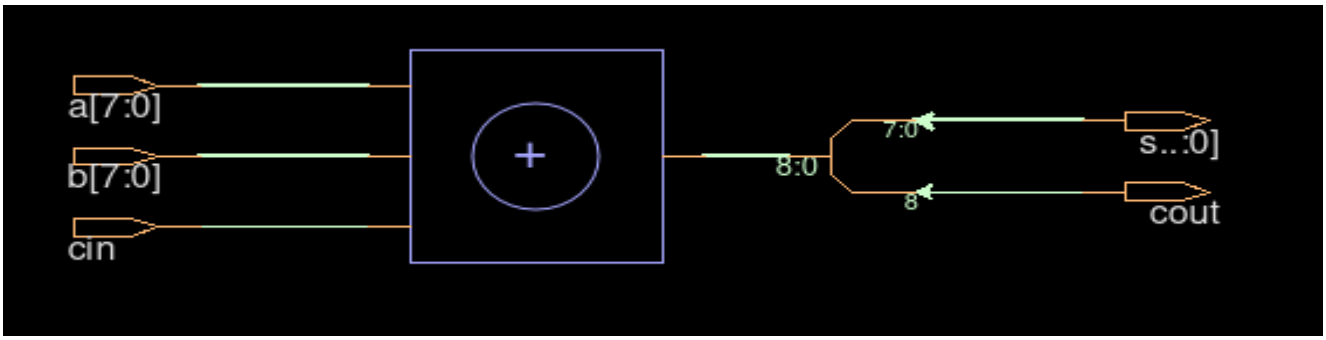
for Time=55,a=9df554c4,b=cac8ccaa,cin=0,and we get sum=68be216e,cout=1

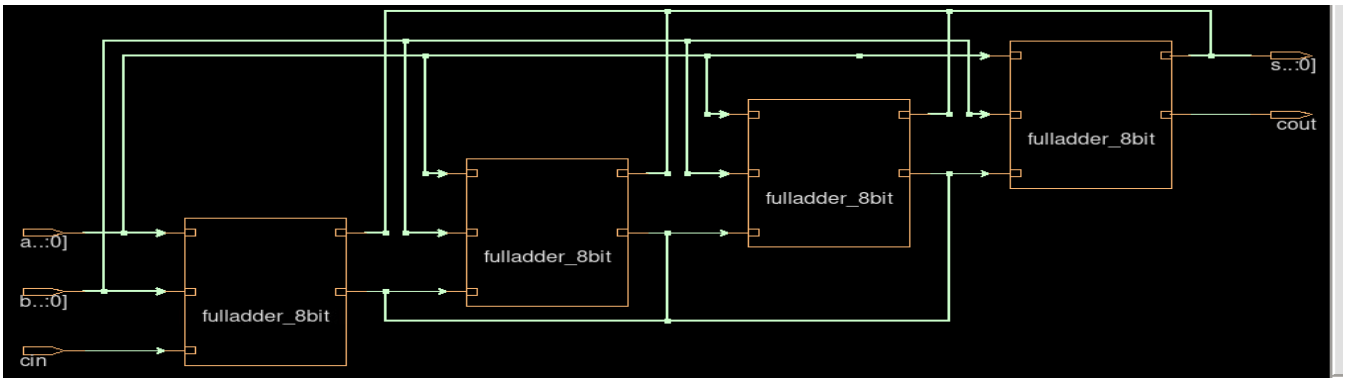
for Time=60,a=11ab26cf,b=38646d4f,cin=0,and we get sum=4a0f941e,cout=0

for Time=65,a=7337dd17,b=76264a88,cin=1,and we get sum=e95e27a0,cout=0

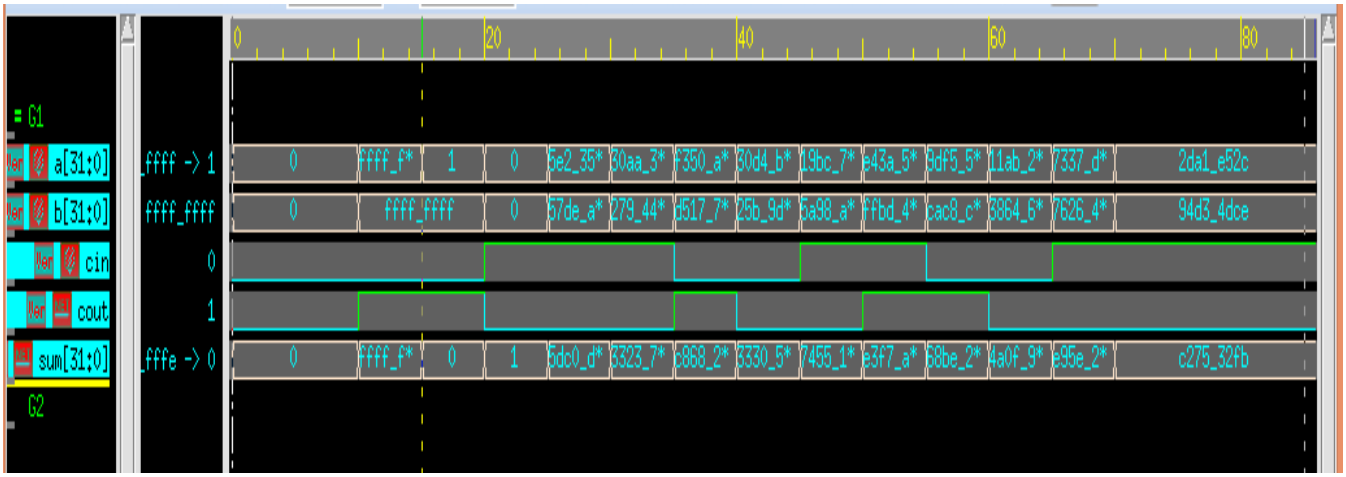
for Time=70,a=2da1e52c,b=94d34dce,cin=1,and we get sum=c27532fb,cout=0

Schematic Design





waveform



8.0 Design a 16-bit full adder using 4-bit full adder by using data flow

File name- FA_16bit_4bit.v

Design code

```
// Design a 16bit full adder using 4-bit full adder


module fulladder_4bit(sum,cout,a,b,cin);
input [3:0]a;
input [3:0]b;
input cin;
output [3:0]sum;
output cout;

assign {cout,sum}= a+b+cin;
endmodule

module fulladder_16bit(sum,cout,a,b,cin);
input [15:0]a;
input [15:0]b;
input cin;
output [15:0]sum;
output cout;
wire [3:0]c;

fulladder_4bit fulladder1(.a(a[3:0]),.b(b[3:0]),.cin(cin),.sum(sum[3:0]),.cout(c[0]));
fulladder_4bit fulladder2(.a(a[7:4]),.b(b[7:4]),.cin(c[0]),.sum(sum[7:4]),.cout(c[1]));
```

```
fulladder_4bit fulladder3(.a(a[11:7]),.b(b[11:7]),.cin(c[1]),.sum(sum[11:7]),.cout(c[2]));

fulladder_4bit
fulladder4(.a(a[15:11]),.b(b[15:11]),.cin(c[2]),.sum(sum[15:11]),.cout(cout));

endmodule
```

Testbench

```
module fulladder_16bit_tb();

reg [15:0] a,b;

reg cin;

wire [15:0]sum;

wire cout;


fulladder_16bit dut(.a(a),.b(b),.cin(cin),.sum(sum),.cout(cout));


initial

begin

      $monitor("for a=%h,b=%h,cin=%b,and we get
sum=%h,cout=%h",a,b,cin,sum,cout);


      a=0;b=0;cin=0;

#5 a=16'h000F;b=16'h0001;cin=0;

#5 a=16'hFFFF;b=16'h0001;cin=0;

#5 a=16'h1234;b=16'h4321;cin=1;

#5 a=16'h1212;b=16'h3541;cin=1;

#10 $finish();

end
```

```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,fulladder_16bit_tb);
end
endmodule
```

Run command

```
vcs FA_16bit_4bit.v FA_16bit_4bit_tb.v -kdb -lca -debug_access+all -full64
```

Simulation Result

output

for a=0000,b=0000,cin=0,and we get sum=0000,cout=0

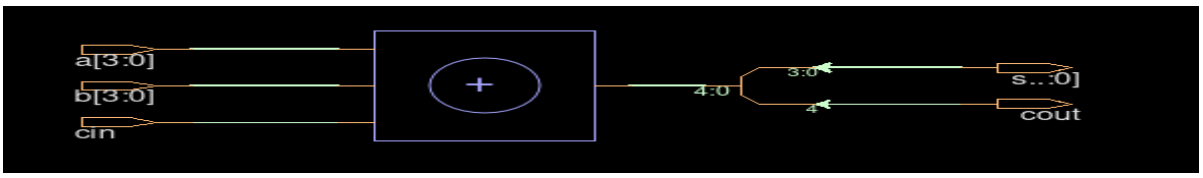
for a=000f,b=0001,cin=0,and we get sum=0010,cout=0

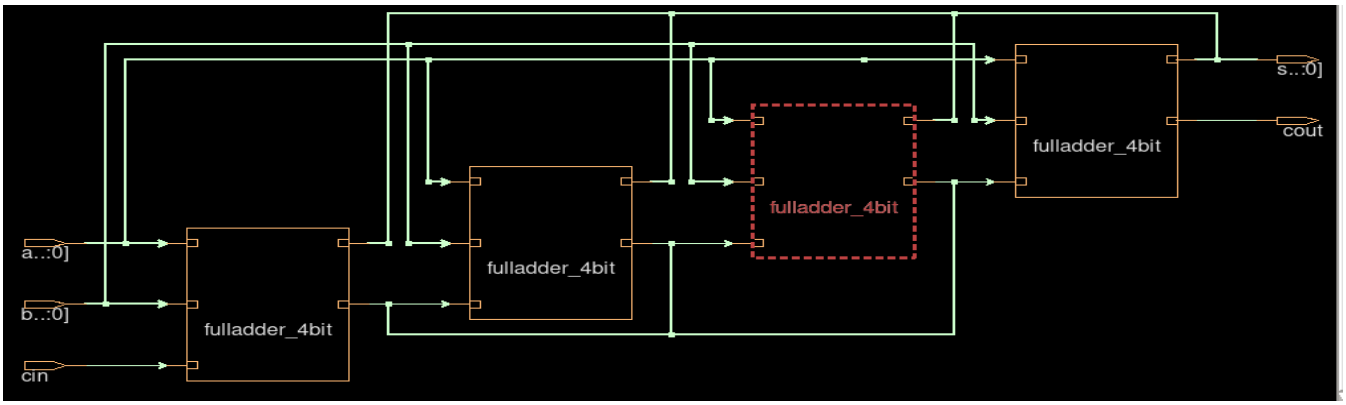
for a=ffff,b=0001,cin=0,and we get sum=0000,cout=1

for a=1234,b=4321,cin=1,and we get sum=5556,cout=0

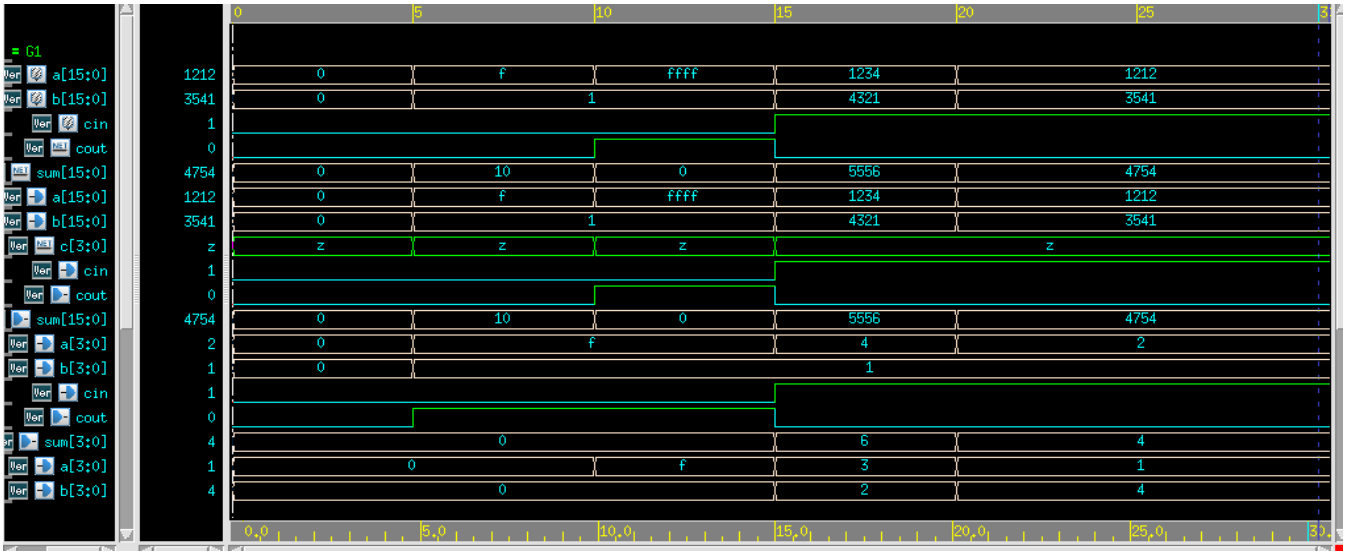
for a=1212,b=3541,cin=1,and we get sum=4754,cout=0

Schematic Design





waveform



9.0 Design an RTL code of 4-bit full adder by using behavioural modelling

Design code

```
// RTL code of 4-bit full adder by using behavioral modelling
module full_adder (sum,cout,a,b,c_in,clk);
input [3:0]a,b;
input c_in,clk;
output reg [3:0]sum;
output reg cout;

reg [3:0] a_reg,b_reg,sum_i;
reg c_reg, cout_i;

always @(posedge clk)
begin
        a_reg <=a;
        b_reg <=b;
```

```
        c_reg <=c_in;
end

always @(posedge clk)
begin
        sum <=sum_i;
        cout <=cout_i;
end

always @(*)
begin
        {cout_i,sum_i}=a_reg +b_reg +c_reg;
end
endmodule
```

Testbench

```
// testbench of 4-bit full adder
//`include "full_adder.v"
module testbench;
reg [3:0]a,b;
reg clk, c_in;
wire [3:0] sum;
wire cout;

full_adder dut (.a(a),.b(b),.clk(clk),.c_in(c_in),.sum(sum),.cout(cout));

always #5 clk = ~clk;
```

```
initial begin
    $fsdbDumpfile("testbench.fsdb");
    $fsdbDumpvars(0,testbench);
    a <= 0;
    b <= 0;
    c_in <= 0;
    clk <= 0;

    #20 a <= 4'b0001; b <= 4'b0001; c_in <= 0; // 2
    $display("a = %b, b = %b, c_in = %b, sum = %b, cout = %b", a, b, c_in, sum, cout);

    #20 a <= 4'b0110; b <= 4'b1010; c_in <= 1; // 17
    $display("a = %b, b = %b, c_in = %b, sum = %b, cout = %b", a, b, c_in, sum, cout);

    #20 a <= 4'b1000; b <= 4'b1111; c_in <= 1; // 24
    $display("a = %b, b = %b, c_in = %b, sum = %b, cout = %b", a, b, c_in, sum, cout);
    #100 $finish;
end
endmodule
```

Run command

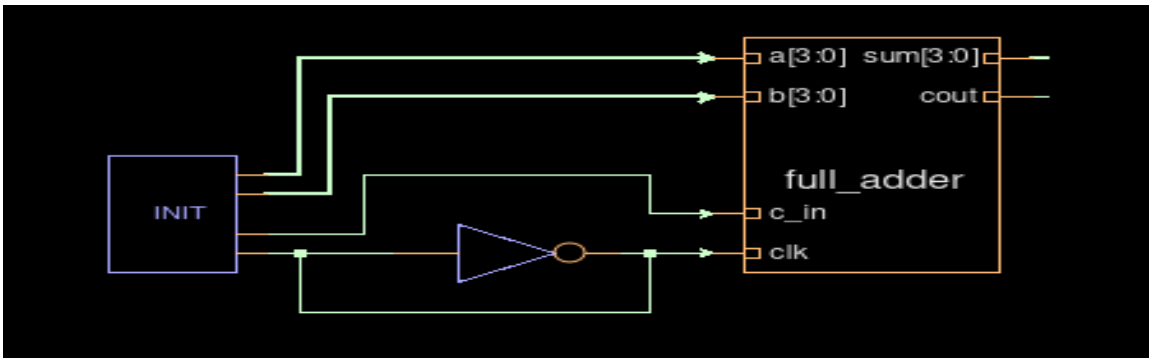
```
vcs full_adder.v full_adder_tb.sv -kdb -lca -debug_access+all -full64
```

Simulation Result

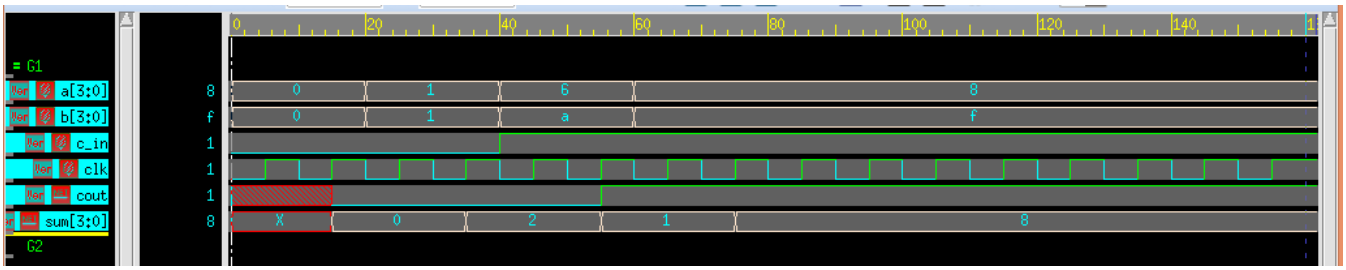
Output

```
a = 0000, b = 0000, c_in = 0, sum = 0000, cout = 0
a = 0001, b = 0001, c_in = 0, sum = 0010, cout = 0
a = 0110, b = 1010, c_in = 1, sum = 0001, cout = 1
```

Schematic Design



waveform



10.0 Design an RTL of a 4-bit full adder and check the functional coverage

File name- rtl_fulladder.v

File name- rtl_fulladder_tb.sv

Design code

```
//RTL FULL ADDER DESIGN
module full_adder (
        input [3:0] A, B,         // 4-bit inputs A and B
        input Clock, C_in,        // Clock and Carry-in
        output reg [3:0] SUM,     // 4-bit sum output
        output reg C_out          // Carry-out output
);


//Internal signals
```

```
reg [3:0] reg1, reg2, sum_i;
reg c_in, c_out;

//Functional Coverage
covergroup cg_full_adder;

coverpoint A;         // Coverpoint for A
coverpoint B;         // Coverpoint for B
coverpoint C_in;      // Coverpoint for Carry-in
coverpoint SUM;       // Coverpoint for the SUM
coverpoint C_out;     // Coverpoint for Carry-out

//You can also add cross-coverage between different signals

cross A, B, C_in;     // Coverage for combinations of A, B, and C_in
cross SUM, C_out;     // Coverage for different SUM and C_out combinations
endgroup

// Create coverage object
cg_full_adder full_adder_cov;

// Always block for latching inputs and storing previous values
always @ (posedge Clock)
begin
        reg1 <= A;         // Store A into reg1
        reg2 <= B;         // Store B into reg2
        c_in <= C_in;      // Store C_in into c_in
```

```
end

//Always block for driving outputs
always @ (posedge Clock)
begin
    SUM <= sum_i;     // Assign sum_i to SUM
    C_out <= c_out;   // Assign c_out to C_out
end
//Always block for the adder logic

always @ *
begin
{c_out, sum_i} = reg1 + reg2 + c_in;   // Perform addition with carry-in
end

// Initialize coverage at the beginning
initial begin
    full_adder_cov = new;  // Create the coverage object
end

// Update coverage during simulation
always @ (A or B or C_in or SUM or C_out)
begin

    full_adder_cov.sample();  //Sample coverage at every change of input/output
end
endmodule
```

Testbench

```
/ tesbench of fulladder
`include "rtl_fulladder.v"
module testbench;
reg [3:0] A, B;
reg Clock, C_in;
wire [3:0] SUM;
wire C_out;

full_adder dut (

        .A(A),
        .B(B),
        .Clock(Clock),
        .C_in(C_in),
        .SUM(SUM),
        .C_out(C_out)
    );

always #5 Clock = ~Clock;

initial begin

        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,testbench);
        A <= 0;
```

```
    B <= 0;
    C_in <= 0;
    Clock <= 0;

    #20 A <= 4'b0001; B <= 4'b0001; C_in <= 0; //1+1 = 2 (binary: 10)
    $display("A = %b, B = %b, C_in = %b, SUM = %b, C_out = %b", A, B, C_in, SUM, C_out);

    #20 A <= 4'b0110; B <= 4'b1010; C_in <= 1; //6+10+1 =17 (binary:10001)
    $display("A = %b, B = %b, C_in = %b, SUM = %b, C_out = %b", A, B, C_in, SUM, C_out);

    #20 A <= 4'b1000; B <= 4'b1111; C_in <= 1; //8+15+1 =24 (binary: 11000)
    $display("A = %b, B = %b, C_in = %b, SUM = %b, C_out = %b", A, B, C_in, SUM, C_out);

    #100 $finish;
end
endmodule
```

Run command

```
vcs -sverilog rtl_fulladder_tb.sv -kdb -lca -debug_access+all -full64
./simv
verdi -cov -covdir simv.vdb/
```

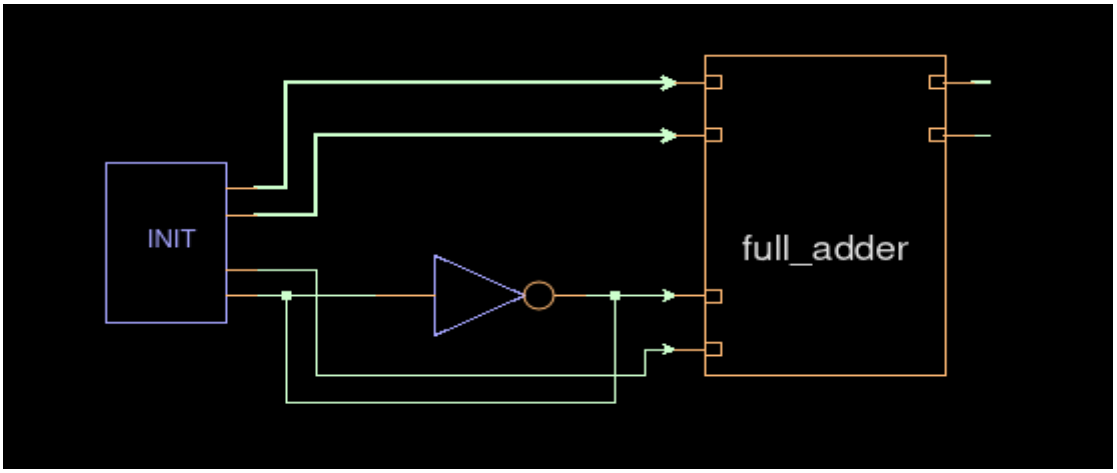
Simulation Result

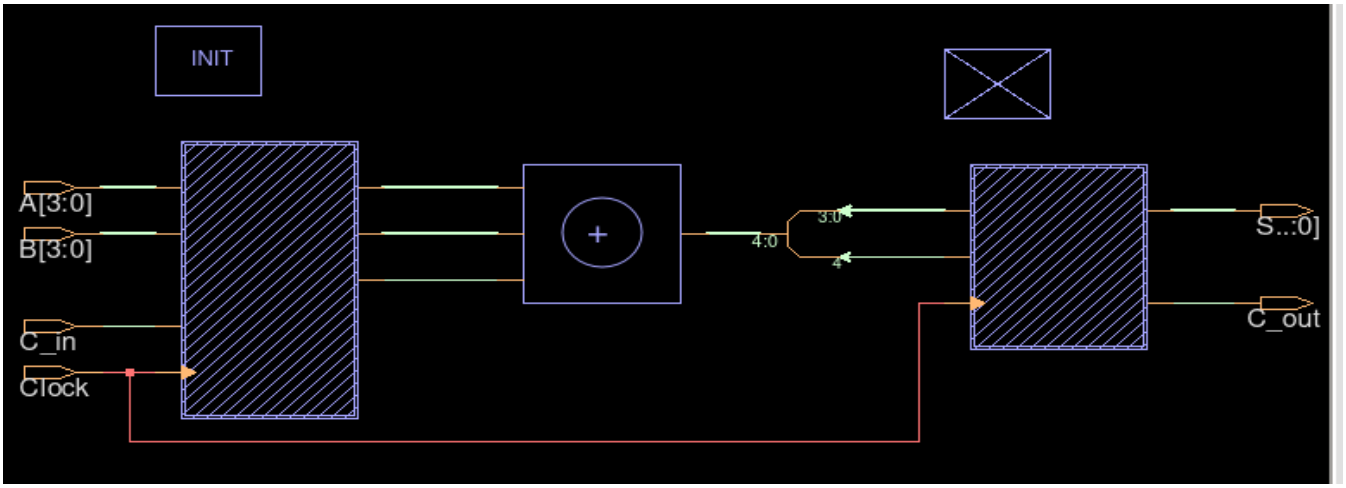
Output

```
A = 0000, B = 0000, C_in = 0, SUM = 0000, C_out = 0
A = 0001, B = 0001, C_in = 0, SUM = 0010, C_out = 0
```

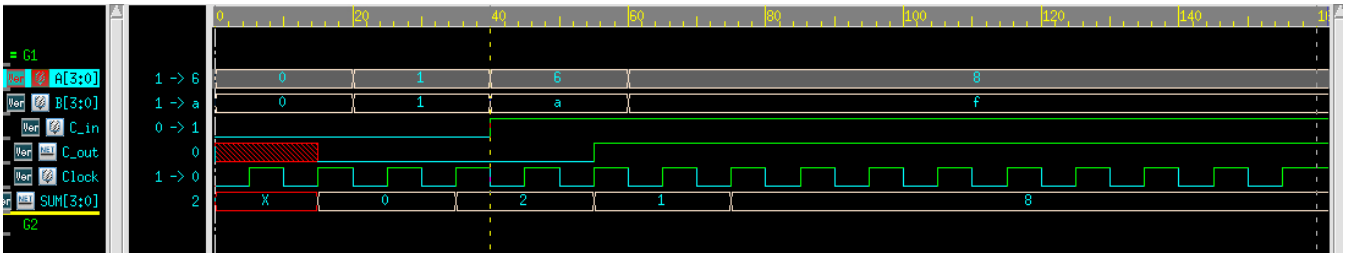
A = 0110, B = 1010, C_in = 1, SUM = 0001, C_out = 1

Schematic Design





waveform



Design a 32-bit full adder using 8-bit full adder by using data flow

Design code

Testbench

Run command

Simulation Result

output

Schematic Design

waveform

Design of 32 bit alu

```
module alu_32(a,b,alu_sel,out);
input [31:0]a,b;
input [3:0]alu_sel,
output [31:0]out,
output reg [31:0]out;

always @(a,b,alu_sel)begin

    case(alu_res)
        4'b0000:out=a+b;
        4'b0001:out=a-b;
        4'b0010:out=a & b;
        4'b0011:out=a |b;
```

```
        4'b0100:out=a ^b;
        4'b0101:out= ~a;
        4'b0110:out= ~(a &b);
        4'b0111:out=~(a |b);
        4'b1000:out= a<<1;
        4'b1001:out=a>>2;
        default:out=0000;
    endcase
end

endmodule
```

```
module full_adder (
input [3:0] A, B, // 4-bit inputs A and B
input Clock, C_in, // Clock and Carry-in
output reg [3:0] SUM, // 4-bit sum output
output reg C_out // Carry-out output
);
// Internal signals
reg [3:0] reg1, reg2, sum_i;
reg c_in, c_out;
// Functional Coverage
covergroup cg_full_adder;
coverpoint A; // Coverpoint for A
coverpoint B; // Coverpoint for B
coverpoint C_in; // Coverpoint for Carry-in
coverpoint SUM; // Coverpoint for the SUM
coverpoint C_out; // Coverpoint for Carry-out
// You can also add cross-coverage between different signals
cross A, B, C_in; // Coverage for combinations of A, B, and C_in
cross SUM, C_out; // Coverage for different SUM and C_out combinations
endgroup
// Create coverage object
cg_full_adder full_adder_cov;


// Always block for latching inputs and storing previous values
always @ (posedge Clock)
begin
reg1 <= A; // Store A into reg1
reg2 <= B; // Store B into reg2
c_in <= C_in; // Store C_in into c_in
```

```
end
// Always block for driving outputs
always @ (posedge Clock)
begin
SUM <= sum_i; // Assign sum_i to SUM
C_out <= c_out; // Assign c_out to C_out
end
// Always block for the adder logic
always @ *
begin
{c_out, sum_i} = reg1 + reg2 + c_in; // Perform addition with carry-in
end
// Initialize coverage at the beginning
initial begin
full_adder_cov = new; // Create the coverage object
end
// Update coverage during simulation
always @ (A or B or C_in or SUM or C_out)
begin
full_adder_cov.sample(); // Sample coverage at every change of input/output
end
endmodule


`include "full_adder.v"
module testbench;
reg [3:0] A, B;
reg Clock, C_in;
wire [3:0] SUM;
wire C_out;
full_adder dut (
.A(A),
.B(B),
.Clock(Clock),
.C_in(C_in),
.SUM(SUM),
.C_out(C_out)
);
always #5 Clock = ~Clock;
initial begin
$fsdbDumpfile("testbench.fsdb");
$fsdbDumpvars(0,testbench);
A <= 0;
B <= 0;
C_in <= 0;
Clock <= 0;


#20 A <= 4'b0001; B <= 4'b0001; C_in <= 0; // 1 + 1 = 2 (binary: 10)
$display("A = %b, B = %b, C_in = %b, SUM = %b, C_out = %b", A, B, C_in, SUM, C_out);
```

```
#20 A <= 4'b0110; B <= 4'b1010; C_in <= 1; // 6 + 10 + 1 = 17 (binary: 10001)
$display("A = %b, B = %b, C_in = %b, SUM = %b, C_out = %b", A, B, C_in, SUM, C_out);
#20 A <= 4'b1000; B <= 4'b1111; C_in <= 1; // 8 + 15 + 1 = 24 (binary: 11000)
$display("A = %b, B = %b, C_in = %b, SUM = %b, C_out = %b", A, B, C_in, SUM, C_out);
#100 $finish;
end
endmodule

module HA(A,B,Clock,Sum,Carry);
input A,B,Clock;
output reg Sum,Carry;

reg A_reg,B_reg,Sum_reg,Carry_reg;

always @ (posedge Clock)
begin
A_reg<=A;
B_reg <=B;
Sum<=Sum_reg;
Carry<=Carry_reg;
end
always @ *
begin
{Carry_reg, Sum_reg} = A_reg + B_reg;
end
endmodule

//Test Bench

module TB();
reg A, B, Clock;
wire Sum, Carry;

HA dut(.A(A), .B(B), .Clock(Clock), .Sum(Sum), .Carry(Carry));

initial Clock = 0;
always #5 Clock = ~Clock;  // posedge at 5, 15, 25, etc.

initial begin
A <= 0; B <= 0; $display("Time=%0t,Clock=%b,A=%b,B=%b",$time,Clock,A,B);
@(posedge Clock);  // Wait for output to update t=5
//@(posedge Clock);//t=15
$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,C
arry);
@(negedge Clock); A <= 0; B <= 1;
$display("Time=%0t,Clock=%b,A=%b,B=%b",$time,Clock,A,B);
@(posedge Clock);
$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,C
arry);
@(negedge Clock); A <= 1; B <= 0;
$display("Time=%0t,Clock=%b,A=%b,B=%b",$time,Clock,A,B);
@(posedge Clock);
$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,C
arry);
@(negedge Clock); A <= 1; B <= 1;
```

```
$display("Time=%0t,Clock=%b,A=%b,B=%b",$time,Clock,A,B);
@(posedge Clock);
$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,C
arry);
@(posedge Clock);
$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,C
arry);
@(posedge Clock);
$display("Time=%0t,Clock=%b,A=%b,B=%b,Sum=%b,Carry=%b",$time,Clock,A,B,Sum,C
arry);

#40 $finish;
end
endmodule
```

input logic clk,
input logic rst,
input logic [31:0] A,
input logic [31:0] B,
input logic [3:0] opcode,
output logic [31:0] result

0 - add
1 - sub
2 - AND
3 - OR
4 - XOR
5 - NOT of A
6 - NAND
7 - NOR
8 - SLL
9 - SRL
Default DEADBEEF

```
module alu32 (
input logic clk,
input logic rst,
input logic [31:0] A,
input logic [31:0] B,
input logic [3:0] opcode,
output logic [31:0] result
);
logic [31:0] opA_reg, opB_reg, res_reg;
logic [3:0] opcode_reg;
// Register inputs and opcode
always_ff @(posedge clk or posedge rst) begin
if (rst) begin
```

```
opA_reg <= 32'd0;
opB_reg <= 32'd0;
opcode_reg <= 4'd0;
end else begin
opA_reg <= A;
opB_reg <= B;
opcode_reg <= opcode;
end
end
// ALU operation
always_comb begin
unique case (opcode_reg)
4'd0 : res_reg = opA_reg + opB_reg; // ADD
4'd1 : res_reg = opA_reg - opB_reg; // SUB
4'd2 : res_reg = opA_reg & opB_reg; // AND
4'd3 : res_reg = opA_reg | opB_reg; // OR
4'd4 : res_reg = opA_reg ^ opB_reg; // XOR
4'd5 : res_reg = ~opA_reg; // NOT A
4'd6 : res_reg = ~(opA_reg & opB_reg); // NAND
4'd7 : res_reg = ~(opA_reg | opB_reg); // NOR
4'd8 : res_reg = opA_reg << 1; // SLL
4'd9 : res_reg = opA_reg >> 1; // SRL
default: res_reg = 32'hDEADBEEF; // Default
endcase
end
// Register output
always_ff @(posedge clk or posedge rst) begin
if (rst)
result <= 32'd0;
else
result <= res_reg;
end
endmodule

module tb_alu32;
logic clk, rst;
logic [31:0] A, B;
logic [3:0] opcode;
logic [31:0] result;
// Instantiate ALU
alu32 call1 (
.clk(clk),
.rst(rst),
.A(A),
.B(B),
.opcode(opcode),
.result(result)
);
// Clock generation
```

```
always #5 clk = ~clk;
initial begin
$display("Starting ALU test...");
clk = 0; rst = 1;
A = 0; B = 0; opcode = 0;
#10 rst = 0;
repeat (10) begin
@(negedge clk);
A = $urandom;
B = $urandom;
opcode = $urandom % 10;
@(posedge clk);
#1;
$display("OP: %0d | A: %0h | B: %0h | RESULT: %0h", opcode, A, B, result);
end
#100 $finish;
end
initial
begin
$fsdbDumpfile("myfile.fsdb");
$fsdbDumpvars(0,tb_alu32);
end
/*
initial
begin
$dumpfile("file.vcd"); // VCD output file
$dumpvars(0, tb_alu32);
end*/
endmodule
```

```
cp -rf /data/labs/po/internship_may_jun_25/ ./
```

```
cp -rf /data/labs/ti/internship_may_jun_25/ ./
```

```
source -echo -verbose ./rm_setup/dc_setup.tcl
```

```
./../ref/lib/stdcell_rvt/saed32rvt_tt0p78vn40c.db
```

Design a 32-bit full adder using 8-bit full adder by using data flow

Design code

Testbench

Run command

Simulation Result

output

Schematic Design

waveform